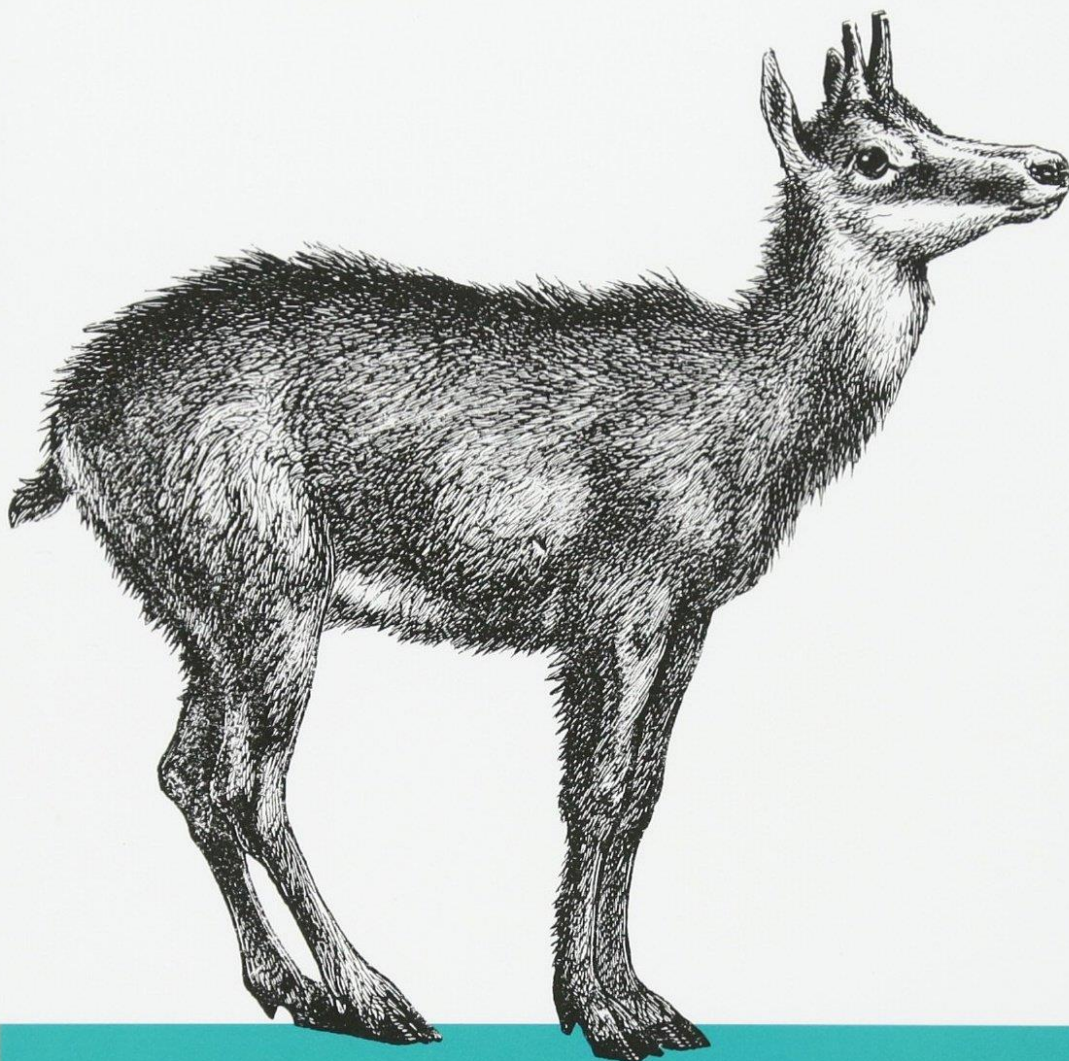


Dive into the Future of Web Development



HTML5

Up and Running

O'REILLY® | Google™ PRESS

Mark Pilgrim

DIVE INTO HTML5 POR MARK PILGRIM

100% TRADUZIDO

PELA COMUNIDADE



[://diveintohtml5.com.br/index.html](https://diveintohtml5.com.br/index.html)

<https://diveintohtml5.com.br/index.html>



ive Into HTML5 procura elaborar uma seleção de funcionalidades da especificação da HTML5 e outras especificações interessantes escolhidas a dedo. O manuscrito final foi publicado no formato em papel pela editora O'Reilly, sobre a marca da Google Press. Compre a obra impressa — intitulada como “HTML5: Up & Running”. Esse trabalho permanece online sob os termos da licença CC-BY-3.0. Seu feedback será sempre bem-vindo.

Nota: Com o súbito abandono de Mark Pilgrim da internet, surgiu a iniciativa de dar continuidade ao seu trabalho; levando-o para ainda mais pessoas através da tradução para língua portuguesa. Nossa ideia é manter ativamente esse projeto; atualizando e refletindo o relevante e atual estado da HTML5, assim como tinha sido feito enquanto estava em posse de Mark. Para mais informações sobre como contribuir nesse processo de tradução ou reportar erros encontrados, acesse nosso Github.

SUMÁRIO

1. Introdução: Cinco Coisas Que Você Deveria Saber Sobre HTML500
2. A Tendenciosa História Da HTML511
3. Detectando Funcionalidades Da HTML5: É Elementar, Meu Caro Watson22
4. O Que Significa Tudo Isso?33
5. Vamos Chamar De Superfície de Desenho44
6. Vídeo Em Um Flash (Sem Aquela Outra Coisa)55
7. Você Está Aqui (Assim Como Todo Mundo Está)66
8. Um Lugar Para Colocar Suas Coisas77
9. Vamos Tornar Isso Offline88
10. Uma Forma De Loucura99
11. “Distribuído”, “Extensível” e Outras Palavras Bonitas1010
12. Manipulando Histórico Para Diversão & Lucro1111
13. O Único Quase-Alfabético Sem-Besteiras Guia Para Detectar TudoAA
14. HTML5 Espiadas, Cutucadas e ApontadoresBB

INTRODUÇÃO: AS CINCO COISAS QUE VOCÊ DEVERIA SABER SOBRE HTML5

exibir índice analítico



1. Não é uma grande coisa só



Você deve estar se perguntando: “Como posso começar a usar HTML5 se os navegadores antigos não a suportam?” Mas a própria questão nos engana. HTML5 não é uma grande coisa só; é uma coleção de funcionalidades individuais. Então você não pode detectar suporte ao “HTML5,” porque isso não faz o menor sentido. Mas você *pode* detectar o suporte a funcionalidades individuais como canvas, vídeo e geolocalização.

Você pode achar que o HTML é um conjunto de tags e colchetes angulados. Essa é uma parte importante, mas não é a história toda. A especificação do HTML5 também define como esses colchetes angulados vão interagir com JavaScript, através do Document Object Model (DOM). A HTML5 não define apenas uma tag `<video>`; também há uma DOM API correspondente para objetos de vídeo no DOM. Você pode usar essa API para detectar o suporte a diferentes formatos de vídeo, executar um vídeo, pausar, dar mute no áudio, identificar quanto o vídeo foi baixado, e todo o resto necessário para criar uma experiência rica ao usuário com relação a própria tag `<video>`.

O Capítulo 2 e o Apêndice A irão te ensinar como detectar de forma adequada o suporte para cada nova funcionalidade da HTML5.

2. Você não precisa jogar nada fora

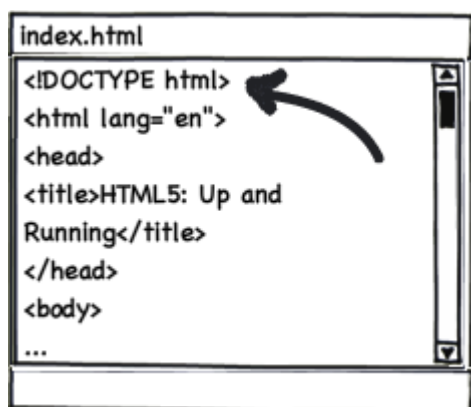


Ame ou odeie, uma coisa você não pode negar é que a HTML 4 é o formato de marcação mais bem sucedido que já existiu. HTML5 é criado sobre esse sucesso. Você não precisa jogar fora sua marcação existente. Você não precisa reaprender coisas que já sabe. Se a sua aplicação web funcionou ontem em HTML 4, continuará funcionando hoje em HTML5. Ponto final.

Agora, se você quiser *aprimorar* suas aplicações web, você veio ao lugar certo. Veja esse exemplo concreto: HTML5 suporta todos os controles de formulário da HTML 4, mas também inclui novos controles de input. Algumas são adições pedidas há tempos como sliders e date pickers; outras são mais sutis. Por exemplo, o tipo de input `email` parece exatamente como uma caixa de texto, porém os navegadores de dispositivos móveis irão customizar sua tela no teclado para facilitar a inserção de endereços de email. Navegadores mais antigos que não suportam o tipo de input `email` irão tratar o tipo de input como um input de texto comum, e o formulário ainda funciona sem mudanças na marcação ou scripts corretores. Isso significa que você pode começar a melhorar seus formulários web *hoje*, mesmo que alguns dos seus visitantes estejam presos ao Internet Explorer 6.

Leia todos os mínimos detalhes sobre formulários em HTML5 no Capítulo 9.

3. É fácil de começar

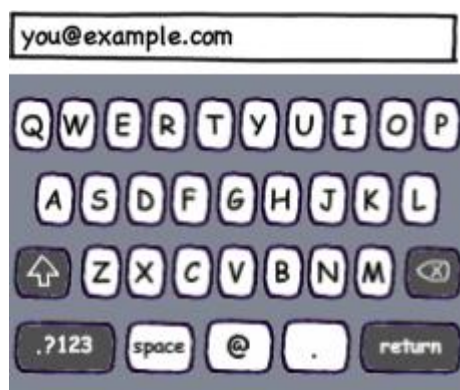


“Atualizar” para HTML5 pode ser tão simples quanto atualizar o seu *doctype*. O *doctype* já deveria estar na primeira linha de toda página HTML. Versões anteriores do HTML definiram muitos *doctype*s, e escolher o *doctype* correto pode ser complicado. Em HTML5, há apenas um *doctype*:

```
<!DOCTYPE html>
```

Atualizar para o *doctype* da HTML5 não irá causar problemas a sua marcação atual, porque todas as tags definidas em HTML4 são suportadas em HTML5. Mas possibilitará você usar — e validar — novos elementos semânticos como `<article>`, `<section>`, `<header>`, e `<footer>`. Você irá aprender sobre todos esses novos elementos no Capítulo 3.

4. Já funciona



Caso você queira desenhar em *canvas*, tocar um vídeo, implementar melhores formulários ou construir aplicações web que funcionem offline, você descobrirá que a HTML5 já é bastante suportada pelos navegadores. Firefox, Safari, Chrome, Opera, e navegadores móveis já suportam *canvas* (Capítulo 4), vídeo (Capítulo 5), geolocalização (Capítulo 6), armazenamento local (Capítulo 7), e mais. Google já suporta *microdata annotations* (Capítulo 10). Até mesmo a Microsoft — raramente conhecida por suportar padrões — suporta a maioria das funcionalidades da HTML5 no Internet Explorer 9.



Cada capítulo desse livro inclui quadros de compatibilidade com os navegadores mais familiares. Porém o mais importante é que cada capítulo inclui uma discussão franca das suas opções se você precisar suportar navegadores antigos. As funcionalidades da HTML5 como geolocalização (Capítulo 6) e vídeo (Capítulo 5) foram primeiramente providas por plugins como Gears ou Flash nos navegadores. Outras funcionalidades, como *canvas* (Capítulo 4), podem ser emuladas inteiramente em JavaScript. Esse livro vai ensinar você como atingir

funcionalidades nativas em navegadores modernos, sem deixar os navegadores mais antigos para trás.

5. Está aqui para ficar

Tim Berners-Lee inventou a world wide web no início dos anos 1990. Mais tarde ele fundou a W3C para agir como um administrador dos padrões da web, o que a organização vem fazendo por mais de 15 anos. Isso é o que a W3C tem a dizer sobre o futuro dos padrões da web, em Julho de 2009:

Hoje o Diretor anunciou que quando a licença do grupo de trabalho sobre XHTML 2 expirar, como programado para o final de 2009, a licença não será renovada. Fazendo isso e aumentando os recursos no grupo de trabalho de HTML, a W3C espera acelerar o progresso da HTML5 e clarificar a posição da W3C acerca do futuro da HTML.

HTML5 está aqui para ficar. Vamos mergulhar.

Nº1.

COMO CHEGAMOS AQUI?

exibir índice analítico



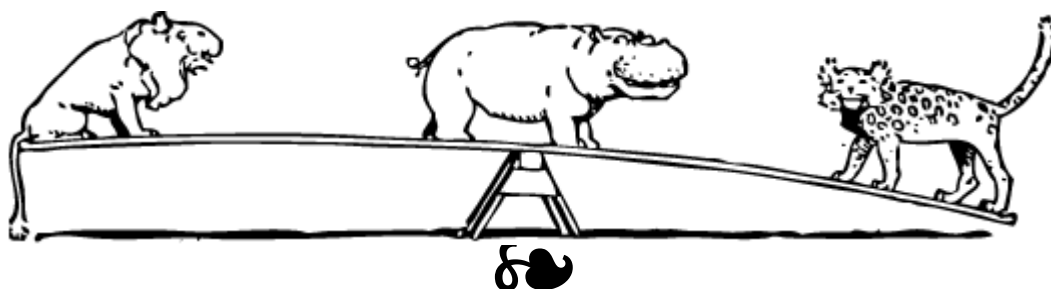
MERGULHANDO



Recentemente, me deparei com uma citação de um desenvolvedor da Mozilla sobre a tensão que existe ao criar padrões:

Implementações e especificações devem dançar delicadamente juntos. Você não quer que uma implementação ocorra antes que a especificação esteja completa, pois as pessoas dependem de detalhes da implementação e isso faz parte da especificação. Entretanto, você também não quer que a especificação esteja completa antes da implementação e dos testes feitos pelo autor com esta implementação, pois você precisa de um feedback. É inevitável que haja uma tensão aqui, mas isso é só uma pequena confusão.

Mantenha esta citação na sua mente, e me deixe explicar como a HTML5 surgiu.



MIME TYPES

Este livro é sobre HTML5, não é sobre as versões anteriores da HTML e não é sobre qualquer versão de XHTML. Mas para entender a história da HTML5 e as motivações por trás dela, você precisa entender primeiro de alguns detalhes técnicos. Especificamente, MIME types.

Toda vez que o seu navegador chama uma página, o servidor web envia “cabeçalhos(headers)” antes de enviar a marcação de página em si. Esses cabeçalhos normalmente são invisíveis, mas existem ferramentas de desenvolvimento que os tornam visíveis caso necessário. Estes cabeçalhos são importantes, pois eles informam ao navegador como interpretar a marcação da página. A parte mais importante do cabeçalho é chamada de Content-Type, e se parece com isso:

Content-Type: text/html

“text/html” é chamado de “content type” (tipo de conteúdo) ou “MIME type” da página. Este cabeçalho é a **única** coisa que determina o que um recurso realmente é, e portanto como ele deve ser renderizado. Imagens tem seu próprio MIME types (image/jpeg para imagens JPEG, image/png para imagens PNG, e por ai vai). Arquivos JavaScript tem seu próprio MIME type. Folhas de estilo CSS tem seu próprio MIME type. Tudo tem seu próprio MIME type. A web funciona por causa dos MIME types.

Claro que a realidade é um pouco mais complicada que apenas isso. A primeira geração de servidores web (e eu estou falando de servidores web de 1993) não enviavam o cabeçalho Content-Type, pois ele ainda não existia. (Ele não foi inventado antes de 1994.) Por causa da compatibilidade até 1993, alguns dos navegadores populares da época ignoravam o cabeçalho Content-Type em alguns momentos. (Isto é chamado de “content sniffing.”) Mas como regra geral, tudo que é procurado na web — páginas HTML, imagens, scripts, videos, PDFs, tudo que tem uma URL — é exibido com seu MIME type específico no cabeçalho Content-Type.

Guarde isto na sua cartola. Nós voltaremos a esse assunto.



UMA LONGA DIGRESSÃO ENTRE OS PADRÕES QUE FORAM FEITOS



Porque existe um elemento `<img>`? Esta não é uma pergunta que você vê todo dia. Obviamente *alguém* criou ela. Estas coisas não aparecem do nada. Todo elemento, todo atributo, toda funcionalidade da HTML que você já usou alguma vez foi criada por alguém, que decidiu como ela deveria funcionar, e escreveu todo o código. Estas pessoas não são deuses ou invencíveis. São apenas pessoas. Pessoas inteligentes, com certeza. Mas apenas pessoas.

Uma das grandes coisas sobre padrões "abertos" é que você pode voltar no tempo e poder responder esse tipo de perguntas. As discussões ocorrem em listas de emails, que geralmente são arquivadas e podem ser procuradas depois. Então eu decidi fazer um pouco de "arqueologia de email" para tentar responder a seguinte pergunta, "Porque nós temos um elemento `<img>`?" E eu tive que voltar para antes que uma organização chamada World Wide Web Consortium (W3C) existisse. Eu voltei aos primeiros dias da web, quando você ainda podia contar o número de servidores web com as duas mãos e talvez alguns dedos do pé.

(Tem alguns erros tipográficos nas citações abaixo. Eu decidi deixar eles intactos para manter a precisão histórica)

Em 25 de Fevereiro de 1993 Marc Andreessen escreveu:

Eu gostaria de propor uma nova tag HTML:

IMG

O argumento obrigatório é: `SRC="url"`.

Ela serve para apontar um arquivo bitmap ou pixmap para o navegador interpretar como uma imagem no meio do texto no ponto de ocorrência da tag.

Como exemplo temos:

```
<IMG SRC="file:///foobar.com/foo/bar/blargh.xbm">
```

(Não há uma tag de fechamento; é uma tag standalone.)

Esta tag pode ser usada como link como qualquer outra coisa; quando isto acontece, ela vira um ícone sensível a ativação exatamente como um texto de âncora comum.

Navegadores devem ter flexibilidade para os formatos de imagem que irão suportar. Xbm e Xpm são bons para suportar, por exemplo. Se um navegador não conseguir renderizar o formato dado, ele pode fazer o que quiser (No X Mosaic irá aparecer um bitmap padrão como substituto da imagem).

Isso é uma função necessária no X Mosaic; nós temos isso funcionando, e nós iremos usar internamente pelo menos. Eu estou bastante aberto a sugestões de como isso pode ser feito com

HTML; se você tem uma ideia melhor que a minha, por favor me informe. Eu sei que é difícil lidar com formatos de imagem, mas eu não vejo uma alternativa além da que eu acabei de falar "deixe o navegador fazer o que ele consegue" e esperar que a solução perfeita apareça (MIME, um dia, quem sabe)

Xbm e Xpm eram formatos gráficos populares no Unix.

"Mosaic" foi um dos primeiros navegadores web. ("X Mosaic" era a versão que rodava no Unix.) Quando ele mandou essa mensagem no início de 1993, Marc Andreessen ainda não havia fundado a companhia que fez ele famoso, Mosaic Communications Corporation, nem tinha começado a trabalhar no principal produto da empresa: "Mosaic Netscape." (Você deve conhecer melhor pelos nomes posteriores, "Netscape Corporation" e "Netscape Navigator.")

"MIME, um dia, quem sabe" é uma referência a negociação de conteúdo, que é uma funcionalidade do HTTP onde um cliente (como um navegador web) diz ao servidor (como um servidor web) qual tipo de recursos ele suporta (como image/jpeg) para que então o servidor retorne para o cliente no seu formato preferido O protocolo HTTP original foi definido em 1991 (a única versão que havia implementada em Fevereiro de 1993) não possuía nenhuma maneira dos clientes dizerem aos servidores que tipo de imagens eram suportadas, o que levou ao dilema de design que Marc encontrou.

Algumas horas depois, Tony Johnson respondeu:

Eu tenho algo bastante similar no Midas 2.0 (em uso aqui no SLAC, e haverá um release público a qualquer semana), exceto que todos os nomes são diferentes, e tem um argumento extra `NAME="name"`. Tem quase exatamente a mesma funcionalidade que você propôs na tag `IMG`, como por exemplo:

```
<ICON name="NoEntry" href="http://note/foo/bar/NoEntry.xbm">
```

A ideia do parâmetro `name` é de permitir ao navegador o uso de imagens internas. Se o nome for o mesmo que uma imagem interna, ele usará ela ao invés de buscar a imagem. O nome pode atuar também como uma dica para quando o navegador rodar no terminal como um tipo de símbolo para substituir a imagem.

Eu não ligo muito sobre os nomes dos parâmetros ou das tags, mas eles devem ser sensíveis se forem utilizados para mais de uma coisa. Eu não ligo muito para abreviações, ou seja porque não `IMAGE=` e `SOURCE=`. Eu prefiro `ICON` uma vez que é menos que `IMAGE`, mas talvez `ICON` seja uma palavra sobrecarregada, não?

Midas foi um outro dos primeiros navegadores, contemporâneo ao X Moisc. Ele era multiplataforma; funcionava tanto no Unix quanto no VMS. "SLAC" refere-se ao Stanford Linear Accelerator Center, agora SLAC National Accelerator Laboratory, foi o lugar que hospedou o primeiro servidor web dos Estados Unidos (na verdade o primeiro servidor web

fora da Europa). Quando Tony escreveu essa mensagem, SLAC era um ancestral da WWW, e hospedava cinco páginas por gritantes 441 dias.

Tony continuou:

Enquanto ainda estamos no assunto de novas tags, eu tenho outra, de algum modo parecida, tag que eu gostaria de suportar no Midas 2.0. A princípio ela é:

```
<INCLUDE HREF="...">
```

A intenção aqui é de poder colocar um segundo documento dentro do primeiro documento no ponto que a tag aparece. A princípio o documento referenciado pode ser qualquer coisa, mas o objetivo principal é de permitir imagens (nesse caso de tamanho arbitrário) para ser incluída dentro de documentos. Novamente a intenção é de que quando o HTTP2 surgir, o formato do documento incluído possa ser negociado separadamente.

“HTTP2” é uma referência ao HTTP básico definido em 1992. Neste ponto, no início de 1993, ele ainda não estava largamente implementado. O rascunho conhecido HTTP2 foi eventualmente padronizado e implementado como "HTTP 1.0" (embora não por mais três anos). HTTP 1.0 não incluía a requisição de cabeçalhos para negociação de conteúdo, a.k.a. “MIME, um dia, quem sabe.”

Tony continuou:

Uma alternativa que considerei foi:

```
<A HREF="..." INCLUDE>See photo</A>
```

Eu não gosto muito de colocar mais funcionalidade na tag <A>, mas a idéia aqui é de manter a compatibilidade entre os navegadores que não podem honrar com o parâmetro INCLUDE. A intenção é que os navegadores que entendam INCLUDE, alterem o texto do link (nesse caso "See photo") com o documento incluído (foto), enquanto os navegadores antigos ou burros ignorem completamente a tag INCLUDE.

Esta proposta nunca foi implementada, no entanto a ideia de colocar um texto quando a imagem não é encontrada, é uma importante técnica de acessibilidade esquecida pela proposta inicial da tag <IMG de Marc. Anos depois esse atributo foi incluído como a tag , que o Netscape mostrava erroneamente o nome quando se colocava o mouse em cima da imagem.

Algumas horas depois que Tony enviou aquela mensagem, Tim Berners-Lee respondeu:

Eu imaginei que figuras poderiam ser representadas como

```
<a name=fig1 href="fghjkdfghj" REL="EMBED, PRESENT">Figure </a>
```

onde a relação entre os valores significaria

EMBED Coloque isso quando apresentar

PRESENT Mostre isso sempre que o documento fonte for apresentado

Veja que você pode ter diversas combinações disso, e se um navegador não suporta nenhuma, ele não quebra

[Eu] vejo que usando esse método para ícones selecionáveis significam links. Hmmm. Mas eu não quero nenhuma tag especial

Esta proposta nunca foi implementada, mas o atributo rel continua por aí.

Jim Davis adicionou:

Seria legal se houvesse um content type específico, por exemplo:

```
<IMG HREF="http://nsa.gov/pub/sounds/gorby.au" CONTENT-TYPE=audio/basic>
```

Mas eu estou completamente disposto a viver com a necessidade de especificar o content type pela extensão do arquivo.

Esta proposta nunca foi implementada, mas o Netscape posteriormente incluiu o suporte de inserir áudio e vídeo com o elemento <embed>.

Jay C. Weber perguntou:

Enquanto imagens estão no topo da minha lista de tipos de mídias num navegador WWW, eu não acho que nós devemos incluir especificações idiossincráticas para cada tipo de mídia. O que aconteceu com o entusiasmo de usar um mecanismo de MIME type?

Marc Andreessen respondeu:

Isto não é um substituto para o futuro uso do MIME como um padrão do mecanismo; Isso apenas dá a implementação necessária e simples da funcionalidade independente do MIME.

Jay C. Weber respondeu:

Vamos temporariamente esquecer do MIME, se é o que está atrapalhando. Minha objeção é a discussão sobre "como nós iremos incluir o suporte a imagens" e não sobre "como nós vamos suportar os diversos problemas nas diversas mídias"

De outra maneira, semana que vem alguém vai sugerir ‘vamos colocar uma nova tag <AUD SRC="file://foobar.com/foo/bar/blargh.snd">’ para áudio.

Isto é muito custo no lugar de usar algo que generalize.

Com o benefício de termos uma retrospectiva, parece que as preocupações de Jay estavam bem fundamentadas. Levou mais de uma semana, mas o HTML5 finalmente inclui os novos elementos <video> e <audio>.

Respondendo a mensagem original de Jay, Dave Raggett disse:

Verdade! Eu gostaria de considerar toda a gama de possibilidade de tipo de arte em imagens, além da possibilidade de negociação do formato. A mensagem de Tim sobre áreas clicáveis nas imagens também são importantes.

Mais tarde em 1993, Dave Raggett propôs HTML+ como uma evolução do padrão HTML. Esta proposta nunca foi implementada, e foi substituída pela HTML 2.0. HTML 2.0 foi uma "retrospectiva", o que significa que formalizou as funcionalidades já em uso “Essa especificação reúne, esclarece e formaliza o conjunto de funcionalidades que grosseiramente corresponde as capacidades do HTML que estavam em uso em Junho de 1994.”

Dave mais tarde escreveu a HTML 3.0, baseado no rascunho feito por eles do HTML+. Fora isso existia a referência da W3C do Arena, HTML 3.0 nunca foi implementado, e foi substituído pelo HTML 3.2, outra "retrospectiva": “HTML 3.2 incluiu largamente outras funcionalidades como tabelas, applets e textos ao redor de imagens, enquanto mantinha a retro-compatibilidade com o padrão existente: HTML 2.0.”

Dave mais tarde foi co-autor do desenvolvimento da HTML 4.0, desenvolveu a HTML Tidy, e ajudou com as especificações do XHTML, XForms, MathML, e outras especificações modernas da W3C.

Voltando para 1993, Marc respondeu a Dave:

Na verdade, talvez nós devêssemos pensar numa linguagem procedural genérica para gráficos que com ela nós possamos incluir hyperlinks aleatórios anexados a ícones, imagens, ou texto, ou qualquer coisa. Alguém vê as capacidades Intermedia disso?

Intermedia foi um projeto de uso de hipertexto da Brown University. Ela foi desenvolvida de 1985 até 1991 e rodava no A/UX, um sistema operacional Unix-like utilizado nos primeiros computadores Macintosh.

A ideia de "uma linguagem procedural genérica para gráficos" foi eventualmente implementada. Navegadores modernos suportam tanto SVG (marcação declarativa com scripts

embutidos) e <canvas> (uma API procedural e direta para gráficos), mesmo que começasse como uma extensão proprietária antes de começar a ser "revista" pela WHATWG.

Bill Janssen respondeu:

Outros sistemas para olharmos que tem alguma noção disso (bastante válida) são Andrew e Slate. Andrew foi feito com _insets_, e cada um deles tem um tipo interessante, como texto, bitmap, desenhos, animações, mensagens, planilhas, etc. A noção de inclusão arbitrária e recursiva está presente, então um inset de qualquer tipo pode ser incluído em qualquer lugar que suporte incorporação. Por exemplo, um inset pode ser incluído em qualquer ponto do texto de um widget de texto, ou em qualquer área retangular de um widget de desenho ou em qualquer célula de uma planilha.

“Andrew” é uma referência a Andrew User Interface System (nessa época era conhecida apenas como Andrew Project).

Ao mesmo tempo, Thomas Fine teve uma ideia diferente:

Esta é a minha opinião. A melhor maneira de usar imagens na WWW é utilizando MIME. Eu tenho certeza que postscript é um tipo suportado no MIME, e que lida tranquilamente com a mistura de texto e imagens.

Mas isto ainda não é clicável, você diz? Sim você está certo. Eu suspeito que já tenha uma resposta para isso ser exibido utilizando display postscript. Mesmo que incluir isto ao padrão postscript seja trivial. Definir um comando de âncora que especifica a URL e o uso do caminho atual como uma região próxima ao botão. Desde que o postscript lide bem com os caminhos, isto faz com que formatos de botões aleatórios sejam triviais.

Display Postscript era uma tecnologia de renderização na tela co-desenvolvida pela Adobe e NeXT.

Esta proposta nunca foi implementada, mas a idéia de um jeito melhor de consertar HTML e substituí-lo por algo completamente diferente aparece de tempos em tempos.

Tim Berners-Lee, 2 de Março de 1993:

HTTP2 permite que um documento contenha qualquer tipo que o usuário diga que pode lidar, não apenas os MIME types registrados. Então pode ser feito um experimento. Sim eu acho que tem algum caso que funciona postscript com hipertexto, eu não entendo de display postscript o suficiente. Eu sei que a Adobe está tentando estabelecer seu próprio tipo de postscript "PDF" que terá links e poderá ser lido pelos leitores proprietários deles.

Eu penso que uma linguagem genérica para links entre camadas (baseados em Hytime?) pode permitir padrões hipertexto e imagens/video que envolva-os separadamente, que pode ajudar ambos.

Deixem que a tag `IMG` possa ser `INCLUDE` (incluída) e deixe ela se referir a um tipo arbitrário de documento. Ou `EMBED` se `INCLUDE` parece com um `include` do `cpp` e que as pessoas possam esperar por um código fonte SGML para ser interpretado — o que não é a intenção.

HyTime foi um dos primeiros sistemas de documentos hipertexto baseado em SGML. Ele teve uma grande importância nas primeiras discussões sobre HTML e posteriormente sobre XML.

A proposta de Tim para uma tag `<INCLUDE` nunca foi implementada, no entanto pode-se ver ecos dela nos elementos `<object>`, `<embed>`, e `<iframe>`.

Finalmente em 12 de Março de 1993, Marc Andreessen revisitou a thread:

Voltando a thread sobre inclusão de imagens — eu estou chegando perto de lançar o Mosaic v0.10, que irá suportar a inclusão de GIF e imagens/bitmaps XBM, como eu disse anteriormente.
...

Nós não estamos preparados para suportar as tags `INCLUDE/EMBED` nesse momento. ... Então nós provavelmente iremos usar as tags `<IMG SRC="url">` (não a tag `ICON`, dado que nem todas as imagens podem ser propriamente chamadas de ícones). Por enquanto as imagens incluídas não deverão possuir um `content-type` específico; futuramente, nós planejamos dar suporte a isto (junto com a adaptação do MIME). Na verdade, a rotina de leitura de imagens que nós estamos utilizando lida com o formato da imagem no momento de renderizar, então a extensão do arquivo não é tão relevante.



UMA LINHA CONTÍNUA

Eu tenho um fascínio incrível por todos os detalhes dessa conversa de quase 17 anos de idade que levou a criação de um elemento HTML que é utilizado em praticamente todas as páginas da internet. Considere que:



- HTTP continua existindo. HTTP evolui com sucesso de 0.9 para 1.0 e posteriormente para 1.1. E continua evoluindo.
- HTML continua existindo. O formato de dados rudimentar — ele sequer suportava imagens em linha! — evoluiu com sucesso para 2.0, 3.2, 4.0. HTML é uma linha contínua. Uma linha torcida, cheia de nós, embolada, com certeza. Existem diversos "branches mortos" na árvore evolutiva, lugares onde pensamentos estavam a frente das próprias pessoas (e na frente dos autores e desenvolvedores). Mas continua. Aqui estamos, em 2010, e as páginas da web de 1990 continuam sendo exibidas corretamente nos navegadores modernos. Eu acabei de carregar uma no navegador em estado-da-arte do meu celular Android e eu nem recebi uma mensagem dizendo “por favor aguarde enquanto estamos importando formatos legados...”
- HTML sempre será uma conversa entre marcações de navegadores, autores, padrões, e outras pessoas que simplesmente apareceram e gostaram de conversar sobre símbolos de maior e menor. A maioria das versões bem sucedidas da HTML foram “retrospectivas,” pegando tudo que existia e tentando empurrar isso para direção certa. Qualquer um que diga a você que a HTML deveria continuar “pura” (presumivelmente ignorando os marcadores dos navegadores, ou ignorando os autores ou ambos) está simplesmente mal informado. A HTML nunca foi pura, e todas as tentativas de purificá-lo foram falhas incríveis, vistas apenas pelas tentativas de substituí-lo.
- Nenhum dos navegadores de 1993 continua existindo de uma forma reconhecível. O Netscape Navigator foi abandonado em 1998 e re-escrito a partir do rascunho para criar a Suite Mozilla, que foi subdivida para criar o Firefox. O Internet Explorer teve diversos “começos” no “Microsoft Plus! for Windows 95,” que foi empacotado junto com alguns temas para área de trabalho e um jogo de pinball. (Mas é claro que também podem ser encontradas referências anteriores a este navegador).
- Alguns dos sistemas operacionais de 1993 continuam existindo, mas nenhum deles é relevante para internet moderna. A maioria das pessoas que “usa” a internet faz em um PC rodando Windows 2000 ou superior, um Mac rodando Mac OS X, um PC rodando algum sabor de Linux, ou um smartphone como um iPhone. Em 1993, o Windows estava na versão 3.1 (e competindo com OS/2), Macs rodavam System 7 e o Linux era distribuído pela Usenet. (Quer se divertir um pouco? Encontre um dinossauro da internet e sussurre “Trumpet Winsock” ou “MacPPP.”)
- Algumas das mesmas *pessoas* continuam por aí e continuam envolvidas no que nós simplesmente chamamos de “web standards (padrões da internet).” Isso após quase 20 anos. Alguns destes estavam envolvidos com os predecessores da HTML, na década de 80 e anteriores.
- Falando dos antecessores... Com a popularidade adquirida da HTML e da internet, é fácil esquecer os formatos e sistemas modernos envolvidos na criação deles. Andrew? Intermedia? HyTime? E o HyTime não foi somente um projeto de pesquisa acadêmico; ele foi um padrão ISO. Ele foi aprovado para uso militar. Ele era um Grande Negócio. E você pode ler sobre ele por você mesmo... na página HTML dele, no seu navegador.

Mas nenhuma dessas coisas responde a pergunta original: por que temos um elemento ? Por que não um elemento <icon>? Ou ainda um elemento <include>? Por que não temos um

link com um atributo `include`, ou alguma combinação de valores em `rel`? Por que um elemento `<img>`? Simples, porque Marc Andreessen enviou um código com ele e código enviado ganha.

Isso não quer dizer que *todos* códigos enviados ganham; afinal, Andrew e Intermedia e HyTime enviaram seus códigos também. Código é necessário, mas não o suficiente para o sucesso. E eu *com certeza* não quis dizer que enviar código antes de um padrão irá produzir a melhor solução. O elemento `<img>` de Marc não funcionava com diversos formatos comuns de figuras; ele não definia como o texto ficaria ao redor da imagem; ele não suportava alternativas de texto ou de substituição de conteúdo em caso de falhas em navegadores antigos. E 17 anos depois, continuamos lidando com o sniffing de conteúdo, e continuam existindo diversas vulnerabilidades loucas. E você tem como voltar atrás 17 anos e ver a Grande Guerra dos navegadores e voltar para 25 de Fevereiro de 1993 quando Marc Andreessen simplesmente comentou, “MIME, um dia, quem sabe,” e enviou seu código de todo modo.

Ganha aquele que é enviado.



UMA LINHA DO TEMPO DO DESENVOLVIMENTO DA HTML DE 1997 À 2004

Em Dezembro de 1997, a World Wide Web Consortium (W3C) publicou a HTML 4.0 e provavelmente fechou o grupo de trabalho da HTML. Menos de dois meses depois, um grupo de trabalho separado da W3C publicou o XML 1.0. Três meses depois apenas, as pessoas que faziam parte da W3C deram um workshop chamado: “Moldando o futuro da HTML” para responder a questão: “A W3C desistiu da HTML?” Esta foi a resposta:

Nas conversas foi concordado que seria difícil ocorrer uma futura extensão da HTML, como a conversão da 4.0 para uma aplicação XML. Foi proposto quebrar com estas restrições e fazer um novo começo para a nova geração da HTML baseada em um conjunto de tags XML.

A W3C recriou o grupo de trabalho da HTML para criação do seu “conjunto de tags XML.” O primeiro passo deles, em dezembro de 1998, foi um rascunho de uma especificação interna que simplesmente reformulou a HTML no XML sem adicionar nenhum novo elemento ou atributo. Esta especificação posteriormente ficou conhecida como “XHTML 1.0.” Ela definiu um novo MIME type para documentos XHTML: `application/html+xml`. Entretanto, para realizar a migração das atuais 4 páginas da HTML existentes, segundo o Apêndice C, resume nas “principais guias de design para os autores que desejam processar documentos XHTML nos agentes HTML existentes.” O apêndice C diz também que é permitido ao autor chamar documentos “XHTML”, mas utilizar ainda o MIME type `text/html`.

O próximo objetivo deles foram os formulários web. Em agosto de 1999, o mesmo grupo de trabalho da HTML publicou o primeiro rascunho da XHTML Extended Forms. Eles colocaram as expectativas no primeiro parágrafo:

Depois de uma cuidadosa consideração, o grupo de trabalho da HTML decidiu que as metas para a nova geração dos formulários são incompatíveis com a retro-compatibilidade com os navegadores desenvolvidos para suportar as versões antigas da HTML. É nosso objetivo criar um novo modelo de formulários do zero (“XHTML Extended Forms”) baseado em um conjunto bem definido de requisitos. Os requisitos descritos nesse documento são baseados na experiência adquirida com um grande espectro de aplicações com formulários.

Alguns meses depois, “XHTML Extended Forms” foi renomeado para “XForms” e movida para seu próprio grupo de trabalho. Este grupo trabalhou paralelamente com o grupo de trabalho da HTML e finalmente publicou em outubro de 2003 a primeira edição do XForms 1.0.

Enquanto isso com a transição para o XML completa, o grupo de trabalho da HTML colocaram seus esforços na “nova geração da HTML.” Em maio de 2001 eles publicaram a primeira edição da XHTML 1.1, que adicionou apenas poucos recursos além dos que haviam na XHTML 1.0, mas também eliminou a brecha existente no “Apêndice C.” A partir da versão 1.1, todos os documentos XHTML passam a funcionar a com o MIME type `application/html+xml`.



TUDO QUE VOCÊ SABE SOBRE XHTML ESTÁ ERRADO

Por que os MIME types são importantes? Por que eu fico voltando a eles? Três palavras: draconian error handling (tratamento de erros draconianos). Navegadores sempre foram “tolerantes” com a HTML. Se você criar uma página HTML mas esquecer a tag `</head>`, os navegadores irão mostrar a página de todo modo. (Algumas tags implicitamente realizam o fechamento da tag `</head>` e o início da tag `<body>`.) Você supostamente aninha as tags de maneira hierárquica — fechando elas da última para primeira — mas, se você criar uma marcação como `<b><i></b></i>`, os navegadores simplesmente vão lidar com isso (de algum jeito) sem mostrar nenhuma mensagem de erro.



Como você pode esperar, o fato dessa marcação HTML “quebrada” continuar funcionando em alguns navegadores, permite que os autores criem páginas HTML quebradas. Muitas páginas quebradas. Segundo algumas estimativas, cerca de 99% das páginas HTML na internet hoje possuem pelo menos um erro nelas. Mas como os navegadores não mostram mensagens de erro para isto, ninguém conserta.

A W3C viu que este era o problema fundamental com a web, e, então, eles decidiram corrigir isso. O XML publicado em 1997 quebrou com a tradição de perdoar os clientes e mandou que todos os programas que consumissem XML deveriam tratar os erros chamados de “boa-formatação” como erros fatais. Este conceito de falha ficou conhecido como “tratamento de erro draconiano” depois do líder grego Draco que foi quem instituiu a pena de morte para quem cometesse menores infrações das suas leis. Quando a W3C reformulou a HTML como um vocabulário XML, eles mandaram que todos os documentos servidos pelo novo MIME type `application/xhtml+xml` deveriam submeter um tratamento de erro draconiano. Caso houvesse apenas um erro de boa-formação na sua página XHTML — Como por exemplo esquecer a tag `</head>` ou uma colocação errada de tags de início e fim — os navegadores não teriam escolha a não ser parar o processamento e mostrar uma mensagem de erro para o usuário final.

Essa ideia não foi popular universalmente. Com uma taxa estimada de erro em 99% das páginas, a sempre presente possibilidade de mostrar mensagens de erro para o usuário final e a escassez de novos recursos no XHTML 1.0 e 1.1 que justificassem o custo, os autores web basicamente ignoraram a `application/xhtml+xml`. Mas isso não significa que eles também ignoraram o XHTML. Definitivamente eles não fizeram isso. O Apêndice C da especificação do XHTML 1.0 deu aos autores da web uma nova brecha: “Usar algo que pareça com a sintaxe do XHTML, mas que continua gerando o MIME type `text/html`.” Isso foi exatamente o que centenas de desenvolvedores web fizeram: eles fizeram um *upgrade* para a sintaxe do XHTML mas que continuassem gerando o MIME type `text/html`

Até hoje, milhões de páginas da internet dizem ser XHTML; Elas começam com o doctype XHTML na primeira linha, usam tags com caixa baixa, usam aspas nos atributos e uma barra simples ao final de elementos vazios como `<br />` e `<hr />`; Mas uma pequena fração

dessas páginas são realmente servidas com o MIME type `application/xhtml+xml` e que façam o tratamento de erro draconiano. Qualquer página que possua o MIME type `text/html` — independente do doctype, sintaxe ou estilo de código — será interpretada usando um parseador HTML tolerante, que silenciosamente ignorará qualquer erro de marcação e nunca alertará os usuários finais (ou qualquer outra pessoa) se a página está tecnicamente quebrada;

O XHTML 1.0 inclui esta brecha, mas o XHTML 1.1 a fechou e o nunca finalizado XHTML 2.0 continuou com a tradição de requerer o tratamento de erro draconiano. É por isso que as bilhões de páginas que dizem ser XHTML 1.0 e apenas algumas dizem ser XHTML 1.1 (ou XHTML 2.0). Então você realmente está usando XHTML? Verifique o seu MIME type. (Na verdade, se você não sabe qual MIME type você está usando eu posso garantir que você continua usando `text/html`.) A menos que as suas páginas estejam utilizando o `application/xhtml+xml`, as páginas que você chama de “XHTML” é XML apenas no nome.



UMA VISÃO CONCORRENTE

Em junho de 2004, o W3C realizou o Workshop sobre Aplicações Web e Documentos Combinados. Estavam presentes nesse workshop representantes de 3 fabricantes de browsers, companhias de desenvolvimento web e outros membros do W3C. Um grupo de partes interessadas, incluindo a Fundação Mozilla e a Opera Software, fizeram uma apresentação sobre sua visão concorrente do futuro da web: uma evolução do existente padrão HTML 4 para incluir novos recursos para desenvolvedores de aplicativos web modernos.

Os 7 princípios a seguir representam o que acreditamos serem os requisitos mais importantes para este trabalho.

Compatibilidade com versões anteriores, com um caminho claro de migração

Tecnologias de aplicação web devem ser baseadas em tecnologias em que os desenvolvedores estão familiarizados, incluindo HTML, CSS, DOM e JavaScript. Características básicas de aplicativos web devem ser implementáveis através de comportamentos e se utilizando scripts e folhas de estilo em IE6 hoje para que os autores tenham um caminho claro de migração. Qualquer solução que não possa ser usada com o atual agente de usuário (user agent) de maior penetração no mercado, sem a necessidade de plugins binários, é altamente improvável de ser bem sucedida.

Tratamento de erro bem definido

O tratamento de erros em aplicações web deve ser definido em um tal nível de detalhe que permita que os agentes de usuário não tenham que bolar seus próprios mecanismos de manipulação de erros ou fazer engenharia reversa em outros agentes de usuário.

Usuários não devem ser expostos a erros de autoria

Especificações devem indicar o comportamento exato de recuperação de erros para cada cenário de erro possível. A manipulação de erro deve ser definida, na maior parte, em termos de recuperação de erro normal (como em CSS) ao invés de erros óbvios e catastróficos (como em XML).

Uso prático

Todo recurso que entra em especificações de aplicações web deve ser justificado por um caso de uso prático. O inverso não é, necessariamente, verdadeiro: cada caso de uso, não necessariamente, precisa ser o fundamento para um novo recurso. Os casos de uso devem ser, preferencialmente, baseados em sites reais, nos quais os desenvolvedores utilizaram, anteriormente, uma solução ruim para contornar a limitação.

Scripting veio pra ficar

Mas deve ser evitado onde marcação mais convenientemente declarativa puder ser usada.

Scripting deve ser dispositiva e representativamente neutro, a não ser quando em um escopo bastante específico de dispositivos (por exemplo, a menos que incluso em XBL).

Profiling específico de dispositivo deve ser evitado

Desenvolvedores devem ser capazes de se valer das mesmas características, sejam elas implementadas em versões desktop ou móveis do mesmo agente de usuário.

Processo aberto

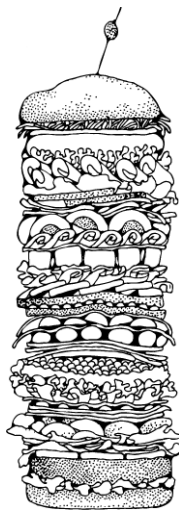
A web foi beneficiada por ter sido desenvolvida num ambiente aberto. Aplicações web serão o *core* da web e seu desenvolvimento também deve se valer desse ambiente aberto. Listas de discussão, arquivos e rascunhos das especificações devem estar visíveis ao público.

Os participantes do workshop participaram de uma enquete com a pergunta “Deve o W3C desenvolver uma extensão declarativa para HTML e CSS e extensões imperativas para o DOM para tratar do nível médio de requisitos de aplicativos web, ao contrário das APIs maduras a nível de sistemas operacionais? (proposto por Ian Hickson, da Opera Software)”. A votação foi de 11 contra 8. No sumário do workshop, o W3C escreveu: “Atualmente, o W3C não pretende colocar quaisquer recursos no terceiro tópico da enquete: extensões para HTML e CSS para aplicativos web, exceto tecnologias que estão sendo desenvolvidos sob a licença dos atuais Grupos de Trabalho do W3C.”

Ante essa decisão, as pessoas que tinham propostas envolvendo HTML e formulários HTML tinham somente duas opções: desistir ou continuar seu trabalho fora do W3C. Eles escolheram a última e registraram o domínio whatwg.org e, em junho de 2004, o WHAT Working Group nasceu.



WHAT WORKING GROUP?



Mas que diabos, afinal, é o WHAT Working Group? Vamos deixá-los explicar:

O Web Hypertext Applications Technology Working Group é um grupo amplo, não oficial e aberto de fabricantes de navegadores web e partes interessadas. O grupo visa desenvolver especificações baseadas em tecnologias HTML e afins para facilitar o desenvolvimento de aplicativos web interoperáveis, com a intenção de submeter os resultados a uma organização de padrões. Esta submissão, então, constitui a base de trabalho em formalmente estender o HTML no caminho dos padrões web.

A criação deste fórum decorre de vários meses de trabalho por e-mail, particularmente sobre as especificações de tais tecnologias. O foco principal, até este ponto, tem sido estender o HTML4 Forms para suportar as características solicitadas pelos autores sem quebrar a compatibilidade com o conteúdo existente. Este grupo foi criado para garantir que o desenvolvimento futuro dessas especificações será completamente aberto, através de uma lista de discussão aberta publicamente arquivada.

A frase-chave aqui é “sem quebrar compatibilidade com versões anteriores”. XHTML (menos o "furo" do apêndice C) não é retrocompatível com HTML. Ele precisa de um tipo de MIME inteiramente novo e trabalha com tratamento de erros draconianos para todo o conteúdo servido com aquele tipo de MIME. XForms não é retrocompatível com formulários HTML porque pode ser usado apenas em documentos disponibilizados com o novo tipo de MIME XHTML, o que significa que XForms também funciona com tratamento de erros de forma draconiana. Todos os caminhos levam ao MIME.

Ao invés de acabar com mais de 1 década de investimentos em HTML e fazer com que 99% das páginas da web existentes fiquem inutilizáveis, o WHAT Working Group optou por uma abordagem diferente: documentar os algoritmos de tratamento de erros "perdoáveis" que os navegadores, efetivamente, utilizam. Web browsers sempre foram indulgentes com erros de HTML, mas ninguém jamais se preocupou em escrever sobre como, exatamente, eles fizeram isso. NCSA Mosaic teve seus próprios algoritmos para lidar com páginas quebradas e a Netscape tentou o mesmo. Então, o Internet Explorer tentou se igualar ao Netscape. Em seguida, o Opera e o Firefox tentaram fazer a mesma coisa que o Internet Explorer. Daí,

o Safari tentou se igualar ao Firefox. E assim por diante, até os dias de hoje. Ao longo do caminho, os desenvolvedores queimaram milhares e milhares de horas tentando tornar seus produtos compatíveis com os de seus concorrentes.

Se isso soa como uma quantidade insana de trabalho, é porque, realmente, é. Ou melhor, era. Demorou 5 anos, mas (exceto por alguns casos obscuros extremos) o WHAT Working Group tem documentado com sucesso como analisar HTML de uma maneira que seja compatível com o conteúdo web já existente. Em nenhum ponto do algoritmo final há um passo que exija que o consumidor de HTML deva parar o processamento a fim de exibir uma mensagem de erro para o usuário final.

Enquanto toda a engenharia reversa estava acontecendo, o WHAT Working Group foi, calmamente, trabalhando em algumas outras coisas, também. Uma delas era uma especificação, inicialmente apelidada Web Forms 2.0, que adicionou novos tipos de controles aos formulários HTML. (Você aprenderá mais sobre formulários web em Uma Forma De Loucura.) Outra foi um rascunho de uma especificação chamada "Web Applications 1.0", que incluía grandes novidades, tais como uma tela para desenhar canvas diretamente e suporte nativo para áudio e vídeo, sem plugins.



DE VOLTA AO W3C



Por dois anos e meio, o W3C e o WHAT Working Group se ignoraram mutuamente. Enquanto o WHAT Working Group focou em formulários web e novos recursos de HTML, o W3C HTML Working Group estava ocupado com a versão 2.0 do XHTML. Mas, em outubro de 2006, ficou claro que o WHAT Working Group teve seu momento crítico, enquanto o XHTML 2 ainda estava definindo em forma de rascunho, não implementado por nenhum navegador principal. Em outubro de 2006, Tim Berners-Lee, o fundador da W3C, em pessoa, anunciou que o W3C poderia trabalhar em conjunto com o WHAT Working Group a fim de evoluir o HTML.

Algumas coisas ficaram mais claras com o passar dos anos. É necessário evoluir o HTML de forma incremental. A tentativa de fazer o mundo mudar para XML, incluindo aspas em torno de valores de atributos e barras em tags vazias e namespaces, de uma só vez, não funcionou. O grande público gerador de HTML não se moveu, em grande parte porque os navegadores não

reclamaram. Algumas grandes comunidades fizeram mudanças e estão colhendo os frutos de sistemas bem-formados, mas não todos. Isso é importante para manter a HTML de forma incremental, bem como continuar a transição para um mundo bem-formado e desenvolver mais poder nesse mundo.

O plano é constituir um grupo completamente novo de HTML. Ao contrário do anterior, este será fretado para fazer melhorias incrementais na HTML, bem como em XHTML, paralelamente. Ele terá uma cadeia e pessoal diferentes. Ele irá funcionar em HTML e XHTML em conjunto. Temos um forte apoio para esse grupo, a partir de muitas pessoas que falaram com, incluindo os fabricantes de browsers.

Haverá trabalho, também, em formulários. Esta é uma área complexa, já que existem formulários tanto em HTML, quanto em XForms. Formulários HTML são ubiquamente implementados e existem muitas implementações e usuários de XForms. Enquanto isso, a apresentação de Webforms sugeriu extensões sensatas para formulários HTML. O plano é, informada pela Webforms, de estender formulários HTML.

Uma das primeiras coisas que o recém re-formado W3C HTML Working Group decidiu mudar é o nome de "Web Applications 1.0" para "HTML5". E aqui estamos nós, mergulhando em HTML5.

POSTSCRIPT

Em outubro de 2009, o W3C descontinuou o Grupo de Trabalho de XHTML 2 e emitiu esse comunicado para explicar sua decisão:

Quando o W3C anunciou os Grupos de Trabalho de HTML e XHTML 2 em março de 2007, indicamos que iríamos continuar a acompanhar o mercado de XHTML 2. O W3C reconhece a importância de um sinal claro à comunidade sobre o futuro do HTML.

Embora reconheçamos o valor do Grupo de Trabalho do XHTML 2 ao longo dos anos, após discussão com os participantes, a gestão W3C optou por permitir a expiração do Grupo de Trabalho no fim de 2009 e não renová-lo.

Os que ganham são aqueles a bordo.



LEITURA COMPLEMENTAR

- A História da Web, um projeto antigo de Ian Hickson
- HTML/História, de Michael Smith, Henri Sivonen e outros
- Uma Breve História do HTML, de Scott Reynen

DETECTANDO FUNCIONALIDADES DO HTML5

exibir índice analítico



MERGULHANDO



Você pode se perguntar: “Como eu posso começar a usar HTML5 se os navegadores não o suportam?” Mas a questão por si só é enganosa. HTML5 não é uma coisa grande e única; é uma coleção de funcionalidades individuais. Então você não pode detectar “suporte ao HTML5,” por que isso não faz sentido. Mas você *pode* verificar o suporte a funcionalidades individuais, como canvas, vídeo, ou geolocalização.



TÉCNICAS DE DETECÇÃO

Quando seu navegador renderiza uma página web, ele constrói um Modelo Objeto Documento (DOM, Document Object Model em inglês), uma coleção de objetos que representam os elementos HTML na página. Cada elemento — todo `<p>`, todo `<div>`, todo `<span>` — é representado no DOM por um objeto diferente. (Também existem objetos globais, como `window` e `document`, que não estão vinculados a elementos específicos.)



Todos os objetos no DOM compartilham um conjunto comum de propriedades, mas alguns objetos têm mais do que outros. Em navegadores que suportam funcionalidades da HTML5, alguns objetos terão propriedades únicas. Uma espiada rápida no DOM irá te dizer quais funcionalidades são suportadas.

Existem quatro técnicas básicas para detectar se o browser suporta uma funcionalidade específica. Do mais simples até o mais complexo:

1. Verifica se uma determinada propriedade existe em um objeto global (como `window` ou `navigator`).

Exemplo: testando o suporte a geolocalização

2. Crie um elemento, então verifique se uma determinada propriedade existe naquele elemento.

Exemplo: testando suporte ao canvas

3. Crie um elemento, verifique se um determinado método existe naquele elemento, então chame o método e verifique o valor que ele retorna.

Exemplo: testando quais formatos de vídeos são suportados

4. Crie um elemento, defina uma propriedade para um determinado valor, então verifique se a propriedade manteve seu valor.

Exemplo: testando quais tipos de `<input>` são suportados



MODERNIZR, UMA BIBLIOTECA DE DETECÇÃO HTML5

Modernizr é um biblioteca JavaScript de código aberto, sob licença MIT, que detecta suporte a várias funcionalidades de HTML5 & CSS3. Você deve usar sempre a última versão. Para usá-la, inclua o seguinte elemento `<script>` no topo da sua página.

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <title>Dive Into HTML5</title>
  <script src="modernizr.min.js"></script>
```

```
</head>
<body>
  ...
</body>
</html>
```

Isso Vai no seu <head>

Modernizr executa automaticamente. Não existe uma função `modernizr_init()` para ser chamada. Quando ela roda, cria um objeto global chamado `Modernizr`, que contém um conjunto de propriedades Booleanas para cada funcionalidade que ela puder detectar. Por exemplo, se seu navegador suporta a API do canvas, a propriedade `Modernizr.canvas` será `true`. Se seu navegador não suportar a API do canvas, a propriedade `Modernizr.canvas` será `false`.

```
if (Modernizr.canvas) {
  // vamos desenhar algumas formas!
} else {
  // o suporte nativo ao canvas não está disponível :(
}
```



CANVAS



Seu navegador possui suporte a API canvas.

A HTML5 define o elemento `<canvas>` como “uma tela bitmap de resolução dependente que pode ser usada para renderizar grafos, gráficos de jogos, ou outras imagens visuais em tempo real.” Um canvas é um retângulo na sua página onde você pode usar JavaScript para desenhar qualquer coisa que você quiser. A HTML5 define um conjunto de funções (“a API canvas”) para desenhar formas, definindo caminhos, criando gradientes e aplicando transformações.

A verificação da API do canvas usa a técnica de detecção #2. Se seu navegador suporta a API do canvas, o objeto DOM o cria para representar um elemento `<canvas>` que terá um método `getContext()`. Se seu navegador não suporta a API do canvas, o objeto DOM o cria

para um elemento <canvas> que terá apenas um conjunto comum de propriedades, mas nada específico do canvas.

```
function supports_canvas() {  
    return !!document.createElement('canvas').getContext;  
}
```

Essa função começa com a criação do elemento <canvas> de teste. Mas o elemento nunca é adicionado a sua página, então ninguém nunca o verá. Está apenas flutuando na memória, indo a lugar nenhum e fazendo nada, como uma canoa em um rio calmo.

```
return !!document.createElement('canvas').getContext;
```

Assim que você cria o elemento <canvas> de teste, você testa a existência de um método `getContext()`. Esse método só existirá se o seu navegador suportar a API do canvas.

```
return !!document.createElement('canvas').getContext;
```

Finalmente, você usa o truque da negação dupla para forçar a conversão do resultado em um valor booleano (`true` ou `false`).

```
return !!document.createElement('canvas').getContext;
```

Essa função irá detectar grande parte da API do canvas, incluindo shapes, paths, gradients & patterns. Ela não detectará a biblioteca de terceiros `explorercanvas` que implementa a API do canvas no Microsoft Internet Explorer.

Ao invés de escrever essa função você mesmo, você pode usar a `Modernizr` para detectar o suporte a API do canvas.

↪ *Verifica o suporte ao Canvas*

```
if (Modernizr.canvas) {  
    // vamos desenhar algumas formas!  
} else {  
    // o suporte nativo ao canvas não está disponível :(  
}
```

Existe um teste separado para a API do canvas `text`, que será demonstrado a seguir.



TEXTO CANVAS



Seu navegador possui suporte API de texto canvas.

Mesmo que o seu navegador suporte a API canvas, ele pode não suportar a API do texto canvas. A API do canvas cresceu através do tempo, e as funções de texto foram adicionadas por último. Alguns navegadores passaram a suportar o canvas antes da API de texto estar completa.

A verificação da API do texto canvas usa a técnica de detecção #2. Se seu navegador suporta a API do canvas, o objeto DOM o cria para representar um elemento <canvas> que terá um método `getContext()`. Se seu navegador não suporta a API do canvas, o objeto DOM o cria para um elemento <canvas> que terá apenas um conjunto comum de propriedades, mas nada específico do canvas.

```
function supports_canvas_text() {  
  if (!supports_canvas()) { return false; }  
  var dummy_canvas = document.createElement('canvas');  
  var context = dummy_canvas.getContext('2d');  
  return typeof context.fillText == 'function';  
}
```

A função começa verificando o suporte ao canvas, usando a função `supports_canvas()` que você acabou de ver na seção anterior. Se seu navegador não suporta a API do canvas, ele certamente não suportará a API de texto canvas!

```
if (!supports_canvas()) { return false; }
```

Depois, você cria um elemento <canvas> de teste e pega seu contexto de desenho. Isso funcionará de forma garantida, porque a função `supports_canvas()` já verificou que o método `getContext()` existe em todo objeto canvas.

```
var dummy_canvas = document.createElement('canvas');  
var context = dummy_canvas.getContext('2d');
```

Finalmente, você verifica se o contexto de desenho tem uma função `fillText()`. Se tiver, a API de texto canvas está disponível. Hooray!

```
return typeof context.fillText == 'function';
```

Ao invés de escrever essa função você mesmo, você pode usar Modernizr para detectar o suporte à API do texto canvas.

↪ *Verifica o suporte ao texto Canvas*

```
if (Modernizr.canvastext) {  
    // vamos desenhar algum texto!  
} else {  
    // sem suporte nativo ao texto canvas disponível :(  
}
```



VÍDEO

A HTML5 define um novo elemento chamado `<video>` para embutir vídeo nas suas páginas web. Embutir vídeos costuma ser impossível sem plugins de terceiros como Apple QuickTime® ou Adobe Flash®.



Seu navegador possui suporte video HTML5.

O elemento `<video>` é projetado para funcionar sem a necessidade de scripts de detecção. Você pode especificar múltiplos arquivos de vídeo, e os navegadores que possuírem suporte ao HTML5 vídeo escolherão um baseado nos formatos suportados por eles.

Navegadores que não possuem suporte a HTML5 vídeo irão ignorar o elemento `<video>` completamente, mas você pode usar isso a seu favor dizendo a eles para tocar o vídeo através de plugins. Kroc Camen desenvolveu uma solução chamada Video for Everybody! que faz uso da HTML5 vídeo onde estiver disponível, porém fazendo tratamento para uso de QuickTime ou Flash em navegadores antigos. Essa solução não usa JavaScript, e funciona praticamente em qualquer navegador, inclusive em mobiles.

Se você quiser fazer mais com vídeo do que apenas colocá-lo em sua página e tocá-lo, terá que usar JavaScript. Checar suporte a vídeo utiliza a técnica de detecção #2. Se seu navegador possui suporte para HTML5 vídeo, o objeto DOM criado para representar o elemento `<video>` terá um método `canPlayType()`. Caso o navegador não tenha suporte para HTML5 vídeo, o objeto DOM criado para o elemento `<video>` terá apenas uma coleção de propriedades comum à todos os elementos. Você pode verificar o suporte a vídeo usando essa função:


```
function supports_video() {  
    return !!document.createElement('video').canPlayType;  
}
```

Ao invés de escrever a função você mesmo, você pode usar o Modernizr para detectar suporte a HTML5 vídeo.

↪ *Verificando suporte a HTML5 vídeo*

```
if (Modernizr.video) {  
    // vamos tocar alguns vídeos!  
} else {  
    // suporte nativo a vídeos indisponível :(  
    // verifique suporte a QuickTime ou Flash talvez  
}
```

No capítulo de Vídeo, explicarei outra solução que faz uso dessas técnicas de detecção para converter elementos <video> para players baseados em Flash, para atender a navegadores que não possuem suporte ao HTML5 vídeo.

Existe um teste separado para detectar quais formatos de vídeo seu navegador pode tocar, o qual demonstrarei em seguida.

FORMATOS DE VÍDEO

Formatos de vídeo são como linguagens escritas. Um jornal em inglês contém a mesma informação que um jornal em espanhol, mas se você só sabe ler inglês, apenas um dos jornais será útil pra você! Para tocar um vídeo, seu navegador precisa entender o “idioma” no qual o vídeo foi escrito.



Seu navegador pode tocar ambos Ogg Theora e H.264 video. Hey, você pode
tocar videos WebM video, também!

O “idioma” de um vídeo é chamado de “codec” — é um algoritmo usado para codificar o vídeo em uma sequência de bits. Existem vários codecs em uso ao redor do mundo. Qual deles você usa? A triste realidade da HTML5 vídeo é que os navegadores não trabalham com um único codec em comum. Contudo, aparentemente eles se limitaram a dois tipos. Um codec é pago (por causa da licença de patente), mas funciona no Safari e no iPhone. (Este também funciona com Flash se você utiliza uma solução como Video for Everybody!) O outro codec é gratuito e funciona em navegadores open source como Chromium and Mozilla Firefox.

A verificação de suporte a formatos de vídeo utiliza a técnica de detecção #3. Se o seu navegador possui suporte a HTML5 vídeo, o objeto DOM criado para representar um elemento <video> terá um método `canPlayType()`. Esse método dirá se seu browser possui suporte a um determinado formato de vídeo.

Essa função verifica formatos suportados por Macs e iPhones.

```
function supports_h264_baseline_video() {
  if (!supports_video()) { return false; }
  var v = document.createElement("video");
  return v.canPlayType('video/mp4; codecs="avc1.42E01E, mp4a.40.2"');
}
```

A função começa verificando suporte a HTML5 vídeo, usando a função `supports_video()` que você acabou de ver na seção anterior. Se o seu navegador não possui suporte a HTML5 vídeo, com certeza não terá suporte para formatos de vídeo!

```
if (!supports_video()) { return false; }
```

Então a função cria um elemento <video> (mas não o inclui na página, assim ele não será visível) e invoca o método `canPlayType()`. Este método está lá seguramente, pois a função `supports_video()` acabou de garantir sua existência.

```
var v = document.createElement("video");
```

Um “formato de vídeo” é na verdade uma combinação de diferentes coisas. Em termos técnicos, você está perguntando ao navegador se ele pode tocar um vídeo de base H.264 e um áudio AAC LC em um container MPEG-4. (Irei explicar o que tudo isso significa no capítulo de Vídeo. Você também pode querer ler Uma breve introdução à codificação de vídeo.)

```
return v.canPlayType('video/mp4; codecs="avc1.42E01E, mp4a.40.2"');
```

A função `canPlayType()` não retorna `true` ou `false`. Considerando a complexidade que os formatos de vídeo possuem, a função retorna uma string:

- "probably" se o navegador está confiante de que possa tocar este formato
- "maybe" se o navegador acha que tem condições de tocar este formato

- "" (uma string vazia) se o navegador tem certeza de que não consegue tocar este formato

Essa segunda função verifica pelo formato de vídeo livre suportado pelo Mozilla Firefox e outros navegadores de código aberto. O processo é exatamente o mesmo; a única diferença é a string que você passa à função `canPlayType()`. Em termos técnicos, você pergunta ao navegador se ele é capaz de tocar vídeo Theora e áudio Vorbis em um container Ogg.

```
function supports_ogg_theora_video() {
  if (!supports_video()) { return false; }
  var v = document.createElement("video");
  return v.canPlayType('video/ogg; codecs="theora, vorbis"');
}
```

Finalmente, WebM é um recente codec de vídeo open-source (e livre de patentes) que será incluído nas próximas versões dos principais navegadores, tais como o Chrome, Firefox, e Opera. Você pode utilizar a mesma técnica para detectar suporte ao vídeo livre WebM.

```
function supports_webm_video() {
  if (!supports_video()) { return false; }
  var v = document.createElement("video");
  return v.canPlayType('video/webm; codecs="vp8, vorbis"');
}
```

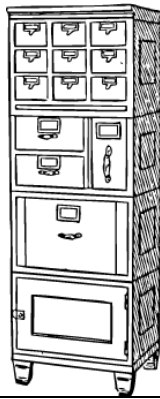
Ao invés de escrever essa função você mesmo, você pode utilizar o Modernizr (1.5 ou superior) para detectar suporte aos diferentes formatos de HTML5 vídeo.

↪ *Verificando suporte a formatos para HTML5 vídeo*

```
if (Modernizr.video) {
  // vamos tocar alguns vídeos! mas de qual tipo?
  if (Modernizr.video.webm) {
    // tentar o WebM
  } else if (Modernizr.video.ogv) {
    // tentar Ogg Theora + Vorbis em um container Ogg
  } else if (Modernizr.video.h264){
    // tentar vídeo H.264 + áudio AAC em um container MP4
  }
}
```



ARMAZENAMENTO LOCAL



Seu navegador possui suporte HTML5 storage.

HTML5 storage fornece aos web sites uma forma de armazenar informações em seu computador e recuperá-lo mais tarde. O conceito é semelhante ao dos cookies, porém ele é projetado para armazenar quantidades de informação superiores. Cookies possuem tamanho bem limitado, e o seu navegador os envia de volta ao servidor web sempre que uma nova página é solicitada (o que leva um tempo extra e uma banda valiosa). O HTML5 storage permanece em seu computador, e os web sites podem acessá-lo através de JavaScript depois que a página é carregada.

P: O armazenamento local é realmente parte da HTML5? Por que ele fica em uma especificação separada?

R: A resposta curta é sim, armazenamento local é parte da HTML5. A resposta um pouco mais completa é que o armazenamento local fazia parte da especificação principal da HTML5, mas foi dividido em uma especificação isolada porque algumas pessoas no Working Group da HTML5 se queixaram que a HTML5 estava ficando muito grande. Se partir um bolo em mais pedaços passa a sensação de reduzir o número de calorias, bem, bem vindo ao mundo estranho dos padrões.

A verificação de suporte ao HTML5 storage utiliza a técnica de detecção #1. Se o seu navegador possui suporte ao HTML5 storage, haverá uma propriedade chamada `localStorage` no objeto `window`. Se o seu navegador não possui suporte ao HTML5 storage, a propriedade `localStorage` será `undefined`. Devido a um bug em versões anteriores do Firefox, este teste causará uma exceção se os cookies estiverem desabilitados, por isso o teste todo é envolvido por um bloco `try...catch`.

```
function supports_local_storage() {  
    try {  
        return 'localStorage' in window && window['localStorage'] !==  
null;  
    } catch(e){  
        return false;  
    }  
}
```

Ao invés de escrever essa função, você pode usar o Modernizr (1.1 ou superior) para detectar suporte ao HTML5 local storage.

↪ *Verificando suporte ao HTML5 local storage*

```
if (Modernizr.localstorage) {  
    // window.localStorage está disponível!  
} else {  
    // sem suporte nativo ao local storage :(  
    // tente usar o Gears ou outra solução, talvez  
}
```

Note que o JavaScript faz distinção entre letras maiúsculas e minúsculas. O atributo do Modernizr é chamado `localStorage` (todas as letras minúsculas), mas a propriedade DOM é chamada `window.localStorage` (letras maiúsculas e minúsculas misturadas).

❗P: Quão seguro é o banco de dados da HTML5 storage? Qualquer um pode acessá-lo?
R: Qualquer um que possua acesso físico ao seu computador provavelmente poderá ler (ou até mesmo editar) seu banco de dados da HTML5 storage. Com seu navegador, qualquer web site pode ler e modificar seus próprios valores, mas sites não podem acessar valores armazenados por outros sites. Isso é chamado de same-origin restriction.

WEB WORKERS

Seu navegador possui suporte web workers.

Web Workers fornecem uma maneira padrão aos navegadores de executarem JavaScript ao fundo. Com web workers, você pode disparar múltiplas “threads” que irão todas ser executadas ao mesmo tempo, mais ou menos. (Pense em como o computador pode executar múltiplas aplicações ao mesmo tempo e você terá praticamente entendido tudo.) Essas “threads em background” podem realizar cálculos matemáticos complexos, fazer requisições HTTP ou acessar o local storage enquanto a página principal atende aos comandos do usuário tais como rolando a página, cliques ou digitação.

Verificar o suporte às web workers utiliza a técnica de detecção #1. Se o seu navegador possui suporte à API do Web Worker, haverá uma propriedade `Worker` no objeto global `window`. Se o

seu navegador não possui suporte à API do Web Worker, a propriedade `Worker` será `undefined`.

```
function supports_web_workers() {  
    return !!window.Worker;  
}
```

Ao invés de escrever essa função, você pode utilizar o Modernizr (1.1 ou superior) para detectar suporte a web workers.

↪ *Verificando suporte a web workers*

```
if (Modernizr.webworkers) {  
    // window.Worker está disponível!  
} else {  
    // sem suporte nativo a web workers :(  
    // tente o Gears ou outra solução  
}
```

Note que o JavaScript faz distinção entre letras maiúsculas e minúsculas. O atributo do Modernizr é chamado `webworkers` (todas as letras minúsculas), mas a propriedade DOM é chamada `window.Worker` (letras maiúsculas e minúsculas misturadas).



APLICAÇÕES WEB OFFLINE



Seu navegador não possui suporte aplicações web offline. :(

Ler páginas web estáticas é fácil: conecte-se na Internet, carregue uma página, se desconecte da Internet, dirija até uma cabine isolada, e leia a página comodamente. (Para ganhar tempo, pule a parte de ir até uma cabine.) Mas e as aplicações como o Gmail ou Google Docs? Graças ao HTML5, qualquer um (não só o Google!) pode construir uma aplicação web que funcione offline.

Aplicações web offline começam como aplicações web online. A primeira vez que você visita um site com conteúdo offline habilitado, o servidor web diz ao navegador quais arquivos ele precisa para trabalhar offline. Esses arquivos podem ser qualquer coisa — HTML, JavaScript, imagens, até mesmo vídeos. Uma vez que o navegador carregou todos os arquivos necessários, você pode revisitar o web site até mesmo se você não estiver conectado a Internet. Seu navegador perceberá que você está offline e usará os arquivos que ele já havia feito download. Quando você se reconectar, quaisquer alterações que você tenha feito podem ser enviadas ao servidor web.

Verificar o suporte a conteúdo offline utiliza a técnica de detecção #1. Se o seu navegador possui suporte à aplicações web offline, haverá uma propriedade `applicationCache` no objeto global `window`. Se o seu navegador não possui suporte à aplicações web offline, a propriedade `applicationCache` será `undefined`. Você pode verificar a disponibilidade de aplicações web offline através da seguinte função:

```
function supports_offline() {  
    return !!window.applicationCache;  
}
```

Ao invés de escrever essa função, você pode utilizar o Modernizr (1.1 ou superior) para detectar suporte a aplicações web offline.

↪ *Verificando suporte a aplicações web offline*

```
if (Modernizr.applicationcache) {  
    // window.applicationCache está disponível!  
} else {  
    // sem suporte nativo a conteúdo offline :(  
    // tente usar o Gears ou outra solução  
}
```

Note que o JavaScript faz distinção entre letras maiúsculas e minúsculas. O atributo do Modernizr é chamado `applicationcache` (todas as letras minúsculas), mas a propriedade DOM é chamada `window.applicationCache` (letras maiúsculas e minúsculas misturadas).



GEOLOCALIZAÇÃO

Geolocalização é a arte de descobrir em que lugar do mundo você está e (eventualmente) compartilhar essa informação com as pessoas de sua confiança. Existe mais de uma maneira de

saber onde você está — seu endereço IP, sua conexão de rede sem fio, com qual torre seu celular está se comunicando, ou o GPS que calcula a latitude e longitude através das informações enviadas pelos satélites no céu.



Seu navegador possui suporte geolocation. [Clique para pesquisar sua localização.](#)

P: A geolocalização faz parte da HTML5? Por que você está falando sobre ela?
R: O suporte a geolocalização foi adicionado aos navegadores agora há pouco, juntamente com o suporte para novos recursos HTML5. Na verdade, a geolocalização está sendo padronizada pelo Geolocation Working Group, o qual está separado da HTML5 Working Group. Porém eu irei falar sobre a geolocalização neste livro assim mesmo, porque ela é parte da evolução da web que está acontecendo nesse momento.

Verificar o suporte à geolocalização utiliza a técnica de detecção #1. Se o seu navegador possui suporte à geolocalização, haverá uma propriedade `geolocation` no objeto global `window`. Se o seu navegador não possui suporte à geolocalização, a propriedade `geolocation` será `undefined`. Veja como você pode verificar se há suporte à geolocalização:

```
function supports_geolocation() {  
    return !!navigator.geolocation;  
}
```

Ao invés de escrever essa função, você pode utilizar o Modernizr para detectar suporte a API de geolocalização.

↪ *Verificando suporte a geolocalização*

```
if (Modernizr.geolocation) {  
    // Vamos ver onde você está!  
} else {  
    // suporte nativo à geolocalização indisponível :(  
    // tente o Gears ou outra solução  
}
```


Se o seu navegador não possui suporte nativo à geolocalização, ainda há esperança. O Gears é um plugin para navegadores livre desenvolvido pelo Google que funciona no Windows, Mac, Linux, Windows Mobile e Android. Ele fornece recursos para navegadores mais antigos que não possuem suporte a todas essas novas coisas estilosas que viemos discutindo neste capítulo. Um dos recursos que o Gears fornece é a API de geolocalização. Não é exatamente a mesma coisa que `navigator.geolocation`, mas funciona como se fosse.

Existem ainda APIs de geolocalização específicas para determinados dispositivos para alguns celulares antigos, incluindo BlackBerry, Nokia, Palm e OMTP BONDI.

O capítulo sobre geolocalização falará nos mínimos detalhes sobre como usar as diferentes APIs.

TIPOS DE ENTRADA



Seu navegador suporta os seguintes tipos de entrada do

HTML5: `search`, `tel`, `url`, `email`, `date`, `month`, `week`, `time`, `datetime-local`, `number`, `range`, `color`

Você sabe tudo sobre formulários web, certo? Fazer um `<form>`, adicionar alguns elementos `<input type="text">` e quem sabe um `<input type="password">`, e por fim um botão `<input type="submit">`.

Você não sabe a metade deles. a HTML5 define um monte de novos tipos de entrada de dados que você pode usar em seus formulários.

1. `<input type="search">` para caixas de busca
2. `<input type="number">` para campos incrementais
3. `<input type="range">` para sliders
4. `<input type="color">` para seleção de cores
5. `<input type="tel">` para números de telefone
6. `<input type="url">` para endereços na web
7. `<input type="email">` para endereços de e-mail
8. `<input type="date">` para calendários

9. `<input type="month">` para meses
10. `<input type="week">` para semanas
11. `<input type="time">` para horas
12. `<input type="datetime">` para precisos e absolutos data+hora
13. `<input type="datetime-local">` para datas e horas locais

Verificando suporte aos tipos de entrada da HTML5 utiliza a técnica de detecção #4. Primeiro, você cria um elemento `<input>` em memória. O tipo padrão para todos os elementos `<input>` é "text".

```
var i = document.createElement("input");
```

Em seguida, configure o atributo `type` ao elemento `<input>` para o tipo de entrada que você quer detectar.

```
i.setAttribute("type", "color");
```

Se o seu navegador possuir suporte para aquele tipo de entrada em particular, a propriedade `type` irá reter o valor que você configurou. Se o seu navegador não possuir suporte para aquele tipo de entrada em particular, ele irá ignorar o valor que você configurou e a propriedade `type` ainda será "text".

```
return i.type !== "text";
```

Ao invés de ter de escrever 13 diferentes funções, você pode usar o Modernizr para detectar suporte para todos os novos tipos de entrada definidos na HTML5. O Modernizr reaproveita um único elemento `<input>` para eficientemente detectar o suporte aos 13 tipos de entrada. Então ele constrói um hash chamado de `Modernizr.inputtypes`, que possui 13 chaves (os atributos `type` da HTML5) e 13 valores booleanos (`true` se for suportado, `false` se não).

↪ *Verificando suporte nativo ao date picker*

```
if (!Modernizr.inputtypes.date) {  
    // sem suporte nativo para <input type="date"> :(  
    // talvez você deva fazer um você mesmo com Dojo ou jQueryUI  
}
```



PLACEHOLDER



Além dos novos tipos de entrada, a HTML5 inclui vários pequenos ajustes aos formulários já existentes. Uma melhoria é colocar um placeholder em um campo de entrada. O placeholder é exibido dentro de um campo de entrada enquanto o campo estiver vazio e sem foco. Assim que você clica no campo (ou navega com o tab até ele), o placeholder some. O capítulo sobre formulários web tem screenshots se você está tendo dificuldades em visualizá-lo.

A verificação de suporte ao placeholder utiliza a técnica de detecção #2. Se o seu navegador possui suporte à colocar placeholders em campos de entrada, o objeto DOM criado para representar o elemento <input> terá uma propriedade placeholder (mesmo que você não inclua um atributo placeholder em seu HTML). Se o seu navegador não possui suporte ao placeholder, o objeto DOM criado para um elemento <input> não terá uma propriedade placeholder.

```
function supports_input_placeholder() {  
    var i = document.createElement('input');  
    return 'placeholder' in i;  
}
```

Ao invés de escrever essa função, você pode utilizar o Modernizr (1.1 ou superior) para detectar suporte ao placeholder.

↪ *checa pelo texto de placeholder*

```
if (Modernizr.input.placeholder) {  
    // seu placeholder já deve estar visível!  
} else {  
    // sem suporte ao placeholder :(  
    // implemente uma solução com script  
}
```



AUTO FOCO DE FORMULÁRIO



Seu navegador possui suporte auto foco.

Os web sites podem usar JavaScript para dar foco ao primeiro campo de um formulário automaticamente. Por exemplo, a página principal do Google.com dá auto foco no campo de busca para você poder iniciar sua busca sem ter que posicionar o cursor do mouse no campo de busca. Enquanto isso pode ser conveniente para a maioria das pessoas, pode vir a ser incômodo para usuários experientes ou pessoas com necessidades especiais. Se você tentar pressionar a barra de espaço esperando que a página seja rolada para baixo, não ocorrerá conforme o esperado, pois o foco já está em um campo. (Acaba digitando um espaço no campo ao invés de rolar a página.) Se você der foco a um campo diferente antes de a página terminar de carregar, o script de auto foco pode “gentilmente” mover o foco de volta ao campo original ao fim do carregamento da página, atrapalhando seu fluxo de trabalho fazendo com que você digite no lugar errado.

Como o auto foco é feito através do JavaScript, pode ser trabalhoso ter que lidar com todos esses delicados casos, e não há muito o que fazer para evitar a página web de “roubar” o foco.

Para resolver esse problema, a HTML5 apresenta um atributo de autofocus em todos os controles de formulário. O atributo autofocus faz exatamente o que o seu nome diz: move o foco para um campo específico. Mas por ser apenas uma marcação ao invés de script, o comportamento será consistente entre todos os sites. Além disso, os desenvolvedores de navegadores podem oferecer formas de desabilitar o comportamento de auto foco.

A verificação de suporte ao auto foco utiliza a técnica de detecção #2. Se o seu navegador possui suporte ao auto foco, o objeto DOM criado para representar um elemento `<input>` terá uma propriedade autofocus (mesmo que você não inclua o atributo autofocus em seu HTML). Se o seu navegador não possui suporte ao auto foco, o objeto DOM criado para um elemento `<input>` não terá uma propriedade autofocus. Você pode detectar suporte ao auto foco usando essa função:

```
function supports_input_autofocus() {  
    var i = document.createElement('input');  
    return 'autofocus' in i;  
}
```

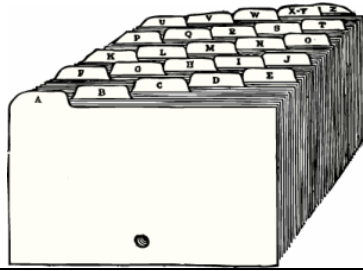
Ao invés de escrever essa função, você pode usar o Modernizr (1.1 ou superior) para detectar o suporte a campos com auto foco.

↪ *Verificando suporte ao auto foco*

```
if (Modernizr.input.autofocus) {  
    // autofocus funciona!  
} else {  
    // sem suporte a autofocus :(  
    // contornar com uma solução com script  
}
```



MICRO DADOS



Seu navegador não possui suporte a API de micro dado do HTML5. :(

Microdata é um jeito padronizado de informar semânticas adicionais em suas páginas. Por exemplo, você pode usar micro dados para declarar que uma foto está disponível sob a licença Creative Commons. Como você pode ver no capítulo de extensibilidade distribuída, você pode usar micro dados para demarcar uma página “Sobre mim”. Navegadores, extensões e motores de busca podem converter suas marcações de micro dados da HTML5 em um vCard, um formato padrão para compartilhamento de contatos. Você também pode definir seu próprio vocabulário de micro dados.

O padrão de micro dados da HTML5 inclui ambos a marcação HTML (a princípio para os motores de busca) e uma coleção de funções DOM (a princípio para os navegadores). Não há problema algum em adicionar marcações de micro dados em suas páginas. Não é nada mais do que alguns poucos atributos bem dispostos, e os motores de busca que não compreendem os atributos de micro dados irão simplesmente ignorá-los. Porém se você precisar acessar ou manipular os micro dados através de DOM, você terá que checar se o navegador possui ou não suporte a API DOM de micro dados.

A verificação de suporte a micro dados da HTML5 utiliza a técnica de detecção #1. Se o seu navegador possui suporte aos micro dados da HTML5, haverá uma função `getItems()` no objeto global `document`. Se o seu navegador não possuir suporte a micro dados, a função `getItems()` será `undefined`.

```
function supports_microdata_api() {  
    return !!document.getItems;  
}
```

O Modernizr ainda não possui suporte aos micro dados, então você terá que usar uma função semelhante a exibida acima.

API DE HISTÓRICO



Seu navegador possui suporte a API de histórico do HTML5.

A API de histórico da HTML5 é uma maneira padronizada de manipular o histórico do navegador via script. Uma parte dessa API — navegando pelo histórico — tem estado disponível em versões anteriores do HTML. A nova parte em HTML5 é uma maneira de adicionar entradas no histórico do navegador, e obter uma resposta quando essas entradas são removidas da pilha quando o usuário pressionar o botão voltar do navegador. Isso quer dizer que a URL pode continuar a fazer o seu trabalho como identificador único para o recurso atual, mesmo em aplicações ricas em scripts que nunca realizam uma recarga total da página.

A verificação de suporte a API de histórico da HTML5 utiliza a técnica de detecção #1. Se o seu navegador possui suporte a API de histórico da HTML5, haverá uma função `pushState()` no objeto global `history`. Se o seu navegador não possui suporte a API de histórico, a função `pushState()` será `undefined`.

```
function supports_history_api() {  
    return !(window.history && history.pushState);  
}
```

Ao invés de escrever sua função, você pode utilizar o Modernizr (1.6 ou superior) para detectar suporte a API de histórico da HTML5.

↪ *Verificando suporte a API de histórico*

```
if (Modernizr.history) {  
    // o gerenciamento de histórico funciona!  
} else {  
    // sem suporte ao histórico :(  
    // contornar com uma solução de script History.js  
}
```



LEITURA ADICIONAL

Especificações e padrões:

- [o elemento <canvas>](#)
- [o elemento <video>](#)
- [tipos de <input>](#)
- [o atributo <input placeholder>](#)
- [o atributo <input autofocus>](#)
- [o storage do HTML5](#)
- [Web Workers](#)
- [Aplicações Web Offline](#)
- [API de Geolocalização](#)
- [Histórico e navegação](#)

Bibliotecas JavaScript:

- [Modernizr](#), uma biblioteca de detecção para HTML5
- [geo.js](#), uma API para geolocalização
- [HTML5 Cross-browser Polyfills](#)

Outros artigos e tutoriais:

- [Video for Everybody!](#)
- [A gentle introduction to video encoding](#)
- [Video type parameters](#)
- [The All-In-One Almost-Alphabetical No-Bullshit Guide to Detecting Everything](#)
- [Internet Explorer 9 Guide for Developers](#)



Nº3.

O QUE SIGNIFICA TUDO ISSO?

exibir índice analítico



MERGULHANDO



Esse capítulo pegará uma página HTML a qual não há nada de errado e irá melhorá-la. Algumas partes ficarão menores, outras partes maiores. Tudo isso ficará com uma melhor semântica. E ficará incrível.

Esta é a página em questão. Abra a página em uma nova aba e não volte até dar uma olhada no código fonte pelo menos uma vez.



O DOCTYPE

A partir do topo:

```
<!DOCTYPE html
  PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN"
  "http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">
```

Esse é o chamado “doctype.” Há uma longa história por trás do doctype. Enquanto trabalhavam no Internet Explorer 5 para Mac, os desenvolvedores da Microsoft depararam-se com um problema. A próxima versão do browser tinha melhorado tanto seus padrões, que páginas antigas não eram mais apresentadas corretamente. Ou até eram apresentadas (de acordo com as especificações), mas eles esperavam que fossem apresentadas *incorretamente*. Essas páginas foram criadas baseadas nas peculiaridades dos browsers que dominavam na época, como Netscape 4 e o Internet Explorer 4. O IE5/Mac estava tão avançado que acabou quebrando a web.

A Microsoft apareceu com uma nova solução. Antes de renderizar uma página, o IE5 verificava o “doctype,” que geralmente era a primeira linha do código HTML (até mesmo antes do próprio elemento <html>). Páginas antigas (que seguiam os padrões dos browsers antigos) geralmente não tinham o doctype. O IE5 renderizava essas páginas como os browsers antigos faziam. Para ativar os novos padrões, os autores das páginas web tinham que optar por inserir o doctype antes do elemento <html>.

Essa idéia se espalhou como fogo, e logo, todos os principais browsers passaram a ter duas opções de interpretação: “modo peculiar (quirks mode)” e “modo padronizado (standards mode)”. É claro que, tratando-se da web, as coisas rapidamente perderam o controle. Quando a Mozilla tentou enviar a versão 1.1 de seu browser, eles descobriram que haviam páginas sendo apresentadas no “modo padronizado” que na verdade dependiam de uma peculiaridade específica. A Mozilla tinha acabado de corrigir sua *rendering engine* para eliminar essa peculiaridade, mas milhares de páginas quebraram de uma vez. Então foi criado — e não estou inventando isso — “o modo quase padronizado (almost standards mode).”

Neste trabalho seminal, Activating Browser Modes with Doctype, Henri Sivonen resume os diferentes modos de renderização:

Quirks Mode (Modo peculiar)

No Quirks mode, os browsers violam as especificações da web contemporânea para evitar “quebrar” páginas criadas de acordo com as práticas que prevaleciam no anos 90.

Standards Mode (Modo padrão)

No Standards mode, os browsers tentam, conforme os documentos da especificação, tratar corretamente a extensão implementada para um browser específico. Para o HTML5 este é o “quirks mode.”

Almost Standards Mode (Modo quase padrão)

Firefox, Safari, Chrome, Opera (a partir da versão 7.5) e o IE8 também possuem o modo conhecido como “modo quase padrão”, que implementa o dimensionamento vertical das células de tabelas tradicionalmente e não rigorosamente de acordo com a especificação CSS2. Para o HTML5 este é o “limited quirks mode.”

(Você deveria ler o resto do artigo de Henri, pois estou apenas simplificando aqui. Até no IE5/Mac, haviam alguns doctypes antigos que não contavam como opção padronizada. Ao longo do tempo, a lista de peculiaridades cresceu, assim como a lista de doctypes que as desencadeava. A última vez que tentei contar, haviam 5 doctypes que disparavam o “almost standards mode,” e 73 que disparavam o “quirks mode.” Mas provavelmente esqueci de alguns e não vou nem falar sobre a besteira que o Internet Explorer 8 faz para trocar entre seus quatro, — quatro! — diferentes modos de renderização. Este é o diagrama de fluxo. Mate-o e queime-o.)

Agora onde estávamos? Ah sim, o doctype:

```
<!DOCTYPE html
  PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN"
  "http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">
```

Este passa a ser um dos 15 doctypes que disparam o “standards mode” em todos os browsers atuais. Não há nada de errado com ele. Se você gosta dele, pode ficar. Ou você pode mudá-lo para o doctype do HTML5, que é menor e melhor, e também dispara o “standards mode” em todos os browsers atuais.

Este é o doctype do HTML5:

```
<!DOCTYPE html>
```

É só isso. Apenas 15 caracteres. É tão fácil que você pode digitá-lo na mão sem medo de errar.



O ELEMENTO RAIZ



Uma página HTML consiste em uma série de elementos. Toda sua estrutura é como uma árvore. Alguns elementos são “irmãos,” como dois galhos que estendem-se de um mesmo tronco. Alguns podem ser “filhos” de outros elementos, como um galho menor que estende-se de um maior. (Também funciona de outro jeito; um elemento que contém outro elemento é chamado de nó “pai” de seus elementos filhos, e o “antecessor” de seus elementos netos). Elementos que não possuem filhos são chamados de nós “folha”. O elemento mais externo, o qual é o antecessor de todos os outros elementos da página, é chamado de “elemento raiz.” O elemento raiz de uma página HTML é sempre o `<html>`.

Nessa página de exemplo, o elemento raiz se parece com isso:

```
<html xmlns="http://www.w3.org/1999/xhtml">
```

```
lang="en"  
xml:lang="en">
```

Não há nada de errado com esta implementação. De novo, se você gosta dela, pode ficar. É válido no HTML5. Mas algumas partes não são mais necessárias no HTML5, então você pode economizar alguns bytes ao removê-las.

A primeira coisa a se discutir é o atributo `xmlns`. É um vestígio do XHTML 1.0. Ele serve para saber que os elementos da página estão dentro do namespace XHTML, <http://www.w3.org/1999/xhtml>. Mas os elementos no HTML5 estão sempre neste namespace, sendo que você não precisa mais declará-lo explicitamente. Sua página HTML5 funcionará exatamente igual em todos os browsers, este atributo estando ou não presente.

Descartando o atributo `xmlns`, ficamos com este elemento raiz:

```
<html lang="en" xml:lang="en">
```

Estes dois atributos, `lang` e `xml:lang`, definem a língua da página HTML. (`en` significa “Inglês.” Não escreve em inglês? [Encontre sua língua.](#)) Mas porque dois atributos para a mesma coisa? De novo, isso é um vestígio do XHTML. Apenas o atributo `lang` tem algum efeito no HTML5. Você pode deixar o atributo `xml:lang` se preferir, mas se deixá-lo, deve garantir que ele contenha o mesmo valor do atributo `lang`.

Para facilitar a migração do XHTML, devemos especificar um atributo sem namespace, sem prefixo e com o localname “`xml:lang`” nos elementos HTML de documentos HTML, mas estes atributos devem apenas serem especificados se o atributo `lang` também for especificado sem namespace. Os dois atributos são case-insensitive e devem conter o mesmo valor. O atributo sem namespace, sem prefixo e com o localname “`xml:lang`” não tem efeito no processamento da língua.

Está pronto para descartá-lo? Tudo bem, deixe-o ir. Indo, indo... já era! Isso nos deixa com este elemento raiz:

```
<html lang="en">
```

E isso é tudo que tenho para falar sobre isso.



O ELEMENTO <HEAD>



O primeiro filho do elemento raiz geralmente é o elemento `<head>`. O `<head>` contém informações — metadata *sobre* a página, em vez do próprio corpo da página. (O corpo da página fica no elemento `<body>`). O elemento `<head>` é um pouco chato, e não mudou nada de interessante no HTML5. A parte boa é o que está *dentro* do `<head>`. E para isso, usaremos novamente nossa página de exemplo:

```
<head>
  <meta http-equiv="Content-Type" content="text/html; charset=utf-
8" />
  <title>My Weblog</title>
  <link rel="stylesheet" type="text/css" href="style-original.css"
/>
  <link rel="alternate" type="application/atom+xml"
        title="My Weblog feed"
        href="/feed/" />
  <link rel="search" type="application/opensearchdescription+xml"
        title="My Weblog search"
        href="opensearch.xml" />
  <link rel="shortcut icon" href="/favicon.ico" />
</head>
```

Primeiramente: O elemento `<meta>`.



CODIFICAÇÃO DE CARACTERES

Quando você pensa em “texto,” provavelmente pensa em “caracteres e símbolos que vejo na tela do meu computador.” Mas os computadores não lidam com caracteres e símbolos, mas sim com bits e bytes. Cada pedaço de texto que vê na tela de seu computador, é na verdade armazenado em uma *codificação de caracteres* (*character encoding*). Há centenas de character encodings diferentes, alguns otimizados para algumas línguas em particular como russo, chinês ou inglês, e outras que podem ser utilizadas para várias línguas. Rudemente falando, o character encoding fornece um mapeamento entre as coisas que você vê em sua tela e as coisas que o computador armazena na memória ou em disco.

Na realidade, é mais complicado que isso. O mesmo caracter pode aparecer em mais de uma codificação, mas cada uma pode usar uma sequência diferente de bytes para realmente armazenar o caracter em memória ou em disco. Então, você pode pensar no character encoding como um tipo de chave de decodificação para o texto. Sempre que alguém lhe der uma sequência de bytes e reivindicar seu “texto”, você precisa saber qual character encoding eles usaram, assim você pode decodificar os bytes em caracteres e exibí-los (ou processá-los, que seja).

Então, como seu browser determina o character encoding de um fluxo de bytes que um servidor web envia? Estou contente que perguntou. Se você está familiarizado com headers (cabeçalhos) HTTP, já deve ter visto um como esse:

```
Content-Type: text/html; charset="utf-8"
```

Brevemente, ele acha que o servidor web está lhe enviando um documento HTML, e pensa que o documento usa o character encoding UTF-8. Infelizmente, no magnífico mundo da World Wide Web, poucos realmente tem controle sobre seus servidores HTTP. Pense no Blogger: o conteúdo é fornecido pelas pessoas, mas os servidores rodam pelo Google. Então o HTML 4 fornecia uma maneira de especificar o character encoding no próprio documento HTML. Você provavelmente já deve ter visto isso também:

```
<meta http-equiv="Content-Type" content="text/html; charset=utf-8">
```

Ele diz que o autor da página web pensa que criaram um documento HTML usando o character encoding UTF-8.

Essas duas técnicas ainda funcionam no HTML5. O header HTTP é o método preferido, e ele substitui a tag <meta> se estiver presente. Mas não é qualquer um que pode definir headers HTTP, então a tag <meta> ainda está por aí. Na verdade, ficou um pouco mais fácil no HTML5. Agora ela é assim.

```
<meta charset="utf-8" />
```

Isso funciona em todos os browsers. Como essa sintaxe abreviada apareceu? Aqui está a melhor explicação que consegui encontrar:

A lógica para a combinação do atributo <meta charset=""> é que UAs já implementam isso, pois as pessoas tendem a deixar as coisas sem aspas como:

```
<META HTTP-EQUIV=Content-Type CONTENT=text/html; charset=ISO-8859-1>
```

Há até alguns casos de teste de <meta charset> se você não acreditar que os browsers já fazem isso.

P: Eu nunca uso caracteres diferentes. Eu ainda preciso declarar meu character encoding?

R: Sim! Você deve *sempre* especificar um character encoding em toda página HTML que escreve. Não especificar um encoding pode levar a vulnerabilidades de segurança.

Resumindo: character encoding é complicado, e ele não foi feito de uma maneira mais fácil devido a décadas de software mal escrito feito por copia-e-cola. Você deve **sempre** especificar um character encoding em **todos** os documentos HTML, ou coisas ruins vão acontecer. Você pode fazer isso com o HTTP Content-Type header, com a declaração `<meta http-equiv>`, ou com a declaração mais curta `<meta charset>`, mas por favor faça. A web agradece.



AMIGOS & (LINK) RELATIONS

Links normais (`<a href>`) simplesmente apontam para uma outra página. Link relations são uma maneira de explicar o *por que* você está apontando para uma outra página. Eles terminam a frase “Estou apontando para esta outra página por que...”

- ...é uma stylesheet que contém regras de CSS que o browser deveria aplicar a esse documento.
- ...é um feed que contém o mesmo conteúdo desta página, mas em um formato padrão registrável.
- ...é uma tradução desta página para outra língua.
- ...é o mesmo conteúdo desta página, mas em formato PDF.
- ...é o próximo capítulo de um livro online o qual esta página também faz parte.

E por aí vai. HTML5 separa os link relations em duas categorias:

Duas categorias de links podem ser criadas usando o elemento link. **Links para recursos externos** são links para recursos que são usados para estender o documento atual, e **hyperlinks** são links para outros documentos. ...

O comportamento exato para links de recursos externos depende da exata relação, como definido para o tipo de link relevante.

Dos exemplos que acabei de dar, somente o primeiro (`rel="stylesheet"`) é um link para um recurso externo. O resto são hyperlinks para outros documentos. Você pode querer seguir esses links, ou não, mas eles não são exigidos para ver a página atual.

Geralmente, link relations são vistos nos elementos `<link>` dentro do elemento `<head>` de uma página. Alguns podem também ser usados em elementos `<a>`, mas isso é incomum mesmo

quando permitido. HTML5 também permite alguns relations em elementos <area>, mais isso é até *menos* comum (HTML 4 não permitia um atributo rel nos elementos <area>). Veja [a tabela completa de link relations](#) para verificar onde você pode usar valores específicos de rel.

☞ P: Posso criar meu próprio link relations?

R: Parece haver um suprimento infinito de idéias para novos link relations. Em uma tentativa para evitar que as pessoas [façam besteira](#), o WHATWG [mantém um registro das propostas para valores rel](#) e [define o processo para aceitá-las](#).

REL = STYLE SHEET

Vamos olhar o primeiro link relation em [nossa página de exemplo](#):

```
<link rel="stylesheet" href="style-original.css" type="text/css" />
```

Este é o link relation mais usado no mundo (literalmente). <link rel="stylesheet"> é para apontar para regras CSS que estão armazenadas em um arquivo separado. Uma pequena otimização que você pode fazer no HTML5 é retirar o atributo type. Há somente uma linguagem stylesheet para a web, o CSS, então esse é o valor padrão para o atributo type. Isso funciona em todos os browsers. (Creio que alguém pode inventar uma nova linguagem stylesheet algum dia, mas se acontecer, apenas acrescente o atributo type de volta).

```
<link rel="stylesheet" href="style-original.css" />
```

REL = ALTERNATE

Continuando com [nossa página de exemplo](#):

```
<link rel="alternate"
      type="application/atom+xml"
      title="My Weblog feed"
      href="/feed/" />
```

Esse link relation também é bastante comum. <link rel="alternate">, combinado com RSS ou Atom media no atributo type, habilita algo chamado “descoberta de feed”. Ele permite que leitores de feeds (como [Google Reader](#)) descubram que um site tem um feed de notícias dos últimos artigos. Alguns browsers também suportam a descoberta de feed exibindo um ícone especial perto da URL. (Ao contrário do rel="stylesheet", o atributo type importa aqui. Não remova-o!)

O link relation `rel="alternate"` sempre foi um caso estranho de uso, até no HTML 4. No HTML5, sua definição foi clareada e estendida para descrever o atual conteúdo da web mais cuidadosamente. Como você acabou de ver, usando `rel="alternate"` em conjunto com `type=application/atom+xml` indica um feed Atom para a página atual. Mas você também pode usar o `rel="alternate"` em conjunto com outros atributos `type` para indicar o mesmo conteúdo em outro formato, como PDF.

HTML5 também encerrou uma confusão de longa data sobre como apontar para documentos de tradução. HTML 4 diz para usar o atributo `lang` em conjunto com `rel="alternate"` para especificar a língua do documento apontado, mas isso é incorreto. O documento HTML 4 Errata lista quatro erros na especificação do HTML 4. Um desses erros é como especificar a língua de um documento apontado com `rel="alternate"`. A maneira correta, descrita no HTML 4 Errata e agora no HTML5, é usar o atributo `hreflang`. Infelizmente, essa errata nunca foi reintegrada na especificação do HTML 4, por que ninguém mais no W3C HTML Working Group estava trabalhando com HTML.

OUTROS LINK RELATIONS NO HTML5

`rel="author"` é usado para apontar para informações sobre o autor da página. Pode ser um endereço `mailto:`, embora não precise ser. Ele pode simplesmente levar a um formulário de contato ou a uma página “sobre o autor”.

`rel="external"` “indica que o link trata-se de um documento que não faz parte do site em que o documento atual está.” Acredito que isso foi popularizado pela WordPress, a qual o utiliza nos links dos comentários deixados pelas pessoas.



HTML 4 usava `rel="start"`, `_rel="prev"`, `_e_rel="next"` para definir relações entre páginas que fazem parte de uma série (como capítulos de um livro ou até posts de um blog). O único que era utilizado corretamente era o `rel="next"`. As pessoas usavam `rel="previous"` ao invés de `rel="prev"`; usavam `rel="begin"` e `rel="first"` ao invés de `rel="start"`; utilizavam `rel="end"` invés de `rel="last"`. Oh, e — por conta própria — eles criaram `rel="up"` para apontar para uma “página pai”.

HTML5 inclui `rel="first"`, que é a variação mais comum das diferentes maneiras para dizer “primeira página da série.” (`rel="start"` é um sinônimo sem conformidade, fornecido para compatibilidade.) Também inclui `rel="prev"` e `rel="next"`, igual ao HTML 4, e suporta `rel="previous"` para compatibilidade, assim como `rel="last"` (a última da série, começada por `rel="first"`) e `rel="up"`.

A melhor maneira de pensar em `rel="up"` é olhar para sua trilha de navegação (ou pelo menos imaginá-la). Sua página principal é provavelmente a primeira página em sua trilha e a página atual está no final. `rel="up"` aponta para a página seguinte a última página da trilha.

`rel="icon"` é o segundo link relation mais popular, depois do `rel="stylesheet"`. Ele é geralmente encontrado junto ao `shortcut`, assim:

```
<link rel="shortcut icon" href="/favicon.ico">
```

Todos os principais browsers suportam seu uso para associar um pequeno ícone a uma página. Geralmente é exibido na barra de endereço do browser próximo a URL, ou na aba do browser, ou em ambos.

Novo também no HTML5: os atributos `sizes` podem ser usados em conjunto em um relacionamento com o ícone para indicar o tamanho do ícone referenciado.

`rel="license"` foi inventado pela comunidade de microformatos. Ele “indica que o documento referenciado fornece os termos da licença sob o qual o documento atual é fornecido.”

`rel="nofollow"` “indica que o link não foi aprovado pelo autor original ou publicador da página, ou que o link para o documento referenciado foi incluído inicialmente por causa de uma relação comercial entre pessoas afiliadas com as duas páginas.” Isso foi inventado pelo Google e padronizado dentro da comunidade de microformatos. WordPress adicionou `rel="nofollow"` para links incluídos nos comentários. A idéia era que se os links “nofollow” não aparecessem no PageRank, spammers desistiriam de tentar postar spams nos comentários dos blogs. Isso não aconteceu, mas `rel="nofollow"` ainda persiste.

`rel="noreferrer"` “indica que nenhuma informação de referência deve ser vazada quando clicar no link.” Atualmente nenhum browser suporta isso, mas o suporte foi adicionado pelo WebKit, sendo que ele aparecerá no Safari, Google Chrome, e outros WebKit browsers. [`rel="noreferrer" test case`]

`rel="pingback"` especifica o endereço de um servidor “pingback”. Como explicado na especificação do Pingback, “O sistema pingback é uma maneira de um blog ser notificado automaticamente quando outros sites chamarem um link para ele. ... Ele permite um vínculo reverso — um modo de voltar em uma corrente de links ao invés de somente fazer um drill down.” Sistemas de blogs, especialmente o WordPress, implementam o mecanismo de pingback

para notificar os autores que você criou um link para a página deles quando criou um novo post em seu blog.



rel="prefetch" “indica que buscar e armazenar um recurso especificado preventivamente é provável que seja benéfico, como o usuário provavelmente irá exigir este recurso.” Mecanismos de busca, às vezes, adicionam `<link rel="prefetch" href="URL do primeiro resultado da busca">` para a página de resultados da busca se eles sentem que o primeiro resultado é freneticamente mais popular que qualquer um. Por exemplo: usando Firefox, procure CNN no Google, olhe o código fonte, e procure pela palavra-chave `prefetch`. Mozilla Firefox é o único browser atual que suporta `rel="prefetch"`.

rel="search" “indica que o documento referenciado fornece uma interface específica para procurar o documento e seus recursos relacionados.” Especificamente, se você quer que o `rel="search"` faça algo útil, ele deve apontar para um documento OpenSearch que descreve como um browser poderia construir uma URL para procurar o site atual para uma dada palavra-chave. OpenSearch (e `rel="search"` ligam aquele ponto para documentos OpenSearch) tem sido suportado no Microsoft Internet Explorer desde a versão 7 e no Mozilla Firefox desde a versão 2.

rel="sidebar" “indica que o documento referenciado, se recuperado, destina-se a ser exibido em um contexto de navegação secundário (se possível), ao invés de no mesmo contexto de navegação atual.” O que isso significa? No Opera e no Mozilla Firefox, isso significa que “quando eu clicar neste link, é solicitado ao usuário para criar um bookmark que, quando selecionado pelo menu de Bookmarks, abre o documento vinculado em uma sidebar do browser.” (Opera atualmente chama isso de “painel” ao invés de “sidebar.”) Internet Explorer, Safari, e Chrome ignoram `rel="sidebar"` e apenas o tratam como um link normal. [rel="sidebar" test case]

rel="tag" “indica que a tag que o documento referenciado representa aplica-se ao documento atual.” Marcação de “tags” (category keywords) com o atributo `rel` foi inventado pela Technorati para ajuda-los a categorizar os posts do blog. Blogs antigos e tutoriais assim se referiu a eles como “Technorati tags.” (Você leu certo: uma empresa comercial convenceu o mundo todo a adicionar metadata que facilitaram o trabalho da empresa. Belo trabalho se você pode te-lo!) A sintaxe foi mais tarde padronizada dentro da comunidade de microformatos,

onde foi simplesmente chamada de `rel="tag"`. A maioria de sistemas de blog que permitem categorias associadas, palavras-chave, ou tags com posts individuais vão marca-las com links `rel="tag"`. Os browsers não fazem nada de especial com eles; eles são realmente desenhados para mecanismos de busca para usar como um sinal sobre do que a página se trata.

NOVOS ELEMENTOS SEMÂNTICOS NO HTML5

HTML5 não se trata somente em reduzir as marcações existentes (embora faça-o em uma quantidade razoável). Ele também define novos elementos semânticos.

<section>	O elemento <code>section</code> representa uma seção genérica de um documento ou aplicação. Uma seção, neste contexto, é um agrupamento de conteúdo, geralmente com um título. Exemplos de seções podem ser capítulos, páginas em abas de uma caixa de diálogo, ou as seções numeradas de uma tese. A página inicial de um website pode ser separada em seções para introdução, itens de notícias, informações para contato.
<nav>	O elemento <code>nav</code> representa a seção de uma página que aponta para outras páginas ou para partes dentro da página: uma seção com links de navegação. Nem todos os grupos de links de uma página precisam estar em um elemento <code>nav</code> — apenas seções que consistem em grandes blocos de navegação são apropriados para o elemento <code>nav</code> . Particularmente, é comum para rodapés (footers) possuir uma pequena lista de links para páginas em comum de um site, tal como termos de uso, página inicial e página de direitos autorais. O elemento <code>footer</code> sozinho é suficiente para esses casos, sem o elemento <code>nav</code> .
<article>	O elemento <code>article</code> representa um componente de uma página que consiste em uma composição de conteúdo próprio em um documento, página, aplicação, ou site e que destina-se a ser independentemente distribuível ou reutilizável, e.g. in syndication. Pode ser uma postagem em um fórum, um artigo de uma revista ou jornal, o comentário de um usuário, um widget ou gadget interativo, ou qualquer outro item independente de conteúdo.
<aside>	O elemento <code>aside</code> representa a seção de uma página que consiste em conteúdo que é tangencialmente relacionado ao conteúdo em torno do elemento <code>aside</code> , e o qual pode ser considerado separado daquele conteúdo. Tais seções são frequentemente representadas como barras laterais em tipografia impressa. O elemento pode ser usado para efeitos tipográficos como citações e barras laterais, para publicidade, para grupos de elementos <code>nav</code> , e para outro conteúdo que é considerado separado do conteúdo principal da página.
<hgroup>	O elemento <code>hgroup</code> representa o título de uma seção. O elemento é usado para agrupar um conjunto de elementos <code>h1</code> – <code>h6</code> quando o título possui vários níveis, tais como subtítulos, títulos alternativos, ou slogans.
<header>	O elemento <code>header</code> representa um grupo de ajuda introdutória ou navegação. Um elemento <code>header</code> geralmente pretende possuir o título da seção (um elemento <code>h1</code> – <code>h6</code> ou um elemento <code>hgroup</code>), mas isso não é obrigatório. O

	elemento <code>header</code> também pode ser usado para cobrir uma seção de tabelas de conteúdo, um formulário de busca, ou qualquer logo relevante.
<code><footer></code>	O elemento <code>footer</code> representa um rodapé para a seção de conteúdo mais próxima ou seção do elemento raiz. Um rodapé tipicamente contém informação sobre sua seção tal como quem a escreveu, links para documentos relacionados, declaração de direitos autorais, e assim por diante. Os rodapés não precisam necessariamente aparecer no fim da seção, embora eles geralmente apareçam. Quando o elemento <code>footer</code> contém seções inteiras, eles representam apêndices, índices, longos termos de uso, e outros conteúdos.
<code><time></code>	O elemento <code>time</code> representa tanto uma hora em um relógio de 24 horas, como uma data precisa no calendário gregoriano, opcionalmente com uma hora e um fuso horário.
<code><mark></code>	O elemento <code>mark</code> representa a execução de texto em um documento marcado ou destacado com o propósito de referência.

Eu sei que você está ansioso para começar a usar esses novos elementos, caso contrário você não estaria lendo este capítulo. Mas primeiro nós precisamos fazer um pequeno desvio.

UM LONGO DESVIO EM COMO OS BROWSERS LIDAM COM ELEMENTOS DESCONHECIDOS

Todo browser tem uma lista de elementos HTML que suporta. Por exemplo, a lista do Mozilla Firefox está armazenada em `nsElementTable.cpp`. Elementos que não estão nesta lista são tratados como “elementos desconhecidos.” Há dois problemas fundamentais com elementos desconhecidos:

1. **Como deve ser o estilo do elemento?** Por padrão, `<p>` tem espaçamento na parte superior e inferior, `<blockquote>` é recuado com uma margem esquerda, e `<h1>` é exibido em uma fonte maior. Mas quais estilos padrão devem ser aplicados aos elementos desconhecidos?
2. **Como o elemento do DOM deve parecer?** O elemento `nsElementTable.cpp` do Mozilla inclui informações sobre quais tipos de outros elementos cada um pode conter. Se você incluir markup como `<p><p>`, o elemento do segundo parágrafo implicitamente fecha o primeiro, assim os elementos terminam como irmãos, e não pai e filho. Mas se você escrever `<p><span>`, o `span` não fecha o parágrafo, porque o Firefox sabe que `<p>` é um elemento de bloco que pode conter o elemento de linha `<span>`. Então, o `<span>` termina como um filho do `<p>` no DOM.

Browsers diferentes respondem a essas perguntas de diferentes maneiras. (Revoltante, Eu sei). Dos principais browsers, a resposta do Microsoft Internet Explorer para as duas perguntas é a mais problemática, mas todo browser precisa de uma pequena ajuda aqui.

A primeira pergunta deveria ser relativamente simples de responder: Não aplique nenhum estilo especial para elementos desconhecidos. Apenas deixe-os herdar qualquer que seja as propriedades CSS que estão em vigor e onde quer que apareça, e deixe que o autor da página especifique todos os estilos com CSS. E isso funciona, com a maioria, mas há uma pequena pegadinha que você precisa estar ciente.

Todos os browsers apresentam elementos desconhecidos como elementos em linha, *i.e.* como se eles tivessem a regra `display:inline` de CSS.

Há diversos elementos novos definidos no HTML5 que são de nível de bloco. Isto é, eles podem conter outros elementos com nível de bloco, e os browsers compatíveis com o HTML5 irão aplicar a propriedade `display:block` por padrão. Se você quiser utilizar esses elementos em browsers antigos, precisará definir a propriedade `display` manualmente:

```
article,aside,details,figcaption,figure,
footer,header,hgroup,menu,nav,section {
    display:block;
}
```

(Esse código é tirado de [HTML5 Reset Stylesheet](#) de Rich Clark, o qual faz muitas outras coisas além do escopo deste capítulo).

Mas espere, fica pior! Antes da versão 9, o Internet Explorer não aplicou *qualquer* estilo nos elementos desconhecidos. Por exemplo, se você tivesse essa implementação:

```
<style type="text/css">
  article { display: block; border: 1px solid red }
</style>
...
<article>
<h1>Welcome to Initech</h1>
<p>This is your <span>first day</span>.</p>
</article>
```

O Internet Explorer (até a versão 8) não trata o elemento `<article>` como um elemento de nível de bloco, nem coloca uma borda vermelha em volta do article. Todas as regras de estilo são simplesmente ignoradas. O Internet Explorer 9 corrige este problema.

O segundo problema é o DOM que os browsers criam quando encontram elementos desconhecidos. Novamente, o browser mais problemático são as versões antigas do Internet Explorer (antes da versão 9, a qual corrige esse problema também). Se o IE 8 não reconhecer explicitamente o nome do elemento, ele irá inserir o elemento no DOM *como um nó vazio sem filhos*. Todos os elementos que você esperaria serem filhos diretos de um elemento desconhecido, na verdade estarão inseridos como irmãos do mesmo.

Aqui está uma arte ASCII para ilustrar a diferença. Este é o DOM que o HTML5 determina:

```
article
|
+--h1 (filho de article)
|   |
|   +--text node "Welcome to Initech"
|
+--p (filho de article, irmão de h1)
|   |
|   +--text node "This is your "
|   |
|   +--span
|   |   |
|   |   +--text node "first day"
|   |
|   +--text node "."
```

Mas esse é o DOM que o Internet Explorer atualmente cria:

```
article (sem filhos)
h1 (irmão de article)
|
+--text node "Welcome to Initech"
p (irmão de h1)
|
+--text node "This is your "
|
+--span
|   |
|   +--text node "first day"
|
+--text node "."
```

Existe uma maravilhosa solução alternativa para este problema. Se você criar um elemento `<article>` falso com JavaScript antes de usá-lo na página, o Internet Explorer irá magicamente reconhecer o elemento `<article>` e deixar você aplicar um estilo CSS nele. Não há necessidade de inserir o elemento falso dentro do DOM. Simplesmente criar o elemento uma

vez (por página) é o suficiente para ensinar o IE como aplicar estilo ao elemento que ele não reconhece.

```
<html>
<head>
<style>
  article { display: block; border: 1px solid red }
</style>
<script>document.createElement("article");</script>
</head>
<body>
<article>
<h1>Welcome to Initech</h1>
<p>This is your <span>first day</span>.</p>
</article>
</body>
</html>
```

Isso funciona em todas as versões do Internet Explorer, até o IE 6! Nós podemos estender essa técnica para criar cópias falsas de todos os novos elementos do HTML5 agora mesmo — novamente, eles nunca são inseridos no DOM, sendo que você nunca verá esses elementos falsos — e apenas começar usá-los sem se preocupar muito com os browsers não compatíveis com o HTML5.

Remy Sharp fez justamente isso, com seu acertadamente chamado [HTML5 enabling script](#). Esse script passou por mais de uma dúzia de revisões desde quando comecei a escrever este livro, mas a idéia é basicamente esta:

```
<!--[if lt IE 9]>
<script>
  var e = ("abbr,article,aside,audio,canvas,datalist,details," +
    "figure,footer,header,hgroup,mark,menu,meter,nav,output," +
    "progress,section,time,video").split(',');
  for (var i = 0; i < e.length; i++) {
    document.createElement(e[i]);
  }
</script>
<![endif]-->
```

Os trechos `<!--[if lt IE 9]>` e `<![endif]-->` são comentários condicionais. O Internet Explorer os interpreta como um `if`: “se o browser atual é uma versão do Internet Explorer anterior a versão 9, então executa este bloco.” Qualquer outro browser irá tratar todo o bloco como um comentário HTML. O resultado é que o Internet Explorer (até e incluindo a versão 8) executará este script, mas os outros browsers irão ignorar este script completamente. Isso faz sua página carregar mais rápido em browsers que não precisam deste hack.

O código JavaScript é relativamente simples. A variável `e` termina como um array de strings como `"abbr"`, `"article"`, `"aside"`, e assim por diante. Depois percorremos o array e criamos cada elemento nomeado chamando `document.createElement()`. Mas uma vez que ignoramos o valor de retorno, os elementos nunca são inseridos dentro do DOM. Mas é o suficiente para o

Internet Explorer tratar esses elementos do jeito que queremos que sejam tratados, uma vez que os usamos mais tarde na página.

Este trecho “mais tarde” é importante. Este script precisa estar no topo de sua página, preferencialmente em seu elemento <head>, e não no final. Dessa forma, o Internet Explorer irá executar este script *antes* de analisar suas tags e atributos. Se você colocar este script no final de sua página, ele o executará tarde. O Internet Explorer já terá mal interpretado sua implementação e construído o DOM errado, e não voltará para ajustá-lo por causa desse script.

Remy Sharp “minificou” este script e o hospedou no Google Project Hosting. (Para o caso de estar se perguntando, o script é open source e possui licença MIT, então pode ser usado em qualquer projeto.) Se preferir, você pode até “linkar” o script apontando diretamente para a versão hospedada, assim:

```
<head>
  <meta charset="utf-8" />
  <title>My Weblog</title>
  <!--[if lt IE 9]>
    <script
src="http://html5shiv.googlecode.com/svn/trunk/html5.js"></script>
    <![endif]-->
</head>
```

Agora estamos prontos para começar a usar os novos elementos no HTML5.

HEADERS



Vamos voltar para nossa página de exemplo. Especificamente, vamos olhar apenas para os headers:

```
<div id="header">
  <h1>My Weblog</h1>
```



```
<p class="tagline">A lot of effort went into making this
effortless.</p>
</div>
```

...

```
<div class="entry">
  <h2>Travel day</h2>
</div>
```

...

```
<div class="entry">
  <h2>I'm going to Prague!</h2>
</div>
```

Não há nada de errado com essa implementação. Se preferir, você pode mantê-la. É válido no HTML5. Mas o HTML5 provê alguns elementos adicionais para headers e sections.

Primeiramente, vamos nos livrar daquele `<div id="header">`. Esse é um padrão comum, mas isso não significa nada. O elemento `div` não possui uma semântica definida, e o atributo `id` também não. (User agents não têm permissão para inferir qualquer significado do valor do atributo `id`). Você pode mudar isso para `<div id="shazbot">` e ele terá o mesmo valor semântico, *i.e.*, nenhum.

HTML5 define um elemento `<header>` para este propósito. A especificação do HTML5 possui exemplos reais do uso do elemento `<header>`. Aqui está como ficaria parecido em nossa página de exemplo:

```
<header>
  <h1>My Weblog</h1>
  <p class="tagline">A lot of effort went into making this
effortless.</p>
  ...
</header>
```

Isso é legal. Ele diz para quem quiser saber que este é o header. Mas e aquela tagline? Outro padrão comum, que até agora não tinha implementação padrão. É uma coisa difícil de implementar. Uma tagline é como um subtítulo, mas está “ligado” ao título principal. Isso é, é um subtítulo que não cria sua própria section.

Elementos Header como `<h1>` e `<h2>` dão estrutura para sua página. Juntos, eles criam um esboço que você pode usar para visualizar (ou navegar) em sua página. Leitores de tela usam esboços de documentos para ajudar usuários cegos a navegar por sua página.

Existem ferramentas online e extensões de browsers que ajudam a visualizar o esboço de seu documento.

No HTML 4, os elementos `<h1>`–`<h6>` são *a única* maneira de criar um esboço do documento. O esboço da página de exemplo se parece com isso:

```
My Weblog (h1)
|
+--Travel day (h2)
|
+--I'm going to Prague! (h2)
```

Tudo bem, mas quer dizer que não tem como implementar a tagline “Um grande esforço foi feito para reduzir o esforço.” Se tentássemos implementá-lo como um `<h2>`, ele iria adicionar um nó fantasma ao esboço do documento:

```
My Weblog (h1)
|
+--A lot of effort went into making this effortless. (h2)
|
+--Travel day (h2)
|
+--I'm going to Prague! (h2)
```

Mas essa não é a estrutura do documento. A tagline não representa uma section, é apenas um subtítulo.

Talvez nós poderíamos implementar a tagline como um `<h2>` e marcar cada título de artigo como um `<h3>`? Não, é até pior:

```
My Weblog (h1)
|
+--A lot of effort went into making this effortless. (h2)
|
+--Travel day (h3)
|
+--I'm going to Prague! (h3)
```

Agora nós ainda temos um nó fantasma no esboço de nosso documento, mas ele “roubou” os filhos que legitimamente pertencem ao nó raiz. E aqui reside o problema: HTML 4 não fornece uma maneira de implementar um subtítulo sem adicioná-lo ao esboço do documento. Não importa o quanto tentarmos mudar as coisas, “um grande esforço feito para reduzir o esforço”

terminará naquele gráfico. Esse é o porque acabamos com marcações semânticas sem significado como `<p class="tagline">`.

HTML5 fornece uma solução para isso: o elemento `<hgroup>`. O elemento `<hgroup>` atua como um empacotador para dois ou mais elementos de título *relacionados*. O que quer dizer “relacionados”? Significa que juntos, eles criam um único nó no esboço do documento.

Dada esta implementação:

```
<header>
  <hgroup>
    <h1>My Weblog</h1>
    <h2>A lot of effort went into making this effortless.</h2>
  </hgroup>
  ...
</header>

...

<div class="entry">
  <h2>Travel day</h2>
</div>

...

<div class="entry">
  <h2>I'm going to Prague!</h2>
</div>
```

Esse é o esboço do documento que é criado:

```
My Weblog (h1 of its hgroup)
|
+--Travel day (h2)
|
+--I'm going to Prague! (h2)
```

Você pode testar suas próprias páginas no [HTML5 Outliner](#) para garantir que você está usando os elementos de título corretamente.

ARTICLES

Continuando com nossa página de exemplo, vamos ver o que podemos fazer sobre esta implementação:

```
<div class="entry">
  <p class="post-date">October 22, 2009</p>
  <h2>
    <a href="#"
      rel="bookmark"
      title="link to this post">
      Travel day
    </a>
  </h2>
  ...
</div>
```

Novamente, isto é HTML5 válido. Mas o HTML5 fornece elementos mais específicos para o caso comum de implementação de um artigo em uma página — chamado apropriadamente de `<article>`.

```
<article>
  <p class="post-date">October 22, 2009</p>
  <h2>
    <a href="#"
      rel="bookmark"
      title="link to this post">
      Travel day
    </a>
  </h2>
  ...
</article>
```

Ah, mas não é tão simples assim. Há mais uma mudança que você deve fazer. Vou lhe mostrar primeiro, depois eu explico:

```
<article>
  <header>
    <p class="post-date">October 22, 2009</p>
    <h1>
      <a href="#"
        rel="bookmark"
        title="link to this post">
        Travel day
```

```
    </a>
  </h1>
</header>
...
</article>
```

Você entendeu isso? Eu mudei o elemento `<h2>` para um elemento `<h1>`, e o coloquei dentro de um elemento `<header>`. Você já viu o elemento `<header>` em ação. Seu propósito é envolver todos os elementos que formam o cabeçalho do artigo (neste caso, a data de publicação e título do artigo). Mas...mas...mas... não deveríamos ter apenas um `<h1>` por documento? Isso não vai estragar o esboço do documento? Não, mas para entender porque não, nós precisamos dar um passo para trás.

No HTML 4, a *única* maneira de criar um esboço do documento era com os elementos `<h1>`–`<h6>`. Se você apenas quisesse um nó raiz em seu esboço, você tinha que limitar-se a um `<h1>` em sua implementação. Mas a especificação do HTML5 define um algoritmo para gerar um esboço de documento que incorpora os novos elementos semânticos no HTML5. O algoritmo do HTML5 diz que um elemento `<article>` cria uma nova seção, que é, um novo nó no esboço do documento. E no HTML5, cada seção pode possuir seu próprio elemento `<h1>`.

Esta é uma mudança drástica do HTML 4, e aqui está o porque isso é uma coisa boa. Muitas páginas da web são realmente geradas por templates. Um pouco de conteúdo é tirado de uma fonte e inserido na página aqui; um pouco de conteúdo é tirado de outra fonte e inserido na página ali. Muitos tutoriais são estruturados da mesma maneira. “Aqui está uma implementação HTML. Apenas copie e cole em sua página.” Tudo bem para pequenos trechos de conteúdo, mas e se a implementação que você está colando é uma seção inteira? Neste caso, o tutorial vai estar como algo do tipo: “Aqui está uma implementação HTML. Apenas copie e cole em seu editor de texto, e corrija as tags de cabeçalho para que possam coincidir com o nível de aninhamento das tags correspondentes da página em que você o está colando.

Deixe-me colocar isso de outra forma. HTML 4 não possui um elemento de cabeçalho *genérico*. Ele possui seis elementos estritamente numerados, `<h1>`–`<h6>`, os quais têm que estar exatamente nesta ordem. Este tipo de coisa fede, especialmente se sua página é “montada” ao invés de “criada.” E este é o problema que o HTML5 resolve com os novos elementos de seção e as novas regras para os elementos de cabeçalho existentes. Se você está usando os novos elementos de seção, posso lhe dar esta implementação:

```
<article>
  <header>
    <h1>A syndicated post</h1>
  </header>
  <p>Lorem ipsum blah blah...</p>
</article>
```

e você pode copiar e colar *em qualquer lugar de sua página* sem modificação. O fato de ele conter um elemento `<h1>` não é um problema, porque a coisa toda está contida dentro de um `<article>`. O elemento `<article>` define um nó contido em si próprio no esboço do documento, o elemento `<h1>` fornece o título para aquele nó, e todos os outros elementos de seção da página vão permanecer em qualquer nível de aninhamento que se encontravam antes.

Como todas as coisas na web, a realidade é um pouco mais complicada do que estou mostrando. Os novos elementos de seção “explícitos” (como `<h1>` contido no `<article>`) podem interagir de inesperadas maneiras com os velhos elementos “implícitos” (`<h1>`–`<h6>`). Sua vida ficará mais simples se você usar um ou outro, mas não os dois. Se você precisar usar os dois na mesma página, tenha certeza de checar o resultado em HTML5 Outliner e verifique se o esboço de seu documento faz sentido.

DATAS E HORAS



Isso é emocionante, não é? Digo, não é como “esquiar pelado no Mount Everest enquanto recita o Star Spangled Banner de trás pra frente”, mas é bem emocionante o quão longe a marcação semântica vai. Vamos continuar com nossa página de exemplo. A próxima linha que quero destacar é esta:

```
<div class="entry">
  <p class="post-date">October 22, 2009</p>
  <h2>Travel day</h2>
</div>
```

A mesma velha estória, certo? Um padrão comum — designando a data de publicação de um artigo — que não contém marcação semântica para apoiá-lo, então autores recorrem às implementações genéricas com alterações em atributos de `class`. Novamente, isso é HTML5 válido. Você não é *obrigado* a mudar isso. Mas o HTML5 fornece uma solução específica para este caso: o elemento `<time>`.

```
<time datetime="2009-10-22" pubdate>October 22, 2009</time>
```

Há três partes no elemento `<time>`:

1. Uma máquina de leitura para timestamp
2. Conteúdo de texto de leitura para humanos
3. Uma flag opcional `pubdate`

Neste exemplo, o atributo `datetime` apenas especifica a data, não o tempo. O formato é de quatro dígitos para o ano, dois dígitos para o mês, e dois dígitos para o dia, separado por traços:

```
<time datetime="2009-10-22" pubdate>October 22, 2009</time>
```

Se você quiser incluir o tempo também, adicione a letra T após a data, o tempo no formato de 24 horas, depois a diferença da timezone.

```
<time datetime="2009-10-22T13:59:47-04:00" pubdate>  
  October 22, 2009 1:59pm EDT  
</time>
```

(O formato data/tempo é bastante flexível. A especificação do HTML5 contém exemplos de data/tempo válidas.)

Note que mudei o texto — entre `<time>` e `</time>` — para combinar com a máquina de leitura para timestamp. Isso não é obrigatório. O conteúdo pode ser o que você quiser, contanto que você forneça uma máquina de leitura para data/timestamp no atributo `datetime`. Então isso é HTML5 válido:

```
<time datetime="2009-10-22">last Thursday</time>
```

E isso também é HTML5 válido:

```
<time datetime="2009-10-22"></time>
```

A peça final do quebra-cabeça é o atributo `pubdate`. É um atributo Boolean, então apenas adicione-o se precisar, assim:

```
<time datetime="2009-10-22" pubdate>October 22, 2009</time>
```

Se você não gostar de atributos “pelados”, isso é equivalente:

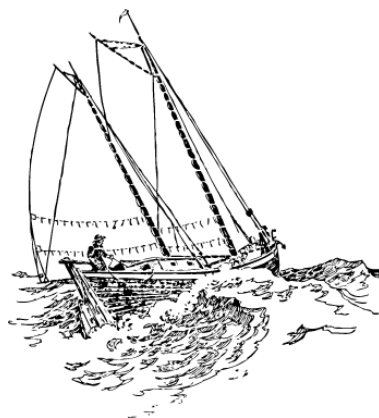
```
<time datetime="2009-10-22" pubdate="pubdate">October 22, 2009</time>
```

O que significa o atributo `pubdate`? Significa uma de duas coisas. Se o elemento `<time>` está em um elemento `<article>`, significa que este timestamp é a data de publicação do artigo. Se o elemento `<time>` não está em um elemento `<article>`, significa que esse timestamp é a data de publicação do documento inteiro.

Este é o artigo inteiro, reformulado para tirar total vantagem do HTML5:

```
<article>
  <header>
    <time datetime="2009-10-22" pubdate>
      October 22, 2009
    </time>
    <h1>
      <a href="#"
        rel="bookmark"
        title="link to this post">
        Travel day
      </a>
    </h1>
  </header>
  <p>Lorem ipsum dolor sit amet...</p>
</article>
```

NAVEGAÇÃO



Uma das partes mais importantes de qualquer site é a barra de navegação. A CNN.com tem “abas” ao longo do topo de cada página que apontam para diferentes seções de notícias — “Tech,” “Health,” “Sports,” &c. As páginas de resultado da busca do Google têm uma faixa semelhante no topo da página para tentar realizar sua busca em diferentes serviços do Google — “Imagens,” “Videos,” “Mapas,” &c. E nossa página de exemplo possui uma barra de navegação no cabeçalho que inclui links para diferentes seções de nosso hipotético site — “home,” “blog,” “gallery,” e “about.”

Assim é como a barra de navegação era originalmente implementada:

```
<div id="nav">
  <ul>
    <li><a href="#">home</a></li>
    <li><a href="#">blog</a></li>
    <li><a href="#">gallery</a></li>
    <li><a href="#">about</a></li>
  </ul>
</div>
```

Novamente, isso é HTML5 válido. Mas enquanto é feito como uma lista de quatro itens, não há nada sobre a lista que lhe diga que ela faz parte da navegação do site. Visualmente, você pode adivinhar pelo fato dela fazer parte do cabeçalho da página, e pelo texto dos links. Mas semanticamente, não há nada para distinguir a lista de links de qualquer outra.

Quem liga para a semântica de navegação do site? Por exemplo, peças com deficiências. Por que isso? Considere este cenário: seu movimento é limitado, e usar o mouse é difícil ou impossível. Para compensar, você pode usar um componente do browser que lhe permite avançar para a maioria dos links de navegação. Ou considere isso: se sua visão é limitada, você pode usar um programa dedicado chamado “leitor de tela” que usa text-to-speech para falar e resumir as páginas da web. Uma vez que passar do título da página, os próximos trechos importantes de informação sobre uma página são os principais links de navegação. Se você quer navegar rápido, você dirá ao seu leitor de tela para pular para a barra de navegação e começar a ler. Se você quiser consultar rápido, você pode dizer ao seu leitor de tela para *pular* a barra de navegação e começar a ler o conteúdo principal. De qualquer forma, ser capaz de determinar os links de navegação programaticamente é importante.

Então, enquanto não há nada de errado em usar `<div id="nav">` para criar a navegação de seu site, Também não há particularmente nada certo sobre isso. O HTML5 fornece uma maneira semântica para implementar seções de navegação: o elemento `<nav>`.

```
<nav>
  <ul>
    <li><a href="#">home</a></li>
    <li><a href="#">blog</a></li>
    <li><a href="#">gallery</a></li>
    <li><a href="#">about</a></li>
  </ul>
</nav>
```

☞P: Há skip links compatíveis com o elemento `<nav>`? Eu ainda preciso de skip links no HTML5?

R: Skip links permitem os leitores pularem entre as seções de navegação. Eles são úteis para usuários com deficiência que usam software de terceiros para ler uma página da web e navegar nela sem mouse. ([Aprenda como e porque fornecer os skip links.](#))

Uma vez que os leitores de tela forem atualizados para reconhecer o elemento <nav>, os skip links ficarão obsoletos, desde que o software de leitor de tela esteja apto a automaticamente saltar sobre uma seção de navegação implementada com o elemento <nav>. Contudo, isso será um pouco antes de todos os usuários com deficiência da web atualizarem para o software de leitor de tela com HTML5, então você deve continuar fornecendo seus próprios skip links para saltarem sobre as seções <nav>.



FOOTERS

Finalmente, nós chegamos ao fim de [nossa página de exemplo](#). A última coisa sobre qual quero falar é a última coisa da página: o footer. O footer era originalmente implementado assim:

```
<div id="footer">
  <p>&#167;</p>
  <p>&#169; 2001&#8211;9 <a href="#">Mark Pilgrim</a></p>
</div>
```

Isso é HTML5 válido. Se você gostar, pode continuar com ele. Mas o HTML5 fornece um elemento mais específico para isso: o elemento <footer>.

```
<footer>
  <p>&#167;</p>
  <p>&#169; 2001&#8211;9 <a href="#">Mark Pilgrim</a></p>
</footer>
```

O que é apropriado colocar em um elemento <footer>? Provavelmente qualquer coisa que você estiver colocando agora em um <div id="footer">. OK, é uma resposta redundante. Mas sério, é isso. A especificação do HTML5 diz, “Um footer geralmente contém informação sobre sua seção como quem a escreveu, links para documentos relacionados, copyright, e assim por diante.” É o que há em nossa página de exemplo: declaração de copyright e um link para uma página de “Sobre o autor”. Olhando ao redor em alguns sites populares, vejo muito potencial nos footers.

- [CNN](#) possui um footer que contém uma declaração de copyright, links para traduções, links para termos de uso e páginas de privacidade, “sobre,” “contato,” e “ajuda”. Todas totalmente apropriadas para o <footer>.
- [Google](#) possui uma home page esparsa, mas no final dela há links para “Programas de publicidade,” “Solução para negócios,” e “Sobre o Google”; uma declaração de

copyright; e um link para a política de publicidade do Google. Tudo isso pode ser incluído em um <footer>.

- My weblog possui um footer com links para meus outros sites, mais uma declaração de copyright. Definitivamente apropriado pra um elemento <footer>. (Note que os links *não* devem ser incluídos em um elemento <nav>, porque eles não são links de navegação do site; eles são apenas uma coleção de links para meus outros projetos em outros sites).

“Footers gordos” são raivosos hoje em dia. Dê uma olhada no footer do site da W3C. Ele contém três colunas, rotuladas “Navigation,” “Contact W3C,” e “W3C Updates.” A implementação se parece mais ou menos com isso:

```
<div id="w3c_footer">
  <div class="w3c_footer-nav">
    <h3>Navigation</h3>
    <ul>
      <li><a href="/">Home</a></li>
      <li><a href="/standards/">Standards</a></li>
      <li><a href="/participate/">Participate</a></li>
      <li><a href="/Consortium/membership">Membership</a></li>
      <li><a href="/Consortium/">About W3C</a></li>
    </ul>
  </div>
  <div class="w3c_footer-nav">
    <h3>Contact W3C</h3>
    <ul>
      <li><a href="/Consortium/contact">Contact</a></li>
      <li><a href="/Help/">Help and FAQ</a></li>
      <li><a href="/Consortium/sup">Donate</a></li>
      <li><a href="/Consortium/siteindex">Site Map</a></li>
    </ul>
  </div>
  <div class="w3c_footer-nav">
    <h3>W3C Updates</h3>
    <ul>
      <li><a href="http://twitter.com/W3C">Twitter</a></li>
      <li><a href="http://identi.ca/w3c">Identi.ca</a></li>
    </ul>
  </div>
  <p class="copyright">Copyright © 2009 W3C</p>
</div>
```

Para converter isso para HTML5 semântico, Eu faria as seguintes alterações:

- Converter o <div id="w3c_footer"> externo para um elemento <footer>.
- Converter as primeiras duas instâncias de <div class="w3c_footer-nav"> para elementos <nav>, e a terceira instância para um elemento <section>.
- Converter o <h3> para <h1>, desde que cada um ficará dentro de um elemento de seção. O elemento <nav> cria uma seção no esboço do documento, assim como o elemento <article>.

A implementação final pode parecer como algo do tipo:

```
<footer>
  <nav>
    <h1>Navigation</h1>
    <ul>
      <li><a href="/">Home</a></li>
      <li><a href="/standards/">Standards</a></li>
      <li><a href="/participate/">Participate</a></li>
      <li><a href="/Consortium/membership">Membership</a></li>
      <li><a href="/Consortium/">About W3C</a></li>
    </ul>
  </nav>
  <nav>
    <h1>Contact W3C</h1>
    <ul>
      <li><a href="/Consortium/contact">Contact</a></li>
      <li><a href="/Help/">Help and FAQ</a></li>
      <li><a href="/Consortium/sup">Donate</a></li>
      <li><a href="/Consortium/siteindex">Site Map</a></li>
    </ul>
  </nav>
  <section>
    <h1>W3C Updates</h1>
    <ul>
      <li><a href="http://twitter.com/W3C">Twitter</a></li>
      <li><a href="http://identi.ca/w3c">Identi.ca</a></li>
    </ul>
  </section>
  <p class="copyright">Copyright © 2009 W3C</p>
</footer>
```

LEITURA COMPLEMENTAR

Páginas de exemplo usadas em todo este capítulo:

- Original (HTML 4)
- Modificado (HTML5)

Sobre codificação de caracteres:

- The Absolute Minimum Every Software Developer Absolutely, Positively Must Know About Unicode and Character Sets (No Excuses!) por Joel Spolsky
- On the Goodness of Unicode, On Character Strings, e Characters vs. Bytes por Tim Bray

Sobre permitir novo HTML5 no Internet Explorer:

- How to style unknown elements in IE por Sjoerd Visscher
- HTML5 shiv por John Resig
- HTML5 enabling script por Remy Sharp

Sobre modos padrões e doctype:

- Activating Browser Modes with Doctype por Henri Sivonen. Este é o único artigo que você deve ler sobre o assunto. Qualquer artigo sobre doctypes que não referencie o trabalho de Henri é garantia de estar desatualizado, incompleto, ou incorreto.

Validador HTML5:

- html5.validator.nu

VAMOS CHAMÁ-LO (SUPERFÍCIE) DE DESENHO

MERGULHANDO



TML 5 define o elemento `<canvas>` como “uma tela bitmap de resolução dependente que pode ser usada para renderizar gráficos, jogos, ou outras imagens em tempo real.” A tag canvas é um retângulo na sua página, onde você pode usar JavaScript para desenhar o que você quiser.

SUPORTE BÁSICO AO <CANVAS>						
IE	FIREFOX	SAFARI	CHROME	OPERA	IPHONE	ANDROID
7.0+*	3.0+	3.0+	3.0+	10.0+	1.0+	1.0+

Internet Explorer 7 e 8 necessitam da biblioteca [explorercanvas](#). Internet Explorer 9 suporta `<canvas>` n

Então como o canvas se parece? Com nada, sério. O elemento `<canvas>` não tem conteúdo nem borda.

ua sintaxe se assemelha a isso:

```
<canvas width="300" height="225"></canvas>
```

Vamos adicionar uma borda pontilhada, então você pode ver com o que estamos lidando.

~ *Canvas com borda*

Você pode ter mais de um elemento `<canvas>` na mesma página. Cada canvas será mostrado no DOM, e cada um mantém seu próprio estado. Se você der para cada canvas um atributo `id`, você pode acessá-lo como qualquer outro elemento.

Vamos expandir nosso exemplo de canvas para incluir o atributo `id`:

```
<canvas id="a" width="300" height="225"></canvas>
```

Agora você pode encontrar facilmente este <canvas> dentro do DOM.

```
var a_canvas = document.getElementById("a");
```

SUPORTE BÁSICO AO <CANVAS>						
IE	FIREFOX	SAFARI	CHROME	OPERA	IPHONE	ANDROID
7.0+*	3.0+	3.0+	3.0+	10.0+	1.0+	1.0+

Todo canvas começa em branco. Que chato! Vamos desenhar alguma coisa.

← *Clique para desenhar nesse Canvas*

O evento onclick dispara essa função:

```
function draw_b() {  
  var b_canvas = document.getElementById("b");  
  var b_context = b_canvas.getContext("2d");  
  b_context.fillRect(50, 25, 150, 100);  
}
```

A 1ª linha dessa função não é nada de especial; apenas encontra o elemento <canvas> no DOM.

E então temos isso →

```
function draw_b() {  
  var b_canvas = document.getElementById("b");  
  var b_context = b_canvas.getContext("2d");  
  b_context.fillRect(50, 25, 150, 100);  
}
```



Toda canvas possui um contexto de desenho, que é onde toda diversão acontece. Uma vez tendo encontrado o elemento <canvas> no DOM (ao usar `document.getElementById()` ou qualquer método que você queira), você chama seu método `getContext()`. Você **deve** passar a string "2d" para o método `getContext()`.

❏P: Existe um canvas 3-D?

R: Ainda não. Fornecedores têm experimentado suas próprias APIs para canvas tridimensionais, mas nenhuma se tornou padrão ainda. De acordo com a especificação da HTML5, "Uma futura versão dessa especificação irá provavelmente definir um contexto 3d."

Então você tem o elemento <canvas> e seu contexto de desenho. O contexto de desenho é onde todos os métodos e propriedades do desenho são definidos. Há um bando de propriedades e métodos dedicados ao desenho de retângulos:

- A propriedade `fillStyle` pode ser uma cor, padrão ou degradê do CSS. (Mais sobre degradês em breve.) O padrão para o `fillStyle` é preto sólido, mas você pode definir o que quiser. Cada contexto de desenho guarda suas próprias propriedades enquanto a página estiver aberta, a não ser que você faça alguma coisa para resetar isso.
- `fillRect(x, y, largura, altura)` desenha um retângulo preenchido com o `fillStyle` atual.
- A propriedade `strokeStyle` é como `fillStyle` — pode ser uma cor, um padrão ou um degradê.
- `strokeRect(x, y, largura, altura)` desenha um retângulo com o `strokeStyle` atual. `strokeRect` não preenche o meio, apenas desenha as bordas.
- `clearRect(x, y, largura, altura)` limpa os pixels no retângulo especificado.

❏P: Eu posso "resetar" um canvas?

R: Sim. Definindo a altura e a largura de um elemento <canvas> irá apagar seu conteúdo e resetar todas as propriedades do seu contexto de desenho para os valores padrão. Você sequer precisa *alterar* a largura; você pode simplesmente definir ele para seu valor atual, como em:

```
var b_canvas = document.getElementById("b");
b_canvas.width = b_canvas.width;
```

Voltando para amostra de código vista no exemplo anterior...

Desenhe um retângulo ➞

```
var b_canvas = document.getElementById("b");
var b_context = b_canvas.getContext("2d");
b_context.fillRect(50, 25, 150, 100);
```

Chamando o método `fillRect()` desenha-se um retângulo e o preenche com o estilo de preenchimento atual, no qual é preto até você alterar isso. Um retângulo é limitado por seu

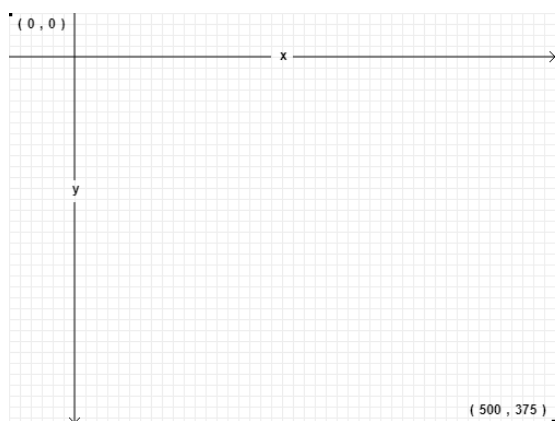
canto superior esquerdo (50, 25), sua largura (150), e sua altura (100). Para ter uma visão melhor de como isso funciona, vamos ver o sistema de coordenadas desse canvas.



COORDENADAS DO CANVAS

O canvas é uma grade bidimensional. A coordenada (0, 0) fica no canto superior esquerdo do canvas. Ao longo do eixo X, os valores aumentam em direção à borda direita da tela. Ao longo do eixo Y, os valores aumentam em direção à borda de baixo do canvas.

Diagrama de coordenadas do Canvas



O diagrama de coordenadas foi desenhado com o elemento <canvas>. Ele compreende:

- um conjunto de linhas verticais esbranquiçadas
- um conjunto de linhas horizontais esbranquiçadas
- duas linhas horizontais pretas
- duas pequenas linhas diagonais pretas que formam uma flecha
- duas linhas verticais pretas
- duas pequenas linhas diagonais pretas que formam uma flecha
- a letra "x"
- a letra "y"
- o texto "(0, 0)" próximo ao canto superior esquerdo
- o texto "(500, 375)" próximo ao canto inferior direito
- um ponto no canto superior esquerdo, e outro no canto inferior direito

Primeiro temos que definir o elemento <canvas>. O elemento <canvas> define a largura (width), a altura (height) e o identificador (id) para que possamos encontrá-lo mais tarde.

```
<canvas id="c" width="500" height="375"></canvas>
```

Depois nós precisamos encontrar o elemento <canvas> no DOM e buscar seu contexto de desenho.

```
var c_canvas = document.getElementById("c");  
var context = c_canvas.getContext("2d");
```

Agora podemos começar a desenhar as linhas.

CAMINHOS

IE	FIREFOX	SAFARI	CHROME	OPERA	IPHONE	ANDROID
7.0+*	3.0+	3.0+	3.0+	10.0+	1.0+	1.0+
* Internet Explorer 7 e 8 necessitam da biblioteca explorercanvas . Internet Explorer 9 suporta caminhos do <canvas> nativamente.						



Imagine que você está desenhando um quadro com tinta. Você não quer começar mergulhando e desenhando o quadro com a tinta, isso porque você pode cometer um erro. Ao invés disso, você rascunha as linhas e curvas com um lápis, e quando você estiver feliz com aquilo, irá traçar com tinta por cima do rascunho.

Cada canvas tem um caminho. Definir um caminho é como desenhar com um lápis. Você pode desenhar o que quiser, mas não irá fazer do produto final até que você pegue a pena e trace seu caminho com tinta.

Para desenhar linhas retas com lápis, use os dois métodos seguintes:

1. `moveTo(x, y)` move o lápis para o ponto inicial especificado.
2. `lineTo(x, y)` desenha a linha para o ponto final especificado.

Quanto mais você chamar `moveTo()` e `lineTo()`, maior será o tamanho do caminho. Esses são métodos “lápiz” — você pode chamá-los o quanto quiser, mas você não irá ver nada no canvas até que você invoque os métodos “tinta”.

Vamos começar desenhando nossa grade esbranquiçada.

```
for (var x = 0.5; x < 500; x += 10) {  
  context.moveTo(x, 0);  
  context.lineTo(x, 375);  
}
```

← *Desenha as linhas verticais*

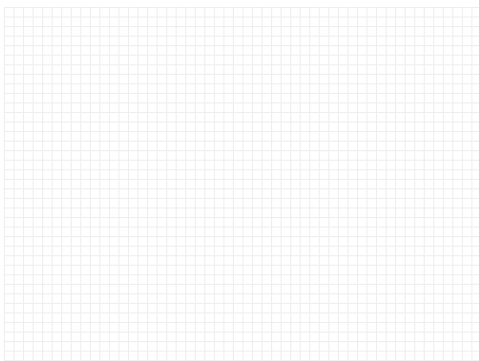
```
for (var y = 0.5; y < 375; y += 10) {  
  context.moveTo(0, y);  
  context.lineTo(500, y);  
}
```

← *Desenha as linhas horizontais*

Esses foram todos métodos “lápiz”. Nada foi desenhado no canvas ainda. Nós precisamos de um método “tinta” para tornar isso permanente.

```
context.strokeStyle = "#eee";  
context.stroke();
```

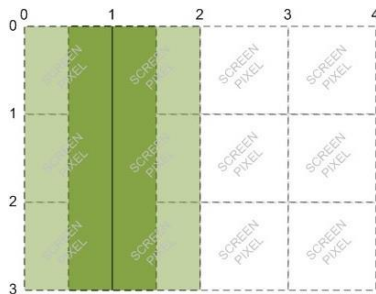
`stroke()` é um dos métodos “tinta”. Ele pega o caminho complexo que você definiu com todos aqueles `moveTo()` e `lineTo()`, e realmente desenha eles no canvas. O `strokeStyle` controla a cor das linhas. Esse é o resultado:



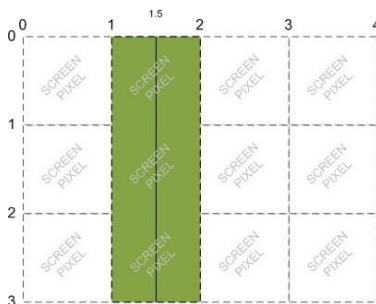
👉 P: Por que você começou com x e y com 0.5? Por que não 0?

R: Imagine cada pixel como um grande quadrado. As coordenadas inteiras (0, 1, 2...) são arestas desse quadrado. Se você desenhar uma linha com uma unidade de largura entre coordenadas inteiras, ele irá sobrepor lados opostos do quadrado de pixel, e a linha resultante será desenhada com dois pixels de largura. Para desenhar uma linha que tenha apenas um pixel de largura, você precisa mudar as coordenadas para 0.5 perpendicular à direção da linha.

Por exemplo, se você tentar desenhar a linha de (1, 0) para (1, 3), o navegador irá desenhar a linha cobrindo 0.5 pixels da tela em ambos os lados $x=1$. A tela não consegue exibir meio pixel, então irá expandir a linha para cobrir um total de dois pixels:



Mas se você tentar desenhar uma linha de (1.5, 0) para (1.5, 3), o navegador irá desenhar a linha cobrindo 0.5 pixels da tela em ambos os lados $x=1.5$, o que resulta na verdade em uma linha de 1 pixel de largura:



Agradecimentos ao Jason Johnson por prover esses diagramas.

Agora vamos desenhar a flecha horizontal. Todas as linhas e curvas no caminho são desenhadas com a mesma cor (ou degradê — sim, nós vamos chegar nisso em breve). Nós queremos desenhar a flecha com uma tinta de cor diferente — preta ao invés de esbranquiçada — então vamos precisar de um novo caminho.

Um novo caminho ~

```
context.beginPath();
context.moveTo(0, 40);
```

```
context.lineTo(240, 40);
context.moveTo(260, 40);
context.lineTo(500, 40);
context.moveTo(495, 35);
context.lineTo(500, 40);
context.lineTo(495, 45);
```

A flecha vertical é praticamente igual. Já que a flecha vertical utiliza a mesma cor que a flecha horizontal, nós **não** vamos precisar criar um novo caminho. As duas flechas farão parte de um mesmo caminho.

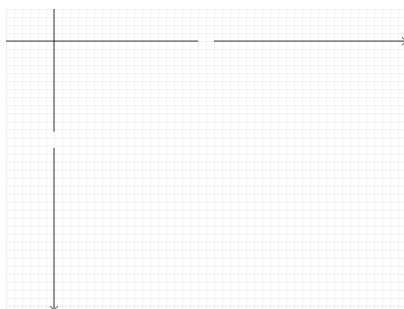
```
context.moveTo(60, 0);
context.lineTo(60, 153);
context.moveTo(60, 173);
context.lineTo(60, 375);
context.moveTo(65, 370);
context.lineTo(60, 375);
context.lineTo(55, 370);
```

~ Não é um novo caminho

Eu disse que essas flechas serão pretas, mas o `strokeStyle` continua esbranquiçado. (O `fillStyle` e o `strokeStyle` não são resetados quando você começa um novo caminho.) Tudo bem, porque nós vamos apenas rodar uma série de métodos “lápiz”. Mas antes de desenhar de verdade, na “tinta”, nós vamos precisar definir o `strokeStyle` para preto. Caso contrário, essas duas flechas ficarão esbranquiçadas, e nós dificilmente vamos ser capazes de vê-las! As seguintes linhas mudam a cor para preto e desenharam as linhas no canvas:

```
context.strokeStyle = "#000";
context.stroke();
```

E o resultado:



TEXTO

Em adição ao desenho de linhas com canvas, você também pode desenhar texto com um canvas. Diferentemente do texto em torno de uma página web, não há box model. Isso significa que nenhuma das técnicas familiares de layout em CSS são válidas: sem floats, sem margins, sem padding, sem word wrapping. (Talvez você pense que isso é uma boa coisa!) Você pode definir alguns poucos atributos de fonte, depois pode pegar um ponto no canvas e começar a desenhar seu texto ali.

Os atributos de fonte a seguir estão disponíveis no contexto do desenho:

- `font` pode ser qualquer coisa que você colocaria na regra `font` do CSS. Incluindo `font style`, `font variant`, `font weight`, `font size`, `line height`, e `font family`.
- `textAlign` controla o alinhamento do texto. É parecido (mas não idêntico) a regra `text-align` do CSS. Os possíveis valores são `start`, `end`, `left`, `right`, e `center`.
- `textBaseline` controla onde o texto é desenhado relativo ao ponto de início. Os possíveis valores são `top`, `hanging`, `middle`, `alphabetic`, `ideographic`, ou `bottom`.

`textBaseline` é complicado, porque texto é complicado (Inglês não é, mas você pode desenhar qualquer caracter Unicode que queira no canvas, e Unicode é complicado). A especificação da HTML5 explica os diferentes textBaselines:

A parte superior do quadrado 'em' fica aproximadamente na parte superior dos glifos em uma fonte, a linha de base é pendurada no lugar onde alguns glifos como आ são âncoradas, o meio é metade do caminho entre a parte superior do quadrado 'em' e a base do quadrado 'em', a linha de base alfabética é onde os caracteres como Á, ÿ, f, e Ω são âncorados, a linha de base ideográfica é onde glifos como 私 e 達 são âncorados, e a base do quadrado 'em' fica aproximadamente na base do glifo em uma fonte. O topo e a base da caixa delimitadora podem ser longe das linhas de base, devido a glifos que se estendem muito além do quadrado.



Para alfabetos simples como o Inglês, você pode usar com segurança o `top`, `middle`, ou `bottom` para a propriedade `textBaseline`.

Vamos desenhar algum texto! O texto desenhado dentro do canvas herda o tamanho da fonte e o estilo do próprio elemento <canvas>, você pode sobrescrever isso definindo a propriedade `font` para o contexto de desenho.

```
context.font = "bold 12px sans-serif";
context.fillText("x", 248, 43);
context.fillText("y", 58, 165);
```

~ *Altera o estilo da fonte*

O método `fillText()` desenha o texto de fato.

```
context.font = "bold 12px sans-serif";
context.fillText("x", 248, 43);
context.fillText("y", 58, 165);
```

☞ *Desenha o texto*

☞P: Posso usar tamanhos de fontes relativos para desenhar texto em um canvas?
R: Sim. Como qualquer outro elemento HTML na sua página, o próprio elemento <canvas> computa o tamanho da fonte baseado nas regras CSS da sua página. Se você definir a propriedade `context.font` para um tamanho relativo de fonte como `1.5em` ou `150%`, seu navegador irá manipular isso para o tamanho computado no próprio elemento <canvas>.

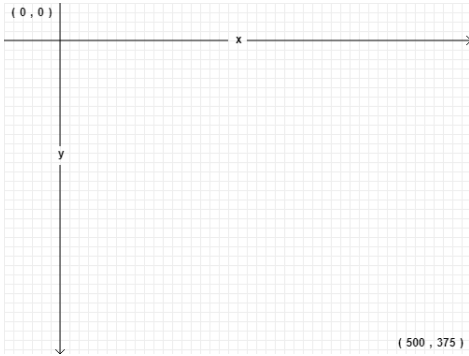
Para o texto no canto superior esquerdo, vamos dizer que eu queira colocar o topo do texto no `y=5`. Mas eu sou um cara preguiçoso — Eu não quero medir a altura do texto e calcular a linha de base. Ao invés disso, eu posso definir o `textBaseline` para `top` e passar para o canto superior esquerdo a coordenada da caixa delimitadora do texto.

```
context.textBaseline = "top";
context.fillText("( 0 , 0 )", 8, 5);
```

Agora para o texto no canto inferior direito. Vamos dizer que eu queira que fique na coordenada (492,370) — somente há alguns pixels de distância do canto inferior direito do canvas — mas eu não quero medir a altura e largura do texto. Posso definir a propriedade `textAlign` para `right` e a `textBaseline` para `bottom`, então chamar `fillText()` com as coordenadas do canto inferior direito da caixa delimitadora do texto.

```
context.textAlign = "right";
context.textBaseline = "bottom";
context.fillText("( 500 , 375 )", 492, 370);
```

E o resultado:

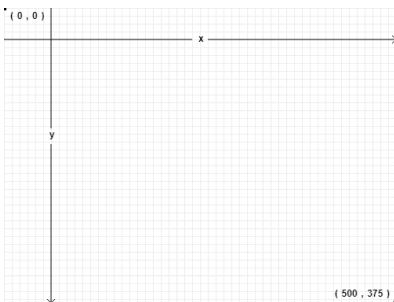


Oops! Nós nos esquecemos dos pontos nos cantos. Nós iremos ver como desenhar círculos um pouco mais tarde. Por enquanto, vou trapacear um pouquinho e desenhar eles como retângulos.

```
context.fillRect(0, 0, 3, 3);  
context.fillRect(497, 372, 3, 3);
```

← *Desenha dois “pontos”*

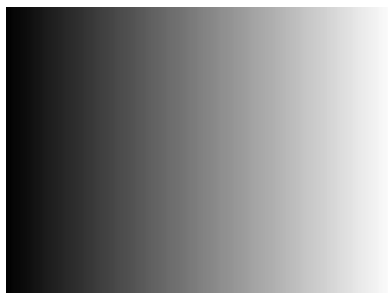
E isso é tudo! Aqui está o produto final:



DEGRADÊS

	IE	FIREFOX	SAFARI	CHROME	OPERA	IPHONE	ANDROID
degradês lineares	7.0+*	3.0+	3.0+	3.0+	10.0+	1.0+	1.0+
degradês radiais	9.0+	3.0+	3.0+	3.0+	10.0+	1.0+	1.0+

Mais cedo nesse capítulo, você aprendeu como desenhar um retângulo preenchido com uma cor sólida, depois uma linha com a borda de uma cor sólida. Mas formas e linhas não são limitadas a cores sólidas. Você pode usar qualquer mágica com degradês. Vamos ver um exemplo.



A marcação parece a mesma que qualquer outro canvas.

```
<canvas id="d" width="300" height="225"></canvas>
```

Primeiro, nós precisamos encontrar o elemento `<canvas>` e seu contexto de desenho.

```
var d_canvas = document.getElementById("d");  
var context = d_canvas.getContext("2d");
```

Uma vez tendo o contexto de desenho, nós começamos a definir o degradê. O degradê é uma transição suave entre duas ou mais cores. O contexto de desenho do canvas suporta dois tipos de degradês:

1. `createLinearGradient(x0, y0, x1, y1)` pinta através de uma linha de $(x0, y0)$ até $(x1, y1)$.
2. `createRadialGradient(x0, y0, r0, x1, y1, r1)` pinta através de um cone entre dois círculos. Os primeiros três parâmetros representam o início do círculo, com origem $(x0, y0)$ e raio $r0$. Os últimos três parâmetros representam o fim do círculo, com origem $(x1, y1)$ e raio $r1$.

Vamos criar um degradê linear. Degradês podem ter qualquer tamanho, mas eu vou criar esse degradê com 300 pixels de largura, como o canvas.

Cria um objeto degradê ↷

```
var my_gradient = context.createLinearGradient(0, 0, 300, 0);
```

Por conta dos valores y (o 2º e 4º parâmetros) são ambos 0, esse degradê irá sombreado uniformemente da esquerda para direita.

Uma vez tendo o objeto degradê, nós podemos definir as cores do degradê. O degradê possui duas ou mais paradas de cor. Paradas de cor podem estar em qualquer lugar através do degradê. Para adicionar

uma parada de cor, você precisa especificar sua posição através do degradê. As posições no degradê podem estar em qualquer lugar entre 0 e 1.

Vamos definir um degradê que irá sombrear do preto ao branco.

```
my_gradient.addColorStop(0, "black");  
my_gradient.addColorStop(1, "white");
```

Definir um degradê não desenha nada no canvas. É só um objeto armazenado em algum lugar na memória. Para desenhar o degradê, você define seu `fillStyle` para o degradê e desenha a forma, como um retângulo ou linha.

Fill style é um degradê ↷

```
context.fillStyle = my_gradient;  
context.fillRect(0, 0, 300, 225);
```

E esse é o resultado:

Supondo que você queira um degradê que irá sombrear de cima para baixo. Quando você criar um objeto degradê, deixe os valores x (1º e 3º parâmetros) constante, e faça que os valores y (2º e 4º parâmetros) alcancem de 0 até a altura do canvas.

Valores x são 0, Valores y Variam ↷

```
var my_gradient = context.createLinearGradient(0, 0, 0, 225);  
my_gradient.addColorStop(0, "black");  
my_gradient.addColorStop(1, "white");  
context.fillStyle = my_gradient;  
context.fillRect(0, 0, 300, 225);
```

E esse é o resultado:

Você também pode criar degradê na diagonal.

ambos valores x e y Variam ↷

```
var my_gradient = context.createLinearGradient(0, 0, 300, 225);  
my_gradient.addColorStop(0, "black");  
my_gradient.addColorStop(1, "white");  
context.fillStyle = my_gradient;  
context.fillRect(0, 0, 300, 225);
```

E esse é o resultado:



IMAGENS

IE	FIREFOX	SAFARI	CHROME	OPERA	IPHONE	ANDROID
7.0+*	3.0+	3.0+	3.0+	10.0+	1.0+	1.0+

* Internet Explorer 7 e 8 necessitam da biblioteca [explorercanvas](#). Internet Explorer 9 suporta <CANVAS> imagens nativamente.

Aqui está um gato:



← *Um elemento *

Aqui está o mesmo gato, só que desenhado em um canvas:

Um elemento <Canvas> →

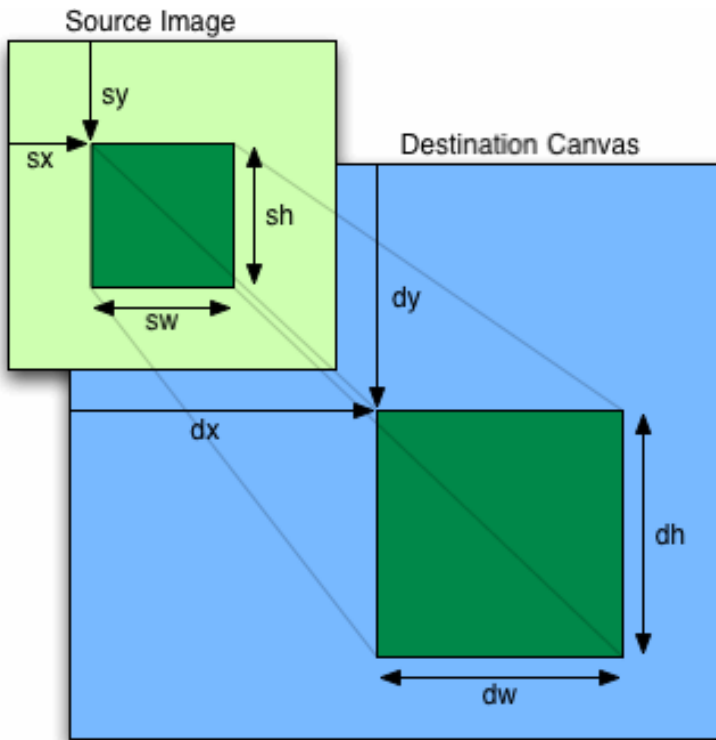
O contexto de desenho do canvas define o método `drawImage()` para desenhar uma imagem no canvas. O método pode ter três, cinco ou nove argumentos.

- `drawImage(image, dx, dy)` pega uma imagem e a desenha no canvas. A coordenada (dx, dy) será o canto superior esquerdo da imagem. Coordenadas (0, 0) devem desenhar a imagem no canto superior esquerdo do canvas.
- `drawImage(image, dx, dy, dw, dh)` pega uma imagem, escala para a largura de dw e altura de dh, e a desenha no canvas nas coordenadas (dx, dy).
- `drawImage(image, sx, sy, sw, sh, dx, dy, dw, dh)` pega uma imagem, ajusta ela para o retângulo (sx, sy, sw, sh), escala para as dimensões (dw, dh), e a desenha no canvas nas coordenadas (dx, dy).

A especificação da HTML5 explica os parâmetros de `drawImage()`:

O retângulo de origem é o retângulo [no interior da imagem fonte] cujos cantos são os quatro pontos (sx, sy) , $(sx+sw, sy)$, $(sx+sw, sy+sh)$, $(sx, sy+sh)$.

O retângulo de destino é o retângulo [no interior do canvas] cujos cantos são os quatro pontos (dx, dy) , $(dx+dw, dy)$, $(dx+dw, dy+dh)$, $(dx, dy+dh)$.



Para desenhar uma imagem no canvas, você precisa de uma imagem. A imagem pode ser um elemento `<img>` existente, ou pode ser criada com o objeto `Image()` do JavaScript. De qualquer maneira, você precisa garantir que a imagem está completamente carregada antes de desenhá-la no canvas.

Se você está usando um elemento `<img>` existente, você pode seguramente desenhá-la no canvas durante o evento `window.onload`.

↪ usando um elemento `<img>`

```

<canvas id="e" width="177" height="113"></canvas>
<script>
window.onload = function() {
    var canvas = document.getElementById("e");
    var context = canvas.getContext("2d");
    var cat = document.getElementById("cat");
```

```

    context.drawImage(cat, 0, 0);
  };
</script>

```

Se você está criando o objeto de imagem inteiramente no JavaScript, você pode seguramente desenhar a imagem no canvas durante o evento `Image.onload`.

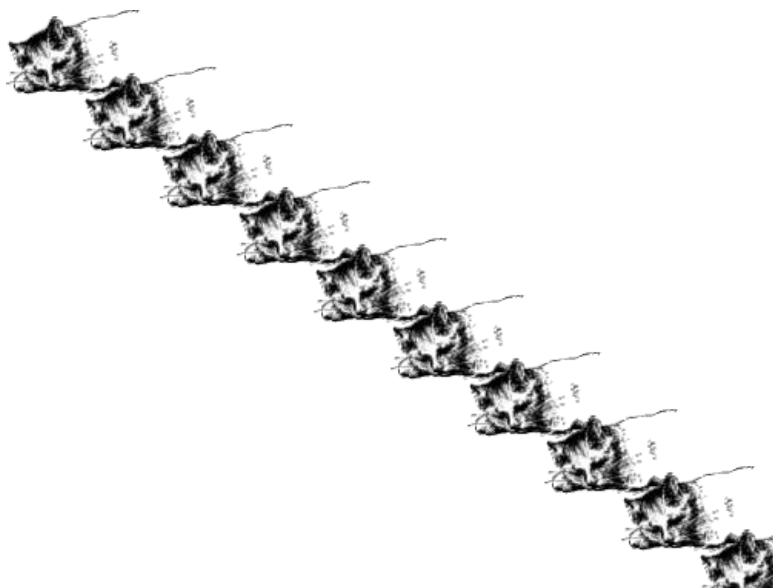
usando um objeto Image() ↷

```

<canvas id="e" width="177" height="113"></canvas>
<script>
  var canvas = document.getElementById("e");
  var context = canvas.getContext("2d");
  var cat = new Image();
  cat.src = "images/cat.png";
  cat.onload = function() {
    context.drawImage(cat, 0, 0);
  };
</script>

```

O 3º e 4º parâmetros opcionais do método `drawImage()` controlam a escala da imagem. Essa é a mesma imagem, escalada pela metade de sua largura e altura e desenhada repetidamente em diferentes coordenadas no mesmo canvas.



Aqui está o script que produz o efeito “multicat”:

```

cat.onload = function() {
  for (var x = 0, y = 0;
    x < 500 && y < 375;
    x += 50, y += 37) {
    context.drawImage(cat, x, y, 88, 56);
  }
};

```

← Escala a imagem

Todo esse esforço levanta a seguinte questão: por que você deveria desenhar uma imagem dentro de um canvas em primeiro lugar? O que faz com que a complexidade extra de uma imagem em um canvas ofereça em cima de um elemento `<img>` e algumas regras CSS? Até mesmo o efeito “multicat” pode ser replicado com 10 elementos `<img>` sobrepostos.

A resposta simples é, pelo mesmo motivo que você iria querer desenhar um texto no canvas. O diagrama de coordenadas do canvas inclui texto, linhas, e formas; o texto no canvas é só uma parte de um grande trabalho. Um diagrama mais complexo poderia facilmente utilizar o `drawImage()` para incluir ícones, sprites, e outros gráficos.

E QUANTO AO IE?

Versões do Internet Explorer antes de 9.0 não suportam a API de canvas. (IE9 suporta completamente a API de canvas.) Entretanto, essas versões antigas do Internet Explorer suportam *sim* uma tecnologia proprietária da Microsoft chamada VML, na qual pode fazer muitas das coisas que o elemento `<canvas>` faz. E assim, nasceu o `excanvas.js`.

Explorercanvas (`excanvas.js`) é uma biblioteca JavaScript de código livre, licenciada pela Apache que implementa a API de canvas no Internet Explorer. Para utilizá-la, inclua o seguinte elemento `<script>` no topo da sua página.

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <title>Dive Into HTML5</title>
  <!--[if lt IE 9]>
    <script src="excanvas.js"></script>
  <![endif]-->
</head>
<body>
  ...
</body>
</html>
```

O `<!--[if lt IE 9]>` e `<![endif]-->` são comentários condicionais. Internet Explorer interpreta eles como uma declaração `if`: “se o navegador é uma versão do Internet Explorer inferior da (mas não inclusa) versão 9, então execute o bloco.” Cada outro navegador vai tratar

o bloco inteiro como um bloco de comentário HTML. O resultado na prática é que o Internet Explorer 7 e 8 irão realizar o download do script `excanvas.js` e executá-los, mas outros navegadores irão ignorar o script (não realizando o download dele, não executando ele, não fazendo nada). Isso irá fazer com que sua página carregue mais rapidamente para aqueles navegadores que implementem a API de canvas nativamente.

Uma vez incluindo o `excanvas.js` no `<head>` da sua página, você não precisa fazer nada mais para acomodar o Internet Explorer. Só inclua os elementos `<canvas>` na sua marcação, ou crie eles dinamicamente com JavaScript. Siga as instruções desse capítulo para capturar o contexto de desenho do elemento `<canvas>`, e você poderá desenhar formas, texto, e padrões.

Bem... não exatamente. Existem algumas limitações:

1. Degradês só podem ser lineares. Degradês radiais não são suportados.
2. Padrões devem se repetir em ambas as direções.
3. Regiões clipping não são suportadas.
4. Escala não uniforme não funciona corretamente com escalas de bordas.
5. É lento. Isso não deve ser um choque violento para ninguém, já que o parser de JavaScript do Internet Explorer é mais lento do que dos outros navegadores para começar. Uma vez começando a desenhar formas complexas usando a biblioteca JavaScript library que traduz comandos para outra tecnologia completamente diferente, coisas vão naturalmente atolar. Você não irá notar degradação de performance em exemplos simples como desenhando algumas linhas e transformando uma imagem, mas você rapidamente irá notar uma vez que comece realizando animações baseadas em canvas ou outras coisas loucas.

Há mais uma advertência sobre o uso de `excanvas.js`, e é um problema que passei enquanto criava o exemplo desse capítulo. ExplorerCanvas inicializa sua própria interface de falso-canvas automaticamente uma vez incluindo o script `excanvas.js` na sua página HTML. Mas isso não significa que o Internet Explorer está pronto para usá-lo imediatamente. Em algumas situações, você pode rodar sob uma condição onde a interface do falso-canvas está *quase*, mas ainda não, pronta para ser usada. O principal sintoma desse estado é que o Internet Explorer irá reclamar que o “objeto não suporta essa propriedade ou método” sempre que você tentar fazer qualquer coisa com o elemento `<canvas>`, como pegar seu contexto de desenho.

A solução mais simples para fazer é adiar todas as suas manipulações relacionadas ao canvas até depois que evento `onload` seja disparado. Isto pode levar um tempo — se a sua página possui muitas imagens ou vídeos, elas irão atrasar o `onload` — mas irá dar ao ExplorerCanvas tempo para trabalhar na sua magia.

UM EXEMPLO COMPLETO AO VIVO

Halma é um jogo de tabuleiro com séculos de idade. Muitas variações existem. Nesse exemplo, criei uma versão solitária de Halma com 9 peças em um tabuleiro 9×9 . No início do jogo, as peças formam um quadrado 3×3 no canto inferior esquerdo do tabuleiro. O objetivo do jogo é mover todas as peças até que formem um quadrado 3×3 no canto superior direito do tabuleiro, com o menor número de movimentos.

Existem dois tipos de movimentos legais no Halma:

- Pegue uma peça e mova para qualquer quadrado vazio adjacente. Um quadrado “vazio” é um que não contém nenhuma peça nele. Um quadrado “adjacente” é um imediatamente ao norte, sul, leste, oeste, noroeste, nordeste, sudoeste ou sudeste da peça na sua posição atual. (O tabuleiro não é girado em torno de um lado para outro. Se uma peça está na coluna mais a esquerda, não pode se mover para oeste, noroeste ou sudoeste. Se uma peça está na linha mais embaixo, não pode se mover para o sul, sudeste ou sudoeste.)
- Pegue uma peça e pule por cima de uma peça adjacente, e possivelmente repita. Isso é, se você pular por cima de uma peça adjacente, então pule por cima de *outra* peça adjacente para sua nova posição, isso conta como um único movimento. Na verdade, qualquer número de puladas por cima continuam contando como um movimento único. (Já que o objetivo é minimizar o número total de movimentos, ir bem no Halma envolve construir, e depois usar, longas cadeias escalonadas para que então outras peças possam pular por cima em uma longa sequência.)

Aqui está o jogo propriamente dito. Você também pode [jogá-lo em uma página separada](#) se você quiser cutucar seu developer tools.

Moves: o

Como isso funciona? Estou feliz que você perguntou. Não irei mostrar *todo* código aqui. (Você pode vê-lo em diveintohtml5.com.br/examples/halma.js.) Na verdade irei pular a maioria do código de jogabilidade, mas quero ressaltar algumas partes do código que realmente mexem o desenho no canvas e a resposta aos cliques de mouse no elemento canvas.

Durante o carregamento da página, nós inicializamos o jogo ao definir as dimensões no `<canvas>` e guardamos as referências no seu contexto de desenho.

```
gCanvasElement.width = kPixelWidth;  
gCanvasElement.height = kPixelHeight;  
gDrawingContext = gCanvasElement.getContext("2d");
```


Depois fazemos algo que você ainda não viu: nós adicionamos uma escuta de evento ao elemento <canvas> para ouvir por eventos de clique.

```
gCanvasElement.addEventListener("click", halmaOnClick, false);
```

A função `halmaOnClick()` é chamada quando o usuário clica em qualquer lugar no canvas. Seu argumento é um objeto `MouseEvent` que contém as informações sobre onde o usuário clicou.

```
function halmaOnClick(e) {
    var cell = getCursorPosition(e);

    // o resto é só lógica do jogo
    for (var i = 0; i < gNumPieces; i++) {
        if ((gPieces[i].row == cell.row) &&
            (gPieces[i].column == cell.column)) {
            clickOnPiece(i);
            return;
        }
    }
    clickOnEmptyCell(cell);
}
```

O próximo passo é pegar o objeto `MouseEvent` e calcular em qual quadrado no tabuleiro Halma acaba de ser clicado. O tabuleiro Halma contempla todo o canvas, ou seja cada clique em *qualquer lugar* do tabuleiro. Nós só precisamos descobrir onde. Isto é complicado, porque eventos de mouse são implementados diferentemente em quase todo navegador.

```
function getCursorPosition(e) {
    var x;
    var y;
    if (e.pageX != undefined && e.pageY != undefined) {
        x = e.pageX;
        y = e.pageY;
    }
    else {
        x = e.clientX + document.body.scrollLeft +
            document.documentElement.scrollLeft;
        y = e.clientY + document.body.scrollTop +
            document.documentElement.scrollTop;
    }
}
```

Nesse ponto, nós temos as coordenadas `x` e `y` que são relativas ao documento (isso é, a página HTML inteira). Isso não é muito útil ainda. Nós queremos as coordenadas **relativas ao canvas**.

```
x -= gCanvasElement.offsetLeft;
y -= gCanvasElement.offsetTop;
```

Agora nós temos as coordenadas x e y que são relativas ao canvas. Isso é, se x é 0 e y é 0 nesse ponto, nós sabemos que o usuário acabou de clicar no pixel do canto superior esquerdo do canvas.

Daqui em diante, nós podemos calcular em qual quadrado do Halma o usuário clicou, e então agir de acordo com isso.

```
var cell = new Cell(Math.floor(y/kPieceHeight),
                    Math.floor(x/kPieceWidth));
return cell;
}
```

Ufa! Eventos de mouse são difíceis. Mas você pode usar a mesma lógica (na verdade, o exato mesmo código) em todas as suas aplicações baseadas em canvas. Lembre-se: mouse click → coordenadas relativas ao documento → coordenadas relativas ao canvas → código específico de aplicação.

OK, vamos olhar para a rotina principal do desenho. Porque os gráficos são simples, escolhi limpar e redesenhar o tabuleiro em cada hora que houver qualquer mudança no jogo. Isso não é estritamente necessário. O contexto de desenho do canvas irá reter qualquer coisa que você tenha desenhado anteriormente nele, até mesmo se o usuário realizar scroll para fora do campo de visão ou mudar de uma aba para outra e depois voltar. Se você está desenvolvendo uma aplicação baseada em canvas com gráficos mais complicados (como um jogo de arcade), você pode otimizar a performance ao identificar quais regiões do canvas estão “sujas” e redesenhar apenas as regiões sujas. Mas isso está fora do escopo desse livro.

```
gDrawingContext.clearRect(0, 0, kPixelWidth, kPixelHeight);
```

A rotina de desenho do tabuleiro deve lhe parecer familiar. É similar como nós desenhamos o digrama de coordenadas do canvas anteriormmente nesse capítulo.

```
gDrawingContext.beginPath();
```

```
/* linhas verticais */
for (var x = 0; x <= kPixelWidth; x += kPieceWidth) {
    gDrawingContext.moveTo(0.5 + x, 0);
    gDrawingContext.lineTo(0.5 + x, kPixelHeight);
}

/* linhas horizontais */
for (var y = 0; y <= kPixelHeight; y += kPieceHeight) {
    gDrawingContext.moveTo(0, 0.5 + y);
    gDrawingContext.lineTo(kPixelWidth, 0.5 + y);
}

/* desenhe! */
gDrawingContext.strokeStyle = "#ccc";
```

```
gDrawingContext.stroke();
```

A brincadeira começa de verdade quando nós vamos desenhar cada peça individual. Uma peça é um círculo, algo que ainda não desenhemos antes. Além disso, se o usuário seleciona a peça em antecipação ao seu movimento, nós queremos desenhar a peça em forma de um círculo preenchido. Aqui, o argumento `p` representa a peça, na qual possui as propriedades de linha e coluna que denotam a posição atual da peça no tabuleiro. Nós usamos algumas constantes do jogo para traduzir (`coluna`, `linha`) em coordenadas (`x`, `y`) relativas ao canvas, então desenhar o círculo, e (se a peça for selecionada) preencher o círculo com uma cor sólida.

```
function drawPiece(p, selected) {  
    var column = p.column;  
    var row = p.row;  
    var x = (column * kPieceWidth) + (kPieceWidth/2);  
    var y = (row * kPieceHeight) + (kPieceHeight/2);  
    var radius = (kPieceWidth/2) - (kPieceWidth/10);
```

Esse é o fim da lógica específica do jogo. Agora nós temos as coordenadas (`x`, `y`), relativas ao canvas, para o centro do círculo que queremos desenhar. Não há um método `circle()` na API do canvas, mas há um método `arc()`. E realmente, o que é um círculo se não um arco que faz uma volta completa? Você lembra de geometria básica? O método `arc()` pega o ponto central (`x`, `y`), o raio, o ângulo inicial e final (em radianos), e a direção (`false` para sentido horário, `true` para sentido anti-horário). Você pode usar o módulo `Math` que está dentro do JavaScript para calcular radianos.

```
gDrawingContext.beginPath();  
gDrawingContext.arc(x, y, radius, 0, Math.PI * 2, false);  
gDrawingContext.closePath();
```

Mas espera aí! Nada foi desenhado ainda. Como `moveTo()` e `lineTo`, o método `arc()` é um método “lápiz”. Para realmente desenhar o círculo, nós precisamos definir o `strokeStyle` e chamar o `stroke()` para traçar a “tinta.”

```
gDrawingContext.strokeStyle = "#000";  
gDrawingContext.stroke();
```

E se a peça estiver selecionada? Nós podemos reutilizar o mesmo caminho que criamos para desenhar o contorno da peça, para preencher o círculo com uma cor sólida.

```
if (selected) {  
    gDrawingContext.fillStyle = "#000";  
    gDrawingContext.fill();  
}
```

E isso é... bem, isso é tudo. O resto do programa é lógica específica do jogo — distinguindo entre movimentos válidos e inválidos, registrando o número de movimentos, detectando se o jogo está terminado. Com 9 círculos, algumas linhas, e 1 onclick handler, nós criamos um jogo inteiramente em <canvas>. Huzzah!

LEITURA COMPLEMENTAR

- [Canvas tutorial](#) no Mozilla Developer Center
- [HTML5 canvas — o básico](#), por Mihai Sucan
- [CanvasDemos.com](#): demos, ferramentas, e tutoriais para o elemento HTML canvas
- [O elemento canvas](#) no rascunho da especificação do HTML5
- [Internet Explorer 9 Guia para desenvolvedores: elemento HTML5 canvas](#)

Nº5

VÍDEO NA WEB

MERGULHANDO



Qualquer um que tenha visitado o Youtube.com nos últimos quatro anos sabe que você pode incorporar um vídeo em uma página web. Mas antes do HTML5, não havia nenhuma maneira baseada nos padrões para fazer isso. Virtualmente todos os vídeos já assistidos “na web” eram afunilados por um plugin de terceiros — talvez QuickTime, talvez RealPlayer, talvez Flash. (YouTube usa Flash.) Esses plugins se integram com seu browser o suficiente que você nem é avisado quando está utilizando eles. Isso é, até o momento em que você tentar assistir um vídeo em uma plataforma que não suporta esse plugin.

HTML5 define uma maneira padrão de incorporar vídeo em uma página web, usando o elemento `<video>`. O suporte para o elemento `<video>` ainda está evoluindo, o que é um modo educado de dizer que ainda não está funcionando. Pelo menos, ele não funciona em todos os lugares. Mas não se desespere! Existem alternativas, fallbacks e uma abundância de opções.

SUPORTE AO ELEMENTO <code><VIDEO></code>						
IE	FIREFOX	SAFARI	CHROME	OPERA	IPHONE	ANDROID
9.0+	3.5+	3.0+	3.0+	10.5+	1.0+	2.0+

Mas o suporte para o elemento `<video>` em si é realmente uma pequena parte da história. Antes de falarmos sobre o vídeo da HTML5, você precisa entender primeiro um pouco sobre vídeo em si. (Se você já sabe sobre vídeo, você pode pular para [O Que Funciona na Web](#).)

EMBALAGENS DE VÍDEO

Você pode pensar em arquivos de vídeo como “arquivos AVI” ou “arquivos MP4.” Na realidade, “AVI” e “MP4” são apenas os formatos das embalagens de vídeo. Assim como um arquivo ZIP pode conter qualquer tipo de arquivo dentro dele, os formatos das embalagens de vídeo definem apenas *como* armazenar as coisas dentro dele, e não *quais* tipos de dados são armazenados. (É um pouco mais complicado que isso, porque nem todos os fluxos de vídeo são compatíveis com todos os formatos de embalagens, mas esqueça isso por enquanto.)

Um arquivo de vídeo usualmente contém múltiplas *faixas* — uma faixa de vídeo (sem áudio), além de uma ou mais faixas de áudio (sem vídeo). As faixas usualmente se interlaçam. Uma faixa de áudio contém marcações dentro dela para ajudar na sincronização entre áudio e vídeo. Faixas individuais podem ter metadados, como relações de aspectos de uma faixa de vídeo, ou a linguagem de uma faixa de áudio. Essas embalagens também podem ter metadados, como o título do próprio vídeo, uma capa para o vídeo, números de episódios (para programas de televisão), e por aí vai.

Existem *vários* formatos para embalagens de vídeo. Os mais populares incluem

- MPEG 4, usualmente com a extensão `.mp4` ou `.m4v`. A embalagem MPEG 4 é baseada na antiga embalagem QuickTime da Apple (`.mov`). Trailers de filmes no site da Apple ainda utilizam a antiga embalagem QuickTime, mas filmes que você aluga no iTunes são entregues na embalagem MPEG 4.
- Flash, usualmente com a extensão `.flv`. Vídeos em Flash são, sem surpresa, usados pelo Adobe Flash. Antes do Flash 9.0.60.184 (a.k.a. Flash Player 9 Update 3), esse era o único formato de embalagem que o Flash suportava. Versões mais recentes do Flash também suportam a embalagem MPEG 4.
- Ogg, usualmente com a extensão `.ogv`. Ogg é um padrão aberto, de código livre, e descoberto de quaisquer patentes conhecidas. Firefox 3.5, Chrome 4, e Opera 10.5 suportam — nativamente, sem qualquer plataforma específica de plugins — o formato de embalagem Ogg, Ogg vídeo (chamado “Theora”), e Ogg áudio (chamado “Vorbis”). No desktop, Ogg é suportado fora-da-caixa por todas as principais distribuições Linux, e você pode usá-lo no Mac e Windows ao instalar os componentes QuickTime ou filtros DirectShow, respectivamente. É ainda executado pelo excelente VLC em todas as plataformas.
- WebM é um novo formato de embalagem. É tecnicamente similar a outro formato, chamado Matroska. WebM foi anunciado em Maio de 2010. Foi projetado para ser usado exclusivamente com o codec de vídeo VP8 e pelo codec de áudio Vorbis. (Mais obre isso em um minuto.) É formato nativamente, sem qualquer plataforma específica de plugins, nas últimas versões do Chromium, Google Chrome, Mozilla Firefox, e Opera. Adobe também anunciou que uma futura versão do Flash irá suportar vídeos em WebM.
- Audio Video Interleave, usualmente com a extensão `.avi`. O formato de embalagem AVI foi inventado pela Microsoft em um tempo mais simples, quando o fato de que computadores podiam tocar vídeos já era considerado incrível. Oficialmente não suporta funcionalidades dos mais recentes formatos de embalagens como metadados incorporados. Oficialmente não suporta até mesmo os mais modernos codecs de áudio e vídeo utilizados hoje em dia. Com o tempo, companhias tentaram extendê-lo de forma incompatível para suportar isso ou aquilo, e ainda é o formato de embalagem padrão para encoders como MEncoder.

CODECS DE VÍDEO

Quando você fala sobre “assistir um vídeo,” você está provavelmente falando da combinação de um fluxo de vídeo e um fluxo de áudio. Mas você não tem dois arquivos diferentes; você tem apenas “um vídeo.” Talvez possa ser um arquivo AVI, ou um arquivo MP4. Esses são apenas formatos de embalagem, como um arquivo ZIP que contém múltiplos tipos de arquivos dentro dele. O formato de embalagem define como serão armazenados os fluxos de vídeo e áudio em um único arquivo.

Quando você “assiste um vídeo,” seu video player está fazendo pelo menos três coisas ao mesmo tempo:

1. Interpretando o formato de embalagem para descobrir quais faixas de vídeo e áudio estão disponíveis, e como elas são armazenadas dentro do arquivo para que possa encontrar os dados que necessitam ser decodificados depois
2. Decodificando o fluxo de vídeo e exibindo uma série de imagens na tela
3. Decodificando o fluxo de áudio e enviando o som para as caixas de som

Um *codec de vídeo* é um algoritmo que será encodificado em um fluxo de vídeo, i.e. ele especifica como fazer a seguir. (A palavra “codec” é um portmanteau, a combinação das palavras “codificar” e “decodificar.”) Seu video player *decodifica* o fluxo de vídeo de acordo com o *codec de vídeo*, depois exibe uma série de imagens, ou “frames,” na tela. A maioria dos codecs de vídeo modernos usam diversas formas para minimizar a quantidade de informação necessária para exibir um frame atrás do outro. Por exemplo, ao invés de armazenar cada frame individualmente (como um screenshot), eles irão armazenar apenas as diferenças entre os frames. A maioria dos vídeos na realidade não mudam completamente entre um frame e o outro, então isso permite alto grau de compressão, resultando em menores tamanhos de arquivo.

Existem codecs de vídeo *com perdas* e *sem perdas*. Vídeos sem perdas são muito grandes para serem usados na web, então irei me concentrar em codecs com perdas. Um *codec com perdas* significa que informação será irremediavelmente perdida durante o processo de encodificação. Como a cópia do áudio de uma fita cassete, você está perdendo informação sobre a fonte do vídeo, e degradando a qualidade, cada vez que você codifica. Ao invés do “assobio” de uma fita cassete de áudio, uma re-re-re-encodificação de vídeo pode parecer bloqueada, especialmente durante cenas com muita ação. (Na verdade, isso pode acontecer até se você codificar direto da fonte original, se você escolher um codec de vídeo pobre ou passar para ele o conjunto errado de parâmetros.) O lado bom é que codecs de vídeo com perdas podem oferecer taxas de compressão incríveis através da suavização sem bloqueios durante a reprodução, para fazer a perda menos perceptível ao olho humano.

Existe uma *porção* de codecs de vídeo. Os três codecs mais relevantes são H.264, Theora, e VP8.

H.264

H.264 mais conhecido como “MPEG-4 part 10,” a.k.a. “MPEG-4 AVC,” a.k.a. “MPEG-4 Advanced Video Coding.” H.264 foi desenvolvido pelo MPEG group e padronizado em 2003. Tem por objetivo oferecer um único codec para banda-larga pobre, CPU de dispositivos pobres (celulares); alta banda-larga, alta CPU de dispositivos (computadores modernos); e qualquer coisa no meio disso. Para realizar isso, o padrão H.264 é dividido em “perfis,” onde cada um define um conjunto de funcionalidades opcionais que negocia a complexidade pelo tamanho do arquivo. Altos perfis usam mais funcionalidades opcionais, oferecem melhor qualidade visual para tamanhos de arquivo menores, levam mais tempo para codificar, e requerem mais poder da CPU para codificar em tempo real.

Para lhe dar uma ideia da variedade dos perfis, o iPhone da Apple suporta o perfil Baseline, a AppleTV suporta os perfis Baseline e Main, e Adobe Flash em um PC suporta os perfis Baseline, Main, e High. YouTube usa agora o H.264 para codificar vídeos de alta definição, tocados a partir do Adobe Flash; YouTube também provê vídeo codificado com H.264 para dispositivos móveis, incluindo iPhone e telefones rodando o sistema operacional móvel Android. Além do H.264 ser um dos codecs de vídeo mandatários pela especificação do Blu-Ray; discos Blu-Ray que usam ele geralmente utilizam no perfil High.

A maioria dos dispositivos, que não são PCs, tocam vídeo em H.264 (incluindo iPhones e reprodutores de Blu-Ray) Most non-PC devices that play H.264 video (including iPhones and standalone Blu-Ray players) na verdade realizam a decodificação em um chip dedicado, uma vez que suas CPUs principais estão longe de ter poder o suficiente para decodificar em tempo real. Atualmente, até mesmo placas gráficas de baixo nível suportam decodificação H.264 no hardware. Existem codificadores H.264 concorrentes, incluindo o open source x264. **O padrão H.264 está coberto por patentes**; licenciamento é intermediado pelo MPEG LA group. Vídeo H.264 pode ser incorporado nos mais populares formatos de contêiner, incluindo MP4 (usado primeiramente pela iTunes Store da Apple) e MKV (usado primeiramente por entusiastas de vídeo não-comerciais).

THEORA

Theora evoluiu do VP3 codec e tem sido subseqüentemente desenvolvido pela Xiph.org Foundation. **Theora é um codec livre de royalties e não é onerado por qualquer patente conhecida** a não ser a patente original VP3, que foi licenciada livre de royalties. Embora o padrão tenha sido “congelado” desde 2004, o projeto Theora (que inclui um referente codificador e decodificador open source) apenas lançou a versão 1.0 em novembro de 2008 e a versão 1.1 em setembro de 2009.

Vídeos em Theora podem ser incorporados em qualquer formato contêiner, embora seja mais visto em Ogg. A maioria das distribuições Linux suportam Theora fora-da-caixa, e o Mozilla Firefox 3.5 inclui suporte nativo a vídeos Theora no contêiner Ogg. E por “nativo”, eu digo “disponível em qualquer plataforma sem plugins específicos daquela plataforma.” Você pode

também reproduzir vídeos Theora no Windows ou no Mac OS X após instalar o software open source decodificador Xiph.org’.

VP8

VP8 é outro codec de vídeo da On2, mesma companhia que originalmente desenvolveu o VP3 (mais tarde Theora). Tecnicamente, ele produz saída em par com o perfil H.264 High, enquanto mantém uma baixa complexidade de decodificação em par com o perfil H.264 Baseline.

Em 2010, a Google adquiriu a On2 e publicou a especificação do codec de vídeo e uma amostra open source do codificador e decodificador. Como parte disso, a Google também “abriu” todas as patentes que a On2 mantinha sobre o VP8, ao licenciar livre de royalties. (Esse é o melhor que você pode esperar para patentes. Na verdade você não pode “lançar” ou anular elas uma vez emitidas. Para torná-las open source, você as licencia livre de royalties, e então qualquer um pode usar a tecnologia que a patente cobre sem pagar qualquer coisa ou negociar as licenças.) A partir de 19 de maio de 2010, **VP8 se tornou livre de royalties, um codec moderno e não onerado por qualquer patente conhecida**, diferente das patentes que a On2 (agora Google) já licenciou livre de royalties.

CODECS DE ÁUDIO

A não ser que você ainda esteja parado nos filmes feitos antes de 1927, você irá querer uma trilha de som no seu vídeo. Como codecs de vídeo, *codecs de áudio* são algoritmos nos quais fluxos de áudio são codificados. Como codecs de vídeo, os codecs de áudio são *com perda* e *sem perda*. E como os codecs de vídeo sem perda, áudios sem perda são realmente muito grandes para colocar na web. Então irei me concentrar em codecs de áudio com perda.

Na verdade, é ainda mais estreito que isso, porque eles estão em diferentes categorias de codecs de áudio com perda. Áudio é usado em lugares onde o vídeo não é (telefonia, por exemplo) e existe toda uma categoria de codecs de áudio otimizados para codificar voz. Você não iria utilizar um desses codecs para um CD de música, porque o resultado seria como uma criança de 4 anos de idade cantando em um viva-voz. Mas você *iria* usar eles em um Asterisk PBX, porque a banda é preciosa, e esses codecs podem comprimir a voz humana em uma fração do tamanho que codecs gerais. Entretanto, graças a falta de suporte em ambos navegadores nativos ou plugins terceiros, codecs de áudio otimizados para voz nunca chegaram realmente na web. Então irei concentrar em *codecs de áudio com propósito geral*.

Eu mencionei mais cedo, quando você “assiste um vídeo,” seu computador está fazendo ao menos três coisas ao mesmo tempo:

1. Interpretando o contêiner do formato
2. Decodificando o fluxo de vídeo

3. Decodificando o fluxo de áudio e enviando o som para os alto-falantes

O *codec de áudio* especifica como #3 — decodificar o fluxo de áudio e torná-lo em ondas digitais que depois seus alto-falantes transformam em som. Assim como os codecs de vídeo, existem todos os tipos de truques para minimizar a quantidade de informação armazenada em um fluxo de áudio. E já que estamos falando sobre codecs de áudio *com perda*, a informação está sendo perdida durante a gravação → codificação → decodificação → ciclo de vida da escuta. Diferentes codecs de áudio jogam fora diferentes coisas, mas eles tem o mesmo objetivo: enganar seus ouvidos para não notar as partes que estão faltando.

Um conceito que o áudio tem que o vídeo não tem são os *canais*. Nós estamos enviando som para os alto-falantes, certo? Bom, quantos alto-falantes você tem? Se você está sentado em um computador, você deve ter apenas dois: um na esquerda e um na direita. Meu desktop tem três: esquerda, direita e mais um no chão. Chamado “surround sound” sistemas podem ter seis ou mais alto-falantes, estrategicamente colocados pelo lugar. Cada alto-falante alimenta um *canal* em particular da gravação original. A teoria é que você pode sentar no meio dos seis alto-falantes, literalmente cercado por seis diferentes canais do som, e seu cérebro sintetiza eles e parece que você está no meio da ação. Isso funciona? Uma indústria multi-bilionária parece acreditar que sim.

Codecs de áudio com propósito geral podem lidar com dois canais de som. Durante a gravação, o som é dividido no canal da esquerda e da direita; durante a codificação, ambos canais armazenam o mesmo fluxo de áudio; durante a decodificação, ambos canais estão decodificados e cada um é enviado para o alto-falante apropriado. Alguns codecs de áudio podem lidar com mais de dois canais, e irão controlar qual canal é qual e então seu reproduutor de áudio pode enviar para o som da direita para o alto-falante da direita.

Existem *muitos* codecs de áudio. Eu disse que haviam muitos codecs de vídeo? Esqueça isso. Existem dezenas e dezenas de codecs de áudio, mas na web, existem apenas que você deveria conhecer mais sobre: MP3, AAC, e Vorbis.

MPEG-1 AUDIO LAYER 3

MPEG-1 Audio Layer 3 é coloquialmente conhecido como “MP3.” Se você nunca ouviu falar de MP3s, eu não sei o que fazer com você. Walmart vende reprodutores de música portáteis e os chama de “MP3 players.” *Walmart*. Em todo caso...

MP3s podem conter **até dois canais** de som. Eles podem ser codificados em diferentes *taxas de bits*: 64 kbps, 128 kbps, 192 kbps, e uma variedade de outros de 32 à 320. Altas taxas de bits significam tamanhos de arquivos maiores e melhor qualidade no áudio, embora a relação da taxa da qualidade com a taxa de bits não seja linear. (128 kbps soa duas vezes melhor que 64 kbps, mas 256 kbps não soa o dobro melhor que 128 kbps.) Além disso, o formato MP3 permite *codificação variável na taxa de bits*, o que significa que algumas partes do fluxo de

codificação são mais comprimidas que outras. Por exemplo, silêncio entre as notas podem ser codificadas em uma baixa taxa de bits, então a taxa de bits pode aumentar um momento mais tarde quando múltiplos instrumentos começam a tocar um acorde complexo. MP3s também podem ser codificados em uma taxa de bits constante, que, sem surpresas, é chamado de *codificação a taxas de bits constantes*.

O padrão MP3 não define exatamente como codificar MP3s (embora defina exatamente como devemos decodificá-los); diferentes codificadores usam diferentes modelos psicoacústicos que produzem resultados descontroladamente diferentes, mas são todos decodificados pelos mesmos players. O projeto open source LAME é o melhor codificador gratuito, e sem dúvida o melhor codificador, ponto final, para todos menos para baixas taxas de bits.

O formato MP3 (padronizado em 1991) é **coberto de patentes**, que explicam porque Linux não pode tocar arquivos MP3 fora da caixa. Praticamente todos os players de música portáteis suportam arquivos MP3s, e os fluxos de áudio MP3 podem ser embutidos em qualquer container de vídeo. Adobe Flash pode ambos, arquivos MP3 e fluxo de áudio MP3 com um container de vídeo MP4.

CODIFICAÇÃO DE ÁUDIO AVANÇADA

Codificação de Áudio Avançada é efetivamente conhecida como "AAC" (Advanced Audio Coding). Padronizada em 1997, deu uma "levantada" proeminente quando a Apple escolheu o MP3 como formato padrão para a iTunes Store. Originalmente, todos os arquivos "AAC" "comprados" pela loja do iTunes eram criptografados com um esquema DRM proprietário da Apple, chamado FairPlay. Músicas selecionadas na loja iTunes agora estão disponíveis como arquivos desprotegidos AAC, que a Apple chama de "iTunes Plus" pois soa muito melhor do que chamar todo o resto de "iTunes Minus." **O formato AAC é coberto de patentes**; taxas de licença são disponibilizadas online.

AAC foi desenvolvido para promover uma qualidade de som melhor que MP3 na mesma *taxa de bits*, e pode codificar áudio em qualquer taxa de bits. (MP3 é limitado a um número fixo de taxa de bits, com um limite superior de 320 kbps.) AAC pode codificar **até 48 canais de som**, embora na prática ninguém faça isso. O formato AAC também difere do MP3 na definição de múltiplos *perfis*, na maioria das vezes como H.264, e pelas mesmas razões. O perfil de "baixa complexidade" é desenvolvido para ser reproduzido em tempo real em dispositivos com poder de CPU limitado, enquanto perfis altos oferecem melhor qualidade na mesma taxa de bits ao preço de lentidão na codificação ou decodificação.

Todos os atuais produtos da Apple, incluindo iPods, AppleTV, e QuickTime suportam alguns perfis de AAC tanto em arquivos de áudio e em fluxos de áudio dentro um container de vídeo MP4. Adobe Flash suporta todos os perfis de AAC em MP4, como fazem os reprodutores de vídeo MPlayer e VLC. Para codificação, a biblioteca FAAC é a opção open source; o suporte é uma opção em tempo de compilação em mencoder e ffmpeg.

VORBIS

Vorbis também chamado de “Ogg Vorbis,” embora esteja tecnicamente incorreto. (“Ogg” e apenas um formato de container, e fluxos de áudio Vorbis podem ser embutido em outros containers.) **Vorbis não é coberto por nenhuma patente conhecida** e portanto é suportado fora da caixa pela maioria das distribuições Linux e dispositivos portáteis rodando o open source Rockbox firmware. Mozilla Firefox 3.5 suporta arquivos de áudio Vorbis em um container Ogg, ou Ogg vídeos com uma faixa de áudio Vorbis. Android celulares também podem reproduzir arquivos de audio Vorbis. streams de audio Vorbis geralmente são embutidas em um Ogg ou em WebM contêiner, mas eles também podem ser embutidos em um MP4 ou MKV container (ou, com algum hacking, em AVI). Vorbis suporta **um número arbitrário de canais de som**.

Existem codificadores e decodificadores Vorbis open source, incluindo OggConvert (codificador), ffmpeg (decodificador), aoTuV (codificador), e libvorbis (decodificador). Também existem Componentes QuickTime para Mac OS X e Filtros DirectShow para Windows.

O QUE FUNCIONA NA WEB

Se os seus olhos não estão vidrados ainda, você está melhor que a maioria. Como você pode dizer, vídeo (e áudio) é um assunto complicado e esta foi a versão abreviada! Eu tenho certeza que você está se perguntando como tudo isso se relaciona com HTML5. Bem, HTML5 inclui um elemento <video> para embutir vídeos em uma página web. Não há restrição no codec do vídeo, codec do áudio ou no formato de container que você pode usar para seu vídeo. Um elemento <video> pode ter um link para múltiplos arquivos de vídeos, e o navegador escolhe qual o primeiro vídeo que irá reproduzir. **Cabe a você saber qual navegador suporta qual containers e codecs.**

Como está escrito, esta é a paisagem do vídeo HTML5:

- Mozilla Firefox (3.5 e superior) suporta vídeo Theora e áudio Vorbis em container Ogg. Firefox 4 também suporta WebM.
- Opera (10.5 e superior) suporta vídeo Theora e áudio Vorbis em um container Ogg. Opera 10.60 também suporta WebM.
- Google Chrome (3.0 e superior) suporta vídeo Theora e áudio Vorbis em um container Ogg. Google Chrome 6.0 também suporta WebM.
- Safari no Macs e PCs Windows (3.0 e superior) irão suportar tudo que o QuickTime suporta. Em teoria, você não pode requerer que seus usuários instalem plugins QuickTime de terceiros. Na prática, alguns usuários estão fazendo isto. Então você é deixado com os formatos que o QuickTime suporta "fora da caixa." Esta é uma lista longa, mas não inclui WebM, Theora, Vorbis, ou o container Ogg. Contudo,

QuickTime *fornece* suporte para vídeo H.264 (perfil principal) e áudio AAC em um container MP4.

- Celulares como iPhone da Apple e celulares com Android da Google suportam vídeo H.264 (perfil de base) e áudio AAC (perfis de "baixa complexidade") em um container de MP4.
- Adobe Flash (9.0.60.184 e superior) suporta vídeos H.264 (todos os perfis) e áudio AAC (todos os perfis) em um container MP4.
- Internet Explorer 9 suporta todos os perfis de vídeos H.264 ou AAC ou áudio MP3 em um container MP4. Também irá reproduzir vídeo WebM se você instalar algum codec de terceiro, que por padrão não é instalado por nenhuma versão do Windows. IE9 não suporta outros codecs de terceiros (ao contrário do Safari, que vai reproduzir tudo que o QuickTime pode reproduzir).
- Internet Explorer 8 não tem suporte de vídeo HTML5 em geral, mas praticamente todos os usuários de Internet Explorer terão o plugin Adobe Flash. Mais adiante neste capítulo, Eu vou mostrar para você como usar vídeo HTML5 e gerar um fallback em Flash.

Isto deve ser mais fácil de digerir em forma de tabela.

SUPORTE DE CODECS DE VÍDEO NOS NAVEGADORES ATUALMENTE							
CODECS/CONTAINER	IE	FIREFOX	SAFARI	CHROME	OPERA	IPHONE	ANDROID
Theora+Vorbis+Ogg	·	3.5+	†	5.0+	10.5+	·	·
H.264+AAC+MP4	·	·	3.0+	5.0-?‡	·	3.0+	2.0+

† Safari irá reproduzir tudo que o QuickTime reproduz. QuickTime vem pre-instalado com suporte H.264/AAC/MP4. Existem plugins de terceiros instaláveis que provêm suporte para Theora e WebM, mas cada usuário precisa instalar esses plugins antes do Safari reconhecer os formatos dos vídeos.‡ Google Chrome irá perder o suporte a H.264 em breve. Ler sobre o por que.

Daqui há um ano, a paisagem vai parecer significativamente diferente com WebM implementado em múltiplos navegadores, estes navegadores habilitarão versões não-experimentais do WebM, e atualizações dos usuários para as novas versões.

SUPORTE DE CODECS DE VÍDEO NOS NAVEGADORES FUTURAMENTE							
CODECS/CONTAINER	IE	FIREFOX	SAFARI	CHROME	OPERA	IPHONE	ANDROID
Theora+Vorbis+Ogg	·	3.5+	†	5.0+	10.5+	·	·
H.264+AAC+MP4	9.0+	·	3.0+	·	·	3.0+	2.0+

* Internet Explorer 9 vai suportar apenas WebM "quando o usuário tiver instalado um codec VP8", o que implica que a Microsoft não enviará o codec deles mesmos.

* Safari irá reproduzir tudo que o QuickTime pode reproduzir, mas QuickTime vem apenas com suporte H.264/AAC/MP4 pré-instalado.

* Embora o Android 2.3 suporte WebM, ainda não existem decodificadores de hardwares, dessa forma a vida da bateria é uma preocupação.

E agora para o soco de nocaute:

Não existem únicas combinações de contêineres e codecs que funcionem em todos os navegadores HTML5.

E isso não vai mudar em um futuro tão próximo.

Para fazer seu vídeo assistível em todos esses dispositivos e plataformas, você precisará codificar seu vídeo mais de uma vez.

Para máxima compatibilidade, aqui está o fluxo de trabalho com o que seu vídeo deve parecer.

1. Fazer uma versão que usa WebM (VP8 + Vorbis).
2. Fazer outra versão que use linhas de bases de vídeo H.264 e audio AAC "baixa complexidade" em um contêiner MP4.
3. Fazer outra versão que use vídeo Theora e audio Vorbis em um contêiner Ogg.
4. Linkar todos os três arquivos de vídeo a partir de um único elemento `<video>`, e um "fallback" para um reprodutor de vídeo flash.

PROBLEMAS DE LICENCIAMENTO COM VÍDEO H.264

Antes de nós continuarmos, eu preciso salientar que há um custo em codificar seu vídeo duas vezes. Bem, existe um custo óbvio, que você tem que codificar seu vídeo duas vezes, o que consome mais computador e mais tempo do que se fosse feito apenas uma vez. Mas há outro custo real associado com vídeo H.264: custos de licenciamento.

Se lembram quando eu expliquei vídeo H.264, e eu mencionei que o codec de vídeo tem patentes-embutidas e o licenciamento foi quebrado pelo consórcio MPEG LA. Isso acaba sendo importante. Para entender porque isso é importante, eu te direciono para O labirinto do licenciamento H.264:

MPEG LA Divide a licença de portfólio H.264 em duas sub-licenças: uma para manufaturas de codificadores e decodificadores e a outra para distribuidores de conteúdo. ...

A sub-licença do lado dos distribuidores fica ainda mais dividida em quatro sub-categorias chaves, duas delas (assinaturas e aquisição título-por-título ou uso remunerado) são ligados ao fato de que o usuário final paga diretamente pelo serviço de vídeos, e dois deles (televisão "gratuita" e broadcast de internet) são ligados a remuneração por outras fontes que não seja o espectador final. ...

A taxa de licenciamento para televisão "gratuita" é baseada em uma das duas opções de royalties. A primeira é um pagamento único de \$2,500 por transmissão codificada AVC, que cobre um codificador AVC "usado por ou a favor da licença de transmissão de vídeo AVC para o usuário final" que é quem irá decodificar e visualizá-lo. Se você está se perguntando se isso é um encargo duplo, a resposta é sim: A taxa da licença já foi cobrada do codificador manufaturado, e o broadcast vai em turnos pagar uma das duas opções de royalties.

A segunda taxa de licenciamento é anual. [A] taxa anual de broadcast é dividida pelo tamanho da audiência:

- \$2,500 por ano por mercado de broadcasts de 100,000–499,999 famílias de televisão
- \$5,000 por ano por mercado de broadcasts de 500,000–999,999 famílias de televisão
- \$10,000 por ano por mercado de broadcasts de 1,000,000 ou mais famílias de televisão

Com toda a questão da televisão "livre" por que alguém envolvido em entregas não-broadcast se importaria? Como eu mencionei antes, as taxas de participação são aplicáveis para cada entrega de conteúdo. Depois de definir que televisão "livre" significa mais do que apenas "over-the-air", MPEG LA passou a definir as taxas de licenciamento para transferência na internet via "vídeo AVC que é entregue em todo mundo para um usuário final para que o usuário final não pague pelo direito de receber ou ver". Em outras palavras, qualquer transmissão pública, seja "over-the-air", cabo, satélite, ou internet estão sujeitas a taxas de participação. ...

As taxas são potencialmente mais íngrimes para transmissões pela internet, talvez sabendo que a entrega na internet irá crescer muito mais rápido do que OTA ou televisão "livre" via cabo ou satélite. Adicionando a televisão "livre" as taxas do mercado da transmissão juntas com a taxa adicional. MPEG LA garantirá um indulto durante o tempo da primeira licença, que acaba em 31 de Dez de 2010, e note que "após a primeira licença o royalties não será mais do que equivalente econômico dos royalties pagos durante o mesmo tempo de televisão livre."

A última parte - Sobre a estrutura das taxas para transmissão pela internet - já foi alterada. A MPEG-LA recentemente anunciou que o streaming de internet não seria cobrado. Que *não* significa que H.264 é livre de royalties para todos os usuários. Em particular, codificadores (Como o que processa o upload de vídeo do YouTube) e codificadores (como o que está incluso no Microsoft Internet Explorer 9) ainda são assuntos de taxas de licenciamento. Veja Livre como em uma tema de fumaça para maiores informações.



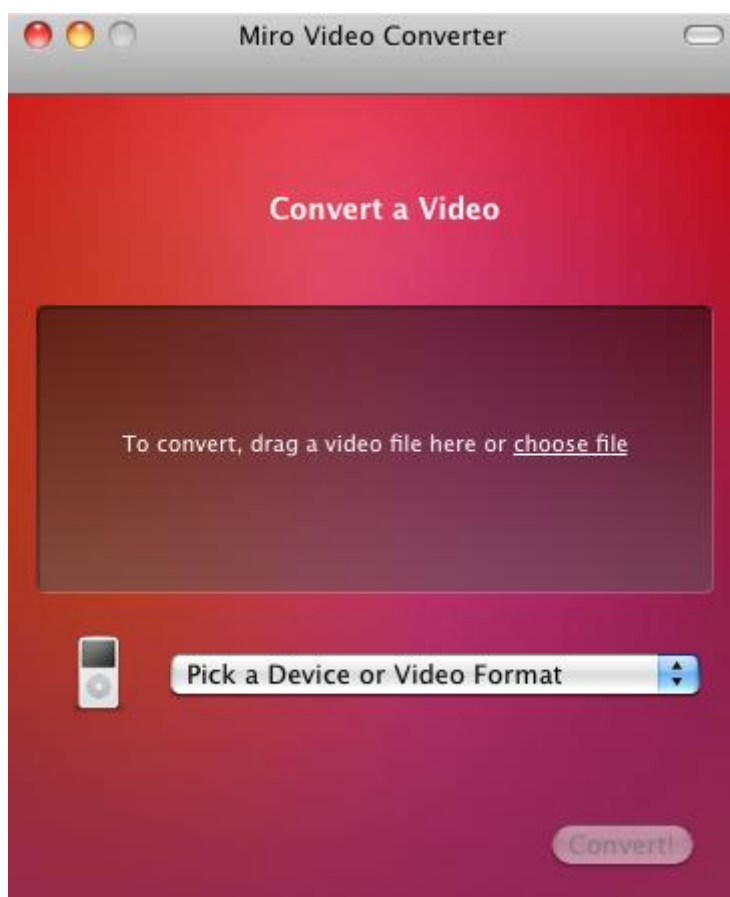
CODIFICANDO VÍDEO COM MIRO VIDEO CONVERTER

Existem muitas ferramentas para codificar vídeo, e há muitas opções de codificação de vídeo que afetam sua qualidade. Se você não deseja tomar o tempo para entender nada sobre codificação de vídeo, esta seção é para você.

Miro Video Converter é programa de código aberto para codificação de vídeo em múltiplos formatos com licença GPL. Faça aqui o download para Mac OS X ou Windows. Ele suporta todos os formatos de saída mencionados neste capítulo. Não oferece opções além de escolher um arquivo de vídeo e escolher um formato de saída. Pode receber virtualmente qualquer arquivo de vídeo como entrada, incluindo vídeo DV produzidos por filmadoras. Produz uma saída com qualidade razoável com a maioria dos vídeos. Devido a sua falta de opções, se você estiver descontente com a saída, você não tem mais recursos além de tentar outro programa.

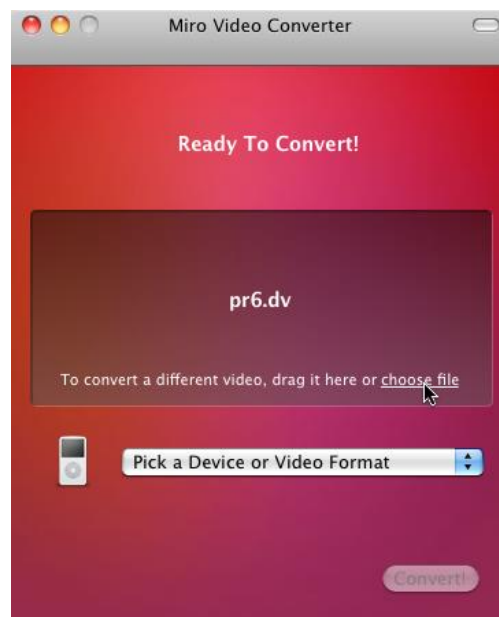
Para começar, Apenas inicie a aplicação Miro Video Converter.

Tela principal do Miro Video Converter ↷



Clique em "Escolher arquivo" e selecione o vídeo que você quer codificar.

"Escolher arquivo" ~

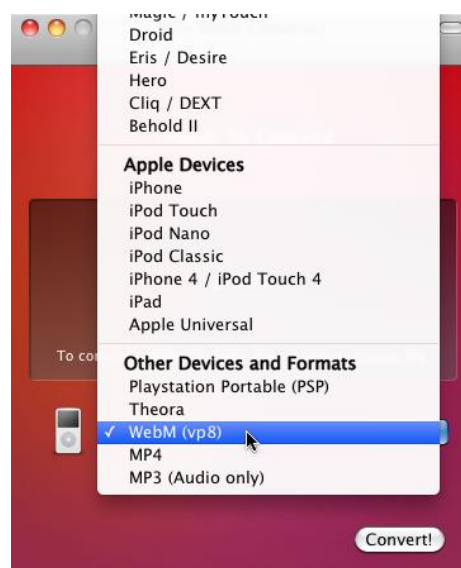


O menu dropdown "Escolha um dispositivo ou um formato de vídeo" lista uma variedade de dispositivos e formatos. Para o propósito deste capítulo, nós estamos interessados em três deles.

1. *WebM (vp8)* é video WebM (video VP8 e audio Vorbis em um contêiner WebM).
2. *Theora* é vídeo Theora e audio Vorbis em um contêiner Ogg.
3. *iPhone* é Perfil de Linha de base de video H.264 e AAC audio de baixa-complexidade em um contêiner MP4.

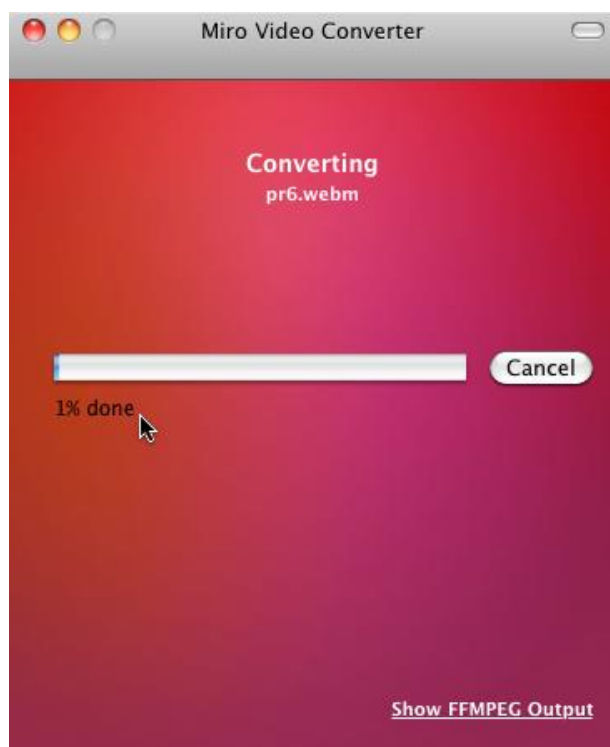
Primeiro selecione "WebM".

Escolha a saída WebM~



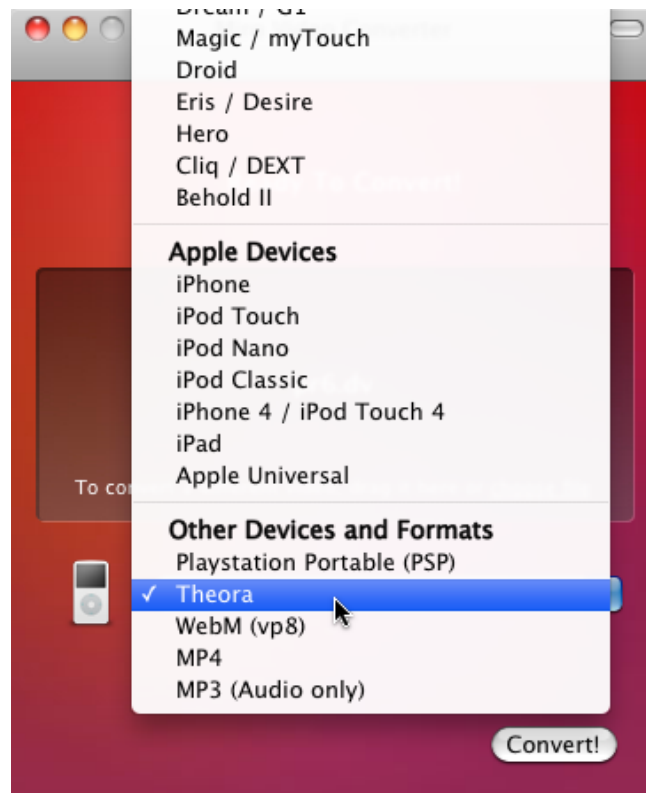
Click no botão "Converter" e o conversor Miro Video vai imediatamente começar a codificar seu vídeo. O arquivo de saída será nomeado ARQUIVOFONTE.webm e vai ser salvo no mesmo diretório do vídeo de origem.

Você ficará olhando para esta tela por um bom tempo ~



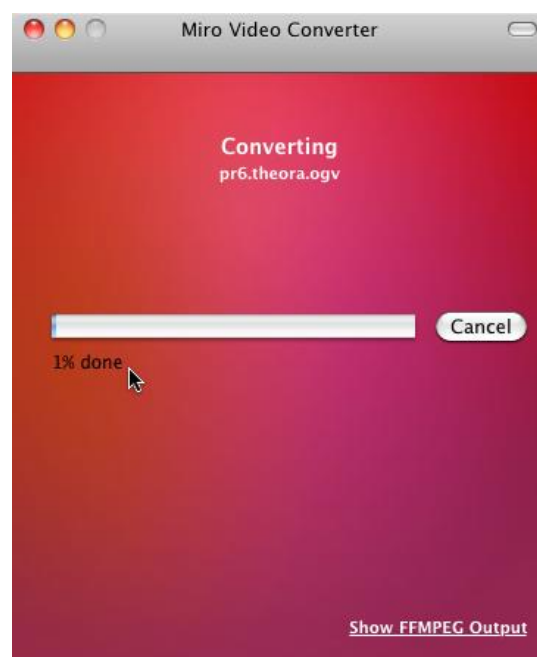
Uma vez que a codificação for completada, você será mandado de volta para a tela principal. Desta vez, selecione "Theora" na lista de dispositivos e formatos.

Hora do Theora ~



É isto; aperte o botão de "Converter" novamente para codificar seu vídeo Theora. O vídeo vai ser nomeado ARQUIVOFONTE.theora.ogv e será salvo no mesmo diretório do arquivo de origem.

Hora de tomar um cafézinho~



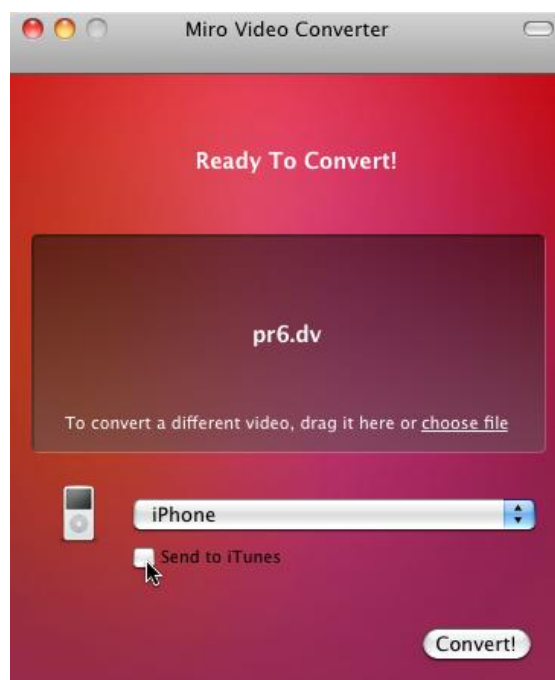
Finalmente, codifique seu vídeo H.264 compatível com iPhone selecionando "iPhone" na lista de dispositivos e formatos.

iPhone, não iPhone 4 ~



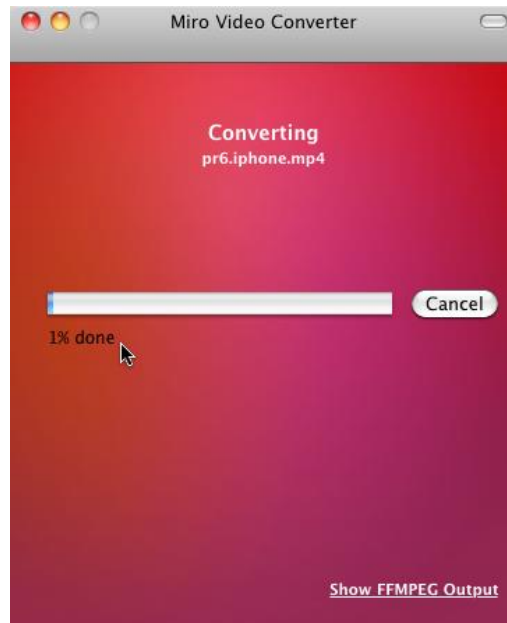
Para vídeo compatível com iPhone, o Conversor Miro Video dará a você uma opção de enviar o arquivo codificado para sua biblioteca iTunes. Eu não tenho uma opinião formada se você gostaria de fazer isto, mas isto não é necessário para publicar vídeo na web.

Não envie para o iTunes~



Aperte o botão mágico "Converter" e aguarde. O arquivo codificado será nomeado ARQUIVOFONTE.iphone.mp4 e vai ser salvo no mesmo diretório do arquivo de origem.

Faça yoga ou algo parecido~



Você deverá ter três arquivos de vídeo ao lado do seu arquivo de vídeo fonte. Se você estiver satisfeito com a qualidade do vídeo, avance para Enfim, a Marcação para ver como montar eles em um único elemento <vídeo> que funcione cross-browsers. Se você gostaria de aprender mais sobre outras ferramentas ou codificação de vídeo, leia.



CODIFICANDO VÍDEO OGG COM FIREFOGG

(Nesta seção, eu vou usar "Vídeo Ogg" como abreviação para "Vídeo Theora e Áudio Vorbis em um contêiner Ogg". Esta é a combinação de codecs+contêiners que funcionam nativamente no Mozilla Firefox e Google Chrome.)

Firefogg é uma extensão do Firefox open source, com licença GPL para codificar vídeo Ogg. Para usar isto, você vai precisar instalar Mozilla Firefox 3.5 ou mais recente, e visitar firefogg.org.

Página inicial do Firefogg ~



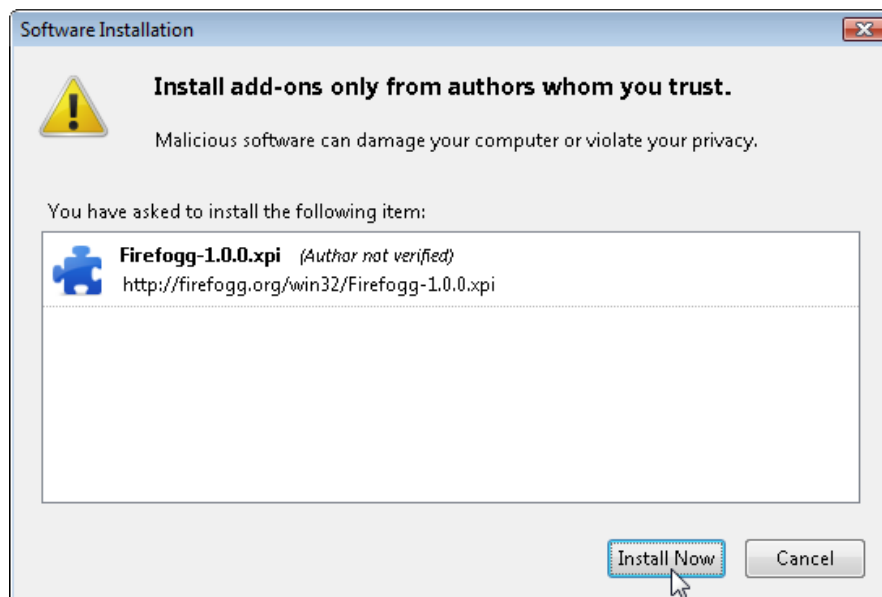
Clique "Instalar Firefogg." Firefox irá perguntar se você realmente quer autorizar o site a instalar uma extensão. Clique "Permitir" para continuar.

Permitir instalação do Firefogg



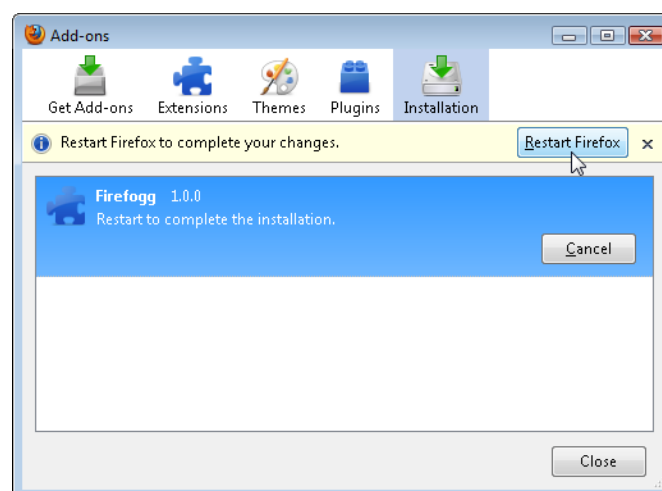
Firefox vai apresentar a janela de instalação do software padrão. Clique "Instalar" para continuar.

Instalar Firefogg~



Clique "Reiniciar Firefox" para completar a instalação.

↪ *Reiniciar Firefox*



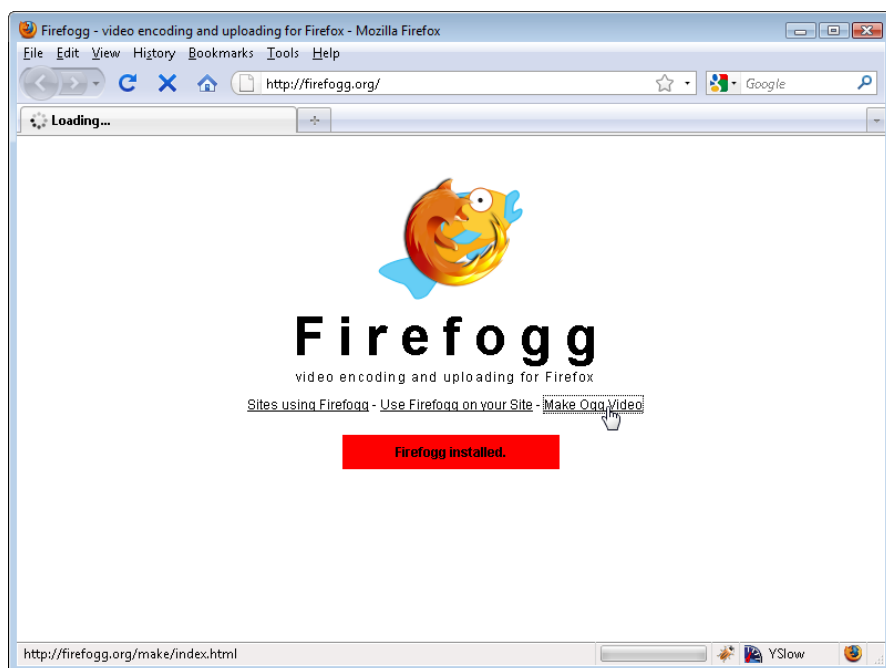
Depois de reiniciar o Firefox, firefogg.org vai confirmar que o Firefogg foi instalado com sucesso.

Instalação completada com sucesso ~



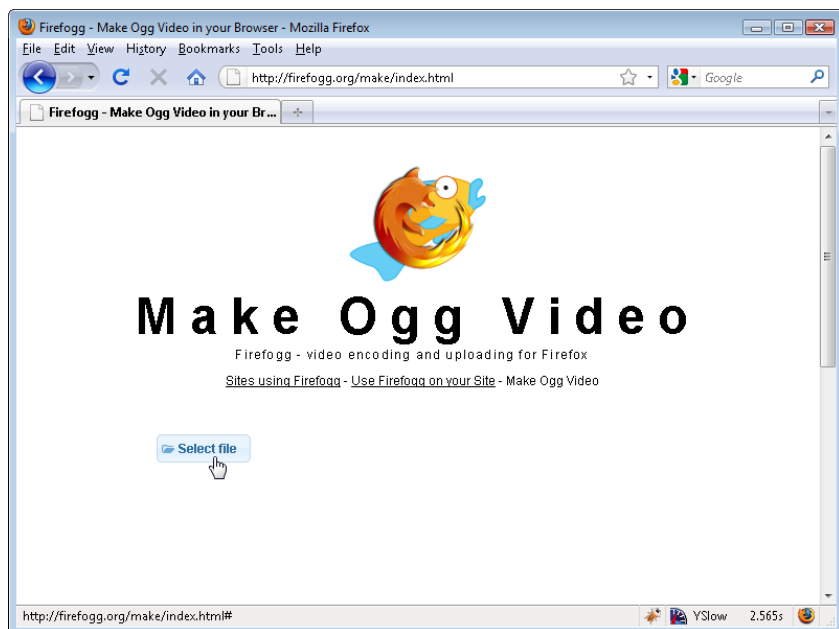
Clique "Criar vídeo Ogg" para iniciar o processo de codificação.

~ *Vamos criar um vídeo!*



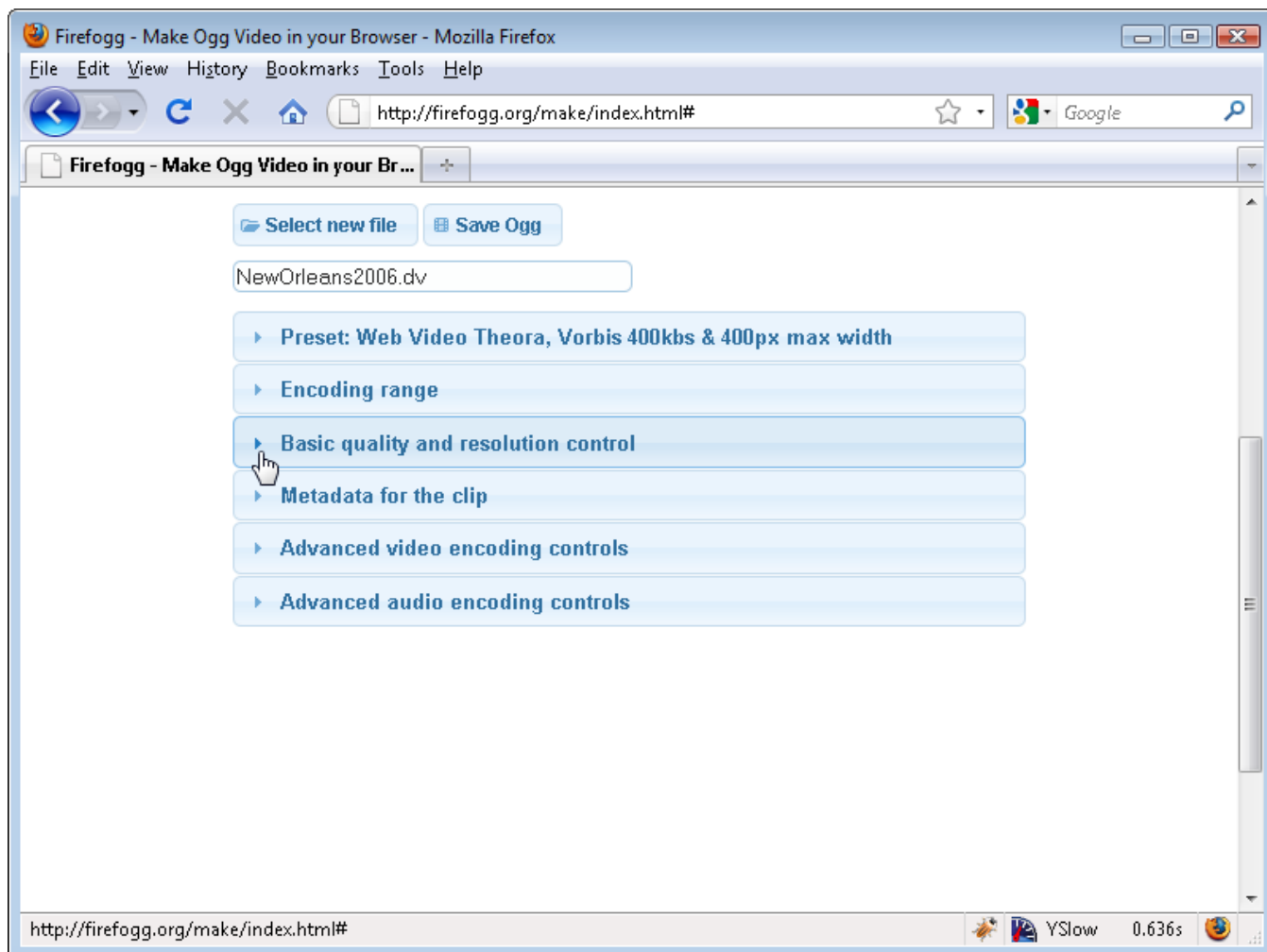
Clique "Selecione o arquivo" para selecionar seu vídeo de origem.

Selecione seu arquivo de vídeo~



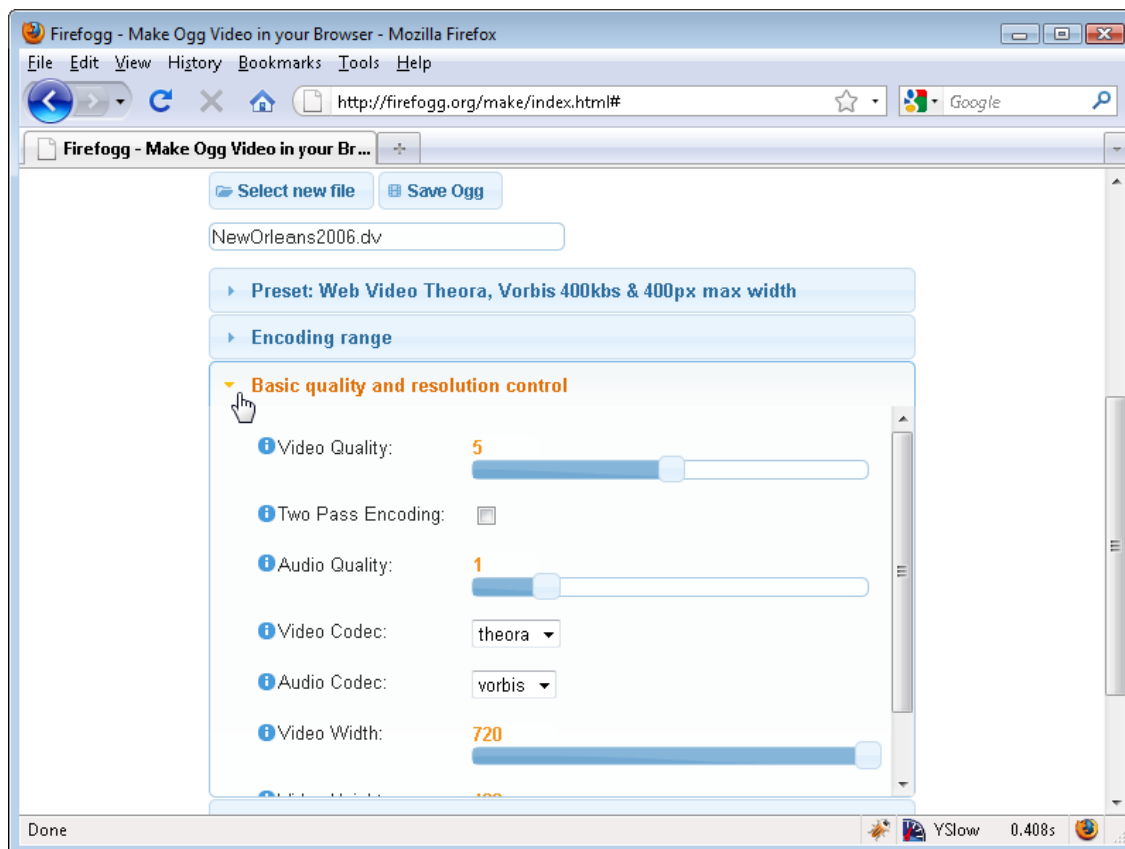
Firefogg tem seis "abas":

1. Predefinições. A predefinição padrão é "vídeo web," o que é bom para nosso propósito.
2. Limite de Codificação. Codificar vídeo pode levar um bom tempo. Quando você estiver começando, talvez você queira codificar apenas partes do seu vídeo (como, os primeiros 30 segundos) até que você encontre uma combinação de configurações que você goste.
3. Qualidade básica e controle de resolução. Esta é onde estão a maioria das opções importantes.
4. Metadata. Eu não vou cobrir isto aqui, mas você pode adicionar metadata ao seu codificador de vídeo como título e autor. Provavelmente você tem adicionado metadata a sua coleção de música com iTunes ou algum outro gerenciador de musicas. É a mesma idéia.
5. Controle avançado de codificação de vídeo. Não mecha com este a menos que saiba o que está fazendo. (Firefogg oferece ajuda interativa na maioria destas opções. Clique no simbolo "i" próximo a cada opção para aprender mais a respeito.)
6. Controle avançado de codificação de áudio. Novamente, Não mecha com este a menos que saiba o que está fazendo.



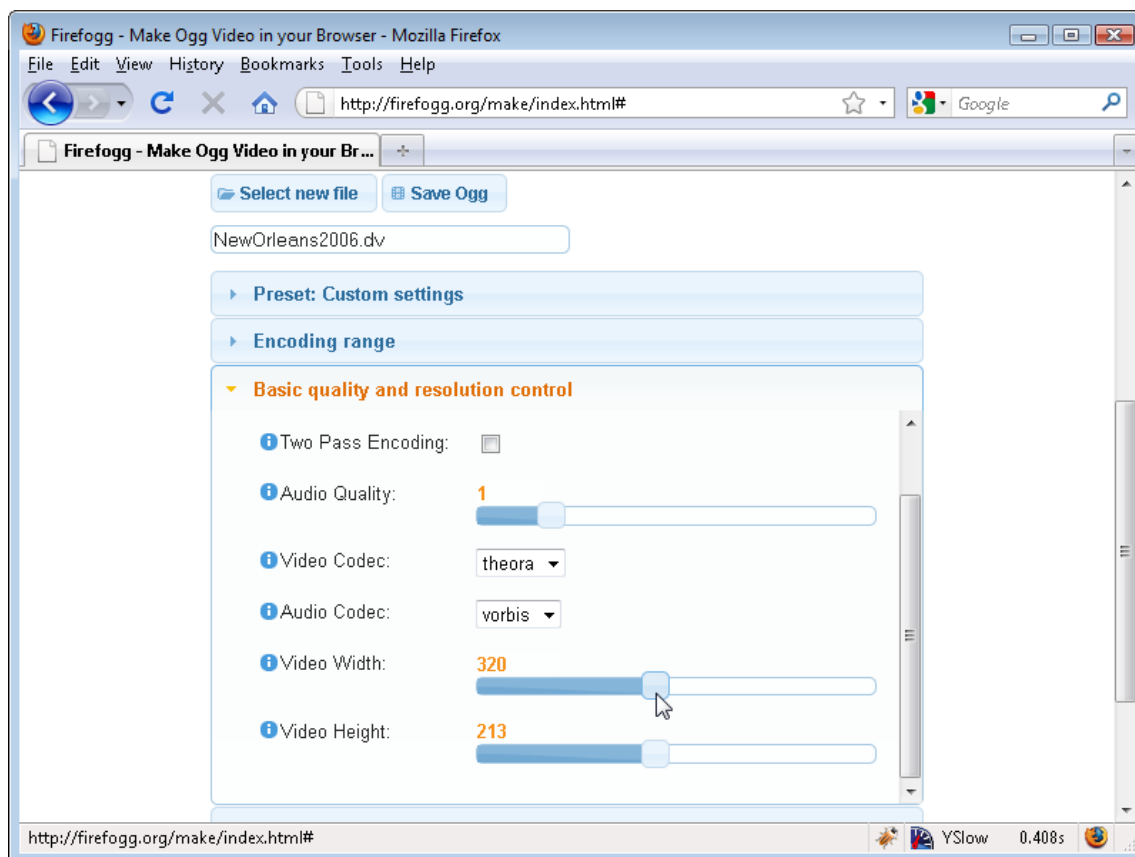
A única opção que eu vou cobrir estão na tabela "Qualidade básica e controle de resolução". Ela contém todas as opções importantes:

- Qualidade de Vídeo. É mensurada em uma escala de 0 (menor qualidade) a 10 (maior qualidade). Números altos significam tamanhos de arquivos maiores, então você terá de experimentar para determinar qual a melhor relação tamanho/qualidade para sua necessidade.
- Qualidade de Áudio. É mensurada em uma escala de -1 (menor qualidade) a 10 (maior qualidade). Números altos significam tamanhos de arquivos maiores, assim como as configurações de qualidade de vídeo.
- Vídeo Codec. Sempre deve ser "Theora"
- Áudio Codec. Sempre deve ser "Vorbis"
- Largura e comprimento de vídeo. Este é o padrão atual de largura e comprimento do seu vídeo fonte. Se você deseja redimensionar o vídeo durante a codificação, você pode trocar a largura (ou comprimento) aqui. Firefogg vai automaticamente ajustar as outras dimensões para manter as proporções originais (então seu vídeo não terminará esticado ou amassado).



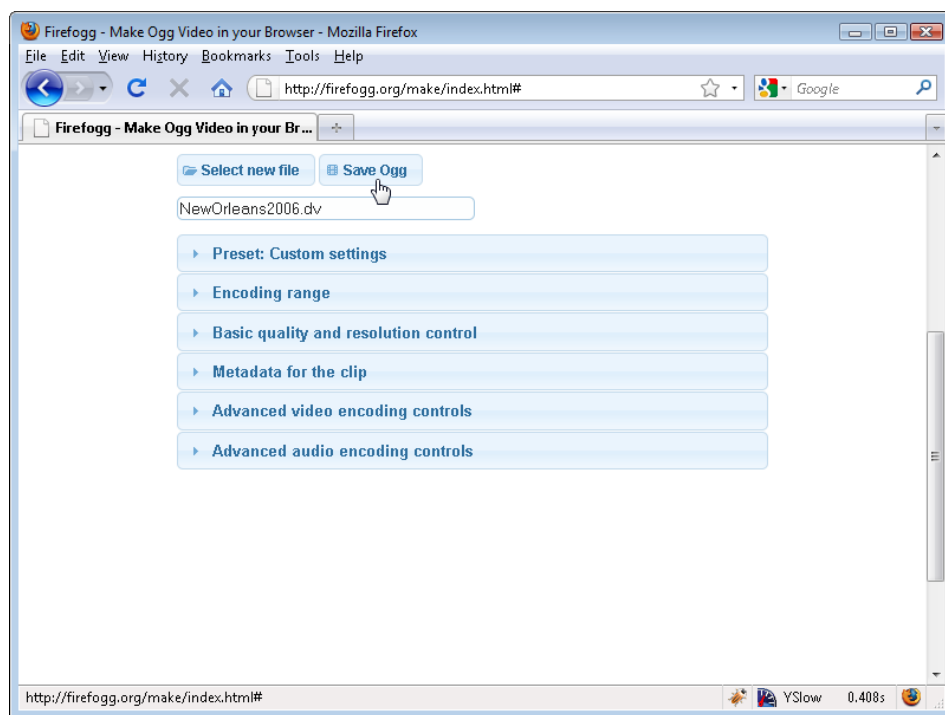
Neste exemplo, vou redimensionar o vídeo para metade do tamanho original de largura. Note como o Firefogg automaticamente ajusta o comprimento.

Ajustando largura e comprimento de vídeo~



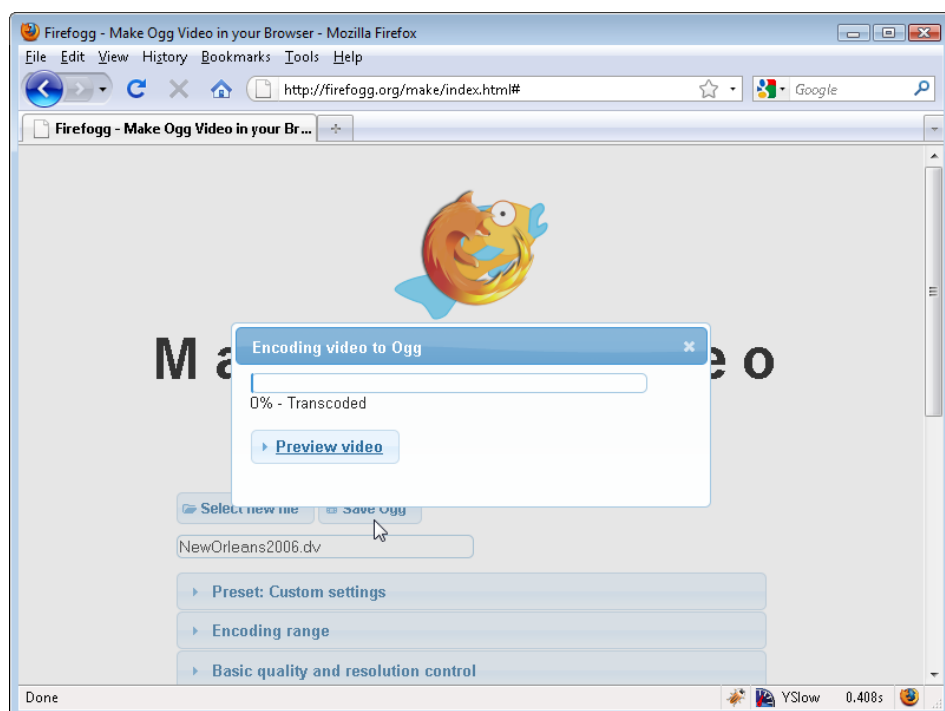
Uma vez expandido com todos os botões, clique em "Save Ogg" para iniciar o processo de codificação atual. Firefogg vai perguntar para você um nome de arquivo para o vídeo codificado.

"Salvar Ogg"~



Firefogg vai mostrar uma bela barra de progresso enquanto ele codifica seu vídeo. Tudo que você tem que fazer é aguardar (e aguardar, e aguardar)!

~ Processo de codificação



CODIFICAÇÃO EM LOTES DE VÍDEOS OGG COM FFMPEG2THEORA

(Como na seção anterior, nesta eu vou usar "Video Ogg", como encurtamento de "Theora vídeo e áudio Vorbis em um contêiner Ogg". Esta é a combinação de codecs+contêiners que funciona nativamente no Mozilla Firefox e Google Chrome.)

Se você está procurando por uma codificação em lotes de diversos arquivos de vídeo Ogg e você quer automatizar o processo, você definitivamente deveria dar uma olhada no [ffmpeg2theora](#).

O ffmpeg2theora é uma aplicação para codificação de vídeos Ogg de código livre e licença GPL. Binários pré-compilados estão disponíveis para Mac OS X, Windows, e distribuições Linux modernas. Ele pode pegar virtualmente qualquer tipo de arquivo como entrada, incluindo vídeos DV produzidos por câmeras simples, não profissionais.

Para usar o ffmpeg2theora, você precisará chamá-lo pela linha de comando. (No Mac OS X, abra Aplicações → Utilitários → Terminal. No Windows, abra o seu menu Iniciar → Programas → Acessórios → Prompt de Comando.)

O ffmpeg2theora tem diversos parâmetros de linha de comando. (Escreva `ffmpeg2theora --help` para ler sobre todos eles.) Eu irei focar em apenas três deles.

- `--video-quality Q`, onde "Q" é um número entre 0–10.
- `--audio-quality Q`, onde "Q" é um número entre -2–10.
- `--max_size=WxH`, onde "W" e "H" são a largura e altura máximos que você deseja para o vídeo (o "x" entre eles é realmente apenas a letra "x"). O ffmpeg2theora irá redimensionar o vídeo proporcionalmente para ajustar a estas dimensões, então o vídeo codificado deverá ser menor que WxH. Por exemplo, realizar a codificação de um vídeo 720x480 com tamanho máximo `--max_size 320x240` irá produzir um arquivo que será 230x213.

Assim, aqui está como você deve codificar um vídeo com as mesmas configurações que nós usamos na seção anterior ([codificação com Firefogg](#)).

```
you@localhost$ ffmpeg2theora --videoquality 5  
                             --audioquality 1  
                             --max_size 320x240  
                             pr6.dv
```

O vídeo codificado será salvo no mesmo diretório do arquivo de vídeo original, com a extensão `.ogv` adicionada. Você pode especificar um lugar diferente e/ou um nome diferente

passando os seguintes parâmetros `--output=/path/to/encoded/video` para o `ffmpeg2theora`.



CODIFICANDO VÍDEOS H.264 COM O HANDBRAKE

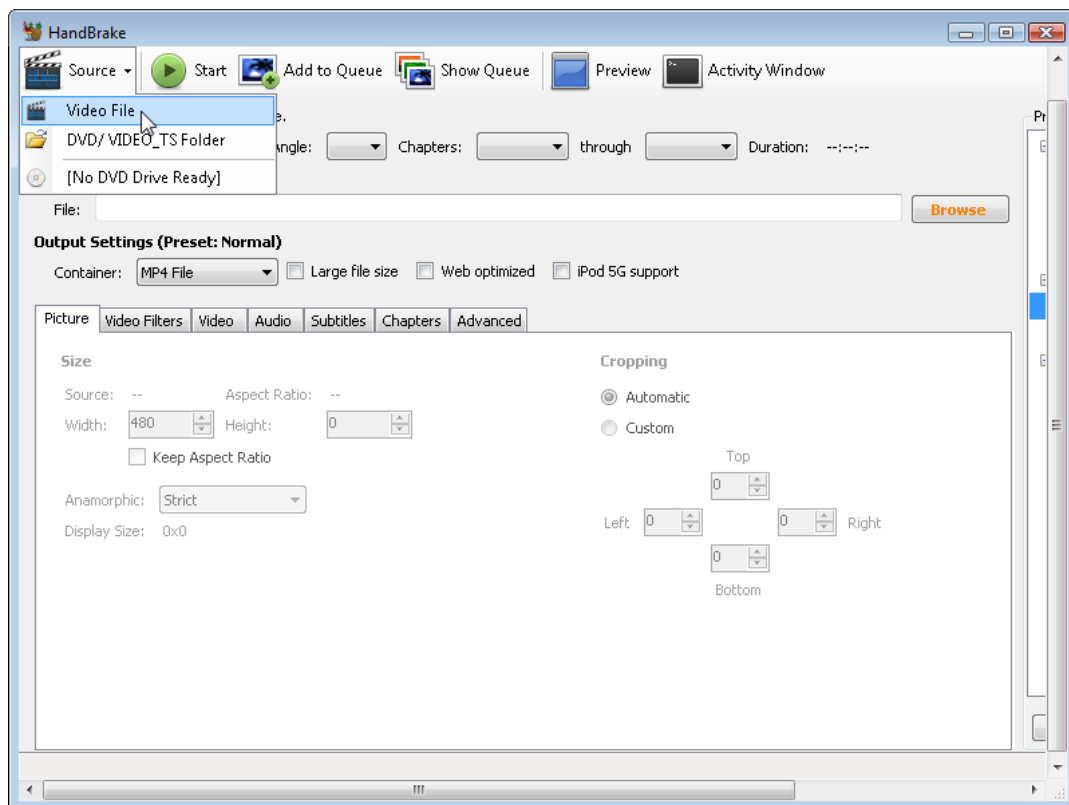
(Nesta seção, eu irei usar "vídeo H.264" como uma abreviação para "Conjunto de especificações e perfis de vídeo H.264 e Perfil de áudio de baixa complexidade AAC em um container MPEG-4". Esta é a combinação de codificadores+containers que funcionam nativamente no Safari, no Adobe Flash, no iPhone e nos aparelhos que utilizam Google Android.)

Tirando as questões de licenciamento, o jeito mais fácil de codificar um vídeo H.264 é usar o HandBrake. O HandBrake é uma aplicação para codificação de vídeos H.264 open source, com licença GPL (as versões anteriores realizavam codificações para outros formatos de vídeo também, mas na última versão os desenvolvedores resolveram retirar o suporte para a maioria dos outros formatos e estão focando todos seus esforços nos vídeos H.264). Os binários pré-compilados estão disponíveis para Windows, Mac OS X e distribuições Linux modernas.

O HandBrake possui duas versões: gráfica e linha de comando. Eu irei fazer um passo-a-passo da interface gráfica primeiro, então veremos quais são as configurações recomendadas traduzidas para a versão de linha de comando.

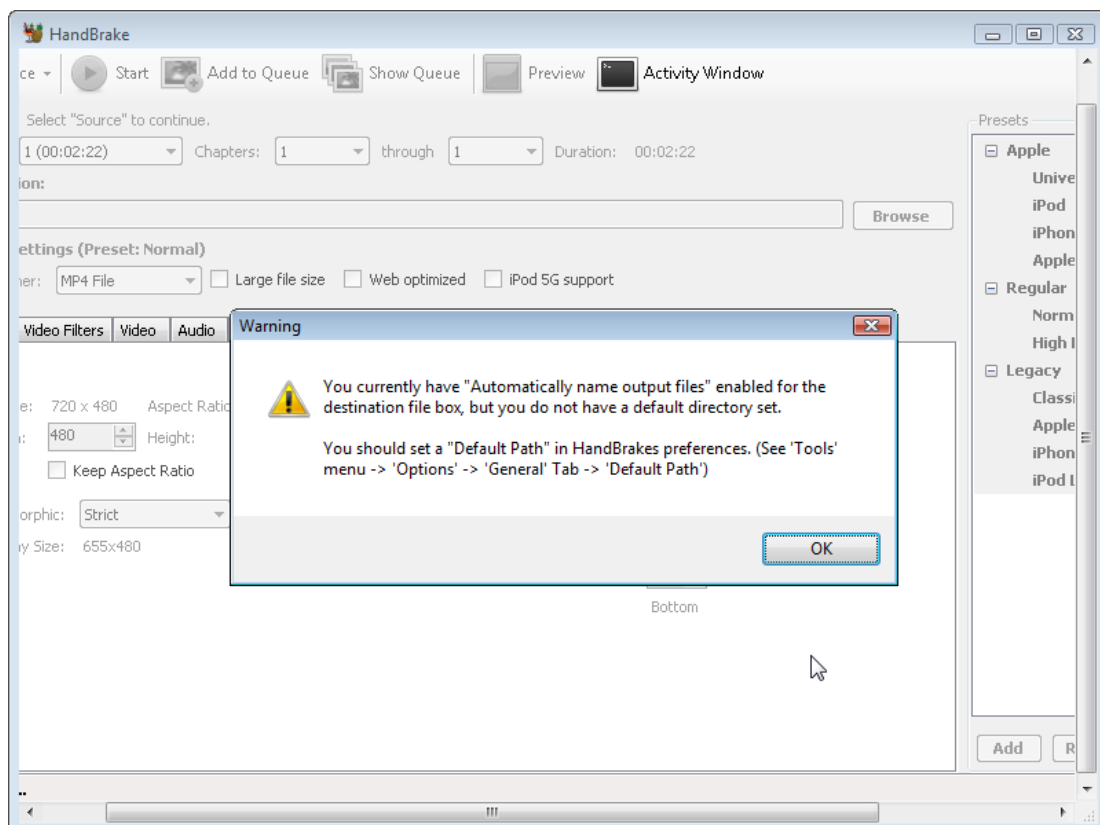
Depois de abrir a aplicação HandBrake a primeira coisa a fazer é selecionar o seu vídeo de origem. Clique em "Origem" no menu dropdown e escolha "Arquivo de Vídeo" para selecionar um arquivo. O HandBrake teoricamente pode pegar qualquer arquivo de vídeo como origem, incluindo vídeos DV produzidos por câmeras simples.

Selecione seu vídeo de Origem ~



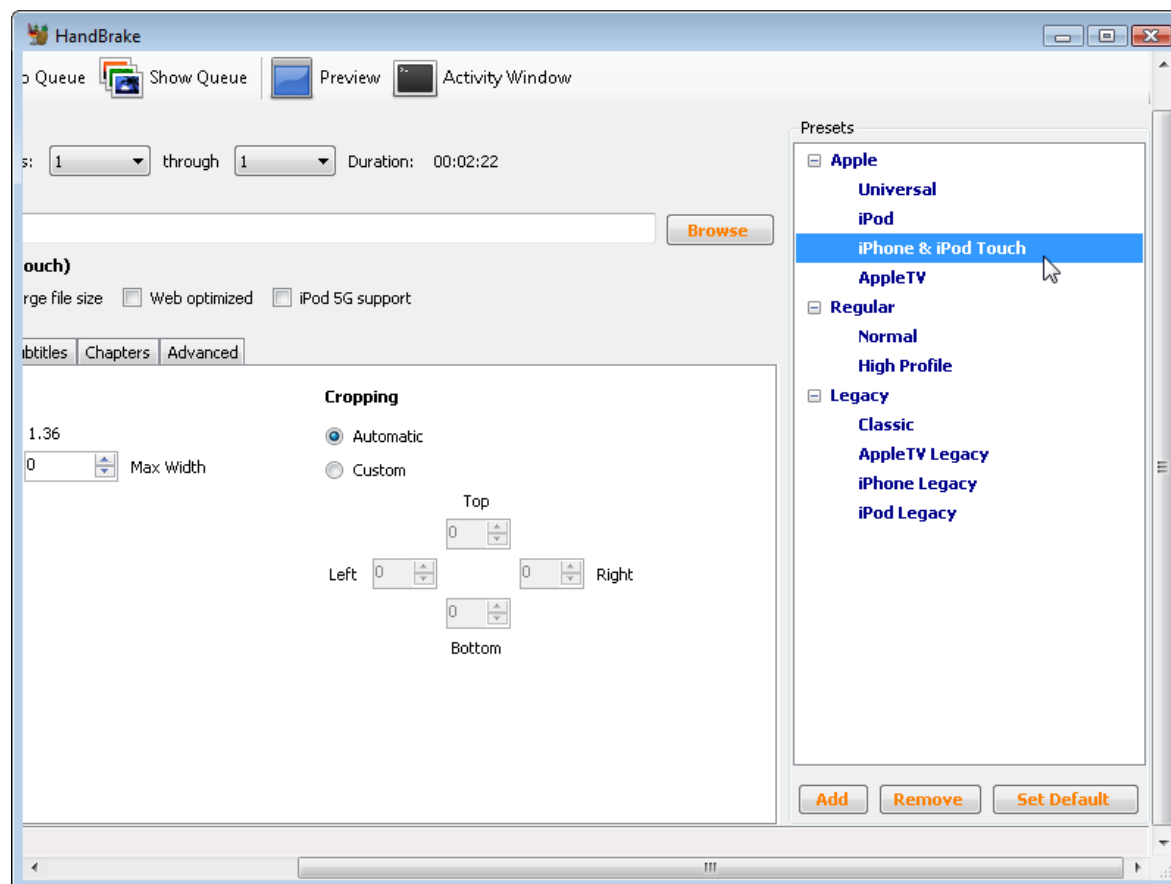
O HandBrake irá reclamar que você não configurou um diretório padrão para salvar os arquivos codificados. Você pode ignorar esse alerta com segurança ou, então, abrir a janela de opções (dentro do menu "Tools") e configurar o diretório padrão de saída.

~ *Ignore isso*



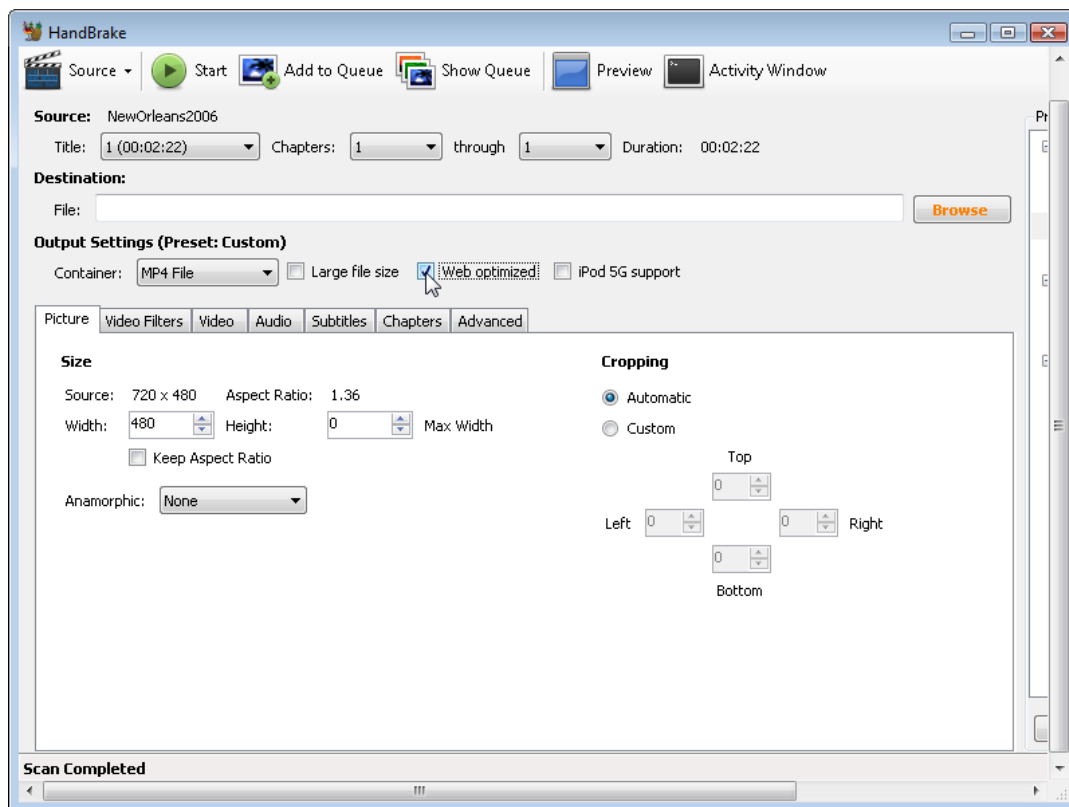
No lado direito há uma lista de predefinições. Selecionando a opção "iPhone & iPod Touch" irá configurar a maioria das opções que você precisa.

Selecione a predefinição iPhone ~



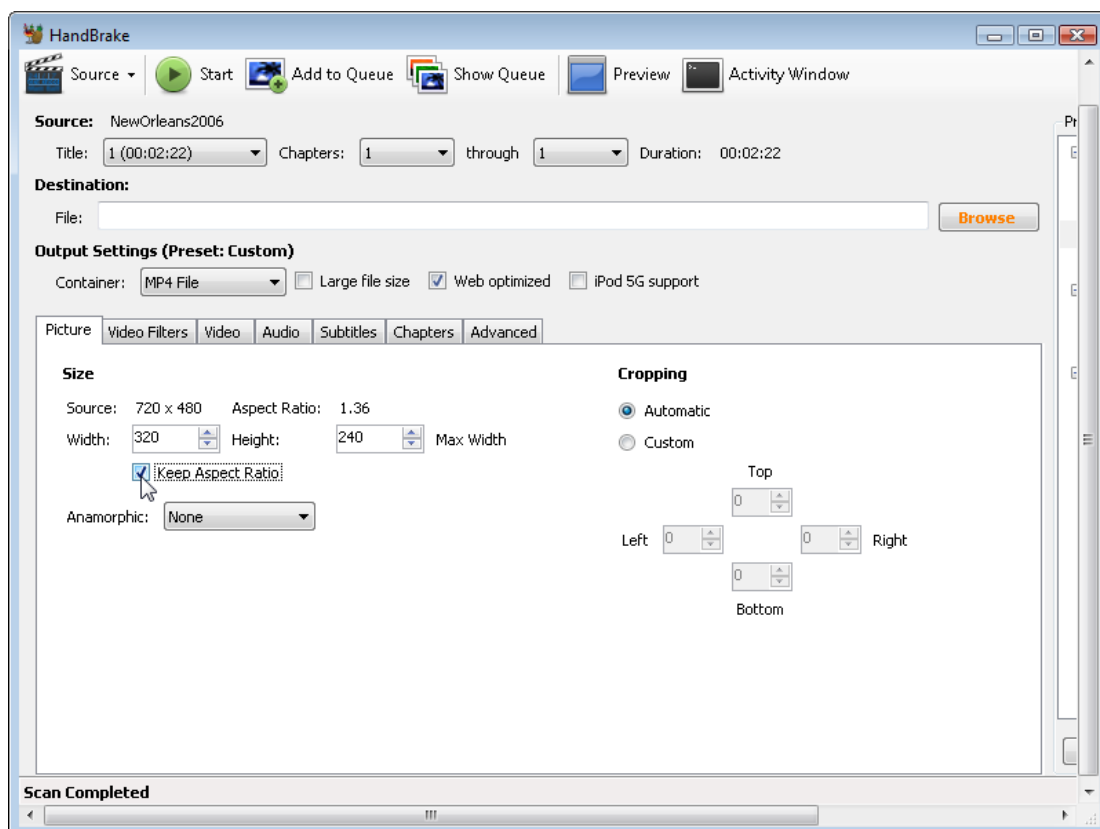
Uma importante opção, e que está desativada por padrão, é a de "Otimizar para Web". Selecionando esta opção os metadados do vídeo codificado serão reordenados, para que você possa assistir o início do vídeo enquanto o resto está sendo baixado em segundo plano. Eu recomendo fortemente sempre conferir esta opção. Ela não afeta a qualidade ou tamanho do arquivo do vídeo codificado, portanto, não há razão para não utilizá-la.

~ Sempre otimizar para web



Na aba de "Imagem" você pode configurar a altura e largura máxima do vídeo codificado. Você também deve selecionar a opção "Manter a proporção" para assegurar que o HandBrake não irá alargar ou esticar o seu vídeo enquanto o redimensiona.

Configure a largura e altura ~



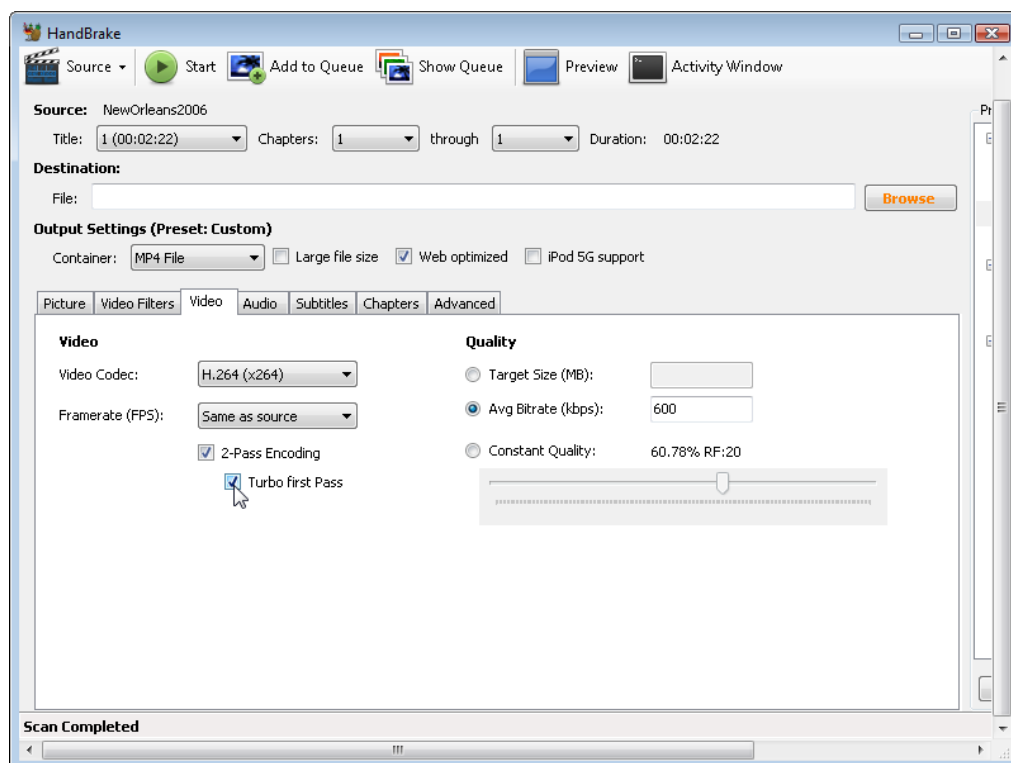
Na aba "Vídeo" você pode configurar quatro opções importantes.

- Codec de Vídeo. Tenha certeza que é "H.264 (x264)"
- Codificação em dois passos. Se esta opção estiver selecionada, o HandBrake irá rodar duas vezes a codificação de vídeo. Na primeira vez, ela apenas analisará o vídeo, procurando por coisas como composição de cores, movimento e paradas de cena. Na segunda vez é que realmente será codificado o vídeo utilizando a informação obtida durante a primeira vez. Como você pode esperar, isto toma duas vezes o tempo do que a codificação em um passo, mas o resultado é um vídeo melhor sem aumentar o tamanho do arquivo. Eu sempre habilito a codificação em dois passos para vídeos H.264. A menos que você esteja fazendo o próximo YouTube e codificando vídeos 24 horas por dia, você provavelmente deveria utilizar a codificação em dois passos também.
- Primeira passagem turbo. Uma vez que você tenha habilitado a codificação em dois passos, você pode recuperar algum tempo habilitando a "primeira passagem turbo." Isto reduzirá a quantidade de trabalho feita durante a primeira passagem (análise do vídeo), enquanto degrada levemente a qualidade. Eu usualmente habilito esta opção, mas se a qualidade é a coisa mais importante para você, você deveria deixar isto desabilitado.
- Qualidade. Existem diferentes modos de especificar a "qualidade" dos seus vídeos codificados. Você pode configurar o tamanho final do arquivo e o HandBrake irá fazer o melhor para que o vídeo codificado não seja maior do que o definido. Você pode configurar a média da "taxa de bits" o que é literalmente o número de bits requeridos para armazenar um segundo valor do vídeo codificado (isto é chamado de "média" da taxa de bits porque alguns segundos irão precisar de mais bits que outros). Ou você pode especificar uma qualidade constante numa escada de 0 à 100%. Números maiores irão resultar em uma melhor qualidade, mas em arquivos maiores. Não há apenas uma resposta certa para qual qualidade que você deve usar.

P: Posso usar codecs em dois passos na codificação de vídeo Ogg?

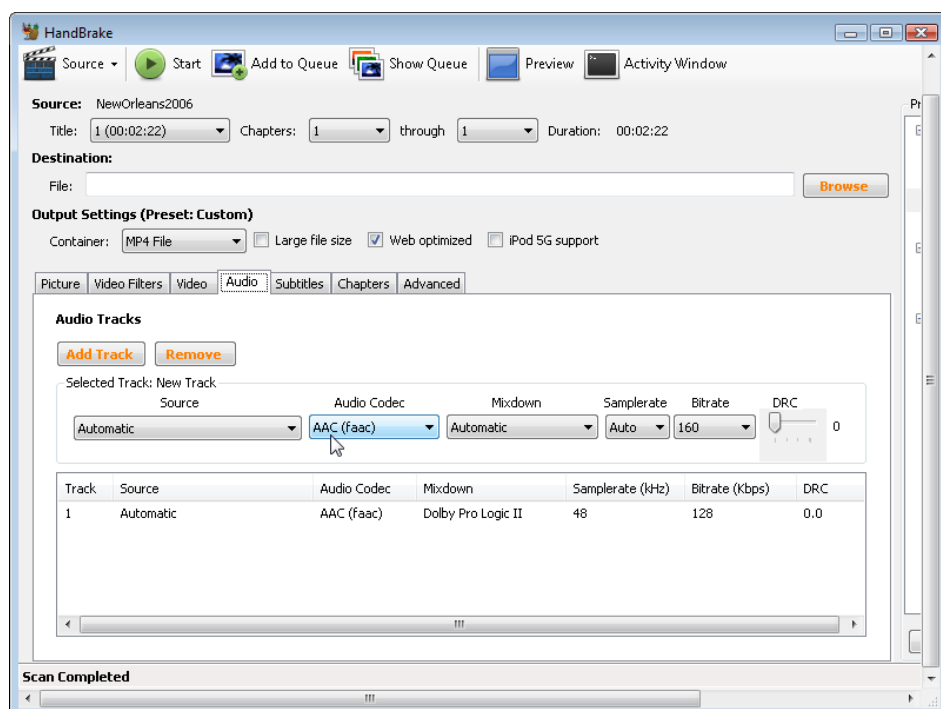
R: Sim, sim mas devido as diferenças fundamentais em como o codificador funciona, você provavelmente não precisa fazê-lo. Codecs em dois passos H.264 quase sempre resultam em vídeos de qualidade mais altas. Codecs Ogg em dois passos de vídeos Ogg só é útil se você está tentando fazer com que seu vídeo codificado tenha um tamanho específico. (Talvez isto é algo em que você esteja interessado, mas não é o que é mostrado nestes exemplos, e provavelmente não vale o tempo extra, codificando o vídeo.) Para a melhor qualidade de vídeo Ogg, use as configurações de qualidade de vídeo, e não se preocupe com codecs em dois passos.

Neste exemplo, Eu escolhi uma taxa de bit médio de 600 kbps, que é bastante alta para um vídeo codificado de 320x240. (Mais tarde neste capítulo, eu vou mostrar um exemplo de vídeo codificado a 200 kbps.) Eu escolhi também codecs em dois passos com um primeiro passo "Turbo".



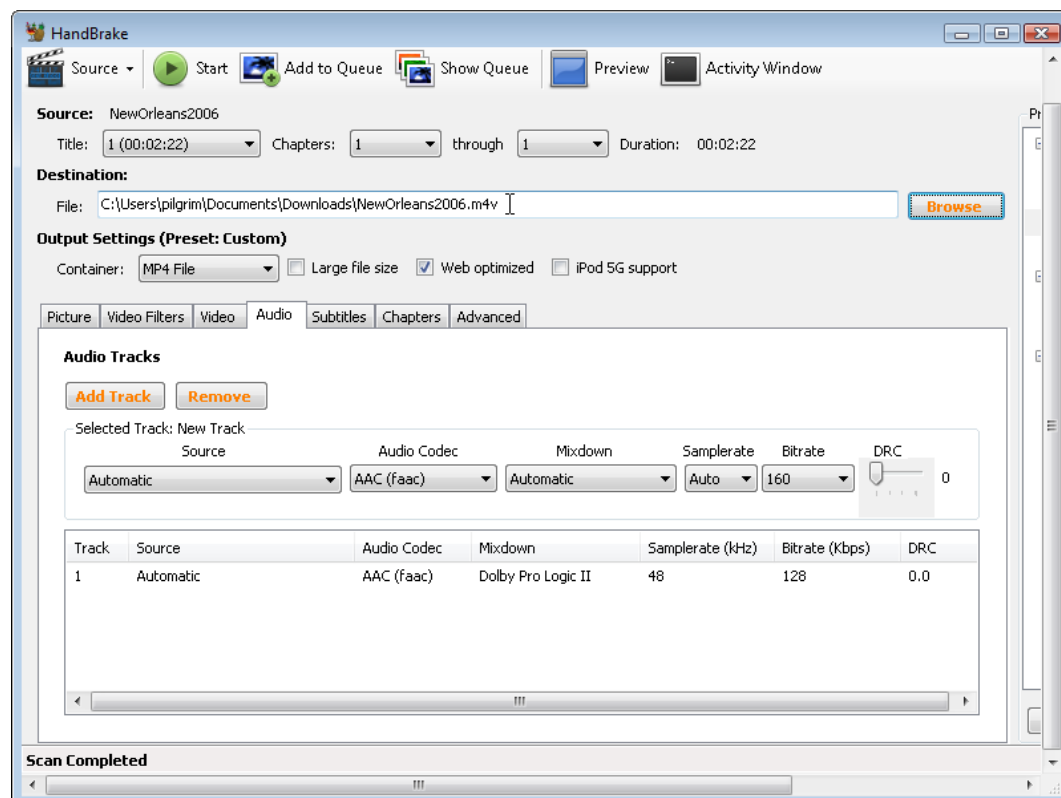
Na aba "Audio", você provavelmente não precisará mudar nada. Se o seu vídeo fonte tiver várias faixas de áudio, você pode precisar selecionar qual delas você deseja no vídeo. Se seu vídeo é na maior parte um discurso pessoal (ao contrário de músicas ou sons ambientes genéricos), você provavelmente pode reduzir a taxa de bit de áudio para 96 kbps or algo parecido. Por outro lado, o padrão predefinido que você herda do "iPhone" deve ser bom.

Opções na qualidade de Áudio ~



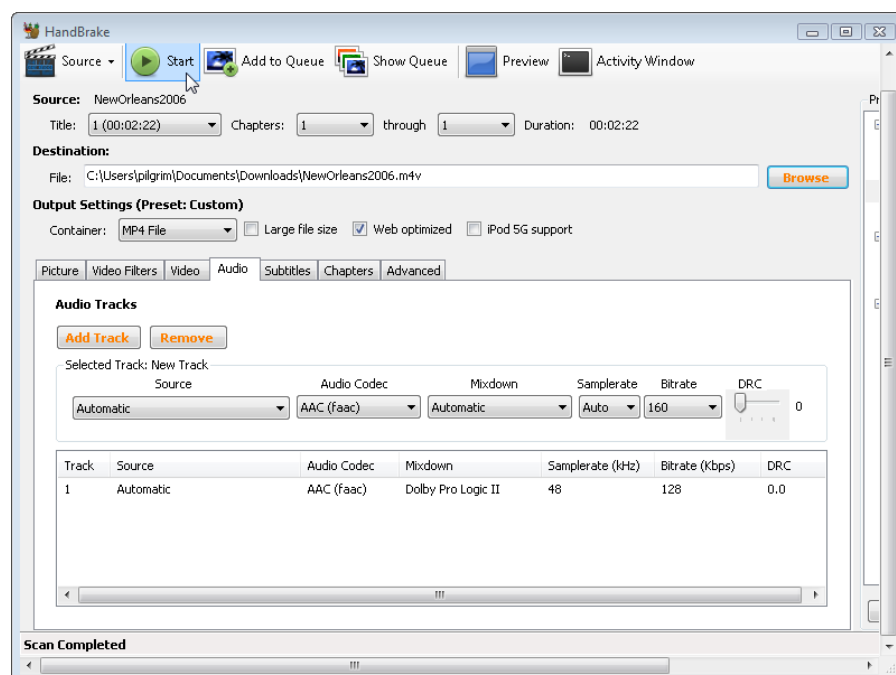
Próximo, clique no botão do "Browse" a escolha o diretório e o nome do arquivo para salvar seu vídeo codificado.

~ Defina o destino e o nome do arquivo



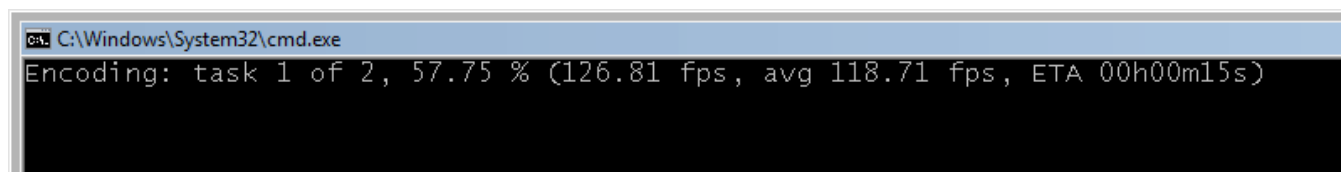
Finalmente, clique "Start" para começar a codificar.

Vamos fazer alguns vídeos! ~



HandBrake irá mostrar a estatística do progresso enquanto codifica seu vídeo.

~ *Paciência, Gafanhoto*



```
C:\Windows\System32\cmd.exe
Encoding: task 1 of 2, 57.75 % (126.81 fps, avg 118.71 fps, ETA 00h00m15s)
```



CODIFICAÇÃO BATCH DE VÍDEOS H.264 COM HANDBRAKE

(Assim como na seção anterior, nesta seção eu vou usar "Vídeo H.264" como abreviação para "Perfil de Linha de base de vídeo H.264 e AAC áudio de baixa-complexidade em um container MP4". Esta é a combinação de codecs + container que funcionam nativamente no Safari, Adobe Flash, iPhone, e nos dispositivos Android.)

HandBrake também vem em uma edição na linha de comandos. Tal como ffmpeg2theora, a edição da linha de comando do HandBrake oferece uma vertiginoso matriz de opções. (Digite `HandBrakeCLI --help` para ler a respeito.) Eu irei focar em apenas alguns:

- `--preset "X"`, onde "X"; é o nome do padrão HandBrake. A predefinição que você quer para vídeo web H.264 é chamada "iPhone" & "iPhone Touch", e é importante colocar o nome inteiro entre aspas.
- `--width W`, onde "W" é a largura do seu vídeo codificado. HandBrake irá automaticamente ajustar a altura para manter a proporção original do vídeo.
- `--vb Q`, onde "Q" é a taxa de bit média (mensurada em kilobits por segundo).
- `--two-pass`, que habilita a codificação em dois passos.
- `--turbo`, que habilita primeiro passo turbo durante a codificação em dois passos.
- `--input F`, onde "F" é o nome do seu vídeo fonte.
- `--output E`, onde "E" é o destino para seu vídeo codificado.

Aqui tem um exemplo da chamada do HandBrake na linha de comando, com marcas na linha de comando que acertam as configurações que escolhemos Com a versão gráfica do HandBrake.

```
you@localhost$ HandBrakeCLI --preset "iPhone & iPod Touch"
                        --width 320
                        --vb 600
                        --two-pass
                        --turbo
```

```
--input pr6.dv
--output pr6.mp4
```

De cima para baixo, estes comandos rodam HandBrake com predefinições "iPhone & iPhone Touch", redimensiona o vídeo para 320x240, define a taxa de bit média para 600 kbps, habilita a codificação em dois passos com primeiro passo turbo, lê o arquivo pr6.dv, e codifica como pr6.mp4. Ufa!



CODIFICANDO VÍDEOS WEBM COM FFMPEG

WebM é completamente suportado por `ffmpeg 0.6+`. Na linha de comando, rode `ffmpeg` sem parâmetros e verifique que foi compilado com suporte VP8:

```
you@localhost$ ffmpeg
FFmpeg version SVN-r23197, Copyright (c) 2000-2010 the FFmpeg developers
  built on May 19 2010 22:32:20 with gcc 4.4.3
  configuration: --enable-gpl --enable-version3 --enable-nonfree --enable-
postproc --enable-pthreads --enable-libfaac --enable-libfaad --enable-
libmp3lame --enable-libopencore-amrnb --enable-libopencore-amrwb --enable-
libtheora --enable-libx264 --enable-libxvid --enable-x11grab --enable-
libvorbis --enable-libvpx
```

Se você não ver as palavras mágicas “`--enable-libvorbis`” e “`--enable-libvpx`,” você não tem a versão certa do `ffmpeg`. (Se você mesmo compilou `ffmpeg`, verifique se não há duas versões instaladas. Isso é tranquilo, eles não irão conflitar entre si. Você precisará apenas usar o caminho completo da versão VP8 do `ffmpeg`.)

Eu vou fazer uma codificação de dois passos. Passo 1 apenas para verificar o arquivo de vídeo inserido (`-i pr6.dv`) e escrever algumas estatísticas no arquivo de log (que será auto-nomeado de `pr6.dv-0.log`). Eu especifico o codec de vídeo com o parâmetro `-vcodec`:

```
you@localhost$ ffmpeg -pass 1 -passlogfile pr6.dv -threads 16 -keyint_min 0
-g 250 -skip_threshold 0 -qmin 1 -qmax 51 -i pr6.dv -vcodec libvpx -b 614400
-s 320x240 -aspect 4:3 -an -y NUL
```

A maioria da linha de comando do `ffmpeg` tem nada a ver com VP8 ou WebM. `libvpx` suporta um número específico de opções VP8 que você pode passar para o `ffmpeg`, mas eu não sei ainda como trabalhar com elas. Assim que encontrar uma boa explicação sobre elas, irei colocar o link aqui e incorporá-la na narrativa se valer a pena.

Para o segundo passo, `ffmpeg` irá ler as estatísticas que foram escritas durante o primeiro passo e realmente codificar o vídeo e áudio. Irá escrever um arquivo `.webm`.

```
you@localhost$ ffmpeg -pass 2 -passlogfile pr6.dv -threads 16 -keyint_min 0
-g 250 -skip_threshold 0 -qmin 1 -qmax 51 -i pr6.dv -vcodec libvpx -b 614400
-s 320x240 -aspect 4:3 -acodec libvorbis -y pr6.webm
```

Existem cinco parâmetros importantes aqui:

- `-vcodec libvpx` especifica que nós estamos codificando com o codec de vídeo VP8. WebM sempre usa vídeo VP8.
- `-b 614400` especifica a taxa de bits. Diferente de outros formatos, `libvpx` espera que a taxa de bits seja mesmo em bits, e não kilobits. Se você quer um vídeo de 600 kbps, multiplique 600 por 1024 e tenha 614400.
- `-s 320x240` especifica o tamanho desejado, largura por altura.
- `-aspect 4:3` especifica a proporção do vídeo. A definição padrão de vídeo normalmente é 4:3, mas a maioria dos vídeos de alta definição são 16:9 ou 16:10. Nos meus testes, encontrei que preciso especificar de forma explícita isso na linha de comando, ao invés de recorrer ao `ffmpeg` para auto-detectar isso.
- `-acodec libvorbis` especifica que nós estamos codificando com codec de áudio Vorbis. WebM sempre usa áudio Vorbis.



ENFIM, A MARCAÇÃO

Eu tenho certeza que esse era pra ser um livro sobre HTML. Então cadê a marcação?

HTML5 lhe dá duas formas de incluir vídeos na sua página web. Ambas envolvem o elemento `<video>`. Se você tem apenas um arquivo de vídeo, você pode simplesmente criar link para ele com o atributo `src`. Isso é muito similar a incluir um vídeo com uma tag ``.

Um arquivo de vídeo ~

```
<video src="pr6.webm"></video>
```

Tecnicamente, isso é tudo que você precisa. Mas assim como uma tag `<img>`, você sempre pode incluir atributos `width` e `height` nas suas tags de `<video>`. Os atributos `width` e `height` podem ser os mesmos que os atributos de largura e altura máxima que você especificou durante o processo de codificação. Não se preocupe se uma dimensão do vídeo é um pouco menor que isso. Seu navegador irá centralizar o vídeo dentro da caixa definida pela tag `<video>`. Nunca será esticado ou achatado fora da proporção.

```
<video src="pr6.webm" width="320" height="240"></video>
```

Por padrão, o elemento `<video>` não irá expor quaisquer controles do player. Você pode criar seus próprios controles com os bons e velhos HTML, CSS, e JavaScript. O elemento `<video>` possui métodos como `play()` e `pause()` e propriedades de leitura/escrita chamados `currentTime`. Também há propriedades de leitura/escrita como `volume` e `muted`. Então você realmente tem tudo o que precisa para criar sua própria interface.

Se você não quer construir sua própria interface, você pode dizer ao navegador para exibir os controles pré-definidos. Para isso, apenas inclua o atributo `controls` na sua tag `<video>`.

```
<video src="pr6.webm" width="320" height="240" controls></video>
```

Há outros dois atributos opcionais que eu quero mencionar antes de ir além: `preload` e `autoplay`. Não dispare o mensageiro; deixe-me explicar porque eles são úteis. O atributo `preload` diz ao navegador que você gostaria de começar o download do arquivo de vídeo assim que a página carregar. Isso faz sentido se o único objetivo da página é visualizar um vídeo. Por outro lado, se é apenas um material suplementar que apenas alguns visitantes irão assistir, então você pode definir o `preload` como `none` para dizer ao navegador que minimize o tráfego de rede.

Aqui está um exemplo de um vídeo que irá começar a fazer o download (mas não tocar) assim que a página carregar:

```
<video src="pr6.webm" width="320" height="240" preload></video>
```

E aqui está um exemplo de um vídeo que *não* irá começar o download assim que a página carregar:

```
<video src="pr6.webm" width="320" height="240" preload="none"></video>
```

O atributo `autoplay` faz exatamente o que parece: diz ao navegador que você gostaria de começar o download do vídeo assim que a página carregar, *e* que você gostaria de começar a tocar o vídeo assim que possível. Algumas pessoas amam isso; outras odeiam. Mas deixe-me explicar por que é importante ter um atributo como esse na HTML5. Algumas pessoas irão querer que seus vídeos toquem automaticamente, mesmo que isso irrite seus visitantes. Se a HTML5 *não* definisse como padrão um modo de tocar automaticamente um vídeo, as pessoas iriam recorrer ao JavaScript com hacks para fazer isso de qualquer forma. (Por exemplo, chamando o método `video's play()` durante o evento da janela de `load`.) Isso poderia ser muito difícil para os usuários contra-atacarem. Por outro lado, basta adicionar uma extensão no seu navegador (ou escrever uma, se necessário) para dizer “ignore o atributo `autoplay`, não quero nunca ver tocar vídeos automaticamente.”

Aqui está um exemplo de vídeo que irá começar o download e tocar assim que for possível, depois de carregar a página:


```
<video src="pr6.webm" width="320" height="240" autoplay></video>
```

E aqui está um script [Greasemonkey](#) que você pode instalar localmente no seu Firefox para prever que o vídeo da HTML5 toque automaticamente. Ele usa o atributo `autoplay` do DOM definido na HTML5, que é equivalente ao atributo `autoplay` da sua marcação HTML. [[disable video autoplay.user.js](#)]

```
// ==UserScript==
// @name          Disable video autoplay
// @namespace      http://diveintomark.org/projects/greasemonkey/
// @description    Ensures that HTML5 video elements do not autoplay
// @include        *
// ==/UserScript==

var arVideos = document.getElementsByTagName('video');
for (var i = arVideos.length - 1; i >= 0; i--) {
    var elmVideo = arVideos[i];
    elmVideo.autoplay = false;
}
```

Mas espere um segundo... Você tem seguido por todo esse capítulo, você não tem apenas um arquivo de vídeo; você tem três. Um é um arquivo `.ogv` criado com [Firefogg](#) ou [ffmpeg2theora](#). O segundo é um arquivo `.mp4` criado com [HandBrake](#). O terceiro é um arquivo `.webm` que você criou com [ffmpeg](#). HTML5 provê um modo de criar links para três deles: o elemento `<source>`. Cada elemento `<video>` pode conter mais que um elemento `<source>`. Seu navegador irá atrás de uma lista de vídeos, em ordem, e tocar apenas o primeiro que estiver habilitado para isso.

Isso levanta outra questão: como o navegador sabe qual vídeo tocar? Bom, no pior cenário, ele carrega cada um dos vídeos e tenta tocar eles. Porém, isso é uma tremenda perda de banda. Você irá economizar muito tráfego de rede se disser primeiro ao navegador sobre cada vídeo. Você diz isso com o atributo `type` no elemento `<source>`.

Aqui está a coisa toda:

Três (!) arquivos de vídeo ~

```
<video width="320" height="240" controls>
  <source src="pr6.mp4" type='video/mp4; codecs="avc1.42E01E, mp4a.40.2"'>
  <source src="pr6.webm" type='video/webm; codecs="vp8, vorbis"'>
  <source src="pr6.ogv" type='video/ogg; codecs="theora, vorbis"'>
</video>
```

Vamos quebrar isso. O elemento `<video>` especifica a largura e altura do vídeo, mas isso não linka para um arquivo de vídeo. Dentro do elemento `<video>` existem três elementos `<source>`. Cada elemento `<source>` linka para um único arquivo de vídeo (com o atributo `src`), e ainda dá informações sobre o formato de vídeo (no atributo `type`).

O atributo `type` parece complicado — droga, e é mesmo complicado. É uma combinação de três pedaços de informações: o formato de container, o codec de vídeo, e o codec de áudio. Vamos começar por baixo. Para o arquivo de vídeo `.ogv`, o formato de container format é Ogg, representado por `video/ogg`. (T tecnicamente falando, isso é o MIME type para arquivos de vídeo Ogg.) O codec de vídeo é Theora, e o codec de áudio é Vorbis. Isso é simples o suficiente, exceto pelo formato do valor do atributo parecer meio estranho. O valor por si precisa incluir marcações por aspas, que significa que você precisa usar um tipo diferente de marcação por aspas para todo valor.

```
<source src="pr6.ogv" type='video/ogg; codecs="theora, vorbis"'>
```

WebM é praticamente a mesma coisa, mas com um diferente MIME type (`video/webm` ao invés de `video/ogg`) e um diferente codec de vídeo (`vp8` ao invés de `theora`) listados no parâmetro de `codecs`.

```
<source src="pr6.webm" type='video/webm; codecs="vp8, vorbis"'>
```

O vídeo H.264 é ainda mais complicado. Lembra quando eu disse que ambos vídeo H.264 e áudio AAC podem vir com diferentes “perfis”? Nós codificamos com o perfil “baseline” H.264 e com o perfil “low-complexity” AAC, quando envolvidos em um container MPEG-4. Toda sua informação está inclusa no atributo `type`.

```
<source src="pr6.mp4" type='video/mp4; codecs="avc1.42E01E, mp4a.40.2"'>
```

O benefício de passar por todos esses problemas é que o navegador irá checar o atributo `type` primeiro para poder tocar um arquivo de vídeo em particular. Se um navegador decide que não pode tocar um vídeo em particular, *ele não irá realizar o download do arquivo*. Nem mesmo uma parte dele. Você irá economizar banda, e seus visitantes irão ver o vídeo que eles vieram pra ver, mais rápido.

Se você seguir as instruções nesse capítulo para codificar seus vídeos, você pode apenas copiar e colar os valores dos atributos `type` desse exemplo. Ou então, você terá que lidar com parâmetros type você mesmo.



iPads rodando iOS 3.x tem um bug que previnem eles de notificar qualquer coisa menos a primeira origem do vídeo listado. iOS 4 (um upgrade gratuito para todos iPads) corrige esse bug. Se você quer entregar vídeo para usuários de iPad que ainda não tenham atualizado para iOS 4, você terá que listar seu arquivo MP4 primeiro, seguido dos formatos de vídeo gratuitos. *Sigh*.



Existem tantas peças no quebra-cabeça do vídeo, eu até mesmo hesitado em trazer isso à tona. Mas é importante, porque uma configuração mal feita no web server pode levar a incontáveis frustrações quando você tenta debugar o porquê do seu vídeo tocar localmente no seu computador mas falhar ao tentar tocar quando você realiza o deploy para o site em produção. Se você se deparar com esse problema, a principal causa é provavelmente MIME types.

Eu mencionei MIME types [na história desse capítulo](#), mas você provavelmente passou por aquilo sem dar a importância devida. Então aqui está em:

ARQUIVOS DE VÍDEO DEVEM SER SERVIDOS COM O MIME TYPE CORRETO!

Qual é o MIME type correto? Você já viu ele; é uma parte do valor do atributo type no elemento <source>. Mas definir o atributo type no seu HTML não é o suficiente. Você também precisa garantir que seu web server inclui o MIME type correto no Content-Type do cabeçalho HTTP.

Se você está usando um servidor web Apache ou qualquer derivado Apache, você pode usar uma diretiva AddType no httpd.conf do seu site ou em um arquivo .htaccess no diretório onde você armazenou seus arquivos de vídeo. (Se você usa outro servidor web, consulte a documentação dele sobre como definir Content-Type no cabeçalho HTTP para tipos específicos de arquivo.)

```
AddType video/ogg .ogg  
AddType video/mp4 .mp4  
AddType video/webm .webm
```

A primeira linha é para vídeos com container Ogg. A segunda linha é para vídeos com container MPEG-4. A terceira é para WebM. Defina uma vez e esqueça isso. Se você esquecer de

definir isso, seus vídeos *irão* falhar ao tocar em alguns navegadores, mesmo que você inclua o MIME no atributo type do seu HTML.

Para ainda mais detalhes sobre como configurar seu servidor web, eu direciono sua atenção para esse excelente arquivo na Mozilla Developer Center: [Configurando servidores para mídia Ogg](#). (O conselho desse artigo se aplica para vídeo MP4 e WebM também.)



E QUANTO AO IE?

Internet Explorer 9 suporta o elemento <video> da HTML5, mas a Microsoft prometeu publicamente que a versão final do IE 9 irá suportar vídeo H.264 video e áudio AAC em um container MPEG-4, assim como Safari e iPhone.

Mas e quanto as versões antigas do Internet Explorer? Tipo, você sabe, todas as versões antigas incluindo o IE 8? A maioria das pessoas que usam Internet Explorer também possuem um plugin Adobe Flash plugin instalado. Versões modernas do Adobe Flash (começando com 9.0.60.184) suportam vídeo H.264 e áudio AAC em um container MPEG-4, assim como Safari e iPhone. Uma vez codificado com vídeo H.264 para Safari, você pode tocar isso em um video player baseado em Flash se você detectar que um dos seus visitantes não possuem navegadores com suporte ao HTML5.

FlowPlayer é de código livre, licenciado com GPL, com video player baseado em Flash. (Licenças comerciais também estão disponíveis.) FlowPlayer não sabe nada sobre seu elemento <video>. Não irá magicamente transformar uma tag <video> em um objeto Flash. Mas um HTML5 bem feito sabe como lidar com isso, porque você pode colocar um elemento <object> dentro do <video>. Navegadores que não suportam vídeo HTML5 irão ignorar o elemento <video> e simplesmente renderizar o elemento <object>, no qual irá invocar o plugin Flash e tocar o vídeo usando FlowPlayer. Navegadores que suportam vídeo HTML5 irão encontrar a origem do vídeo que eles pode tocar e tocá-los, e *ignorar o elemento <object> junto com isso.*

Esse último pedaço é a chave para todo quebra-cabeça: HTML5 especifica que todos os elementos (além dos elementos <source>) são filhos de um elemento <video> devem ser ignorados. Isso permite que você possa usar vídeos HTML5 em navegadores mais novos e

prover um fall back em Flash para navegadores mais antigos, sem precisar de qualquer hack JavaScript. Você pode ler mais sobre essa técnica em: [Vídeos Para Todos](#).



PROBLEMAS EM IPHONES E IPADS

iOS é o sistema operacional da Apple's que equipa iPhones, iPod Touches, e iPads. iOS 3.2 tem um número grande de problemas com vídeo HTML5.

1. iOS não irá reconhecer o vídeo se você incluir um atributo `poster`. O atributo `poster` do elemento `<video>` permite exibir uma imagem personalizada enquanto o vídeo é carregado, ou até que o usuário pressione o "play." Esse bug é consertado no iOS 4.0, mas irá levar tempo para usuários atualizarem-se.
2. Se você tem múltiplos elementos `<source>`, iOS irá reconhecer qualquer coisa menos o primeiro. Já que dispositivos iOS suportam apenas H.264+AAC+MP4, isso é efetivamente significa que você precisa listar seu MP4 primeiro. Esse bug também é corrigido no iOS 4.0.



PROBLEMAS EM DISPOSITIVOS ANDROID

Android é o sistema operacional da Google's que equipa diferentes telefones e dispositivos de mão. Versões do Android antes da 2.3 tinham uma série de problemas com vídeo HTML5.

1. O atributo `type` nos elementos `<source>` confundem muito o Android. A única forma para reconhecer uma origem de vídeo é, ironicamente, omitindo o atributo `type` e garantir que arquivos de vídeo H.264+AAC+MP4 terminem com o formato `.mp4`. Você pode ainda incluir o atributo `type` nas outras origens de vídeo, já que H.264 é o único formato de vídeo que Android 2.2 suportam. (Esse bug é corrigido no Android 2.3.)
2. O atributo `controls` não foi suportado. Não há efeitos problemáticos nisso, mas o Android não irá mostrar qualquer controle de interface para um vídeo. Você irá precisar do seu próprio controle na interface. No mínimo, você deveria prover um script que comece tocando o vídeo quando o usuário clicar no vídeo. Esse bug também é corrigido no Android 2.3.



UM COMPLETO E VIVO EXEMPLO

Aqui está um exemplo vivo de um vídeo que usa essas técnicas. Eu extendi o código do "Vídeo para Todos" para incluir suporte a vídeo WebM. Codifiquei a mesma origem do vídeo em três formatos, com esses comandos:

```
## Theora/Vorbis/Ogg
you@localhost$ ffmpeg2theora --videobitrate 200 --max_size 320x240 --output
pr6.ogv pr6.dv

## H.264/AAC/MP4
you@localhost$ HandBrakeCLI --preset "iPhone & iPod Touch" --vb 200 --width
320 --two-pass --turbo --optimize --input pr6.dv --output pr6.mp4

## VP8/Vorbis/WebM
you@localhost$ ffmpeg -pass 1 -passlogfile pr6.dv -threads 16 -keyint_min 0
-g 250 -skip_threshold 0 -qmin 1 -qmax 51 -i pr6.dv -vcodec libvpx -b 204800
-s 320x240 -aspect 4:3 -an -f webm -y NUL
you@localhost$ ffmpeg -pass 2 -passlogfile pr6.dv -threads 16 -keyint_min 0
-g 250 -skip_threshold 0 -qmin 1 -qmax 51 -i pr6.dv -vcodec libvpx -b 204800
-s 320x240 -aspect 4:3 -acodec libvorbis -ac 2 -y pr6.webm
```

A marcação final usa o elemento <video> para HTML5, um elemento <object> dentro para fallback em Flash, e um pequeno pedaço de script para beneficiar dispositivos Android:

```
<video id="movie" width="320" height="240" preload controls>
  <source src="pr6.webm" type='video/webm; codecs="vp8, vorbis"' />
  <source src="pr6.ogv" type='video/ogg; codecs="theora, vorbis"' />
  <source src="pr6.mp4" />
  <object width="320" height="240" type="application/x-shockwave-flash"
    data="flowplayer-3.2.1.swf">
    <param name="movie" value="flowplayer-3.2.1.swf" />
    <param name="allowfullscreen" value="true" />
    <param name="flashvars" value='config={"clip": {"url":
"http://wearehugh.com/dih5/pr6.mp4", "autoPlay":false,
"autoBuffering":true}}' />
    <p>Download video as <a href="pr6.mp4">MP4</a>, <a
href="pr6.webm">WebM</a>, or <a href="pr6.ogv">Ogg</a>.</p>
  </object>
</video>
<script>
  var v = document.getElementById("movie");
  v.onclick = function() {
    if (v.paused) {
      v.play();
    } else {
      v.pause();
    }
  };
</script>
```

Com essa combinação de HTML5 e Flash, você deve conseguir assistir esse vídeo em quase todos os navegadores e dispositivos:



- [HTML5: The <video> element](#)
- [Video for Everybody](#)
- [A gentle introduction to video encoding](#)
- [Theora 1.1 is released — what you need to know](#)
- [Configuring servers for Ogg media](#)
- [Encoding with the x264 codec](#)
- [Video type parameters](#)
- [Everything you need to know about HTML5 audio and video](#)
- [Making HTML5 video work on Android phones. *Le sigh.*](#)
- [Internet Explorer 9 Guide for Developers: HTML5 video and audio elements](#)

Controles customizados para vídeo HTML5:

- [VideoJS](#)
- [MediaElement.js](#)
- [Kaltura HTML5 Video & Media JavaScript Library](#)

Nº6

VOCÊ ESTÁ AQUI

(ASSIM COMO TODO MUNDO ESTÁ)

MERGULHANDO



Geolocalização é a arte de descobrir aonde você está no mundo e (opcionalmente) compartilhar essa informação com as pessoas que você confia. Há mais de uma maneira de descobrir aonde você está — seu endereço IP, sua conexão de rede sem fio, com que torre de celular seu telefone está falando, ou hardware GPS dedicado que calcula latitude e longitude da informação enviada por satélites no céu.

P: Geolocalização parece assustador. Posso desligá-la?

R: Privacidade é uma preocupação óbvia quando se está falando sobre compartilhar sua localização física com um servidor web remoto. A API de geolocalização diz explicitamente: “User Agents não devem enviar informação de localização para sites na Web sem a expressa permissão do usuário.” Em outras palavras, compartilhar a sua localização é sempre opcional. Se você não quiser, você não tem que fazê-lo.

API DE GEOLOCALIZAÇÃO

A API de geolocalização permite que você compartilhe sua localização em sites confiáveis. A latitude e longitude são disponibilizadas na página via JavaScript, que por sua vez pode ser enviado a um servidor web e fazer coisas como encontrar locais ao seu redor ou mostrar sua posição em um mapa.

Como você pode ver na tabela a seguir, a API de geolocalização é suportada pela maioria dos navegadores desktop e dispositivos móveis. Além disso, vários navegadores e dispositivos antigos podem oferecer suporte via bibliotecas de terceiros, como veremos depois neste capítulo.

SUPORTE DA API DE GEOLOCALIZAÇÃO						
IE	FIREFOX	SAFARI	CHROME	OPERA	IPHONE	ANDROID
9.0+	3.5+	5.0+	5.0+	10.6+	3.0+	2.0+

Junto com o suporte padrão da API de geolocalização, há também várias APIs de dispositivos específicos em outras plataformas móveis. Falarei sobre tudo isso ainda neste capítulo.



MOSTRE-ME O CÓDIGO

A API de geolocalização centraliza tudo em volta da sua nova propriedade global chamada `navigator` com o objeto: `navigator.geolocation`.

O simples uso da API de geolocalização se parece com isso:

```
function get_location() {
    navigator.geolocation.getCurrentPosition(show_map);
}
```

Porém isso não detecta o posicionamento (latitude e longitude), não há tratamento de erros e não retorna opções. Você pode incluir uma verificação para detectar se há suporte para a API de geolocalização. Para detectar se há suporte, você pode usar o Modernizr:

```
function get_location() {
    if (Modernizr.geolocation) {
        navigator.geolocation.getCurrentPosition(show_map);
    } else {
        // no native support; maybe try Gears?
    }
}
```

“EU POSSO TER GEOLOCALIZAÇÃO?”

O que você faz sem o suporte da geolocalização é contigo. Eu explicarei como fazer um fallback em um minuto, mas antes eu quero falar sobre o que acontecerá *durante* a chamada do método `getCurrentPosition()`. Como eu mencionei no começo deste capítulo, o suporte da geolocalização é *opcional*. Isso significa que seu browser nunca irá forçar você revelar sua localização física atual para um servidor remoto. A experiência do usuário é diferente de browser pra browser. No Mozilla Firefox, a chamada do método `getCurrentPosition()` da API de geolocalização fará com que o browser mostre uma “barra de notificação” no topo da janela do navegador. Essa barra se parece com isso:



Há muita coisa acontecendo aqui. Você, como um usuário final,

- é dito que um site pergunta sua localização
- é dito *qual* o site que pergunta sua localização
- em Mozilla's "Location-Aware Browsing" explicará que diabos está acontecendo (lembre-se: o Google fornece sua localização e salva seus dados de acordo com sua Política de Privacidade sobre Serviços de Localização)
- pode escolher para compartilhar sua localização
- pode escolher para *não* compartilhar sua localização
- pode falar pro seu browser para lembrar sua escolha (espera aí, compartilhar ou não compartilhar sua localização) então você nunca mais verá está barra de notificação novamente neste site

Além disso, essa barra é

- não modal, então não irá impedi-lo de mudar para uma outra janela ou tab
- tab específica, irá desaparecer se você mudar para uma outra janela ou tab e reaparecerá quando você voltar para a janela ou tab em que estava
- incondicional, então não há como um site roubar sua localização sem sua permissão
- segura, não há chance de um site poder determinar sua localização enquanto ele estiver esperando sua resposta

Você acabou de ver o código em JavaScript com que fez essa barra de notificação aparecer. É uma chamada de função única que tem uma função de callback (no qual chamei de `show_map`). A chamada do método `getCurrentPosition()` retornará a localização imediatamente, mas isso não quer dizer que você terá a localização do usuário. O primeiro lugar que é garantido para você o acesso a localização do usuário é na função de callback. A função de callback se parece com isso:

```
function show_map(position) {  
    var latitude = position.coords.latitude;  
    var longitude = position.coords.longitude;  
    // let's show a map or do something interesting!  
}
```

A função de callback será chamada com um único parâmetro, e retornará um objeto com duas propriedades: `coords` e `timestamp`. O `timestamp` é somente a data e o horário de quando a localização foi calculada. (Uma vez que tudo isso está acontecendo de forma assíncrona, você poderá realmente não saber quando irá acontecer com antecedência. Pode demorar algum tempo para que o usuário visualize a barra de notificação e aceite compartilhar sua localização. Dispositivos com GPS dedicado podem demorar mais tempo para se conectar com o satélite do GPS. E assim por diante.) O objeto `coords` possui propriedades

como `latitude` e `longitude` que são exatamente o que parece ser: a localização física do usuário no mundo.

POSIÇÃO DO OBJETO		
Propriedade	Tipo	Observação
<code>coords.latitude</code>	<code>double</code>	graus decimais
<code>coords.longitude</code>	<code>double</code>	graus decimais
<code>coords.altitude</code>	<code>double</code> ou <code>null</code>	metros em elipsoide de referência
<code>coords.accuracy</code>	<code>double</code>	metros
<code>coords.altitudeAccuracy</code>	<code>double</code> ou <code>null</code>	metros
<code>coords.heading</code>	<code>double</code> ou <code>null</code>	graus em sentido do norte verdadeiro
<code>coords.speed</code>	<code>double</code> ou <code>null</code>	metros/segundos
<code>timestamp</code>	<code>DOMTimeStamp</code>	como um objeto de <code>Date()</code>



Somente três de uma das propriedades são garantidas (`coords.latitude`, `coords.longitude`, e `coords.accuracy`). O restante pode voltar `null`, dependendo da capacidade do seu dispositivo e a distância que o servidor está de você. As propriedades `heading` e `speed` serão calculadas baseadas na posição anterior do usuário, se possível.



LIDANDO COM ERROS

Geolocalização é complicado. Coisas podem dar errado. Eu havia dito que o “consentimento do usuário” é estranho. Se sua aplicação precisa da localização do usuário mas o usuário não permitir, você está ferrado. O usuário sempre ganha. Se ele não permitir, o que apareceria

como mensagem? Veja o segundo parâmetro do método `getCurrentPosition()`: um erro é chamado na função de callback.

```
navigator.geolocation.getCurrentPosition(
  show_map, handle_error)
```

Se qualquer coisa sair errado, sua função callback de erro será chamada com um objeto `PositionError`.

OBJETO POSITIONERROR		
Propriedade	Tipo	Observações
code	short	um valor enumerado
message	DOMString	não usado para usuários finais

A propriedade `code` retornará um desses resultados

- `PERMISSION_DENIED` (1) se o usuário clicar em “não compartilhar” ou qualquer outra coisa que bloqueará seu acesso e sua localização.
- `POSITION_UNAVAILABLE` (2) se a rede cair ou a posição do satélite não poder ser resgatada.
- `TIMEOUT` (3) se a rede não cair mas o tempo para calcular a posição do usuário for longo demais. Como saber o que é “longo demais”? Vou mostrar como definir isso numa próxima seção.

↪ *Seja educado na derrota*

```
function handle_error(err) {
  if (err.code == 1) {
    // user said no!
  }
}
```

P: A API de geolocalização pega na Estação Espacial Internacional, na Lua, ou em outros planetas?

R: Os estados específicos da geolocalização, “O sistema de referência geográfica de coordenadas usado pelos atributos é referente ao World Geodetic System (2d) [WGS84]. Nenhum outro sistema de referência é suportado.” A Estação Espacial Internacional está em órbita na Terra, então os astronautas na estação podem dizer suas localizações por latitude, longitude e altitude. No entanto, o World Geodetic System é centralizado somente na Terra, por isso não podemos usar para saber localizações na Lua ou em outros planetas.



ESCOLHAS! EU QUERO ESCOLHAS!

Alguns aparelhos — como iPhone e Android — suportam *dois* métodos para mostrar sua localização. O primeiro método triangula a sua posição baseando-se na sua localização relativa das diferentes torres da sua operadora de celular. Este método é rápido e não necessita de qualquer hardware de GPS dedicado, mas ele só pega uma ideia aproximada de onde você está.



O segundo método atualmente usa algum hardware de GPS dedicado em seu aparelho para se comunicar com algum satélite de GPS dedicado que está orbitando na Terra. O GPS normalmente pode identificar a sua localização a poucos metros. O lado negativo de um chip de GPS dedicado em seu aparelho é que consome muita energia, então telefones e outros dispositivos geralmente desligam esse chip quando precisa. Isso significa que terá um atraso quando o chip for inicializado para se conectar com o satélite. Se você sempre usa o Google Maps em um iPhone ou outro smartphone, você já viu os dois métodos em ação. Primeiro você vê um grande círculo que aproxima sua posição (procurando uma torre de celular próxima), então um círculo menor (triangulando com outras torres de celulares), então um único ponto com sua posição exata (pego por um satélite de GPS).

A razão de eu falar isso é que, dependendo da sua aplicação web, você talvez não terá uma grande precisão. Se você estiver somente procurando por alguns cinemas nas proximidades, uma precisão menor provavelmente será o suficiente pra você. Não há muitas salas de cinema, mesmo em cidades maiores, e você provavelmente vai listar mais de uma. Por outro lado, se voce está querendo direções em tempo real, você realmente terá que saber onde o usuário está exatamente para poder dizer “vire à direita em 20 metros” ou qualquer outra coisa.

O método `getCurrentPosition()` tem como um terceiro argumento opcional, um objeto `PositionOptions`. Existem três propriedades que você pode definir no objeto `PositionOptions`. Todas as propriedades são opcionais. Você pode definir qualquer uma ou todas, ou nenhuma delas.

OBJETO POSITIONOPTIONS			
Propriedade	Tipo	Padrão	Observações

<code>enableHighAccuracy</code>	Booleano	false	true pode ser mais lento
<code>timeout</code>	long	(sem padrão)	em milissegundos
<code>maximumAge</code>	long	0	em milissegundos

A propriedade `enableHighAccuracy` é exatamente o que se parece. Se for verdadeiro, e o dispositivo tiver suporte para tal, e o usuário permitir compartilhar sua posição, então o dispositivo irá tentar fornecer uma precisão maior. Tanto iPhones quanto Androids tem uma permissão separada para baixa e alta precisão, por isso, é possível que a chamada `getCurrentPosition()` com `enableHighAccuracy:true` possa falhar, mas chamar com `enableHighAccuracy:false` poderá dar certo.

A propriedade `timeout` é um número em milissegundos que sua aplicação irá esperar pela posição. Este tempo não será contado *antes* do usuário dar permissão para tentar calcular sua posição. Não é o tempo do usuário; é o tempo da rede.

A propriedade `maximumAge` permite que os dispositivos respondam imediatamente com uma posição em cache. Por exemplo, vamos chamar o método `getCurrentPosition()` por um período, o usuário permitiu, e a função de callback chamou a posição que foi calculada exatamente às 10:00 horas da manhã, você chamou o método `getCurrentPosition()` novamente com a propriedade `maximumAge` em 75000.

```
navigator.geolocation.getCurrentPosition(
    success_callback, error_callback, {maximumAge: 75000});
```

O que você está dizendo é que você não precisa necessariamente da posição atual do usuário. Você provavelmente ficará satisfeito sabendo onde ele esteve em 75 segundos atrás (75000 milissegundos). O dispositivo sabe onde o usuário foi em 60 segundos atrás (60000 milissegundos), porque sua posição foi calculada antes da primeira chamada do método `getCurrentPosition()`. Então o dispositivo não precisa recalcular a localização do usuário novamente. Ele somente retorna exatamente a mesma informação que foi retornada na primeira chamada: mesma latitude, mesma longitude, mesma precisão, e mesmo timestamp (10:00 da manhã).



Antes que você pergunte pela localização do usuário, deve-se pensar sobre o quanto de precisão você precisa, e definir `enableHighAccuracy` de acordo. Se você precisa encontrar sua localização mais de uma vez, você deve pensar sobre como uma informação antiga pode vir ser útil, e definir `maximumAge` de acordo. Se você precisa encontrar sua localização *continuamente*, então o método `getCurrentPosition()` não é pra você. Você precisa utilizar o método `watchPosition()`.

O método `watchPosition()` tem a mesma estrutura que o método `getCurrentPosition()`. Tem duas funções de callback, uma necessária para sucesso e uma opcional para qualquer erro que possa dar, e também pode ter um objeto opcional `PositionOptions` que terá todas as mesmas propriedades que você aprendeu. A diferença é que sua função de callback irá ser chamada *toda vez que a localização do usuário mudar*. Não há necessidade de pesquisar sua posição toda hora. O dispositivo irá determinar o melhor intervalo pra pesquisa da localização, e ele irá chamar sua função de callback sempre que a posição do usuário alterar. Você pode usar isto para atualizar um marcador em um mapa, fornecendo instruções sobre onde deve ir, ou o que quiser.

O método `watchPosition()` retornará sempre um número. Você provavelmente deve guardar este número em algum lugar. Se você quiser parar de ver a mudança de localização do usuário, você pode chamar o método `clearWatch()` e passar este número, e o dispositivo irá parar de chamar a função de callback. Isso funciona de forma parecida com as funções Javascript `setInterval()` e `clearInterval()`.



E O IE?

Antes da versão 9 (tecnicamente 9.0RC1), o Internet Explorer não tem suporte para a API de geolocalização da W3C. Mas não se desespere! Gears é um plugin de código aberto da Google que funciona em Windows, Mac, Linux, Windows Mobile, e Android, que fornece recursos para os navegadores antigos. Um desses recursos que o Gears fornece é a API de geolocalização. Ele não é bem a API de geolocalização da W3C, mas tem o mesmo resultado.

Enquanto estamos em um assunto de plataformas legadas, gostaria de deixar bem claro que as plataformas antigas tem suas próprias APIs de geolocalização. BlackBerry, Nokia, Palm, e OMTP BOMDI todas fornecem suas próprias APIs. É claro, todas elas trabalham de forma diferente do Gears, que por sua vez funciona de forma diferente da API de geolocalização da W3C. Wheeeeeee!



GEO.JS AO RESGATE

`geo.js` é uma biblioteca JavaScript, de código aberto, com licença MIT que facilita o uso entre as diferentes APIs de geolocalização da W3C, a Gears API, e as APIs fornecidas pelas próprias plataformas. Para usá-la, você irá precisar adicionar dois scripts no rodapé da sua página. (T tecnicamente, você poderia colocar eles em qualquer lugar, mas scripts no `<head>` vão fazer com que sua página carregue mais devagar. Então, não faça isso!)

O primeiro script é o `gears_init.js`, que inicializará o Gears caso estiver instalado. O segundo script é o `geo.js`.

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <title>Dive Into HTML5</title>
</head>
<body>
  ...
  <script src="gears_init.js"></script>
<script src="geo.js"></script>
</body>
</html>
```

« *Não deixe isso no seu `<head>`*

Agora você está pronto para usar qualquer API de geolocalização.

```
if (geo_position_js.init()) {
  geo_position_js.getCurrentPosition(geo_success, geo_error);
}
```

Vamos começar devagar. Primeiro, você precisa chamar o método `init()`. O método `init()` retorna `true` se o suporte para a API de geolocalização estiver disponível.

```
if (geo_position_js.init()) {
```

Chamando o método `init()` ele não irá buscar sua localização. Ele somente verifica que é possível procurar sua localização. Para encontrar sua localização, você precisa chamar o método `getCurrentPosition()`.


```
geo_position_js.getCurrentPosition(geo_success, geo_error);
```

O método `getCurrentPosition()` irá fazer seu navegador pedir sua permissão para encontrar e compartilhar sua localização. Se sua geolocalização está sendo fornecida pelo Gears, irá aparecer uma janela perguntando se você permite o Gears utilizar alguns dados. Se seu navegador tem suporte nativo para geolocalização, a janela que irá aparecer, é diferente. Por exemplo, o Firefox 3.5 tem suporte nativo para geolocalização. Se você tentar procurar sua localização no Firefox 3.5, aparecerá uma barra de notificação no topo da página perguntando se você quer compartilhar sua localização com este site.

O método `getCurrentPosition()` passa dois parâmetros, ambos funções. Se o método `getCurrentPosition()` conseguiu encontrar sua localização — isto é, se você deu permissão, a API de geolocalização irá funcionar como mágica — a função passada no primeiro parâmetro será chamada. Neste exemplo, a função de callback de sucesso é chamada de `geo_success`.

```
geo_position_js.getCurrentPosition(geo_success, geo_error);
```

A função de callback tem um único parâmetro, e que contém a posição do usuário.

↪ *Chamada de sucesso*

```
function geo_success(p) {  
    alert("Encontrado na latitude " + p.coords.latitude +  
        ", longitude " + p.coords.longitude);  
}
```

Se o método `getCurrentPosition()` não encontrar sua localização — porque você não deu permissão, ou a API de geolocalização falhou por algum motivo — será chamada a função no segundo parâmetro. Neste exemplo, o callback de falha é chamado de `geo_error`.

```
geo_position_js.getCurrentPosition(geo_success, geo_error);
```

O callback de falha não tem parâmetros

↪ *Chamada de erro*

```
function geo_error() {  
    alert("Não conseguimos encontrar você!");  
}
```

`geo.js` não tem suporte para o método `watchPosition()`. Se você precisar da localização contínua, irá precisar ativar o método `getCurrentPosition()` você mesmo.



UM EXEMPLO COMPLETO E REAL

Aqui está um exemplo real usando a `geo.js` para tentar pegar sua localização e mostrar um mapa de onde você está quase que instantaneamente:



Seu navegador suporta geolocation. [Clicar para ver sua localização.](#)

Como isso funciona? Vamos dar uma olhada. No carregamento da página, esta página chama `geo_position_js.init()` para determinar se a geolocalização está disponível em qualquer uma das interfaces que `geo.js` suporta. Se tiver suporte, ele cria um link que você pode clicar para ver sua localização. Clicando no link é chamado então o método `lookup_location()`, aqui está:

```
function lookup_location() {  
    geo_position_js.getCurrentPosition(show_map, show_map_error);  
}
```

Se você permitir rastrear sua localização, e o serviço de backend conseguir determinar sua localização, `geo.js` chama a primeira função de callback, `show_map()`, com um único parâmetro, `loc`. O objeto `loc` tem uma propriedade `coords` no qual contém sua latitude, longitude e precisão. (Este exemplo não usa a precisão.) O resto do método `show_map()` usa a API de mapas do Google Maps para mostrar um mapa.

```
function show_map(loc) {  
    $("#geo-wrapper").css({'width':'320px','height':'350px'});  
    var map = new GMap2(document.getElementById("geo-wrapper"));  
    var center = new GLatLng(loc.coords.latitude, loc.coords.longitude);  
    map.setCenter(center, 14);  
    map.addControl(new GSmallMapControl());  
    map.addControl(new GMapTypeControl());  
}
```

```
    map.addOverlay(new GMarker(center, {draggable: false, title: "Você está  
aqui (mais ou menos)"}));  
}
```

Se `geo.js` não conseguir determinar sua localização, ele chamará o segundo callback, `show_map_error()`.

```
function show_map_error() {  
    $("#live-geolocation").html('Não foi possível determinar sua  
localização.');
```



LEITURA ADICIONAL

- [W3C geolocation API](#)
- [Gears](#)
- [BlackBerry geolocation API](#)
- [Nokia geolocation API](#)
- [Palm geolocation API](#)
- [OMTP BOMDI geolocation API](#)
- [geo.js](#), the geolocation API wrapper script
- [Internet Explorer 9 Guide for Developers: Geolocation](#)

O PASSADO, PRESENTE & FUTURO DE ARMAZENAMENTO LOCAL PARA APLICAÇÕES WEB MERGULHANDO



Armazenamento local persistente é uma das áreas onde aplicações nativas de cliente têm mantido uma vantagem sobre aplicações web. Para aplicações nativas, o sistema operacional tipicamente provê uma camada de abstração para armazenar e recuperar dados específicos do aplicativo como preferências ou estado de tempo de execução. Estes valores podem ser armazenados no registro, arquivos INI, arquivos XML ou algum outro lugar de acordo com a convenção da plataforma. Se o seu aplicativo cliente nativo necessita de armazenamento local diferente dos pares chave/valor, você pode inserir o seu próprio banco de dados, inventar o seu próprio formato de arquivo ou qualquer número de outras soluções.

Historicamente, aplicações web não tiveram nenhum desses luxos. Cookies foram inventados no início da história da web e, de fato, eles podem ser usados para armazenamento local persistente de pequenas quantidades de dados. Mas eles têm três desvantagens potencialmente [potentially dealbreaking downsides]:

- Cookies são incluídos em toda requisição HTTP, atrasando assim sua aplicação web por desnecessariamente sempre transmitir os mesmos dados;
- Cookies são incluídos em toda requisição HTTP, enviando assim dados não criptografados através da internet (a menos que sua aplicação web inteira seja servida sobre SSL);
- Cookies são limitados a certa de 4KB de dados — suficiente para atrasar sua aplicação (ver acima), mas não o suficiente para ser muito útil;

O que nós realmente queremos é

- bastante espaço de armazenamento
- no cliente
- que persista depois da atualização da página
- e não seja transmitido para o servidor

Antes da HTML5, todas as tentativas de obtenção foram fundamentalmente insatisfatórias de diferentes maneiras.



UMA BREVE HISTÓRIA DOS HACKS DE ARMAZENAMENTO LOCAIS ANTES HTML5

No início, existia apenas o Internet Explorer. Ou ao menos, era isso que a Microsoft gostaria que o mundo acreditasse. Para esse fim, como parte da Primeira Grande Guerra de Browsers, a Microsoft inventou uma grande quantidade de coisas e as incluiu no seu "vencedor de todas as batalhas", o Internet Explorer. Uma dessas coisas foi conhecida como DHTML Comportamentos, e um desses comportamentos foi conhecido como userData.

userData possibilita que páginas web armazenem até 64KB de dados por domínio, em uma estrutura hierárquica baseada em XML. (Domínios confiáveis, tais como sites de intranet, podem armazenar 10 vezes esta quantidade. E, 640 KB deveria ser o suficiente para qualquer um.) O IE não apresenta qualquer forma de diálogo de permissões e não há previsão para o aumento da quantidade de armazenamento disponível.

Em 2002, a Adobe introduziu uma funcionalidade no Flash 6 que ganhou o infeliz e enganoso nome de "Flash cookies." Dentro do ambiente do Flash, a funcionalidade é propriamente conhecida como Local Shared Objects - Objetos de local compartilhado. Resumidamente, isso permitia que objetos Flash armazenassem até 100KB de dados por domínio. Brad Neuberg desenvolveu um pré protótipo de uma ponte Flash-to-JavaScript chamada AMASS (AJAX Massive Storage System), porém ficou limitado por algumas peculiaridades de design do Flash. Em 2006, com o advento da ExternalInterface presente no Flash 8, acessar LSOs a partir do JavaScript tornou-se uma coisa de magnitude mais fácil e rápida. Brad reescreveu o AMASS e o integrou dentro do popular Dojo Toolkit com o nome de `dojox.storage`. Flash garante a cada domínio 100 KB de armazenamento "livre." Além disso, ele solicita ao usuário para cada ordem de magnitude um aumento no armazenamento de dados (1 Mb, 10 Mb, e assim por diante).

Em 2007, o Google lançou o Gears, um plugin de código aberto com o objetivo de providenciar capacidades adicionais nos navegadores. (Nós já tínhamos discutido sobre Gears no contexto de providenciar uma API de geolocalização para o Internet Explorer.) Gears possibilita uma API para uma base de dados SQL incorporada baseada em SQLite. Após obter a permissão de um usuário uma vez, Gears pode armazenar ilimitada quantidade de dados por um domínio em uma tabela de banco de dados SQL.

Entretanto, Brad Neuberg e outros continuaram a trabalhar em cima do `dojox.storage` para providenciar uma interface unificada para todos esses diferentes plugins e APIs. Em 2009, `dojox.storage` podia auto-detectar (e providenciar uma interface unificada em cima de) Adobe Flash, Gears, Adobe AIR, e um pré protótipo da HTML5 storage que era apenas implementado em antigas versões do Firefox.

Quando você analisa essas soluções, um padrão surge: todos eles são específicos de um único navegador, ou dependentes de plugins de terceiros. Apesar do esforço heróico para deixar clara as diferenças (no `dojox.storage`), eles todos apresentam interfaces totalmente diferentes, têm diferentes limitações de armazenamento, e apresentam diferentes experiências de usabilidade. Portanto, esse é o problema que a HTML5 se propõe a resolver: providenciar uma padronizada API, implementada de forma nativa e consistente em vários navegadores, sem a necessidade e dependência de plugins de terceiros.



INTRODUÇÃO AO HTML5 STORAGE

O que me refiro como “HTML5 Storage” é a especificação nomeada de Web Storage, que estava na especificação da HTML5 por um tempo, mas era tratado separadamente por desinteresse e razões políticas. Certos navegadores se referenciavam como “Local Storage” ou “DOM Storage.” A nomeação ficava ainda mais complicada em navegadores relacionados com padrões emergentes, que vou discutir mais adiante neste capítulo.

Então o que é HTML5 Storage? Simplesmente armazenar, é a forma de páginas da web armazenarem pares de chave/valor localmente, dentro do navegador do cliente. Semelhante aos cookies, esses dados persistem mesmo depois de você sair do site, fechar a guia ou o navegador. Diferente dos cookies, esses dados nunca são transmitidos ao servidor (a menos que você queria enviá-los manualmente). Diferente de todas as tentativas anteriores de fornecimento de armazenamento local persistente, este é implementado de forma nativa nos navegadores, para que esteja disponível mesmo quando os plugins de terceiros não estiverem.

Quais navegadores? Bom, as últimas versões de quase todos os navegadores suportam HTML5 Storage... até o Internet Explorer!

SUPORTE AO HTML5 STORAGE						
IE	FIREFOX	SAFARI	CHROME	OPERA	IPHONE	ANDROID
8.0+	3.5+	4.0+	4.0+	10.5+	2.0+	2.0+

Direto do seu código JavaScript, você acessa o HTML5 Storage através do objeto `localStorage` no objeto global `window`. Antes de usá-lo, você deve detectar se o navegador tem suporte.

↪ *Verificando suporte HTML5 Storage*

```
function supports_html5_storage() {  
    try {
```

```

    return 'localStorage' in window && window['localStorage'] !==
null;
  } catch (e) {
    return false;
  }
}

```

Ao invés de escrever essa função, você pode usar Modernizr para detectar o suporte da HTML5 Storage.

```

if (Modernizr.localstorage) {
  // window.localStorage é suportado pelo seu navegador!
} else {
  // não há suporte nativo ao HTML5 storage no seu navegador! :(
  // tente talvez dojox.storage ou alguma solução de terceiros
}

```



USANDO HTML5 STORAGE

HTML5 Storage é baseado em pares de chave/valor nomeadas. Você pode armazenar dados em uma chave nomeada, e com essa mesma chave, recuperar os dados armazenados. A chave nomeada é uma string. E seu valor, é qualquer um suportado pelo JavaScript, incluindo Strings, Booleanos, Inteiros, ou Flutuantes. Conteúdo, os dados são armazenados como uma sequência de strings. Se você estiver armazenando e recuperando qualquer outro tipo de dado diferente de Strings, você precisa usar funções como `parseInt()` ou `parseFloat()` para recuperar os dados em um tipo esperado pelo JavaScript.

```

interface Storage {
  getter any getItem(in DOMString key);
  setter creator void setItem(in DOMString key, in any data);
};

```

Chamando `setItem()` com o nome de uma chave que já existe, ela sobrescreve o valor anterior. Chamando `getItem()` com o nome de uma chave que não existe, retornará `null` ao invés de de uma exceção.

Como outros objetos JavaScript, você pode tratar o objeto `localStorage` como um array associativo. Ao invés de usar os métodos `getItem()` e `setItem()`, você pode usar simplesmente colchetes. Por exemplo, no trecho seguinte:

```

var foo = localStorage.getItem("bar");
// ...

```

```
localStorage.setItem("bar", foo);
```

...pode ser reescrito para usar a sintaxe de colchetes ao invés:

```
var foo = localStorage["bar"];  
// ...  
localStorage["bar"] = foo;
```

Existem métodos para remover um valor já dado a uma chave, e limpar o armazenamento local inteiro (isto é, apagar todas as chaves/valor existentes de uma só vez).

```
interface Storage {  
    delete void removeItem(in DOMString key);  
    void clear();  
};
```

Chamando `removeItem()` com uma chave inexistente, não retornará nada.

Finalmente, há uma propriedade para obter o número total de chaves/valor na área de armazenamento local, e também, para percorrer todas as chaves pelo seu índice (e retornar o nome de cada uma).

```
interface Storage {  
    readonly attribute unsigned long length;  
    getter DOMString key(in unsigned long index);  
};
```

Se você chamar `key()` com um índice que não está entre `0–(length-1)`, a função retornará `null`.

CONTROLANDO ALTERAÇÕES NA ÁREA DO HTML5 STORAGE

Se quiser manter o controle das constantes mudanças no armazenamento local, você pode capturar o evento `storage`. O evento `storage` é ativado no objeto `window` sempre que `setItem()`, `removeItem()`, ou `clear()` é chamado *e, geralmente, muda alguma coisa*. Por exemplo, se você definir um valor a uma chave existente ou chamar `clear()` em chaves não nomeadas, o evento `storage` não será chamado, porque realmente, nada mudou na área de armazenamento local.

O evento `storage` é suportado em todos os lugares em que o objeto `localStorage` funciona, incluindo Internet Explorer 8. IE 8 não suporta o padrão W3C `addEventListener` (será adicionado, finalmente, no IE 9). Portanto, para pegar o evento `storage`, você vai precisar checar qual mecanismo de evento o navegador suporta. (Se você já fez isso antes, para outros

eventos, pode pular essa seção. Pegando o evento `storage`, funciona da mesma forma como todos os outros eventos, você estará sempre preso. Se você preferir usar jQuery ou qualquer outra biblioteca JavaScript para registrar os eventos de manipulação, você pode fazer isso com o evento `storage`, também.)

```
if (window.addEventListener) {  
    window.addEventListener("storage", handle_storage, false);  
} else {  
    window.attachEvent("onstorage", handle_storage);  
};
```

A função de callback `handle_storage` é chamada junto ao objeto `StorageEvent`, exceto no Internet Explorer que os objetos de evento são armazenados no `window.event`.

```
function handle_storage(e) {  
    if (!e) { e = window.event; }  
}
```

Nesse ponto, a variável `e` será o objeto `StorageEvent`, que tem as seguintes propriedades:

OBJETO STORAGEEVENT		
PROPRIIDADE	TIPO	DESCRIÇÃO
key	string	o nome da chave que foi adicionada, removida ou alterada
oldValue	qualquer	o valor antigo (agora atualizado), ou null se for um novo item adicionado
newValue	qualquer	o novo valor, ou null se um item foi removido
url*	string	a página que chamou o método que realizou a mudança
* Nota: a propriedade <code>url</code> é originalmente chamada de <code>uri</code> . Alguns navegadores lançaram essa propriedade antes das mudanças na especificação. Para uma compatibilidade máxima, você deve checar se a propriedade <code>url</code> existe, se não, checar se a propriedade <code>uri</code> existe.		

O evento `storage` não é cancelável. Dentro da função callback `handle_storage`, não há maneira de parar essa mundaça. É uma simples forma do navegador dizer, “ei, isso já aconteceu. Não há nada que você possa fazer, eu só queria te avisar sobre isso.”

LIMITAÇÕES NOS NAVEGADORES ATUAIS

Ao falar sobre a história dos hacks no armazenamento local usando plugins de terceiros, fiz questão de mencionar a limitação de cada técnica, tal como limite de armazenamento. Apenas percebi que não tinha mencionado nada sobre as limitações do agora padronizado armazenamento da HTML5. Vou dar a vocês as respostas primeiro, em seguida explicarei elas. As respostas, em ordem de importância, são “5 megabytes,” “QUOTA_EXCEEDED_ERR,” e “Não.”

“5 megabytes” é a quantidade em espaço que cada origem tem por padrão. Isto é surpreendentemente consistente em todos os navegadores, embora seja formulada como não mais do que uma sugestão na especificação do Armazenamento local da HTML5. Um aspecto a ter em mente é que você está armazenando strings, não as informações no seu formato original. Se você está armazenando muitos inteiros ou floats, a diferença de representação pode realmente aumentar. Cada dígito naquele float está sendo armazenado como um caractere, não na representação usual de um ponto de número flutuante.

“QUOTA_EXCEEDED_ERR” é a exceção que será lançada se você exceder sua cota de armazenamento de 5 megabytes. “Não” é a resposta para a próxima questão óbvia, “Eu posso pedir permissão do usuário para usar mais espaço de armazenamento?” No momento em que escrevo (fevereiro 2011), não há navegador que suporte qualquer mecanismo para desenvolvedores web requisitarem mais espaço de armazenamento. Alguns navegadores (como Opera) permitem que o usuário controle a cota de armazenamento de cada site, mas isto é uma ação puramente iniciada pelo usuário, não algo que você como desenvolvedor pode construir na sua aplicação web.



ARMAZENAMENTO HTML5 EM AÇÃO

Vamos ver o Armazenamento da HTML5 em ação. Recorde o jogo Halma que nós construímos no capítulo do canvas. Existe um pequeno problema com o jogo: se você fechar o navegador no meio do jogo, você perderá seu progresso. Porém, com o armazenamento da HTML5, podemos salvar o progresso localmente, dentro do próprio navegador. Aqui está uma demonstração no ar. Faça alguns movimentos, depois feche a aba do navegador, em seguida reabra ela. Se o seu navegador suporta Armazenamento da HTML5, a página de demonstração deve magicamente recordar a sua exata posição dentro do jogo, incluindo o número de movimentos que você fez, a posição de cada uma das peças do tabuleiro, e até mesmo se uma determinada peça está selecionada.

Como isto funciona? Cada momento que uma mudança acontece dentro do jogo, chamamos esta função:

```
function saveGameState() {  
    if (!supportsLocalStorage()) { return false; }  
}
```

```

localStorage["halma.game.in.progress"] = gGameInProgress;
for (var i = 0; i < kNumPieces; i++) {
    localStorage["halma.piece." + i + ".row"] = gPieces[i].row;
    localStorage["halma.piece." + i + ".column"] = gPieces[i].column;
}
localStorage["halma.selectedpiece"] = gSelectedPieceIndex;
localStorage["halma.selectedpiecehasmoved"] = gSelectedPieceHasMoved;
localStorage["halma.movecount"] = gMoveCount;
return true;
}

```

Como você pode ver, ela usa o objeto `localStorage` para salvar se há um jogo em progresso (`gGameInProgress`, um booleano). Se assim for, percorre as peças (`gPieces`, um Array JavaScript) e salva o número de linha e coluna de cada uma. Em seguida ele salva alguns estados adicionais do jogo, incluindo a peça que está selecionada (`gSelectedPieceIndex`, um inteiro), se a peça está no meio de uma série potencialmente longa de saltos (`gSelectedPieceHasMoved`, um booleano), e o número total de movimentos feito até então (`gMoveCount`, um inteiro).

No carregamento da página, em vez de chamar automaticamente uma função `newGame()` que resetaria estas variáveis para valores "hard-coded", chamamos uma função `resumeGame()`. Usando Armazenamento HTML5, a função `resumeGame()` checa se um estado relacionado ao jogo em progresso está armazenado localmente. Se está, ele recupera esses valores usando o objeto do `localStorage`.

```

function resumeGame() {
    if (!supportsLocalStorage()) { return false; }
    gGameInProgress = (localStorage["halma.game.in.progress"] == "true");
    if (!gGameInProgress) { return false; }
    gPieces = new Array(kNumPieces);
    for (var i = 0; i < kNumPieces; i++) {
        var row = parseInt(localStorage["halma.piece." + i + ".row"]);
        var column = parseInt(localStorage["halma.piece." + i + ".column"]);
        gPieces[i] = new Cell(row, column);
    }
    gNumPieces = kNumPieces;
    gSelectedPieceIndex = parseInt(localStorage["halma.selectedpiece"]);
    gSelectedPieceHasMoved = localStorage["halma.selectedpiecehasmoved"] ==
    "true";
    gMoveCount = parseInt(localStorage["halma.movecount"]);
    drawBoard();
    return true;
}

```

A parte mais importante desta função é a ressalva que mencionei no início deste capítulo, que vou repetir aqui: *Informações são armazenadas como string. Se você está armazenando algo diferente de string, você vai precisar forçar uma conversão quando você recuperá-la.* Por exemplo, a bandeira para verificar se existe um jogo em progresso (`gGameInProgress`) é um booleano. Na

função `saveGameState()`, nós apenas armazenamos e não nos preocupamos em relação ao datatype:

```
localStorage["halma.game.in.progress"] = gGameInProgress;
```

Pórem, na função `resumeGame()`, precisamos tratar o valor que pegamos da área de armazenamento local como uma string e construir manualmente o valor booleano apropriado:

```
gGameInProgress = (localStorage["halma.game.in.progress"] == "true");
```

Semelhantemente, o número de movimentos está armazenado no `gMoveCount` como um inteiro. Na função `saveGameState()`, apenas armazenamos:

```
localStorage["halma.movecount"] = gMoveCount;
```

Mas na função `resumeGame()`, precisamos forçar o valor para um inteiro, usando a função nativa do JavaScript `parseInt()`:

```
gMoveCount = parseInt(localStorage["halma.movecount"]);
```



ALÉM DE PARES DE CHAVES-VALORES NOMEADOS: VISÕES CONFLITANTES

Enquanto o passado é cheio de truques e soluções alternativas, a condição presente do Armazenamento da HTML5 é surpreendentemente otimista. Uma nova API vem sendo padronizada e implementada em todos os principais navegadores, plataformas e dispositivos. Como um desenvolvedor web, isto não é algo que você vê todo dia, não é verdade? Porém, há mais vida além dos “5 megabytes de pares de chaves/valores nomeados” e o futuro do armazenamento local persistente é...como poderei colocá-lo... bom, existem visões conflitantes.

Uma visão é uma sigla que você provavelmente já sabe: SQL. Em 2007, a Google lançou Gears, um plugin de código aberto que incluía um banco de dados incorporado com base de dados no SQLite. Esse protótipo inicial influenciou a criação da especificação do Web SQL Database. Web SQL Database (formalmente conhecida como “WebDB”) fornece uma fina camada em torno de uma base de dados SQL, permitindo você fazer coisas como essa pelo JavaScript:

↪Código atual funcionando em 4 navegadores

```
openDatabase('documents', '1.0', 'Local document storage', 5*1024*1024,
function (db) {
  db.changeVersion('', '1.0', function (t) {
    t.executeSql('CREATE TABLE docids (id, name)');
  }, error);
});
```

Você pode ver, maior parte da ação reside na sequência que você passa para o método `executeSql`. Esta string pode ser qualquer comando suportado pelo SQL, incluindo `SELECT`, `UPDATE`, `INSERT` and `DELETE`. É como programação de base de dados backend, exceto o fato de você está fazendo isto via JavaScript! Oh alegria!

A especificação da Base de Dados SQL vem sendo implementada por quatro navegadores e plataformas.

SUPORTE A BASE DE DADOS SQL WEB						
IE	FIREFOX	SAFARI	CHROME	OPERA	IPHONE	ANDROID
.	.	4.0+	4.0+	10.5+	3.0+	2.0+

Claro, se você já usou mais de um produto de banco de dados em sua vida, você está ciente que “SQL” é mais um termo de marketing do que um rígido e rápido padrão. (Alguns podem dizer o mesmo da “HTML5,” mas não se importe.) Claro, existe uma especificação atual do SQL (ela é chamada de SQL-92), mas não há nenhuma base de dados do servidor no mundo que está em conformidade com esta e apenas esta especificação. Existe o SQL da Oracle, SQL da Microsoft, SQL do MySQL, SQL do PostgreSQL e SQL do SQLite. De fato, cada um desses produtos adiciona novas funcionalidades ao SQL no passar do tempo, então mesmo dizendo “SQL do SQLite” não é suficiente para determinar exatamente o que você está falando. Você precisa dizer a versão do SQL que acompanha o SQLite versão X.Y.Z.”

Tudo o que nos leva ao seguinte aviso, atualmente residindo no topo da especificação da Base de Dados SQL Web:

Esta especificação chegou a um impasse: todos os implementadores interessados têm utilizado o mesmo backend do SQL (Sqlite), mas precisamos de múltiplas implementações independentes para prosseguir ao longo de um caminho de normalização. Até outro implementador estar interessado em implementar esta especificação, a descrição do dialeto SQL foi deixada como simplesmente uma referência para Sqlite, o que não é aceitável para um padrão.

Isto é contra esse cenário que vou apresentá-lo para uma outra visão concorrente avançada, persistente, armazenamento local para aplicações web: A API de “Indexed Database”, formalmente conhecida como “WebSimpleDB,” agora carinhosamente conhecida como “IndexedDB.”

A API do “Indexed Database” expõe o que é chamado um *objeto de armazenamento*. Um objeto de armazenamento compartilha vários conceitos com o a base de dados SQL. Existem “base de

dados” com “registros,” e cada registro tem um número definido de “campos.” Cada campo tem um “datatype” específico, que é definido quando a base de dados é criada. Você pode selecionar um subconjunto de registros, depois enumerá-los com um “índice.” Alterações no armazenamento de objetos são tratados dentro de “transações”.

Se você já fez alguma programação de banco de dados SQL, esses termos provavelmente serão familiares. A principal diferença é que o objeto de armazenamento não tem linguagem de consulta estruturada. Você não constrói uma declaração como "SELECT * from USERS where ACTIVE = 'Y'". Em vez disso, você usa métodos fornecidos pelo objeto de armazenamento para adicionar um índice na base de dados nomeada “USERS,” enumerar os registros, filtrar os registros de usuários inativos, e usar métodos de acesso para obter os valores de cada campo nos registros restantes. Uma visão geral do IndexedDB é um bom tutorial de como IndexedDB funciona, fazendo comparações lado a lado entre o IndexedDB e base dados SQL.

No momento que escrevo, IndexedDB está implementado apenas na versão beta do Firefox 4. (Em contraste, a Mozilla afirmou que eles nunca irão implementar Base de Dados SQL Web.) Google afirmou que estão considerando o suporte ao IndexedDB para o Chromium e o Google Chrome. E até mesmo a Microsoft disse que o IndexedDB “é uma ótima solução para a web.”

Então o que você, como desenvolvedor web, pode fazer com IndexedDB? Neste momento, praticamente nada além de algumas demonstrações técnicas. Um ano a frente? Talvez algo mais. Para iniciar, confira os links na seção “Leitura complementar” com alguns bons tutoriais.

LEITURA COMPLEMENTAR

HTML5 storage:

- Especificação de Armazenamento HTML5
- Introdução ao DOM Storage no MSDN
- Web Storage: armazenamento de dados no lado do cliente mais fácil e mais poderoso powerful em Opera Developer Community
- DOM Storage no Mozilla Developer Center. (Nota: a maior parte dessa página é devota ao protótipo da implementação do Firefox’ do objeto `globalStorage`, um não-padrão precursor do `localStorage`. Mozilla adicionou suporte ao padrão `localStorage` no Firefox 3.5.)
- Desabilite local storage para aplicações web mobile com HTML5, um tutorial na IBM DeveloperWorks

Trabalho prévio de Brad Neuberg *et. al.* (pré-HTML5):

- Internet Explorer possui suporte nativo para persistência?!?! (sobre o objeto `userData` do IE)

- Dojo Storage, parte de um tutorial maior sobre a (agora não continuada) biblioteca Dojo Offline
- `dojox.storage.manager` referência da API
- `dojox.storage` repositório Subversion

Web SQL Database:

- Especificação Web SQL Database
- Introdução à Web SQL Databases
- Demonstração de Web Database
- `persistence.js`, um “ORM assíncrono em JavaScript” construído sob Web SQL Database e Gears

IndexedDB:

- Especificação do Indexed Database API
- Além de HTML5: Database APIs e o caminho para IndexedDB
- Firefox 4: Uma prévia caminhada por IndexedDB

VAMOS TORNAR ISSO OFFLINE

MERGULHANDO



que são aplicações web offline? Em um primeiro momento, parece uma contradição de termos. Páginas da web são coisas que você faz download e renderiza. Download implica conexão com a rede. Como você pode fazer download quando você está offline? É claro, você não pode. Mas você *pode* fazer download quando você está online. E é assim que as aplicações offline da HTML5 funcionam.

De forma simples, uma aplicação web offline é uma lista de URLs — HTML, CSS, JavaScript, imagens, ou qualquer tipo de recurso. A página inicial de uma aplicação web offline aponta para essa lista, chamado de arquivo manifesto, que é apenas um arquivo texto localizado em qualquer lugar no servidor web. Um navegador web que implementa aplicações offline em HTML5 irá ler a lista de URLs do arquivo manifesto, fazer download dos recursos, realizar o cache deles localmente, e automaticamente manter os arquivos locais atualizados à medida que são alterados. Quando chegar a hora em que você tentar acessar as aplicações web sem uma conexão de rede, seu navegador web irá automaticamente apontar para as cópias locais.

A partir disso, a maioria do trabalho está com você, desenvolvedor web. Existe uma indicação no DOM que irá dizer se você está online ou offline. Existem eventos que são disparados quando seu status offline muda (em um minuto você está offline e no próximo minuto está online, ou vice-versa). Mas é basicamente isso. Se a sua aplicação cria dados ou salvo estados, está com você a missão de armazenar dados localmente enquanto está offline e sincronizar isso com o servidor remoto uma vez estando online novamente. Em outras palavras, HTML5 pode levar suas aplicações web offline. O que você faz quando chegar lá é com você.

SUPORTE OFFLINE						
IE	FIREFOX	SAFARI	CHROME	OPERA	IPHONE	ANDROID
.	3.5+	4.0+	5.0+	10.6+	2.1+	2.0+

O MANIFESTO CACHE

Uma aplicação web offline utiliza um arquivo de manifesto cache. O que é um arquivo manifesto? É uma lista de todos os recursos que a sua aplicação web irá precisar acessar quando estiver desconectada de uma rede. Afim de iniciar o processo de download e cache desses recursos, você irá precisar apontar para o arquivo manifesto, utilizando o atributo `manifest` no seu elemento `<html>`.

```
<!DOCTYPE HTML>
<html manifest="/cache.manifest">
<body>
...
</body>
</html>
```

Seu arquivo de manifesto cache pode estar localizado em qualquer lugar do seu servidor web, mas precisa estar servido sob o tipo de conteúdo `text/cache-manifest`. Se você está rodando um servidor web baseado no Apache, você provavelmente precisa adicionar uma diretiva `AddType` no seu arquivo `.htaccess` na raiz do seu diretório web:

```
AddType text/cache-manifest .manifest
```

Depois certifique-se que o nome do seu arquivo de manifesto cache termina com `.manifest`. Se você usa um servidor web diferente ou uma configuração diferente do Apache, consulte a documentação do seu servidor em controle de cabeçalho `Content-Type`.

P: Minha aplicação web abrange mais de uma página. Preciso de um atributo `manifest` em cada página, ou posso colocar apenas na home page?

R: Cada página da sua aplicação web precisa de um atributo `manifest` que aponta para o manifesto cache para toda aplicação.

OK, então cada uma das suas páginas HTML apontam para seu arquivo de manifesto cache, e seu arquivo de manifesto cache sendo servido com o devido cabeçalho `Content-Type`. Mas o que acontece *dentro* do arquivo manifesto? É aqui onde as coisas começam a ficar interessante.

A primeira linha de todo arquivo de manifesto cache é assim:

CACHE MANIFEST

Depois disso, todos os arquivos manifesto são divididos em três partes: a seção “explícita”, a seção “fallback”, e a seção “lista branca online”. Cada seção tem um header, na sua própria

linha. Se o arquivo manifesto não tiver nenhum desses cabeçalhos de seção, todos os recursos listados estão implicitamente na seção “explícita”. Tente não se debruçar sobre a terminologia, se não sua cabeça irá explodir.

Aqui está um arquivo manifesto válido. Ele lista três recursos: um arquivo CSS, um arquivo JavaScript, e uma imagem JPEG.

```
CACHE MANIFEST
/clock.css
/clock.js
/clock-face.jpg
```

Esse arquivo de manifesto cache não possui cabeçalhos de seção, então todos os recursos listados estão na seção “explícita” por padrão. Recursos na seção “explícita” serão feitos download e cache localmente, e serão usados no lugar de seus respectivos semelhantes sempre que você estiver desconectado de uma rede. Assim que for carregado esse arquivo de manifesto cache, seu navegador irá realizar o download de `clock.css`, `clock.js`, e `clock-face.jpg` do diretório raiz do seu servidor web. Então você pode desplugar seu cabo de rede e atualizar a página, e todos aqueles recursos estarão disponíveis offline.

P: Preciso listar minhas páginas HTML no meu manifesto cache?
R: Sim e não. Se a sua aplicação web inteira está contida em uma única página, apenas certifique-se que a página aponta para o manifesto cache usando o atributo `manifest`. Quando você navega em uma página HTML com o atributo `manifest`, a página em si assume ser parte da aplicação web, então você não precisa listá-la no arquivo manifesto. Entretanto, se a sua aplicação web agrega múltiplas páginas, você deve listar todas as páginas HTML no arquivo manifesto, caso contrário o navegador não saberá que existem outras páginas HTML que precisam fazer download e cache.

SEÇÕES DE REDE

Aqui está um exemplo um pouquinho mais complicado. Suponha que você queira que sua aplicação relógio rastreie os visitantes, usando um script `tracking.cgi` que é carregado dinamicamente por um atributo `<img src>`. Realizando o cache desse recurso irá acabar com o propósito desse rastreamento, então esse recurso nunca deve realizar cache e nunca deve estar disponível offline. Aqui está o que você deve fazer:

```
CACHE MANIFEST
NETWORK:
/tracking.cgi
CACHE:
/clock.css
/clock.js
/clock-face.jpg
```

Esse arquivo de manifesto cache inclui cabeçalhos de seção. A linha marcada com **NETWORK:** é o começo dessa seção “lista branca online”. Recursos nessa seção nunca sofrerão cache e não estarão disponíveis offline. (Tentar carregá-los enquanto estiver offline irá resultar em um erro.) A linha marcada com **CACHE:** é o começo da seção “explícita”. O resto do arquivo de manifesto cache é o mesmo do exemplo anterior. Cada um dos três recursos listados estará em cache e disponível offline.

SEÇÕES DE FALLBACK

Esse é mais um tipo de seção em um arquivo de manifesto cache: a seção fallback. Na seção fallback, você pode definir substituições para recursos online que, por uma razão qualquer, não devem realizar cache ou não realizaram cache com sucesso. A especificação da HTML5 oferece esse exemplo usando a seção fallback:

```
CACHE MANIFEST
FALLBACK:
/ /offline.html
NETWORK:
*
```

O que isso faz? Primeiro, considere um site que contém milhões de páginas, como no [Wikipedia](#). Você não poderia fazer download do site inteiro, nem você iria querer. Mas suponha que você consiga deixar *parte* disso disponível offline. Como você iria decidir quais página fazer cache? Que tal isso: cada página que você já tenha visto em uma hipotética Wikipedia offline seriam feitos download e cache. Isso incluiria cada entrada da enciclopédia que você já visitou, cada página de discussão (onde você encontra discussões improvisadas sobre uma entrada particular da enciclopédia), e cada página de edição (onde você consegue, de fato, realizar alterações sobre uma entrada particular).

É isso que o manifesto cache faz. Suponha que cada página HTML (entrada, discussão, edição, histórico) no Wikipedia apontasse para esse arquivo de manifesto cache. Quando você visita qualquer página que aponta para o manifesto cache, seu navegador diz “ei, essa página é parte de uma aplicação web offline, essa é alguma que eu conheço?” Se o seu navegador nunca fez download desse arquivo de manifesto cache, ele irá definir um novo “appcache” offline; (nome curto para “aplicação cache”), fazer download de todos os recursos listados no manifesto cache, e depois adicionar a página corrente para o appcache. Se o seu navegador conhece o manifesto cache, irá simplesmente adicionar a página corrente para o appcache existente. Dessa forma, a página que você acabou de visitar acaba no appcache. Isso é importante. Significa que você pode ter uma aplicação web offline que, de forma “preguiçosa”, adiciona páginas à medida que as visita. Você não precisa listar cada uma das suas páginas HTML no seu manifesto cache.

Agora veja a seção fallback. A seção fallback nesse manifesto cache tem apenas uma única linha. A primeira parte da linha (antes do espaço) não é uma URL. É na verdade um padrão de URL. O único caractere (/) irá definir qualquer página do seu site, não apenas a página de entrada.

Quando você tentar visitar a página enquanto estiver offline, seu navegador irá procurar por ele no appcache. Se o seu navegador encontra a página no appcache (porque você a visitou enquanto esteve online, e a página foi adicionada de forma implícita no appcache naquela hora), então seu navegador irá exibir uma cópia da página em cache. Se o seu navegador não encontrar a página no appcache, ao invés de exibir uma mensagem de erro, irá exibir a página `/offline.html`, como especificado na segunda parte daquela linha na seção fallback.

Finalmente, vamos examinar a seção de rede. A seção de rede nesse arquivo de manifesto cache também tem apenas uma única linha, a linha que contém apenas um único caractere (*). Esse caractere tem um significado especial na seção de rede. É chamado de “curinga da lista branca online.” É um modo elegante de dizer que qualquer coisa que não esteja no appcache pode ainda realizar download do endereço web original, desde que você tenha uma conexão de internet. Isso é importante para uma aplicação web offline sem data de volta marcada. Significa que, enquanto você estiver navegando *online* nessa hipotética Wikipedia offline, seu navegador irá buscar imagens, vídeos e outros recursos embutidos normalmente, mesmo se estiverem em um domínio diferente. (Isso é comum em websites grandes, mesmo se eles não fizerem parte de aplicações web offline. Páginas HTML são geradas e servidas localmente, enquanto que imagens e vídeos são servidos a partir de um CDN em outro domínio.) Sem esse curinga, nossa hipotética Wikipedia offline iria se comportar estranhamente enquanto você estivesse online — especificamente, não iria carregar nenhuma imagem ou vídeo hospedados externamente!

Esse exemplo está completo? Não. A Wikipedia é mais do que arquivos HTML. Ela comumente usa CSS, JavaScript, e imagens em cada página. Cada um desses recursos precisariam ser listados explicitamente na seção **CACHE**: do arquivo manifesto, para que cada página exibisse e se comportasse propriamente offline. Mas o ponto da seção fallback é que você pode ter uma aplicação web offline sem data de volta marcada que estende além dos recursos que você listou explicitamente no arquivo manifesto.



O FLUXO DE EVENTOS

Até agora, falei sobre aplicações web offline, sobre cache manifest e vagamente sobre cache de aplicações offline (“appcache”), com termos quase mágicos. Coisas são carregadas, navegadores fazem decisões e tudo simplesmente funciona. Você sabe melhor que isso, certo? Quero dizer, desse desenvolvimento web que estamos falando. Nada sempre “apenas funciona”.

Primeiro, vamos falar sobre o fluxo dos eventos. Especificamente, eventos DOM. Quando seu navegador acessa uma página que aponta para um arquivo de cache manifest, ele aciona uma série de eventos no objeto `window.applicationCache`. Eu sei que parece complicado mas, confie em mim, essa é a versão mais simples que eu pude pensar e que não deixa de fora informações importantes.

1. Assim que o navegador percebe o atributo `manifest` no elemento `<html>`, ele dispara um evento `checking`. (Todos os eventos listados aqui são acionados pelo objeto `window.applicationCache`). O evento `checking` é sempre disparado, independente de você ter visitado esta página ou outra página que aponte para o mesmo arquivo de cache manifest.
2. Se seu navegador nunca viu este cache manifest antes...
 - Ele irá acionar o evento `downloading`, e então iniciar o download dos arquivos listados no cache manifest.
 - Enquanto ele está baixando, seu navegador irá disparar periodicamente o evento `progress`, que contém informações de quantos arquivos já foram baixados e quantos arquivos ainda estão na fila download.
 - Depois que todos os arquivos listados no cache manifest forem baixados com sucesso, o navegador dispara um evento final, o `cached`. Este é seu sinal de que a aplicação web offline está completamente cacheada e pronta para ser usada sem conexão. Pronto, você terminou.
3. Por outro lado, se você visitou esta página anteriormente ou alguma outra página que aponte para o mesmo arquivo de cache manifest, então seu navegador já sabe sobre esse cache manifest. Ele pode já ter alguns arquivos no appcache. Ele pode até conter toda a aplicação web offline funcionando no appcache. Sendo assim, a questão é: o cache manifest mudou desde a última vez que seu navegador o acessou?
 - Se a resposta for não, o cache manifest não mudou, seu navegador irá imediatamente disparar o evento `noupdate`. E é isso, está tudo pronto.
 - Se a resposta for sim, o cache manifest *mudou*, seu navegador irá disparar o evento `downloading` e começar a baixar todos os arquivos listados no cache manifest novamente.
 - Enquanto os arquivos são baixados, seu navegador irá disparar periodicamente o evento `progress`, que contém informações como quantos arquivos foram baixados até agora e quantos arquivos ainda estão na fila para serem baixados.
 - Depois que todos os arquivos listados no cache manifest forem baixados novamente com sucesso, o navegador irá acionar um evento final chamado `updateready`. Este é o sinal de que uma nova versão de sua aplicação web offline está totalmente cacheada e pronta para ser usada sem conexão. *A nova versão ainda não está sendo usada.* Para fazer uma “troca rápida” para a nova versão sem forçar o usuário a atualizar a página atual, você pode chamar manualmente a função `window.applicationCache.swapCache()`.

Se, em qualquer ponto deste processo, alguma coisa terrível acontecer de errado, seu navegador irá acionar o evento `error` e parar. Aqui está uma lista resumida do que pode ter dado errado:

- O cache manifest retornou um erro 404 de HTTP (Page Not Found) ou 410 (Permanently Gone).
- O cache manifest foi encontrado e não foi alterado, mas a página HTTP que aponta para o arquivo cache manifest não foi carregada corretamente.
- O cache manifest mudou enquanto o carregamento estava em andamento.

- O cache manifest foi encontrado e alterado, mas o navegador falhou em baixar um dos arquivos listados no arquivo de cache manifest.

A FINA ARTE DE DEPURAR, TAMBÉM CONHECIDA COMO “ME MATE! ME MATE AGORA!”

Eu quero falar sobre dois pontos importantes aqui. O primeiro é algo que você acabou de ler, mas aposto que não absorveu muito bem, então, lá vai novamente: mesmo que um único arquivo listado em seu arquivo de cache manifest falhar na hora de ser carregado corretamente, todo o processo de cache de sua aplicação web offline irá falhar. Seu navegador irá disparar o evento `error`, mas não existe nada indicando qual é o problema. Isso pode tornar a depuração de aplicações web offline ainda mais frustrante do que normalmente é.

O segundo ponto importante é algo que, tecnicamente falando, não é um erro, mas vai parecer um problema sério do navegador até que você descubra o que está acontecendo. Tem relação exatamente com o modo que seu navegador checa como um arquivo cache manifest foi alterado. Este é um processo de três fases. É chato mas muito importante, então preste atenção.

1. Via semântica normal de HTTP, seu navegador vai checar se o cache manifest expirou. Bem como qualquer outro arquivo sendo servido em HTTP, seu servidor vai incluir meta-informações, como faz tipicamente, sobre o arquivo nos cabeçalhos de resposta HTTP. Alguns desses cabeçalhos HTTP (`Expires` e `Cache-Control`) dizem para seu navegador como é permitido cachear arquivos sem ao menos ter que perguntar o que mudou pro servidor. Este tipo de cache não tem relação nenhuma com aplicações web offline. Isso acontece para praticamente todas as páginas HTML, folhas de estilo, script, imagens ou qualquer outro arquivo na web.
2. Se o cache manifest expirou (de acordo com seu cabeçalho HTTP), então o navegador irá pedir ao servidor qual a versão mais nova, e se existir uma, o navegador irá baixar a mesma. Para isso, seu navegador aponta uma requisição HTTP que inclui a data da última modificação do cache manifest, a qual seu servidor inclui na resposta do cabeçalho HTTP da última vez que seu navegador baixou o arquivo de cache manifest. Se o servidor determinar que o arquivo de cache manifest não mudou desde tal data, ele vai retornar, simplesmente, um status 304 (`Not Modified`). Novamente, nada disso é específico para aplicações web offline. Isso acontece, essencialmente, para todos os tipos de arquivos na web.
3. Se o servidor acha que o arquivo de cache manifest mudou desde tal data, ele vai retornar um código de status HTTP 200 (`OK`), seguido pelo conteúdo do novo arquivo, junto de um novo cabeçalho `Cache-Control` e uma nova data de última modificação do arquivo, então o passo 1 e 2 irão funcionar devidamente da próxima vez. (HTTP é legal; servidores web estão sempre preparados para o futuro. Se seu servidor web definitivamente deve te enviar um arquivo, ele fará tudo que pode para ter certeza que ele não lhe enviará duas vezes, atoa.) Uma vez baixado o novo arquivo de cache manifest, seu navegador irá verificar o conteúdo do seu arquivo baixado com o arquivo baixado da última vez. Se o

conteúdo dos arquivos de cache manifest forem iguais, seu navegador não irá fazer o download novamente de qualquer um dos arquivos nele listados.

Qualquer um desses passos pode te confundir enquanto estiver programando e testando sua aplicação web offline. Por exemplo, digamos que você publicou uma versão do seu arquivo de cache manifest; então, dez minutos depois, você percebeu que precisa adicionar outro arquivo nele. Sem problema, certo? Apenas adicione uma linha e publique novamente. OPA! Irá acontecer o seguinte: você atualiza a página, seu navegador percebe que existe o atributo `manifest`, ele dispara o evento `checking`, e então... nada. Seu navegador insiste que o arquivo de cache manifest não mudou. Por que? Porque, por padrão, seu servidor web provavelmente está configurado para avisar o navegador para cachear arquivos estáticos por algumas horas (via semântica HTTP, usando cabeçalhos `Cache-Control`). Isso significa que seu navegador nunca vai passar o passo 1 daquele processo de três passos. Certamente, o servidor web sabe que o arquivo mudou, mas seu navegador nunca vai passar perto de fazer a solicitação ao servidor. Por que? Porque a última vez que seu navegador baixou o arquivo de cache manifest, o servidor pediu para cachear os arquivos por algumas horas (via semântica HTTP, usando cabeçalhos `Cache-control`). E agora, 10 minutos depois, é exatamente isso que seu navegador está fazendo.

Para deixar claro, isso não é um bug, é uma característica. Tudo está funcionando exatamente do jeito que deveria. Se servidores web não tivessem uma maneira de dizer aos navegadores (e intermediar proxies) a cachear coisas, a web poderia entrar em colapso do dia para noite. Mas isso não é confortável, depois de gastar horas tentando imaginar por que seu navegador não avisa sobre a atualização no arquivo de cache manifest. (E ainda melhor, se você esperar o suficiente, isso vai começar a funcionar novamente! Porque o cache HTTP expirou! Bem como deveria ser! Me mate! Me mate AGORA!)

Então aqui está uma coisa que você deveria, definitivamente, fazer: reconfigure seu servidor web para que arquivos de cache manifest não sejam cacheados por semânticas HTTP. Se você está usando um servidor baseado em Apache, estas duas linhas em seu `.htaccess` irão fazer a mágica:

```
ExpiresActive On
ExpiresDefault "access"
```

Isso vai, na verdade, desabilitar o cache para todos os arquivos desse tipo do diretório e subdiretórios. Provavelmente, isso não é o que você quer em produção, então você deve qualificar isso com uma diretiva `<Files>` para que, assim, irá afetar apenas seu arquivo de cache manifest, ou criar um subdiretório que não contenha nada além desse arquivo `.htaccess` e seus arquivos de cache manifest. Como sempre, detalhes de configuração variam dependendo do servidor, então consulte a documentação do seu para como controlar headers de cache HTTP.

Uma vez que tenha desabilitado o cache HTTP para arquivos de cache manifest, você ainda irá encontrar situações onde você atualiza um dos arquivos listados no appcache, mas ele irá

continuar com a mesma URL em seu servidor. Aqui, o segundo passo do processo de três fases irá te sacanear. Se seu arquivo de cache manifest não mudou, o navegador nunca vai te avisar que um dos arquivos cacheados anteriormente mudou. Considere o exemplo a seguir:

```
CACHE MANIFEST
# rev 42
clock.js
clock.css
```

Se você mudou o arquivo `clock.css` e publicou novamente, você não vai ver mudanças, porque o arquivo de cache manifest não mudou. Toda vez que você faz uma alteração em um dos arquivos da sua aplicação web offline, você vai precisar mudar também o arquivo de cache manifest. Isso pode ser tão fácil quanto mudar um caractere. A maneira mais fácil que encontrei para fazer isso, é incluir uma linha de comentário com um número de revisão. Mude o número da revisão no comentário e, então, o servidor irá entregar as mudanças dos arquivos listados no arquivo de cache manifest, seu navegador irá perceber que o conteúdo do arquivo mudou e irá iniciar o processo de baixar novamente os arquivos listados no arquivo manifest.

```
CACHE MANIFEST
# rev 43
clock.js
clock.css
```



VAMOS CONSTRUIR UM!

Lembra-se do jogo Halma que apresentei no capítulo de canvas, e depois foi melhorado salvando o estado com persistência de armazenamento local? Vamos deixar nosso jogo, Halma, offline.

Para fazer isso, precisamos de um arquivo manifest que liste todos os arquivos que o jogo precisa acessar. Bem, temos a página principal em HTML, um único arquivo JavaScript que contém todo o código do jogo e... só. Não existem imagens, porque todos os desenhos são feitos programaticamente via canvas API. Todo o CSS necessário está em um elemento `<style>`, no topo da página HTML. Então, nosso arquivo de cache manifest fica assim:

```
CACHE MANIFEST
halma.html
../halma-localstorage.js
```

Uma palavrinha sobre caminhos. Criei um subdiretório `offline/` no diretório `examples/`, e este arquivo de cache manifest mora dentro desse subdiretório. Devido a página HTML, vamos precisar acrescentar uma coisinha para que funcione offline (mais sobre isso em um minuto), criei também uma cópia separada do arquivo HTML, que também mora dentro do

subdiretório `offline/`. Porém, graças a não haver mudanças no código JavaScript, adicionamos suporte a armazenamento local, então estou literalmente reutilizando o mesmo arquivo `.js`, que mora no diretório pai (`examples/`). Todos juntos, os arquivos parecem com essa estrutura:

```
/examples/localstorage-halma.html
/examples/halma-localstorage.js
/examples/offline/halma.manifest
/examples/offline/halma.html
```

No arquivo de cache manifest (`/examples/offline/halma.manifest`), nós queremos referenciar dois arquivos. Primeiro, a versão offline do HTML (`/examples/offline/halma.html`). Desde que estes dois arquivos estejam no mesmo diretório, será listado no arquivo de manifest sem qualquer prefixo de caminho. Segundo, o arquivo JavaScript que mora no diretório pai (`/examples/halma-localstorage.js`). Este está listado no arquivo de manifest usando notação de URL relativa: `../halma-localstorage.js`. Isso está do jeito que você deve usar uma URL em um atributo `<img src>`. Como você verá no próximo exemplo, também é possível usar caminhos absolutos (que começa na raiz do domínio atual) ou até URLs absolutas (que apontam arquivos em outros domínios).

Agora, no arquivo HTML, precisamos adicionar o atributo `manifest` que aponta para o arquivo de cache manifest.

```
<!DOCTYPE html>
<html lang="en" manifest="halma.manifest">
```

Pronto! Quando um navegador de suporte offline carregar pela primeira vez a página HTML com suporte offline habilitado, será baixado o arquivo de cache manifest e, também baixar todos os arquivos referenciados, armazenando todos no cache da aplicação offline. Deste momento em diante, o algoritmo offline da aplicação tomará conta de tudo, sempre que a página for visitada. Você pode jogar offline, e já que o jogo lembra de seu estado localmente, você pode sair e voltar quantas vezes quiser.



LEIA MAIS (EM INGLÊS)

Padrões:

- Offline web applications na especificação HTML5

Documentação de browser vendors:

- [Offline resources in Firefox](#)
- [HTML5 offline application cache, parte do Safari client-side storage and offline applications programming guide](#)

Tutoriais e demos:

- [Gmail for mobile HTML5 series: using appcache to launch offline - part 1](#)
- [Gmail for mobile HTML5 series: using appcache to launch offline - part 2](#)
- [Gmail for mobile HTML5 series: using appcache to launch offline - part 3](#)
- [Debugging HTML5 offline application cache](#)
- [an HTML5 offline image editor and uploader application](#)
- [20 Things I Learned About Browsers and the Web](#), uma demonstração avançada que usa aplicações cache e outras técnicas HTML5.

Nº9

UMA FORMA DE LOUCURA

MERGULHANDO



Todo mundo conhece sobre formulários web, certo? Crie uma marcação `<form>`, alguns campos `<input type="text">`, ou talvez `<input type="password">`, no final um botão `<input type="submit">`, e pronto está feito.

Você não conhece nem a metade deles. A HTML5 definiu mais de meia dúzia de novos tipos de campos que você pode usar em seus formulários. E quando digo "usar", é usar agora - sem nenhuma gambiarra, hack ou soluções alternativas. Só não se entusiasme tanto; Eu não disse que todos estes recursos são suportados por todos os navegadores da atualidade. Oh, meu Deus, eu também não disse em nenhum. Em navegadores modernos, seus formulários irão arrebentar. Nos mais antiquados eles irão apenas cumprir sua função. Ou seja, os recursos irão se adaptar e degradar para funcionar em todos os navegadores. Até no IE6.

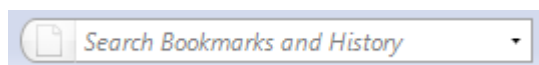


PLACEHOLDER

SUPORTE AO PLACEHOLDER						
IE	FIREFOX	SAFARI	CHROME	OPERA	IPHONE	ANDROID
.	3.7+	4.0+	4.0+	11.0+	4.0+	.

O primeiro aperfeiçoamento que a HTML5 trouxe aos formulários web é o recurso para definir um texto como placeholder, em campos input. Este texto é exibido dentro do input enquanto o campo estiver vazio ou sem foco. Logo que é clicado (ou acessado por tab) o texto placeholder desaparece.

Você provavelmente já viu o placeholder antes. Por exemplo, o Mozilla Firefox possui um texto placeholder na barra de endereço onde se lê "Search Bookmarks and History":



Quando voce clica (ou acessa por tab) a barra de endereço, o texto desaparece:



Aqui como você pode incluir um texto placeholder em seu formulário web:

```
<form>
  <input name="q" placeholder="Search Bookmarks and History">
  <input type="submit" value="Search">
</form>
```

Navegadores que não suportam o atributo placeholder irão simplesmente ignorá-lo. Veja se o seu navegador suporta o texto placeholder.

P: Eu posso usar marcações HTML no atributo placeholder? Eu gostaria de inserir uma imagem, ou talvez mudar as cores.

R: O atributo placeholder só pode conter texto, nenhuma marcação HTML. Porém há algumas extensões do CSS especificamente proprietárias que permitem estilizar o texto placeholder em alguns navegadores.

CAMPOS COM FOCO AUTOMÁTICO

SUPORTE AO AUTOFOCUS						
IE	FIREFOX	SAFARI	CHROME	OPERA	IPHONE	ANDROID
.	4.0+	4.0+	3.0+	10.0+	.	.



Web sites podem usar JavaScript para criar um foco automático no primeiro campo de um formulário. Por exemplo, a página inicial do Google foca automaticamente no campo de busca para você poder digitar suas palavras-chaves. Enquanto isto é conveniente para a maioria das pessoas, pode ser irritante para usuários experientes ou pessoas com necessidades especiais. Caso você pressione a barra de espaço esperando que a página role, ela não irá, pois o foco está em um campo de formulário. (Você terá um espaço no campo ao invés da rolagem). Se seu foco estiver em um outro campo enquanto a página ainda estiver carregando, o código de foco automático, que deveria ajudar, irá mover o foco para o campo original, interrompendo seu fluxo e fazendo com que digite no lugar errado.

Por causa do foco automático ser processado pelo JavaScript, torna complicado lidar com estes casos específicos, e isto é um pequeno ajuste para as pessoas que não querem que o website “roube” o foco.

Para resolver este problema, a HTML5 introduziu um atributo de autofocus para todos elementos de formulário web. O atributo `autofocus` executa exatamente o que ele diz, foco automático. O truque: logo que a página carrega, ele move o foco para um campo em particular. Mas como ele é apenas uma marcação ao invés de um script, o comportamento se torna consistente durante todo o website. Além disso, navegadores (ou extensões) podem oferecer que o usuário desabilite o foco automático.

Abaixo como você pode definir um campo de formulário com foco automático:

```
<form>
  <input name="q" autofocus>
  <input type="submit" value="Search">
</form>
```

Navegadores que não suportam o atributo `autofocus` simplesmente irão ignorá-lo. Veja se o seu navegador suporta o campo de foco automático.

Como assim? Você quer dizer que deseja que o autofocus funcione em todos os navegadores, não apenas nestes navegadores cheios de HTML5? Bom, Você pode manter seu script atual de foco automático. Faça apenas estas duas pequenas alterações:

1. Adicione o atributo `autofocus` na sua marcação HTML
2. Detecte se o navegador suporta o atributo `autofocus`, e execute o seu próprio script de foco automático, apenas se o navegador não suportar autofocus nativamente.

↪ *Autofocus com fallback*

```
<form name="f">
  <input id="q" autofocus>
  <script>
    if (!("autofocus" in document.createElement("input"))) {
      document.getElementById("q").focus();
    }
  </script>
  <input type="submit" value="Go">
</form>
...
```

Veja um exemplo de autofocus com fallback.

DEFINA UM FOCO O MAIS CEDO POSSÍVEL

Muitas páginas da web esperam o carregamento do `window.onload` para definir um foco. Mas o evento `window.onload` não executa nenhuma ação *até que* todas as imagens tenham sido carregadas. Se sua página possui diversas imagens, seu script pode atrapalhar e refazer o foco em um campo, enquanto o usuário já estiver interagindo em outra parte da página. Por isso que usuários experientes odeiam scripts de foco automático.

O exemplo a seguir executa o script de foco automático imediatamente depois que os campos de formulário são declarados. Esta é a solução ideal, mas pode ofender seu ego, por colocar um bloco de código JavaScript no meio da página. (Ou, mais prático, seu sistema de back-end, pode não ter esta flexibilidade.) Se você não pode colocar um script no meio da página, você deverá definir o foco durante um evento personalizado. Por exemplo, o `$(document).ready()` do jQuery ao invés de `window.onload`.

↳ Autofocus com jQuery fallback

```
<head>
<script src=jquery.min.js></script>
<script>
  $(document).ready(function() {
    if (!("autofocus" in document.createElement("input"))) {
      $("#q").focus();
    }
  });
</script>
</head>
<body>
<form name="f">
  <input id="q" autofocus>
  <input type="submit" value="Go">
</form>
```

Veja um exemplo do autofocus com jQuery fallback.

jQuery executa o evento personalizado `ready` logo que o DOM estiver disponível — isto é, ele espera até que todo o texto da página carregue, mas não espera até todas as imagens. Não está perto do ideal - se a página for consideravelmente grande ou a conexão de internet for extremamente baixa, o usuário pode começar a interagir com a página antes do seu script de foco automático execute. Mas é muito melhor que esperar o `window.onload`.

Se você estiver hesitante ou não puder inserir um bloco isolado de código em seu HTML, há um meio termo menos ofensivo que a primeira opção e mais otimizado que a segunda. É possível utilizar eventos personalizados do jQuery para definir seu próprio, digamos `autofocus_ready`.

Assim você pode acioná-lo manualmente, imediatamente depois que o campo com autofocus fique disponível. *Obrigado a E. M. Sternberg por me ensinar sobre esta técnica.*

~ *Autofocus com fallback utilizando evento personalizado*

```
<head>
<script src=jquery.min.js></script>
<script>
  $(document).bind('autofocus_ready', function() {
    if (!("autofocus" in document.createElement("input"))) {
      $("#q").focus();
    }
  });
</script>
</head>
<body>
<form name="f">
  <input id="q" autofocus>
  <script>$(document).trigger('autofocus_ready');</script>
  <input type="submit" value="Go">
</form>
```

Veja um exemplo de autofocus com fallback utilizando evento personalizado.

Esta abordagem é melhor que a primeira; o foco será definido o mais cedo que a técnica permitir, enquanto o texto da página ainda estiver carregando. Mas ele transfere a maior parte da lógica da sua aplicação (foco do campo de formulário) para fora do corpo da página. Este exemplo é baseado em jQuery, mas o conceito de eventos personalizados não é único do jQuery. Outras bibliotecas JavaScript como YUI e Dojo oferecem capacidades similares.

Para resumir:

- Definir o foco corretamente é importante.
- De todas as possibilidades, deixe o navegador trabalhar definindo o atributo autofocus no campo de formulário no qual deseja o foco.
- Se codificar um fallback para navegadores antigos, detecte o suporte do atributo autofocus e garanta que o fallback será executado *apenas* em navegadores antigos.
- Defina o foco o mais cedo possível. Insira o script de foco dentro da marcação imediatamente após o campo de formulário. Se isto ofender você, use bibliotecas JavaScript que suportem eventos personalizados para acionar um evento personalizado imediatamente depois que o campo de formulário estiver disponível. Se não for possível, use apenas algo como o evento `$(document).ready()`.
- Sobre outras circunstâncias espere até carregar `window.onload` para ter o foco definido.



ENDEREÇOS DE EMAIL

Por mais de uma década, existiam apenas alguns tipos de campos nos formulários web. Os mais comuns eram:

Tipo do Campo	Código HTML	Notas
checkbox	<code><input type="checkbox"></code>	pode ser marcado ou não
radio button	<code><input type="radio"></code>	pode ser agrupado com outros
password field	<code><input type="password"></code>	exibe pontos ao invés dos caracteres digitados
drop-down lists	<code><select><option>...</code>	
file picker	<code><input type="file"></code>	abre uma janela para “abrir arquivos”
submit button	<code><input type="submit"></code>	
plain text	<code><input type="text"></code>	o atributo type pode ser omitido

Todos esses campos continuam funcionando em HTML5. Se você “atualizar para HTML5” (quem sabe, mudando o DOCTYPE), não vai ser preciso fazer nenhuma mudança nos formulários. Viva a compatibilidade!

No entanto, há 13 novos tipos de campos na HTML5, e por razões que ficarão claras em breve *não* há porque não começar a utilizá-los.

O primeiro destes novos campos é o endereço de email. Se declara assim:

```
<form>
  <input type="email">
  <input type="submit" value="Go">
</form>
```

Eu estava prestes a escrever uma frase que começava com “nos navegadores que não suportam `type="email"...`” mas me impedi. Por que? Porque eu não tenho certeza do que quer dizer um navegador não suportar `type="email"`. Todos os navegadores “suportam” `type="email"`. Eles só não fazem nada de especial com ele (você verá alguns exemplos especiais de tratamento em breve), mas navegadores que não

reconhecem `type="email"` irão tratá-lo como `type="text"` e renderizar um tipo de campo simples.

Eu não consigo enfatizar quão importante ele é. A internet possui milhões de formulários que pedem para digitar um endereço de email, e todos eles usam `<input type="text">`. Você vê um campo de texto, você digita um endereço de email na caixa, e é isto. Com a HTML5 utilize a definição `type="email"`. Os navegadores vão surtar? Não. Todo o tipo de navegador da Terra trata um atributo `type` nulo como `type="text"` — inclusive o IE6. Então você pode “atualizar” seu formulário para `type="email"` agora.

O que você quer dizer com os navegadores *AINDA* suportam `type="email"`? Bom, significa várias coisas. A especificação da HTML5 não exige nenhuma interface particular para este novo tipo de campo. A maioria dos navegadores desktop como Safari, Chrome, Opera e Firefox simplesmente renderizam uma caixa de texto — exatamente como `type="text"` — logo, seu usuário nunca irá notar a diferença (até tentar enviar o formulário).

Então surge o iPhone.

O iPhone não possui um teclado físico. A digitação é feita por toque no teclado virtual, que surge nos momentos apropriados. Por exemplo, quando você seleciona um campo de um formulário web. A Apple fez algo inteligente no navegador do iPhone. Ele reconhece vários dos novos tipos de campos da HTML5 e *dinamicamente altera o teclado virtual* adaptado para o campo específico.

Por exemplo, o campo de endereço de email é um texto, certo? Certo, mas ele precisa de um tipo especial de texto, praticamente todos os endereços de email contém o símbolo @ e pelo menos um ponto (.), e não costumam ter nenhum espaço. Então quando você usa o iPhone e foca em um campo `<input type="email">`, você tem um teclado virtual que possui uma barra de espaço menor que a usual, mais teclas específicas para os caracteres @ e (.).



Faça você o teste do campo `type="email"`.

Para resumir: Não há nenhuma desvantagem em converter todos os seus campos de email para `type="email"`. Praticamente ninguém vai notar, exceto os usuários do iPhone, que provavelmente não vão notar também. Mas aqueles que notarem irão sorrir timidamente e agradecer a você por tornar a experiência na web um pouco mais fácil.



ENDEREÇOS WEB

Endereços web - que a maioria dos usuários avançados de internet chamam de URLs, exceto por alguns poucos pedantes que chamam de URIs — outro tipo de texto especializado. A sintaxe dos endereços web é definida pelos padrões de relevância da Internet. Se alguém pedir para você digitar um endereço web em um formulário, é esperado algo como “`http://www.google.com/`”, não “Rua Getulio Vargas, 160.” Barras são comuns — inclusive a página inicial do Google possui três delas. Pontos são tão comuns quanto, mas espaços são proibidos. E todos os endereços web possuem um sufixo relacionado a um domínio como: “.com” ou “.org”.

Observem... (rufar de tambores, por favor)... o `<input type="url">`. No iPhone:



Faça você o teste do campo `type="url"`.

O iPhone altera seu teclado virtual, do mesmo modo que ele faz para os campos de endereço de email, só que desta vez otimizado para endereços web. A barra de espaço é trocada por três teclas virtuais: um ponto, uma barra, e um botão “.com”. (Você pode pressionar por um tempo o botão “.com” para escolher outros sufixos comuns como “.org” ou “.net”).

A maioria dos navegadores modernos, simplesmente irão renderizar um campo `type="url"` como uma caixa de texto, logo seus usuários não irão notar a diferença até enviarem o formulário. Navegadores que suportam a HTML5 irão tratar o `type="url"` exatamente como um `type="text"`, por isso não há desvantagem em usá-los para todos os campos que requerem um endereço web.



NÚMEROS EM CAIXAS ROTATIVAS

Em seguida: Números. Pedir um número é mais complicado do que pedir um endereço de e-mail ou endereço de web. Primeiro de tudo, números são mais complicados do que você imagina. Rápido: escolha um número. -1? Não, eu quero um número entre 1 e 10. 7½? Não, não uma fração, bobo. π ? Agora você está começando a ser irracional.

Meu ponto é, você geralmente não pede “apenas um número.” É mais provável que você queira um número em determinado intervalo. E você deseja apenas certo tipo de número neste intervalo - talvez todos os números, mas não frações ou decimais, ou algo mais esotérico como números divisíveis por 10. A HTML5 lhe dá cobertura.

↪ *Escolha um número, (quase) qualquer número*

```
<input type="number"
      min="0"
      max="10"
      step="2"
      value="6">
```

Vamos analisar um atributo por vez. (Você pode acompanhar com este exemplo real, se quiser.)

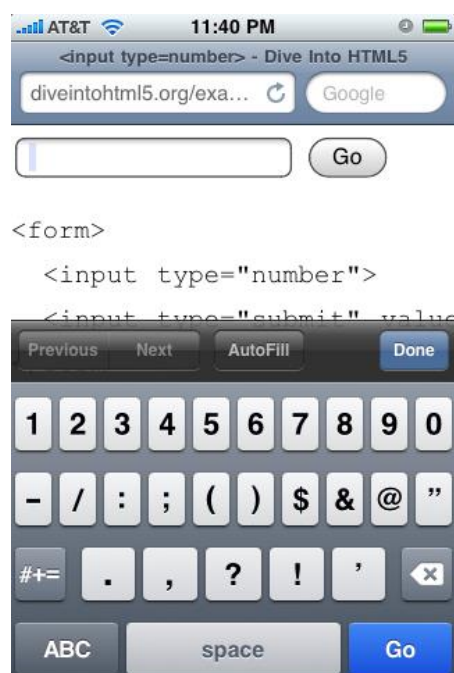
- `type="number"` significa que este é um campo numérico.
- `min="0"` especifica o valor mínimo aceitável para este campo.
- `max="10"` é o valor máximo aceitável.
- `step="2"`, combinado com o valor `min` define o número aceitável dentro do intervalo: 0, 2, 4, e adiante, até o valor `max`
- `value="6"` é o valor padrão. Isto deve parecer familiar. É o mesmo atributo que você sempre usou para especificar valores em um campo de formulário. (Eu mencionei ele aqui para chegar no ponto que a HTML5 é baseada nas versões anteriores da HTML. Você não precisa reaprender como fazer as coisas que você já está fazendo.)

Está é a marcação para campos numéricos. Tenha em mente que todos os atributos são opcionais. Se você tiver um mínimo e não um máximo, você pode especificar o atributo `min` mas não o `max`. O intervalo padrão é 1, podendo omitir o atributo `step`, se não for preciso um intervalo diferente. Se não existir um valor padrão, então o atributo `value` pode ser vazio ou até mesmo omitido.

Mas a HTML5 não acaba aqui. Pelo mesmo preço, de banana, você recebe estes métodos JavaScript super úteis:

- `input.stepUp(n)` aumenta o valor do campo por `n`.
- `input.stepDown(n)` diminui o valor do campo por `n`.
- `input.valueAsNumber` retorna o valor atual como ponto flutuante. (A propriedade `input.value` sempre retorna como string.)

Problemas pra visualizar? Bem, a interface exata do controle numérico depende do seu navegador, os fabricantes dos navegadores implementaram o suporte de diferentes formas. No iPhone, onde digitar é incômodo, o navegador mais uma vez otimiza o teclado virtual para entradas numéricas.



Na versão desktop do Opera, o mesmo campo `type="number"` é renderizado com uma "caixa rotativa", com pequenas setas para cima e para baixo que você pode clicar e alterar o valor.

<form>

```
<input type="number"
      min="0"
      max="10"
      step="2"
      value="6">
```

O Opera respeita os atributos `min`, `max`, and `step`, você sempre irá acabar com um valor numérico aceitável. Caso você aumente o valor até o máximo, a seta superior da caixa rotativa fica desabilitada.

Como acontece com todos os tipos de campos que comentei neste capítulo, navegadores que não suportam `type="number"` irão trata-lo como `type="text"`. O valor padrão irá aparecer no campo (já que é armazenado no atributo `value`), mas os outros atributos como `min` e `max` serão ignorados. Você é livre para implementá-los como quiser, ou pode reutilizar uma biblioteca Javascript que já possui um controle de caixa rotativa. Apenas cheque o suporte nativo da HTML5 primeiro, deste jeito:

```
if (!Modernizr.inputtypes.number) {
  // sem suporte nativo ao campo type=number
  // talvez use Dojo ou outra biblioteca Javascript
}
```



NÚMEROS EM CONTROLES DESLIZANTES

Números em caixas rotativas não são o único modo de representar campos numéricos. Você provavelmente já deve ter visto "controles deslizantes" como este:

Faça você o teste do campo `type="range"`.

Agora você pode ter controles deslizantes no seu formulário web também. A marcação é bem semelhante com as caixas rotativas:

↪ *A imagem para comprovar*

```
<input type="range"
      min="0"
      max="10"
      step="2"
      value="6">
```

Todos os atributos válidos são os mesmos do `type="number"` — `min`, `max`, `step`, `value` — e funcionam do mesmo modo. A única diferença é a interface. Ao invés de ser um campo para digitar, é esperado dos navegadores renderizarem `type="range"` como um controle deslizante. Safari, Chrome e Opera fazem isto. (Infelizmente, o iPhone renderiza uma caixa de texto simples. Que nem ao menos otimiza o seu teclado virtual para entradas numéricas). Todos os outros navegadores simplesmente tratam o campo como `type="text"`, então não há razão para não começar a usá-lo imediatamente.



SELECCIONADORES DE DATA

HTML não possui selecionadores de data. As bibliotecas JavaScript tiveram que resolver esta negligência ([Dojo](#), [jQuery](#), [UI](#), [YUI](#), [Closure Library](#)), mas é claro estas soluções exigem assumir o uso da biblioteca que o selecionador de data é construído.

HTML5 finalmente definiu um modo para incluir selecionadores de data nativos. Sem uso de scripts personalizados. Na verdade, é definido seis: data, mês, semana, hora, data + hora, e data + hora - fuso horário.

Até agora, o suporte é... escasso.

SUPORTE AO SELECCIONADORES DE DATA		
TIPO DE CAMPO	OPERA	QUALQUER OUTRO NAVEGADOR
<code>type="date"</code>	9.0+	.

SUPORTE AO SELECIONADORES DE DATA		
TIPO DE CAMPO	OPERA	QUALQUER OUTRO NAVEGADOR
type="month"	9.0+	.
type="week"	9.0+	.
type="time"	9.0+	.
type="datetime"	9.0+	.
type="datetime-local"	9.0+	.

Assim é como o Opera renderiza `<input type="date">`:

The screenshot shows the Opera browser's date picker for the `type="date"` input. At the top, there is a text input field containing "2009-12-25" and a blue "Go" button. Below this is a calendar interface. The calendar has a header with "December" and "2009", with left and right navigation arrows. The main body is a grid of days from Sunday to Saturday. The date "25" is highlighted with a mouse cursor. At the bottom of the calendar, there are two buttons: "Today" and "None".

Se você precisar da hora junto com a data. O Opera também suporta `<input type="datetime">`:

The screenshot shows the Opera browser's datetime picker for the `type="datetime"` input. At the top, there is a text input field containing "2009-12-25", a time input field containing "16:20", a dropdown menu set to "UTC", and a blue "Go" button. Below this is a calendar interface, identical to the one in the previous screenshot, showing "December 2009" with the date "25" highlighted. At the bottom are "Today" and "None" buttons.

Se você só precisar de mês e ano (por exemplo, a data de vencimento de um cartão de crédito). O Opera pode renderizar `<input type="month">`:

2009-12

December							2009
Mon	Tue	Wed	Thu	Fri	Sat	Sun	
30	1	2	3	4	5	6	
7	8	9	10	11	12	13	
14	15	16	17	18	19	20	
21	22	23	24	25	26	27	
28	29	30	31	1	2	3	
4	5	6	7	8	9	10	

Menos comum, mas também válido, é a possibilidade de selecionar uma semana específica do ano com `<input type="week">`:

2009-W52

December							2009
Week	Mon	Tue	Wed	Thu	Fri	Sat	Sun
49	30	1	2	3	4	5	6
50	7	8	9	10	11	12	13
51	14	15	16	17	18	19	20
52	21	22	23	24	25	26	27
53	28	29	30	31	1	2	3
1	4	5	6	7	8	9	10

Último e não menos importante, você poder selecionar uma hora com `<input type="time">`:

16:20

É provável que os outros navegadores, eventualmente, venham a suportar os selecionadores de data. Mas como `type="email"` e os outros tipos campos, eles serão renderizados como uma caixa de texto, nos navegadores que não reconhecerem o `type="date"` e as outras variações. Se você preferir, pode simplesmente usar `<input type="date">` e companhia, e fazer usuários do Opera felizes, e esperar o suporte dos outros navegadores. Mais realístico, você pode usar `<input type="date">`, detectar se o navegador possui suporte nativo para seletores de data, e utilizar um fallback para um script utilizando a biblioteca a sua escolha ([Dojo](#), [jQuery UI](#), [YUI](#), [Closure Library](#), ou outra solução).

↪ *Selecionador de data com fallback*

```
<form>
  <input type="date">
</form>
...
```



```
<script>
  var i = document.createElement("input");
  i.setAttribute("type", "date");
  if (i.type == "text") {
    // Sem suporte nativo ao selecionador de data :[
    // Utilize Dojo/jqueryUI/YUI/Closure para criar um,
    // então dinamicamente substitua este <input> element.
  }
</script>
```

CAIXAS DE BUSCA

OK, este assunto é sutil. Bem, a ideia é bastante simples, mas as implementações requerem algumas explicações. Aqui vai...

Busca. Não só o buscador do Google ou do Yahoo. (Os demais.) Pense em qualquer caixa de busca de uma página de um site. A Amazon possui uma caixa de busca. Newegg possui uma caixa de busca. A maioria dos blogs possui uma caixa de busca. Como eles são marcados? `<input type="text">`, assim como todas as outras caixas de texto da web. Vamos corrigir isto.

```
<form>
  <input name="q" type="search">
  <input type="submit" value="Find">
</form>
```

« *Nova era dos campos de pesquisa*

Teste `<input type="search">` no seu próprio navegador. Em alguns navegadores, você não irá notar diferença de uma caixa de texto normal. Mas se você utilizar Safari no Mac OS X, ele irá aparecer assim:



```
<form>
  <input type="search">
  <input type="submit" value="Go">
</form>
```

Você pode ver a diferença? O campo possui cantos redondos! Eu sei, Eu sei, você mal consegue conter a empolgação. Mas espere, tem mais! Quando você começa digitar na caixa `type="search"`, o Safari insere um pequeno botão "x" no canto direito da caixa. Clicando

no "x" o conteúdo do campo é limpo. (O Google Chrome, que divide muitas tecnologias com o Safari debaixo do capô, também possui este comportamento.) Ambas dessas pequenas modificações são feitas para combinar com a aparência e comportamento das caixas de buscas nativas no iTunes e outras aplicações do Mac OS X.



```
<form>
  <input type="search">
  <input type="submit" \
</form>
```

Apple.com usa `<input type="search">` para os campos de pesquisa dos seus sites, de forma a incrementar a experiência "Mac". Mas não há nenhuma especificação do Mac sobre. É apenas uma marcação, então cada navegador em cada plataforma pode escolher a renderização de acordo com as especificações e convenções. Como todos os outros novos tipos de campo, navegadores que não reconhecem `type="search"` irão tratá-lo como `type="text"`, logo não existe absolutamente nenhuma razão para na começar usar `type="search"` para todas as suas caixas de pesquisa hoje.

Por padrão, versões antigas do Safari não aplicam nem os mais básicos estilos CSS no campo `<input type="search">`. Se você quer forçar o Safari tratar seus campos de pesquisa como um campo de texto normal (para poder inserir seus próprios estilos CSS), adicione esta regra em sua folha de estilo:

```
input[type="search"] {
  -webkit-appearance: textfield;
}
```

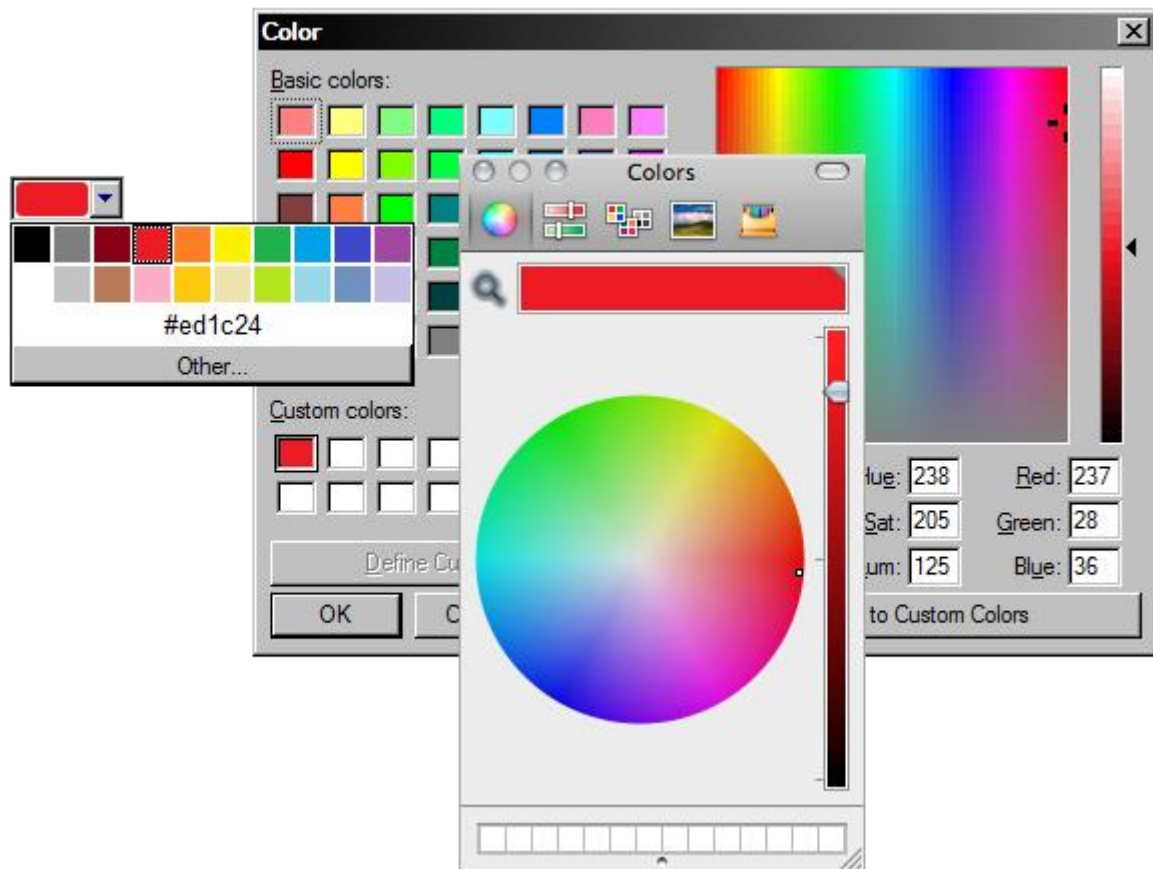
Obrigado a Jonh Lein por me ensinar este truque.



SELECIONADORES DE COR

HTML5 também possui `<input type="color">`, que permite você selecionar uma cor e retornar sua representação hexadecimal. ~~Nenhum navegador suporta por enquanto, o que é uma vergonha, porque eu sempre adorei o selecionador de cor do Mac OS. Talvez algum dia.~~ Boas notícias,

peçoal! Opera 11 agora suporta `type=color`. No Mac e no Windows, o campo é integrado com o selecionador de cor nativo da plataforma. No Linux, desce um seletor de cores básico. Em todas as outras plataformas, o campo possui um limite de seis caracteres hexadecimal de cor RGB, apropriado para utilizar onde aceitar uma cor CSS.



Teste você mesmo o campo `type="color"`.

Obrigado ao Patrick Lauke e Chris Mills por liberar esta imagem para ser inclusa no livro. Você deveria ler o artigo deles sobre as novas características dos formulários no Opera 11.



VALIDAÇÃO DE FORMULÁRIOS

SUPORTE A VALIDAÇÃO DE FORMULÁRIO						
IE	FIREFOX	SAFARI	CHROME	OPERA	IPHONE	ANDROID
.	4.0+	5.0+	10.0+	9.0+	.	.

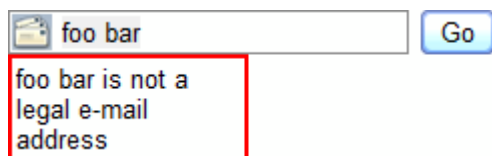
Neste capítulo, eu falo sobre os novos tipos de campos e suas características, como foco automático, mas eu não mencionei o que é, talvez, a parte mais emocionante nos formulários HTML5: validação automática dos campos. Considerando o problema comum de digitar um endereço de email em um formulário web. Você provavelmente possui uma validação no lado do cliente em JavaScript, seguido por uma no lado do servidor em PHP ou Python ou outra linguagem de servidor. HTML5 nunca poderá substituir a validação do servidor, mas pode um dia substituir a do lado do cliente.

Há dois grandes problemas em validar um endereço de email em JavaScript:

1. Um número surpreendente do seus visitantes (provavelmente cerca de 10%) não possui JavaScript habilitado
2. Você vai fazer errado.

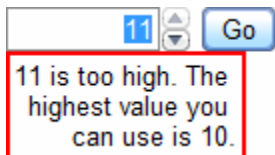
Sério, você vai fazer errado. Determinar se uma sequência aleatória de caracteres é um endereço de email válido é inacreditavelmente complicado. Quanto mais você olha, mais complicado fica. Eu já falei que é muito, muito complicado? Não seria mais fácil deixar esta dor de cabeça para o navegador?

Navegadores modernos validam o type="email" ~



A imagem de tela pertence ao Opera 10, porém a funcionalidade está presente desde o Opera 9. Firefox 4 e Chrome 10 também fornecem a mesma funcionalidade. A única marcação envolvida é setar o atributo type para "email". Quando o usuário tentar enviar o formulário com um campo `<input type="email">` o navegador automaticamente oferece uma validação RFC-compliant, mesmo se os scripts estiverem desabilitados.

HTML5 também oferece validação para endereços web no campo `<input type="url">` e de números no `<input type="number">`. A validação de números também leva em conta os atributos `min` e `max` attributes, ou seja os navegadores não irão permitir enviar o formulário se você digitar um número que seja muito grande.



Não há nenhuma marcação para ativar a validação de formulário do HTML5; é feita por padrão. Para desligar, use o atributo novalidate.

Não me valide, parceiro ~

```
<form novalidate>
  <input type="email" id="addr">
  <input type="submit" value="Subscribe">
</form>
```



CAMPOS OBRIGATÓRIOS

SUPORTE AOS CAMPOS OBRIGATÓRIOS						
IE	FIREFOX	SAFARI	CHROME	OPERA	IPHONE	ANDROID
.	4.0+	.	10.0+	9.0+	.	.

Validação de formulários com HTML5 não se limita a cada tipo de campo. Você pode também especificar que certo campo é obrigatório. Os campos obrigatórios precisam de um valor antes de serem enviados.

A marcação para um campo obrigatório é a mais simples possível:

```
<form>
  <input id="q" required>
  <input type="submit" value="Search">
</form>
```

Teste `<input required>` em seu navegador. Navegadores podem alterar a aparência padrão dos campos obrigatórios. Por exemplo, assim é como um campo obrigatório parece no Mozilla Firefox 4.0

A screenshot of a web form. It consists of a single-line text input field with a red border and a 'Go' button to its right. The input field is empty.

```
<form>
  <input name="q" required>
  <input type="submit" value="Go">
</form>
```

Além disso, se você tentar enviar o formulário sem preencher o valor requerido, o Firefox irá mostrar uma mensagem informando que o campo é obrigatório e não pode ser deixado em branco.

LEITURA COMPLEMENTAR

Especificações e normas:

- tipos de `<input>`
- o atributo `<input placeholder>`
- o atributo `<input autofocus>`
- o atributo `<form novalidate>`
- o atributo `<input required>`

Bibliotecas JavaScript

- Modernizr, biblioteca de detecção de HTML5

Artigos Úteis

- Pensando pra frente Validação de formulários
- Novos recursos de formulários no Opera 11
- Mozilla Developer Center: Formulários na HTML5
- Formulários na HTML5 no Mozilla Firefox 4.0+
- Validação de formulários na HTML5

Nº10

“DISTRIBUÍDO”, “EXTENSÍVEL”, E OUTRAS PALAVRAS BONITAS MERGULHANDO



Existem mais de 100 elementos na HTML5. Alguns são puramente semânticos, outros são apenas recipientes para APIs de script. Ao longo da história da HTML, conhecedores dos padrões vêm discutindo sobre que elementos devem ser incluídos na linguagem. O elemento `<figure>` deveria ser incluso na HTML? O elemento `<person>`? Que tal um elemento `<rant>`? Decisões são tomadas, especificações são escritas, atores atuam, implementadores implementam, e a web dá um grande passo à frente.

Claro, HTML não pode agradar a todos. Nenhum padrão pode. Algumas ideias não atendem aos requisitos. Por exemplo, não existe o elemento `<person>` na HTML5 (Não há também o elemento `<rant>`, droga!). Nada impede de você usar o elemento `<person>` numa página web, porém não vai validar, não vai funcionar consistentemente nos navegadores, e poderá conflitar com especificações futuras da HTML, se o elemento for adicionado mais à frente.

Certo, então se fazer seu próprio elemento não é a resposta, o que faz um escritor semanticamente correto? Houveram tentativas de estender as versões anteriores da HTML. O método mais popular são os microformats, que usam os atributos `class` e `rel` na HTML. Outra opção é RDFa, que foi originalmente projetado para ser usado na XHTML mas também foi portado para HTML.

Microformats e RDFa cada um tem seus pontos fortes e fracos. Eles usam abordagens completamente diferentes em direção ao mesmo objetivo: estender as páginas web com semântica adicional que não é parte do núcleo da linguagem HTML. Eu não pretendo transformar este capítulo em um formato “flamewar”. (Isso definitivamente precisaria de um elemento `<rant>`!) Em vez disso, quero focar em uma terceira opção desenvolvida usando as lições aprendidas a partir de microformats e RDFa, e projetada para ser íntegra na própria HTML5: microdata.



O QUE É MICRODATA?

Cada palavra na sentença seguinte é importante, então preste atenção.

Microdata escreve o DOM com escopo dos pares de nome/valor a partir de vocabulários personalizados.

Sim, mas o que isso significa? Vamos começar do fim e vamos retrocedendo. Microdados giram em torno de vocabulários personalizados. Pense no “conjunto de todos os elementos da HTML5” como um vocabulário. Este vocabulário inclui elementos para representar uma seção ou um artigo, mas isto não inclui elementos para representar uma pessoa ou um evento. Se você deseja representar uma pessoa na página web, você vai precisar definir seu próprio vocabulário. Microdata deixa você fazer isso. Qualquer um pode definir um vocabulário microdata e começar a incorporar propriedades personalizadas em suas próprias páginas web.

A próxima coisa a fazer para conhecer o microdata é que ele funciona com pares de nome/valor. Todo vocabulário microdata define um conjunto de propriedades nomeadas. Por exemplo, um vocabulário Pessoa poderia definir propriedades como `nome` e `foto`. Para incluir um microdata específico na sua página web, você pode fornecer o nome da propriedade em um lugar específico. Dependendo de onde você declara o nome da propriedade, microdata tem regras sobre como extrair o valor da propriedade. (Mais sobre isto na próxima seção)

Junto com propriedades nomeadas, microdados dependem muito do conceito de “escopo.” A maneira mais simples de pensar no escopo do microdata é pensar na relação pai-filho natural dos elementos no DOM. O elemento `<html>` geralmente contém dois filhos, `<head>` e `<body>`. O elemento `<body>` geralmente contém vários filhos, cada um deles pode ter seus próprios elementos-filho. Por exemplo, sua página pode incluir um elemento `<h1>` dentro de um elemento `<hgroup>` dentro de um elemento `<header>` que está dentro de um elemento `<body>`. Uma tabela de informações pode conter `<td>` dentro de `<tr>` dentro de um `<table>` (Dentro do `<body>`). Microdata reutiliza a estrutura hierarquica do próprio DOM para fornecer uma forma de dizer “todas as propriedades dentro *deste* elemento são tomadas *deste* vocabulário.” Isso permite que você use mais de um vocabulário microdata na mesma página. Você pode até aninhar vocabulários microdata dentro de outros vocabulários, tudo pelo reuso da estrutura natural do DOM. (Irei mostrar vários exemplos de vocabulários aninhados ao longo deste capítulo)

Agora, já toquei no assunto DOM, mas deixe-me dissertar sobre isso. Microdata se trata de aplicar semântica adicional para *informações que já são visíveis na sua página web*. Microdata não foi projetada para ser um formato de dados independente. Ele é um complemento da HTML. Como você verá na próxima seção, microdata funciona melhor quando você já está usando a HTML corretamente, porém o vocabulário da HTML não é expressivo o bastante. Microdata

é excelente para aprimorar a semântica da informação que já está no DOM. Se a informação que você está tentando tornar semântica não está no DOM, você deveria voltar atrás e reavaliar se microdata é a solução correta.

Essa sentença faz mais sentido agora? “Microdata escreve o DOM com escopo dos pares de nome/valor a partir de vocabulários personalizados.” Espero que sim. Vamos ver isto na prática.



MODELO DE DADOS DO MICRODATA

Definir seu próprio vocabulário microdata é fácil. Primeiro, você precisa de um namespace, que é apenas uma URL. O namespace URL pode realmente apontar para uma página web funcionando, apesar de não ser estritamente necessário. Vamos dizer que eu quero criar um vocabulário microdata que descreve uma pessoa. Se tenho o domínio `data-vocabulary.org`, irei usar a URL `http://data-vocabulary.org/Person` como o namespace para meu vocabulário microdata. Essa é uma maneira fácil de criar um identificador global exclusivo: escolha uma URL em um domínio que você tem controle.

Nesse vocabulário, eu preciso definir algumas propriedades nomeadas. Vamos começar com três propriedades básicas:

- `name` (seu nome completo)
- `photo` (um link para uma foto sua)
- `url` (um link para um site associado a você, como um blog ou um perfil do Google)

Algumas dessas propriedades são URLs, outras são texto plano. Cada uma delas se presta a forma natural de marcação, mesmo antes de você começar a pensar em microdata ou vocabulários ou outros enfeites. Imagine que você tem um perfil ou uma página “sobre”. Seu nome provavelmente é marcado como um título, como um elemento `<h1>`. Sua foto provavelmente é um elemento `<img>`, já que você quer que as pessoas vejam. E quaisquer URLs associadas ao seu perfil provavelmente já estão marcadas como hyperlinks, pois você precisa que as pessoas sejam aptas a clicar neles. Para fins de discussão, vamos dizer que todo seu perfil também é englobado em um elemento `<section>` para separá-lo do resto do conteúdo da página. Assim:

↪ *Minhas informações*

```
<section>
  <h1>Mark Pilgrim</h1>
  <p></p>
  <p><a href="http://diveintomark.org/">blog</a></p>
</section>
```

O modelo microdata são pares de nome/valor. A propriedade microdata name (como name ou photo ou url neste exemplo) é sempre declarado no elemento HTML. A propriedade value correspondente é então retirada do elemento DOM. Para muitos dos elementos HTML, a propriedade value é simplesmente o conteúdo textual do elemento. Porém, há uma boa quantidade de exceções.

DE ONDE VÊM AS PROPRIEDADES VALUE?	
Elemento	Valor
<meta>	Atributo content
• <audio> • <embed> • <iframe> • • <source> • <video>	Atributo src
• <a> • <area> • <link>	Atributo href
<object>	Atributo data
<time>	Atributo datetime
Demais elementos	Conteúdo textual

“Adicionar microdata” na sua página é uma questão de adicionar alguns atributos nos elementos HTML que você já tem. A primeira coisa que você sempre faz é declarar que vocabulário microdata você está usando, adicionando um atributo `itemtype`. A segunda coisa que você sempre faz é declarar o escopo do vocabulário, usando um atributo `itemscope`. Neste exemplo, toda a informação que queremos tornar semântica é um elemento `<section>`, então vamos declarar os atributos `itemtype` e `itemscope` no elemento `<section>`.

```
<section itemscope itemtype="http://data-vocabulary.org/Person">
```

Seu nome é o primeiro pedaço de informação dentro do elemento `<section>`. Ele está englobado em um elemento `<h1>`. O elemento `<h1>` não tem nenhum tratamento especial no modelo de dados microdata da HTML5, por isso se enquadra na regra “demais elementos” onde a propriedade microdata value é simplesmente conteúdo textual de um elemento. (Isso funcionaria igualmente bem se o seu nome for envolvido em um elemento `<p>`, `<div>`, ou `<span>`.)

```
<h1 itemprop="name">Mark Pilgrim</h1>
```

Traduzindo, isto quer dizer “aqui está a propriedade name do vocabulário <http://data-vocabulary.org/Person>, e o valor da propriedade é Mark Pilgrim.”

Em seguida: a propriedade photo. Isto deveria ser uma URL. De acordo com o modelo de dados microdata da HTML5, o “value” de um elemento `<img>` é seu atributo `src`. Ei! Olhe, a URL da foto do seu perfil já está em um atributo `<img src>`. Tudo que você precisa fazer é declarar que o elemento `<img>` é a propriedade photo.

```
<p></p>
```

Traduzindo, isto quer dizer “aqui está a propriedade photo do vocabulário <http://data-vocabulary.org/Person>, e o valor da propriedade é <http://www.example.com/photo.jpg>.”

Finalmente, a propriedade url também é uma URL. De acordo com o modelo de dados microdata da HTML5, o “value” de um elemento `<a>` é seu atributo `href`. E mais uma vez, isto funciona perfeitamente com a sua marcação existente. Tudo que você precisa fazer é dizer que seu elemento `<a>` existente é a propriedade url:

```
<a itemprop="url" href="http://diveintomark.org/">dive into mark</a>
```

Traduzindo, isto quer dizer “aqui está a propriedade url do vocabulário <http://data-vocabulary.org/Person>, e o valor da propriedade é <http://diveintomark.org/>.”

Claro, se sua marcação parece um pouco diferente, não tem problema. Você pode adicionar propriedades e valores microdata em qualquer marcação HTML, até mesmo “deformado do século 20”, “tabelas para layout”, “meu Deus por que eu concordei em manter esta” marcação.

↪ *Pelo amor de Deus, não faça isso*

```
<TABLE>
  <TR><TD>Nome<TD>Mark Pilgrim
  <TR><TD>Link<TD>
    <A href=# onclick=goExternalLink()>http://diveintomark.org/</A>
</TABLE>
```

Para marcar a propriedade name, apenas adicione um atributo `itemprop` na célula da tabela que contém o nome. Células de tabela não têm regras especiais no valor da propriedade microdata table, então elas pegam o valor padrão, “a propriedade microdata é o conteúdo textual.”

```
<TR><TD>Nome<TD itemprop="name">Mark Pilgrim
```

Adicionar a propriedade url parece complicado. Essa marcação não usa o elemento `<a>` corretamente. Em vez de colocar o destino do link no atributo `href`, não tem nada

de útil no atributo href e usa Javascript no atributo onclick para chamar a função (não visível) que extrai a URL e navega até ela. Para extra “caralho, pare de fazer isso por favor” pontos de bônus, vamos supor que a função abre também o link numa minúscula janela popup sem barra de rolagem. Não foi a diversão da internet do século passado?

De qualquer forma, você ainda pode converter isso em propriedade microdata, você precisa apenas ser um pouco criativo. Usar o elemento <a> diretamente está fora de questão. O destino do link não está no atributo href, e não há forma de passar por cima da regra que diz “em um elemento <a>, procure o valor da propriedade microdata no atributo href.” Mas você *pode* adicionar um elemento englobador em torno de toda a bagunça, e usar ele para adicionar a propriedade microdata url.

↪ Isto é o que você ganha por subverter a HTML

```
<TABLE itemscope itemtype="http://data-vocabulary.org/Person">
  <TR><TD>Nome<TD>Mark Pilgrim
  <TR><TD>Link<TD>
    <span itemprop="url">
      <A href=#
onclick=goExternalLink()>http://diveintomark.org/</A>
    </span>
</TABLE>
```

Já que o elemento não tem processamento especial, ele usa regras padrões, “a propriedade microdata está no conteúdo textual.” “Conteúdo textual” não significa “toda marcação dentro deste elemento” (como você pegaria com, digamos, a propriedade DOM innerHTML). Isso significa “apenas o texto, senhora”. Nesse caso, http://diveintomark.org/, o conteúdo textual do elemento <a> dentro do elemento .

Para resumir: você pode adicionar propriedades microdata em qualquer marcação. Se você está usando HTML corretamente, você vai achar mais fácil adicionar microdata do que se a sua marcação HTML for uma sujeira, mas o microdata pode sempre ser aplicado.



MARCANDO “PEOPLE”

A propósito, os exemplos iniciais na seção anterior não foram completamente formados. Existe realmente um vocabulário microdata para marcar informação sobre pessoas, e ele é realmente assim fácil. Vamos dar uma olhada.

A maneira mais fácil de integrar microdata em um website pessoal é na sua página “sobre”. Você *com certeza* tem uma página “sobre”, não é verdade? Se não, você pode seguir como eu estudando este exemplo de página “sobre” com semânticas adicionais. O resultado final está aqui: person-plus-microdata.html.

Primeiro vamos olhar para a marcação crua, antes de qualquer propriedade microdata ser adicionada:

```
<section>
  

  <h1>Informação para Contato</h1>
  <dl>
    <dt>Nome</dt>
    <dd>Mark Pilgrim</dd>

    <dt>Cargo</dt>
    <dd>Developer Advocate na Google, Inc.</dd>

    <dt>Endereço Postal</dt>
    <dd>
      100 Main Street<br>
      Anytown, PA 19999<br>
      USA
    </dd>
  </dl>
  <h1>Meus rastros digitais</h1>
  <ul>
    <li><a href="http://diveintomark.org/">weblog</a></li>
    <li><a href="http://www.google.com/profiles/pilgrim">Google
profile</a></li>
    <li><a href="http://www.reddit.com/user/MarkPilgrim">Reddit.com
profile</a></li>
    <li><a href="http://www.twitter.com/diveintomark">Twitter</a></li>
  </ul>
</section>
```

A primeira coisa que você sempre precisa fazer é declarar o vocabulário que você está usando, e o escopo das propriedades que você quer adicionar. Você faz isso adicionando os atributos `itemtype` e `itemscope` no elemento exterior que contém os outros elementos que contém os dados reais. Neste caso, ele é um elemento `<section>`.

```
<section itemscope itemtype="http://data-vocabulary.org/Person">
```

[Acompanhe comigo! Antes: person.html, depois: person-plus-microdata.html]

Agora você pode começar definindo propriedades microdata do vocabulário <http://data-vocabulary.org/Person>. Mas o que são essas propriedades? Como acontece, você pode ver a lista de propriedades navegando no data-vocabulary.org/Person em seu navegador. A especificação do microdata não necessita disso, mas eu diria que isso é certamente uma “melhor prática.” Afinal, se você deseja que os desenvolvedores *usem* de verdade seu vocabulário microdata, você precisa documentá-lo. E onde mais colocar sua documentação do que na própria URL do vocabulário?

VOCABULÁRIO PERSON	
Propriedade	Descrição
name	Nome
nickname	Apelido
photo	Um link de imagem
title	O cargo da pessoa (por exemplo, “Gerente Financeiro”)
role	O papel da pessoa (Por exemplo, “Contador”)
url	Link para uma página web, tal como a página pessoal da pessoa
affiliation	O nome da organização com que a pessoa é associada (por exemplo, um empregador)
friend	Identifica uma relação social entre a pessoa descrita e outra pessoa
contact	Identifica uma relação social entre a pessoa descrita e outra pessoa
acquaintance	Identifica uma relação social entre a pessoa descrita e outra pessoa
address	A localização da pessoa. Pode ter as subpropriedades <code>street-address</code> , <code>locality</code> , <code>region</code> , <code>postal-code</code> , e <code>country-name</code> .

A primeira coisa neste exemplo de página “sobre” é uma foto minha. Naturalmente, ela é marcada com elemento `<img>`. Para declarar que este elemento `<img>` é a foto do meu perfil, tudo que precisamos fazer é adicionar `itemprop="photo"` no elemento `<img>`.

```

```

[Acompanhe comigo! Antes: [person.html](#), depois: [person-plus-microdata.html](#)]

Onde está o valor da propriedade microdata? Já está lá, no atributo `src`. Se você chamar novamente no modelo de dados microdata da HTML5, o “valor” de um elemento `<img>` é seu atributo `src`. Cada atributo `<img>` tem um atributo `src` — caso contrário, seria apenas uma imagem quebrada — e o `src` é sempre uma URL. Viu? Se você está usando HTML corretamente, microdata é fácil.

Além disso, esse elemento `<img>` não está sozinho na página. Ele é um elemento filho de um elemento `<section>`, aquele que acabamos de declarar com o atributo `itemscope`. Microdata reusa o relacionamento pai-filho dos elementos na página para definir o escopo das propriedades microdata. Em linguagem clara, estamos dizendo, “Este é elemento `<section>` representa uma pessoa. Qualquer propriedade microdata que você venha encontrar nos filhos do elemento `<section>` são propriedades desta pessoa.” Se isso ajuda, você pode pensar que o elemento `<section>` tem o assunto de uma frase. O atributo `itemprop` representa o verbo da frase, algo como “é fotografada em.” O valor da propriedade microdata representa o objeto da frase.

Esta pessoa [explícito, do `<section itemscope itemtype="...">`]

[é fotografada em] [explícito, do `<img itemprop="photo">`]

https://diveintohtml5.com.br/examples/2000_05_mark.jpg [implícito, do atributo `<img src>`]

O assunto precisa ser definido apenas uma vez, colocando os atributos `itemscope` e `itemtype` no elemento externo `<section>`. O verbo é definido colocando o atributo `itemprop="photo"` no elemento `<img>`. O objeto da sentença não precisa de nenhuma marcação especial, pois o modelo de dados microdata da HTML5 diz que a propriedade valor de um elemento `<img>` é o seu atributo `src`.

Passando para o próximo pedaço de marcação, vemos um título `<h1>` e o começo de uma lista `<dl>`. Nem o `<h1>` nem o `<dl>` precisam ser marcados com microdata. Não é toda parte da HTML que precisa ser uma propriedade microdata. Microdata trata-se das próprias propriedades, não da marcação ou cabeçalhos em torno das propriedades. Esse `<h1>` não é uma propriedade; é apenas um título. Semelhantemente, o código `<dt>` que diz “Nome” não é uma propriedade; é apenas um rótulo.

↪ *Chato*

Chato ↪

```
<h1>Informações de contato</h1>
<dl>
  <dt>Nome</dt>
  <dd>Mark Pilgrim</dd>
```

Então onde está a verdadeira informação? Ela está no elemento `<dd>`, então é nele que precisamos colocar o atributo `itemprop`. Que propriedade é essa? É a propriedade `name`. Onde está o valor da propriedade? Ele é o texto dentro do elemento `<dd>`. Ele precisa ser marcado?

o modelo de dados microdata da HTML5 diz que não, elementos <dd> não têm processamento especial, então o valor da propriedade é apenas o texto dentro do elemento.

↪ *Este é meu nome, não use-o por aí*
<dd itemprop="name">Mark Pilgrim</dd>

[Acompanhe comigo! Antes: person.html, depois: person-plus-microdata.html]

O que acabamos de dizer, traduzindo? O nome “dessa pessoa é Mark Pilgrim.” Bom, Ok então. Adiante.

As próximas duas propriedades são um pouco complicado. Essa é a marcação, pré-microdata:

```
<dt>Cargo</dt>
<dd>Developer Advocate na Google, Inc.</dd>
```

Se você olhar na definição do vocabulário Pessoa, o texto “Developer Advocate na Google, Inc.” na verdade engloba *duas* propriedades: **title** (“Developer Advocate”) e **affiliation** (“Google, Inc.”). Como você pode expressar isso em microdata? A resposta curta é, você não pode. Microdata não tem uma forma de quebrar textos corridos em propriedades separadas. Você não pode dizer “os primeiros 18 caracteres desse texto é uma propriedade de microdata, e os 12 caracteres finais desse texto são outra propriedade microdata.”

Mas nem tudo está perdido. Imagine que você gostaria de estilizar o texto “Developer Advocate” em uma fonte diferente do texto “Google, Inc.” CSS também não consegue fazer isso. Então o que você faria? Primeiramente você precisaria englobar os diferentes pedaços de texto em elementos “fictícios”, como , em seguida aplicar regras CSS diferentes para cada elemento .

Essa técnica também é útil para microdata. Existem dois pedaços distintos de informação aqui: um **title** e um **affiliation**. Se você engloba cada pedaço em um elemento “fictício”, você pode declarar que cada é uma propriedade microdata separada.

```
<dt>Cargo</dt>
<dd><span itemprop="title">Developer Advocate</span> na
    <span itemprop="affiliation">Google, Inc.</span></dd>
```

[Acompanhe comigo! Antes: person.html, depois: person-plus-microdata.html]

Tada! O título “dessa pessoa é 'Developer Advocate.' Essa pessoa é contratada pela Google, Inc.” Duas sentenças, duas propriedades microdata. Um pouco mais de marcação, mas uma compensação que vale a pena.

A mesma técnica é útil para marcar logradouro. O vocabulário `Person` define uma propriedade `address`, que é um próprio item microdata. Isso significa que o endereço tem seu próprio vocabulário (<http://data-vocabulary.org/Address>) e define suas próprias propriedades. O vocabulário `Address` define 5 propriedades: `street-address`, `locality`, `region`, `postal-code`, e `country-name`.

Se você é um programador, você provavelmente está familiarizado com notação de ponto para definir objetos e suas propriedades. Pense na relação como esse:

- `Person`
- `Person.address`
- `Person.address.street-address`
- `Person.address.locality`
- `Person.address.region`
- `Person.address.postal-code`
- `Person.address.country-name`

Nesse exemplo, todo o logradouro está contido em um único elemento `<dd>`. (Novamente, o elemento `<dt>` é apenas um rótulo, logo ele não desempenha nenhum papel na adição de semântica com microdata.) Notar a propriedade `address` é fácil. Adicione apenas um atributo `itemprop` no elemento `<dd>`.

```
<dt>Endereço Postal</dt>
<dd itemprop="address">
```

[Acompanhe comigo! Antes: `person.html`, depois: `person-plus-microdata.html`]

Mas lembre-se, a propriedade `address` é propriamente um item microdata. Isso significa que também precisamos adicionar os atributos `itemscope` e `itemtype`.

```
<dt>Endereço Postal</dt>
<dd itemprop="address" itemscope
    itemtype="http://data-vocabulary.org/Address">
```

[Acompanhe comigo! Antes: `person.html`, depois: `person-plus-microdata.html`]

Vemos isso tudo antes, porém para itens de alto nível. Um elemento `<section>` define `itemtype` e `itemscope`, e todos elementos dentro do elemento `<section>` que definem as propriedades microdata são “escopadas” dentro desse vocabulário específico. Mas esse é o primeiro caso que vemos escopos *aninhados* — definir um novo `itemtype` e `itemscope` (no elemento `<dd>`) dentro de um já existente (no elemento `<section>`). Esse escopo aninhado funciona exatamente como o DOM HTML. O elemento `<dd>` tem um certo número de elementos filho, todos que estão no escopo do vocabulário definido no elemento `<dd>`. Uma vez que o elemento `<dd>` é fechado com uma tag `</dd>` correspondente, o escopo reverte para o vocabulário definido pelo elemento pai (`<section>`, nesse caso).

As propriedades do Address sofrem o mesmo problema que nós encontramos com as propriedades `title` e `affiliation`. Existe apenas um longo texto corrido, porém nós queremos quebrá-lo em cinco propriedades microdata separadas. A solução é a mesma: englobe cada pedaço destinto de informação é um elemento `<span>` fictício, em seguida declare propriedades microdata em cada elemento `<span>`.

```
<dd itemprop="address" itemscope
    itemtype="http://data-vocabulary.org/Address">
    <span itemprop="street-address">100 Main Street</span><br>
    <span itemprop="locality">Anytown</span>,
    <span itemprop="region">PA</span>
    <span itemprop="postal-code">19999</span>
    <span itemprop="country-name">USA</span>
</dd>
</dl>
```

[Acompanhe comigo! Antes: [person.html](#), depois: [person-plus-microdata.html](#)]

Traduzindo: “Esta pessoa tem um endereço postal. O logradouro parte do endereço postal é '100 Main Street.' A localidade é 'Anytown.' A região é 'PA.' O código postal é '19999.' O nome do país é 'USA.'” Extremamente simples.

P: Esse formato de endereço postal é específico dos EUA?
R: Não. As propriedades do vocabulário Address são genéricas o suficiente que podem descrever vários endereços postais do mundo. Nem todo endereço terá os valores para cada propriedade, mas não tem problema. Alguns endereços podem exigir mais de uma “linha” em uma única propriedade, mas sem problemas também. Por exemplo, se o seu endereço postal tem uma rua e um número de suíte, eles podem ir juntos dentro da subpropriedade `street-address`:

```
<p itemprop="address" itemscope
    itemtype="http://data-vocabulary.org/Address">
    <span itemprop="street-address">
        100 Main Street
        Suite 415
    </span>
    ...
</p>
```

Não há nada de mais nessa página “sobre”: uma lista de URLs. O vocabulário Person tem a propriedade para isso, chamada de `url`. A propriedade `url` pode ser qualquer coisa, de verdade. (Bem, ela deve ser uma URL, mas você provavelmente já adivinhou isso.) O que quero dizer é que a propriedade `url` é vagamente definida. A propriedade pode ser qualquer tipo de URL que você queira associar a uma pessoa: um blog, uma galeria de fotos, ou um perfil em um outro site como Facebook ou Twitter.

Outra coisa importante de notar aqui é que uma única pessoa pode ter várias propriedades `url`. Tecnicamente, qualquer propriedade pode aparecer mais de uma vez, mas até agora, não tiramos proveito disso. Por exemplo, poderíamos ter duas propriedades `photo`, cada uma apontando para uma URL de imagem diferente. Aqui, gostaria de listar quatro URLs diferentes: meu blog, meu perfil do Google, meu perfil de usuário no Reddit, e minha conta no Twitter. Na HTML, está uma lista de links: quatro elementos `<a>`, cada um em seu próprio elemento `<li>`. Em microdata, cada elemento `<a>` leva um atributo `itemprop="url"`.

```
<h1>Meus rastros digitais</h1>
<ul>
  <li><a href="http://diveintomark.org/"
    itemprop="url">blog</a></li>
  <li><a href="http://www.google.com/profiles/pilgrim"
    itemprop="url">Perfil do Google</a></li>
  <li><a href="http://www.reddit.com/user/MarkPilgrim"
    itemprop="url">Perfil do Reddit.com</a></li>
  <li><a href="http://www.twitter.com/diveintomark"
    itemprop="url">Twitter</a></li>
</ul>
```

De acordo com o modelo de dados microdata da HTML5, elementos `<a>` tem processamento especial. O valor da propriedade microdata é o atributo `href`, não o conteúdo textual filho. O texto de cada link é na verdade ignorado pelo processador microdata. Assim, traduzindo, isso quer dizer “Essa pessoa tem uma URL no `http://diveintomark.org/`. Essa pessoa tem outra URL no `http://www.reddit.com/user/MarkPilgrim`. Ela tem outra URL no `http://www.twitter.com/diveintomark`.”

APRESENTANDO O GOOGLE RICH SNIPPETS

Quero voltar atrás apenas um momento e perguntar, “Por que estamos fazendo isso?” Estamos adicionando semântica apenas por questão de adicionar semântica? Não me leve a mal; Gosto de mexer com colchetes tanto quanto o internauta ao lado. Mas por que microdata? Por que se preocupar?

Existem duas principais classes de aplicações que consomem HTML, e por extensão, HTML5 microdata:

1. Navegadores Web
2. Motores de busca

Para navegadores, HTML5 define um conjunto de APIs DOM para extrair itens microdata, propriedades, e valores de propriedades de uma página web. No momento que escrevo

(Fevereiro 2011), nenhum navegador suporta essa API. Nenhuma delas. Então, isso é... uma espécie de beco sem saída, pelo menos até os navegadores pegarem e implementarem as APIs no lado do cliente.

Outro grande consumidor de HTML são os motores de busca. O que poderia um motor de busca fazer com propriedades microdata sobre uma pessoa? Imagine isso: em vez de simplesmente exibir o título da página e um resumo de texto, o motor de busca poderia integrar algumas das informações estruturadas e mostrá-las. Nome completo, cargo, empregador, talvez até uma miniatura da foto de um perfil. Isso chamaria sua atenção? Isso chamaria a minha.

Google apoia microdata como parte de seu programa Rich Snippets. Quando web crawler do Google analisa sua página e encontra propriedades microdata que obedecem ao vocabulário <http://data-vocabulary.org/Person>, ele analisa essas propriedades e armazena elas juntamente com o restante dos dados da página. Google ainda oferece uma ferramenta útil para ver como o Google “enxerga” suas propriedades microdata.

Item

```
Type: http://data-vocabulary.org/person
photo = https://diveintohtml5.com.br/examples/2000_05_mark.jpg
name = Mark Pilgrim
title = Defensor do Desenvolvedor
affiliation = Google, Inc.
address = Item( 1 )
url = http://diveintomark.org/
url = http://www.google.com/profiles/pilgrim
url = http://www.reddit.com/user/MarkPilgrim
url = http://www.twitter.com/diveintomark
```

Item 1

```
Type: http://data-vocabulary.org/address
street-address = 100 Main Street
locality = Anytown
region = PA
postal-code = 19999
country-name = USA
```

Está tudo lá: a propriedade photo do atributo , todas as quatro URLs da lista de atributos <a href>, até o objeto endereço (listado como “Item 1”) e todos suas cinco subpropriedades.

E como o Google usa toda essa informação? Isso depende. Não há regras rápidas e rígidas sobre como as propriedades microdata devem ser exibidas, qual delas devem ser exibida, ou se todas elas devem ser exibida. Se alguém buscar por “Mark Pilgrim,” e o Google determina que essa página “sobre” deve ser ranqueado nos resultados, e o Google decide que aquela propriedade microdata originalmente encontrados na página valem exibição, então o resultado de busca deve parecer algo parecido com isto: s

Sobre Mark Pilgrim

Anytown PA - Defensor do Desenvolvedor - Google, Inc.

Trecho da página aparecerá aqui.

Trecho da página aparecerá aqui.

diveintohtml5.com.br/examples/person-plus-microdata.html - Cached - Páginas similares

A primeira linha, “Sobre Mark Pilgrim,” é na verdade o título da página, dado no elemento `<title>`. Isso não é terrivelmente maravilhoso; Google faz isso para todas as páginas. Mas a segunda linha é cheia de informação tirada diretamente das anotações microdata que adicionamos à página. “Anytown PA” foi parte do endereço postal, marcado com o vocabulário `http://data-vocabulary.org/Address`. “Defensor do Desenvolvedor” e “Google, Inc.” foram duas propriedades do vocabulário `http://data-vocabulary.org/Person` (title e affiliation, respectivamente).

Isso realmente é bastante surpreendente. Você não precisa ser uma grande empresa fazendo negócios especiais com os fornecedores de motores de busca para customizar suas listagens no resultado de busca. Apenas tire dez minutos e adicione alguns atributos da HTML para anotar os dados que você já estava publicando de qualquer forma.

P: Fiz tudo que você falou, mas minha listagem no resultado de busca no Google não apresenta nenhuma diferença. O que houve?

R: “Google não garante que marcação em qualquer página ou site será usada nos resultados de busca.” Mas, mesmo se o Google decidisse não usar suas anotações microdata, outro motor busca pode usá-las. Como o resto da HTML5, microdata é um padrão aberto que qualquer um pode implementar. É seu trabalho fornecer o máximo de informação possível. Deixe o resto do mundo decidir o que fazer com isso. Eles podem lhe surpreender!



MARCANDO “ORGANIZATIONS”

Microdata não está limitado a um único vocabulário. Páginas “Sobre” são legais, mas você tem provavelmente apenas uma dela. Ainda sedento por mais?

Aqui está uma página exemplo de listagens de empresas. Vamos olhar para a marcação HTML original, sem microdata.

```
<article>
  <h1>Google, Inc.</h1>
  <p>
    1600 Amphitheatre Parkway<br>
    Mountain View, CA 94043<br>
    USA
  </p>
```

```
<p>650-253-0000</p>
<p><a href="http://www.google.com/">Google.com</a></p>
</article>
```

[Acompanhe comigo! Antes: [organization.html](#), depois: [organization-plus-microdata.html](#)]

Curto e suave. Toda a informação sobre a organização está contida dentro do elemento `<article>`, então vamos começar por aqui.

```
<article itemscope itemtype="http://data-vocabulary.org/Organization">
```

Tal como com marcando “people”, você precisa configurar os atributos `itemscope` e `itemtype` no elemento externo. Nesse caso, o elemento externo é o elemento `<article>`; O atributo `itemtype` declara o vocabulário microdata que você está usando (nesse caso, <http://data-vocabulary.org/Organization>), e o atributo `itemscope` declara que todas as propriedades que você configura em elementos filho relacionados a esse vocabulário.

Então o que está dentro do vocabulário “Organization”? É simples e direto. Na verdade, alguns deles já deve parecer familiar.

VOCABULÁRIO ORGANIZATION

Propriedade	Descrição
<code>name</code>	O nome da empresa (por exemplo, “Initech”)
<code>url</code>	Link para a página da empresa
<code>address</code>	A localização da empresa. Pode conter as subpropriedades <code>street-address</code> , <code>locality</code> , <code>region</code> e <code>country-name</code> .
<code>tel</code>	O número do telefone da empresa
<code>geo</code>	Especifica as coordenadas geográficas da localização. Sempre contém duas subpropriedades: <code>latitude</code> e <code>longitude</code> .

O primeiro pedaço de marcação dentro do elemento externo `<article>` é um `<h1>`. Esse elemento `<h1>` contém o nome de uma empresa, então vamos colocar um atributo `itemprop="name"` diretamente no elemento `<h1>`.

```
<h1 itemprop="name">Google, Inc.</h1>
```

[Acompanhe comigo! Antes: [organization.html](#), depois: [organization-plus-microdata.html](#)]

De acordo com o modelo de dados microdata da HTML5, elementos `<h1>` não precisam de nenhum processamento especial. O valor da propriedade microdata é simplesmente o conteúdo textual do elemento `<h1>`. Traduzindo, acabamos de dizer “o nome da empresa é 'Google, Inc.'”

Próximo é um logradouro. Marcar endereço de uma empresa funciona exatamente da mesma forma como marcar o endereço de um pessoa. Primeiro, adicione um atributo `itemprop="address"` no elemento externo do endereço (nesse caso, um elemento `<p>`). Que indica que esta é a propriedade `address` da “Organization”. Mas e as propriedades do próprio endereço? Também precisamos definir os atributos `itemtype` e `itemscope` para dizer que esse é um item Address que tem suas próprias propriedades.

```
<p itemprop="address" itemscope  
  itemtype="http://data-vocabulary.org/Address">
```

[Acompanhe comigo! Antes: [organization.html](#), depois: [organization-plus-microdata.html](#)]

Finalmente, precisamos englobar cada parte distinta de informação em um elemento simulado `<span>` assim podemos adicionar o nome apropriado da propriedade microdata (`street-address`, `locality`, `region`, `postal-code`, e `country-name`) em cada elemento `<span>`.

```
<p itemprop="address" itemscope  
  itemtype="http://data-vocabulary.org/Address">  
  <span itemprop="street-address">1600 Amphitheatre Parkway</span><br>  
  <span itemprop="locality">Mountain View</span>,&br/>  <span itemprop="region">CA</span>  
  <span itemprop="postal-code">94043</span><br>  
  <span itemprop="country-name">USA</span>  
</p>
```

[Acompanhe comigo! Antes: [organization.html](#), depois: [organization-plus-microdata.html](#)]

Traduzindo, acabamos de dizer “Essa empresa tem um endereço. A parte de logradouro é '1600 Amphitheatre Parkway'. A localidade é 'Mountain View'. A parte da região é 'CA'. O código postal é '94043'. O nome do país é 'USA'.”

Em seguida: um número de telefone para a empresa. Números de telefone são notoriamente complicados, e a sintaxe exata é específica da localização. (E se você gostaria de ligar para outro lugar, é ainda pior.) Nesse exemplo, temos um número de telefone dos Estados Unidos, em um formato adequado para ligar de qualquer lugar nos Estados Unidos.

```
<p itemprop="tel">650-253-0000</p>
```

[Acompanhe comigo! Antes: [organization.html](#), depois: [organization-plus-microdata.html](#)]

(Ei, caso você não tenha percebido, o vocabulário Address saiu do escopo quando seu elemento <p> foi fechado. Agora estamos voltando a definir propriedades no vocabulário Organization.)

Se você gostaria de listar mais de um número de telefone — talvez um para clientes dos Estados Unidos e outro para clientes internacionais — você pode fazer isso. Qualquer propriedade microdata pode ser repetida. Apenas certifique-se que cada número de telefone está em seu próprio elemento HTML, separado de qualquer rótulo que você tenha dado a ele.

```
<p>
  US customers: <span itemprop="tel">650-253-0000</span><br>
  UK customers: <span itemprop="tel">00 + 1* + 6502530000</span>
</p>
```

De acordo com o [modelo de dados microdata da HTML5](#), nem o elemento <p> nem o elemento tem processamento especial. O valor da propriedade microdata tel é apenas o conteúdo textual. O vocabulário microdata Organization não faz nenhuma tentativa de subdividir as diferentes partes de um número de telefone. A propriedade tel inteira é apenas texto aberto. Se você quiser colocar código de área em parênteses, ou usar espaços em vez de traços para separar os números, você pode fazer isso. Se o cliente consumidor de microdata quiser transformar o número de telefone, isso fica inteiramente a cargo deles.

Seguindo, temos outra propriedade familiar: url. Assim como [associar uma URL com uma Person](#), você pode associar uma URL com uma empresa. Isso pode ser a página da empresa, uma página de contato, ou qualquer outra coisa. Se for uma URL sobre, da, ou pertencente a empresa, marque-a com um atributo itemprop="url".

```
<p><a itemprop="url" href="http://www.google.com/">Google.com</a></p>
```

[Acompanhe comigo! Antes: [organization.html](#), depois: [organization-plus-microdata.html](#)]

De acordo com o [modelo de dados microdata da HTML5](#), o elemento <a> tem processamento especial. O valor da propriedade microdata é o valor do atributo href, não o texto do link. Traduzindo, isso quer dizer “essa empresa é associada com a URL <http://www.google.com/>.” Isso não diz nada mais específico sobre a associação, e não inclui o texto do link “Google.com.”

Finalmente, gostaria de falar sobre geolocalização. Não, não é a [API W3C de Geolocalização](#). Se trata de como marcar a localização física de uma empresa, usando microdata.

Até o momento, todos os nossos exemplos têm-se focado na marcação de dados *visíveis*. Isto é, você tem um <h1> com um nome de uma empresa, então você adiciona um

atributo `itemprop` no elemento `<h1>` para declarar que o título textual (visível) é, na verdade, o nome de uma empresa. Ou você tem um elemento `<img>` que aponta para uma foto, então você adiciona um atributo `itemprop` no elemento `<img>` para declarar que a imagem (visível) é a foto de uma pessoa.

Nesse exemplo, informação de geolocalização não é assim. Não há texto visível que mostra a exata latitude e longitude (com quatro casas decimais!) da empresa. Na verdade, o exemplo [organization.html](#) (sem microdata) não tem nenhuma informação de geolocalização. Tem um link para o Google Maps, mas até mesmo a URL desse link não contém as coordenadas de latitude e longitude. (Ele contém informações semelhantes num formato específico do Google.) Porém, mesmo se tivéssemos um link para um hipotético serviço online de mapas que pegasse as coordenadas de latitude e longitude como parâmetros da URL, microdata não tem como separar diferentes partes de uma URL. Você não pode declarar que o primeiro parâmetro query da URL é a latitude e o segundo parâmetro query da URL é a longitude e o resto dos parâmetros são irrelevantes.

Para lidar com casos extremos como este, HTML5 fornece uma maneira de anotar dados *invisíveis*. Essa técnica deve ser usada apenas como último recurso. Se existir uma forma de mostrar ou processar os dados que você se importa com, você deve fazer assim. Dados invisíveis que apenas máquinas podem ler tendem a “banalizar-se” rapidamente. Isto é, alguém chegará mais tarde e atualizará o texto visível mas esquecerá de atualizar as informações invisíveis. Isso acontece com mais frequência do que você pensa, e isso *vai* acontecer com você também.

Ainda, existem casos que dado invisível é inevitável. Talvez o seu chefe queira *realmente* informação de geolocalização legível para máquina mas não queira bagunçar a interface com pares de incompreensíveis números de seis dígitos. Dado invisível é a única opção. A única salvação aqui é que você pode colocar a informação invisível imediatamente depois do texto visível que o descreve, o que pode ajudar a pessoa que vem mais tarde e atualiza o texto visível que eles precisam atualizar a informação invisível logo depois disso.

Nesse exemplo, podemos criar elemento simulado `<span>` dentro do mesmo elemento `<article>` como todas outras propriedades da empresa, em seguida colocar o dado de geolocalização invisível dentro do elemento `<span>`.

```
<span itemprop="geo" itemscope
  itemType="http://data-vocabulary.org/Geo">
  <meta itemprop="latitude" content="37.4149" />
  <meta itemprop="longitude" content="-122.078" />
</span>
</article>
```

[Acompanhe comigo! Antes: [organization.html](#), depois: [organization-plus-microdata.html](#)]

Informação de geolocalização é definida em seu próprio vocabulário, como o endereço de uma pessoa ou empresa. Portanto, esse elemento `<span>` precisa de três atributos:

1. `itemprop="geo"` diz que o elemento representa a propriedade `geo` da “Organization” ao redor
2. `itemtype="http://data-vocabulary.org/Geo"` diz que vocabulário microdata essas propriedades do elemento estão em conformidade
3. `itemscope` diz que esse elemento é o elemento delimitador para um item microdata com seu próprio vocabulário (dado no atributo `itemtype`). Todas as propriedades dentro desse elemento são propriedades de `http://data-vocabulary.org/Geo`, não a do em torno `http://data-vocabulary.org/Organization`.

A próxima grande questão que esse exemplo responde é, “Como você anota dado invisível?” Você usa o elemento `<meta>`. Nas versões anteriores da HTML, você poderia usar apenas o elemento `<meta>` dentro do `<head>` de sua página. Na HTML5, você pode usar o elemento `<meta>` em qualquer lugar. E é exatamente isso que estamos fazendo aqui.

```
<meta itemprop="latitude" content="37.4149" />
```

[Acompanhe comigo! Antes: [organization.html](#), depois: [organization-plus-microdata.html](#)]

De acordo com o modelo de dados microdata da HTML5, o elemento `<meta>` tem processamento especial. O valor da propriedade microdata é o atributo `content`. Já que esse atributo nunca é visivelmente mostrado, temos a configuração perfeita para quantidades ilimitadas de dados invisíveis. Com grandes poderes vem grandes responsabilidades. Nesse caso, a responsabilidade é sua de certificar-se de que esse dado invisível fique sincronizado com o texto visível em volta dele.

Não há suporte direto para o vocabulário Organization no Google Rich Snippets, então não tenho nenhum bom exemplo de listagem no resultado de busca para mostrá-lo. Mas as organizações aparecem fortemente nos próximos dois estudos de caso: eventos e revisões, e essas *são* suportadas pelo Google Rich Snippets.



MARCANDO “EVENTS”

Merda acontece. Algumas merdas acontecem em pré-determinadas vezes. Não seria legal se você pudesse dizer aos motores de busca exatamente quando a merda estava prestes a acontecer?

Vamos começar olhando para um cronograma exemplo de amostra das minhas palestras.

```

<article>
  <h1>Google Developer Day 2009</h1>
  
  <p>
    Google Developer Day é uma chance de aprender sobre o
    desenvolvimento dos produtos Google pelos engenheiros que os
    constroem. Essa conferência de um dia inclui seminários e “horário
    de trabalho” em tecnologias web como Google Maps, OpenSocial,
    Android, APIs AJAX, Chrome, e Google Web Toolkit.
  </p>
  <p>
    <time datetime="2009-11-06T08:30+01:00">2009 November 6,
8:30</time>
    &ndash;
    <time datetime="2009-11-06T20:30+01:00">20:30</time>
  </p>
  <p>
    Congress Center<br>
    5th května 65<br>
    140 21 Praha 4<br>
    Czech Republic
  </p>
  <p><a
href="http://code.google.com/intl/cs/events/developerday/2009/home.
html">GDD/Prague home page</a></p>
</article>

```

[Acompanhe comigo! Antes: [event.html](#), depois: [event-plus-microdata.html](#)]

Toda informação sobre o evento está contida dentro do elemento `<article>`, então é nele que precisamos colocar os atributos `itemtype` e `itemscope`.

```

<article itemscope itemtype="http://data-vocabulary.org/Event">

```

[Acompanhe comigo! Antes: [event.html](#), depois: [event-plus-microdata.html](#)]

A URL para o vocabuário Event é <http://data-vocabulary.org/Event>, que ocorre de conter também um pequeno bonito gráfico

descrevendo as propriedades do vocabulário. E o que são essas propriedades?

VOCABULÁRIO EVENT	
Propriedade	Descrição
summary	O nome do evento
url	Link para a página de detalhes do evento
location	A localização ou lugar do evento. Pode opcionalmente ser representado por um <u>Organization</u> ou <u>Address</u> aninhado.
description	Uma descrição do evento
startDate	A data e hora inicial do evento no <u>formato de data ISO</u>
endDate	A data e hora final do evento no <u>formato de data ISO</u>
duration	A data de duração no <u>formato de duração ISO</u>
eventType	A categoria do evento (por exemplo, “Show” ou “Palestra”). Esta é uma string de forma livre, não um atributo enumerado.
geo	Especifica as coordenadas geográficas da localização. Sempre contém duas subpropriedades, <u>latitude</u> e <u>longitude</u> .
photo	Um link para uma foto ou imagem relacionada ao evento

O nome do evento está em um elemento <h1>.

De acordo com o modelo de dados microdata da HTML5, elementos <h1> não tem processamento especial. O valor da propriedade microdata é apenas o conteúdo textual do elemento <h1>. Tudo que precisamos adicionar é o atributo `itemprop` para declarar que esse elemento <h1> contém o nome do evento.

```
<h1 itemprop="summary">Google Developer Day 2009</h1>
```

[Acompanhe comigo! Antes: [event.html](#), depois: [event-plus-microdata.html](#)]

Traduzindo, isso quer dizer, “O nome desse evento é Google Developer Day 2009.”

Essa listagem de evento tem uma foto, que pode ser marcada com uma propriedade `photo`. Como você poderia esperar, a foto já está marcada com um elemento `<img>`. Como a propriedade `photo` no vocabulário `Person`, uma foto de um evento é uma URL. Já que o modelo de dados microdata da HTML5 diz que o valor da propriedade de um elemento `<img>` está em seu atributo `src`, a única coisa que precisamos fazer é adicionar o atributo `itemprop` no elemento `<img>`.

```

```

[Acompanhe comigo! Antes: [event.html](#), depois: [event-plus-microdata.html](#)]

Traduzindo, isso quer dizer, “A foto para esse evento está aqui <https://diveintohtml5.com.br/examples/gdd-2009-prague-pilgrim.jpg>.”

Em seguida está uma descrição mais longa do evento, que é apenas um parágrafo de texto livre.

```
<p itemprop="description">Google Developer Day é uma chance de aprender
sobre o desenvolvimento dos produtos Google pelos engenheiros que os constroem.
Essa conferência de um dia inclui seminários e “horário de trabalho” em
tecnologias web como Google Maps, OpenSocial, Android, APIs AJAX, Chrome, e
Google Web Toolkit.</p>
```

[Acompanhe comigo! Antes: [event.html](#), depois: [event-plus-microdata.html](#)]

O próximo pedaço é algo novo. Eventos geralmente ocorrem em datas específicas e começam e terminam em horas específicas. Na HTML5, data e hora deve ser marcada com o elemento `<time>`, e já estamos fazendo isso aqui. Então fica a questão, como adicionamos propriedades microdata nesses elementos `<time>`? Olhando atrás no modelo de dados microdata da HTML5, vemos que o elemento `<time>` tem processamento especial. O valor da propriedade microdata em um elemento `<time>` é o valor do atributo `datetime`. E olhe, as propriedades `startDate` e `endDate` do vocabulário `Event` leva um estilo de data ISO, assim como a propriedade `datetime` de um elemento `<time>`. Mais uma vez, a semântica do principal vocabulário HTML se encaixa muito bem com a semântica de nosso vocabulário microdata personalizado. Marcar data inicial e final com microdata é tão simples como:

1. Usar HTML corretamente em primeiro lugar (usar elemento `<time>` para marcar data e hora), e
2. Adicionar um simples atributo `itemprop`

```
<p>
  <time      itemprop="startDate"      datetime="2009-11-06T08:30+01:00">2009
November 6, 8:30</time>
```

–
<time itemprop="endDate" datetime="2009-11-06T20:30+01:00">20:30</time>
</p>

[Acompanhe comigo! Antes: [event.html](#), depois: [event-plus-microdata.html](#)]

Traduzindo, isso quer dizer, “Esse evento começa em 6 de Novembro, 2009, às 8:30 da manhã, e vai até 6 de Novembro, 2009, às 20:30 (horário local para Prague, GMT+1).”

Em seguida está a propriedade `location`. A [definição do vocabulário Event](#) diz que isso pode ser tanto uma “Organization” ou um “Address”. Nesse caso, o evento está sendo realizado em um local especializado em conferências, o Congress Center in Prague. Marcar isso como uma “Organization” nos permite incluir o nome do local, bem como seu endereço.

Primeiro, vamos declarar que o elemento `<p>` que contém o endereço é a propriedade `location` do evento, e que esse elemento é também seu próprio item microdata que está em conformidade com o vocabulário <http://data-vocabulary.org/Organization>

```
<p itemprop="location" itemscope  
  itemtype="http://data-vocabulary.org/Organization">
```

[Acompanhe comigo! Antes: [event.html](#), depois: [event-plus-microdata.html](#)]

Depois, marcar o nome da empresa através de um elemento `<span>` externo e adicionar um atributo `itemprop` no elemento `<span>`.

```
<span itemprop="name">Congress Center</span><br>
```

[Acompanhe comigo! Antes: [event.html](#), depois: [event-plus-microdata.html](#)]

Devido as regras microdata de escopo, esse `itemprop="name"` está definindo uma propriedade no vocabulário Organization, não no vocabulário Event. O elemento `<p>` definido no começo do escopo das propriedades Organization, e que elemento `<p>` não foi fechado ainda com uma tag `</p>`. Qualquer propriedade microdata que definimos aqui são propriedades do mais recente vocabulário com escopo. Vocabulários aninhadas são como uma [pilha](#). Não desempilhamos ainda, então ainda estamos falando sobre propriedades da “Organization”.

Na verdade, vamos adicionar um terceiro vocabulário na pilha: um “Address” da “Organization” para o “Event”.

```
<span itemprop="address" itemscope  
  itemtype="http://data-vocabulary.org/Address">
```

[Acompanhe comigo! Antes: [event.html](#), depois: [event-plus-microdata.html](#)]

Mais uma vez, queremos marcar cada pedaço de um endereço como uma propriedade microdata separada, então precisamos uma série de elementos `<span>` fictícios para pendurar nossos atributos `itemprop` dentro. (Se estou indo muito rápido para você aqui, volte e leia sobre [marcando o endereço de uma pessoa](#) e [marcando o endereço de uma empresa](#).)

```
<span itemprop="street-address">5th května 65</span><br>
<span itemprop="postal-code">140 21</span>
<span itemprop="locality">Praha 4</span><br>
<span itemprop="country-name">Czech Republic</span>
```

[Acompanhe comigo! Antes: [event.html](#), depois: [event-plus-microdata.html](#)]

Não existem mais propriedades do Address, então fechamos o elemento `<span>` que começou o escopo Address, e desempilhamos.

```
</span>
```

Não existe mais propriedades da “Organization”, então fechamos o elemento `<p>` que começou o escopo Organization, e desempilhamos novamente.

```
</p>
```

Agora voltamos para definir propriedades no evento. A próxima propriedade é `geo`, para representar a localização física do evento. Ela usa o mesmo vocabulário Geo que usamos para [marcar a localização física de uma empresa](#) na seção anterior. Precisamos de um elemento `<span>` para agir como um container; ele leva os atributos `itemtype` e `itemscope`. Dentro desse elemento `<span>`, precisamos de dois elementos `<meta>`, um para a propriedade `latitude` e outro para a propriedade `longitude`.

```
<span itemprop="geo" itemscope itemtype="http://data-vocabulary.org/Geo">
  <meta itemprop="latitude" content="50.047893" />
  <meta itemprop="longitude" content="14.4491" />
</span>
```

[Acompanhe comigo! Antes: [event.html](#), depois: [event-plus-microdata.html](#)]

E fechamos o `<span>` que guardava as propriedades Geo, então estamos voltando a definir propriedades do evento. A última propriedade é a propriedade `url`, que deve parecer familiar. Associar uma URL com um evento funciona da mesma forma que [associar uma URL com uma pessoa](#) e [associar uma URL com uma empresa](#). Se você está usando HTML corretamente (marcando hyperlinks com `<a href>`), então declarar que o hyperlink é uma propriedade `url` microdata é uma simples questão de adicionar o atributo `itemprop`.

```
<p>
  <a itemprop="url"
```

```
href="http://code.google.com/intl/cs/events/developerday/2009/home.html">
    GDD/Prague home page
</a>
</p>
</article>
```

[Acompanhe comigo! Antes: [event.html](#), depois: [event-plus-microdata.html](#)]

A página de evento exemplo também lista um segundo evento, minha palestra no ConFoo Conference em Montréal. Para ser breve, não vou passar por essa marcação linha a linha. É essencialmente o mesmo que o evento em Prague: um item Event com itens Geo e Address aninhados. Estou mencionando apenas de passagem para recordar que uma única página pode ter vários eventos, cada um marcado com microdata.

O RETORNO DO GOOGLE RICH SNIPPETS

De acordo com a ferramenta de teste do Google Rich Snippets, essa é a informação que os rastreadores do Google vão imprimir do nosso exemplo de eventos na página de listagem:

Item

```
    Type: http://data-vocabulary.org/Event
    summary = Google Developer Day 2009
    eventType = conference
    photo = https://diveintohtml5.com.br/examples/gdd-2009-prague-
pilgrim.jpg
    description = Google Developer Days é uma chance de aprender
sobre o desenvolvimento dos produtos Google pelos engenheiros que os
constrói. Essa conferência de um dia inclui seminários e “horário
de trabalho” em tecnologias web como Goo..
    startDate = 2009-11-06T08:30+01:00
    endDate = 2009-11-06T20:30+01:00
    location = Item(__1)
    geo = Item(__3)
    url =
http://code.google.com/intl/cs/events/developerday/2009/home.html
```

Item

```
    Id: __1
    Type: http://data-vocabulary.org/Organization
    name = Congress Center
    address = Item(__2)
```


Item

Id: __2

Type: http://data-vocabulary.org/Address
street-address = 5th května 65
postal-code = 140 21
locality = Praha 4
country-name = Czech Republic

Item

Id: __3

Type: http://data-vocabulary.org/Geo
latitude = 50.047893
longitude = 14.4491

Como você pode ver, toda informação que adicionamos no microdata está lá. Propriedades que são itens separados de microdata recebem IDs internos (Item(__1), Item(__2) e assim por diante). Isso não faz parte da especificação microdata. É apenas uma convenção que ferramentas de teste do Google usam para linearizar a saída de exemplo e mostrar-lhe o agrupamento de itens aninhados e suas propriedades.

Aqui está como Google pode escolher representar essa página exemplo em seus resultados de busca. (Novamente, tenho que introduzir isso com o aviso que isso é apenas um exemplo. Google pode mudar o formato dos seus resultados de busca a qualquer momento, e não há garantia que o Google vai prestar atenção em sua marcação microdata. Desculpa parecer uma recordação chata, mas nossos advogados me fazem dizer essas coisas.)

Calendário de Eventos de Mark Pilgrim

Trecho da página aparecerá aqui.

Trecho da página aparecerá aqui.

Google Developer Day
2009

Fri, Nov 6

Congress Center, Praha 4, Czech Republic

ConFoo.ca 2010

Wed, Mar
10

Hilton Montreal Bonaventure, Montréal, Québec,
Canada

diveintohtml5.com.br/examples/event-plus-microdata.html - Cached - Páginas similares

Após o título da página e o auto-gerado texto resumido, o Google começa usar a marcação microdata que adicionamos na página para mostrar uma pequena tabela de eventos. Observe o formato de data: "Fri, Nov 6." Isso não é uma string que apareceu em algum lugar do nosso HTML ou marcação microdata. Usamos duas completamente qualificadas strings formatadas ISO. 2009-11-06T08:30+01:00 e 2009-11-06T20:30+01:00. Google pegou essas duas datas, descobriu que se tratava do mesmo dia, e decidiu mostrar uma única data em um formato mais amigável.

Agora olhe para o endereço físico. Google escolheu mostrar apenas o nome do local + localização + país, não exatamente o logradouro. Isso é possível pelo fato de que nós

separamos o endereço em cinco subpropriedades — `name`, `street-address`, `region`, `locality`, e `country-name` — e marcamos cada parte do endereço como uma propriedade microdata diferente. Google se aproveita disso para mostrar um endereço abreviado. Outros consumidores da mesma marcação microdata podem fazer diferentes escolhas sobre o que mostrar ou como mostrar isso. Não existe escolha certa ou errada aqui. Está a seu cargo prover o máximo de informação possível, o mais fielmente possível. Interpretar isso está a cargo do resto do mundo.



MARCANDO “REVIEWS”

Aqui está outro exemplo de fazer a web melhor (e possivelmente ser listado no resultado de busca) através de marcação: avaliação de produtos e negócios.

Esse é uma pequena avaliação que escrevi da minha pizzaria favorita perto da minha casa. (A propósito, isso é um restaurante de verdade. Se você alguma vez estiver em Apex, NC, recomendo altamente ele.) Vamos olhar para a marcação original:

```
<article>
  <h1>Anna’s Pizzeria</h1>
  <p>★★★★☆ (4 estrelas de 5)</p>
  <p>Nova pizza no estilo Nova York logo ali no centro histórico
Apex</p>
  <p>
    A comida é de primeira linha. A atmosfera está ali para a
“pizzaria do bairro.”
    O restaurante em si é um pouco apertado; se você está acima do
peso,
    você pode ter dificuldade de entrar e sair de sua cadeira e
caminhar entre outras mesas.
    Costumava dar gratuitamente “garlic knots” quando você sentava;
agora eles lhe dão pão
    simples e você tem que pagar pelas coisas boas. No geral, é um
vencedor.
  </p>
  <p>
    100 North Salem Street<br>
    Apex, NC 27502<br>
    USA
  </p>
  <p>– avaliado por Mark Pilgrim, última atualização 31 de Março,
2010</p>
</article>
```

[Acompanhe comigo! Antes: [review.html](#), depois: [review-plus-microdata.html](#)]

Essa avaliação está contida em um elemento `<article>`, então é nele que vamos colocar os atributos `itemtype` e `itemscope`. A URL namespace para esse vocabulário é <http://data-vocabulary.org/Review>.

```
<article itemscope itemtype="http://data-vocabulary.org/Review">
```

[Acompanhe comigo! Antes: [review.html](#), depois: [review-plus-microdata.html](#)]

Quais são as propriedades disponíveis no vocabulário Review? Estou feliz que você perguntou.

VOCABULÁRIO REVIEW	
Propriedade	Descrição
<code>itemreviewed</code>	O nome do item que está sendo avaliado. Pode ser um produto, serviço, negócio, entre outros.
<code>rating</code>	Uma classificação numérica para o item, em uma escala de 1 até 5. Pode ser também um vocabulário http://data-vocabulary.org/Rating aninhado para usar uma escala diferente do padrão.
<code>reviewer</code>	O nome do autor que escreveu a avaliação
<code>dtreviewed</code>	A data que o item foi avaliado no <u>formato de data ISO</u>
<code>summary</code>	Um pequeno resumo da avaliação
<code>description</code>	O corpo da avaliação

A primeira propriedade é simples: `itemreviewed` é apenas texto, e aqui ele está contido no elemento `<h1>`, então é aqui onde devemos colocar o atributo `itemprop`.

```
<h1 itemprop="itemreviewed">Anna's Pizzeria</h1>
```

[Acompanhe comigo! Antes: [review.html](#), depois: [review-plus-microdata.html](#)]

Vou pular a classificação real e voltarei a ela no final.

As próximas duas propriedades também são simples. A propriedade `summary` é uma pequena descrição do que você está avaliando, e a propriedade `description` é o corpo da avaliação.

`<p itemprop="summary">Nova pizza no estilo Nova York logo ali no centro histórico Apex</p>`

`<p itemprop="description">`

A comida é de primeira linha. A atmosfera está ali para a “pizzaria do bairro.”

O restaurante em si é um pouco apertado; se você está acima do peso,

você pode ter dificuldade de entrar e sair de sua cadeira e caminhar entre outras mesas.

Costumava dar gratuitamente “garlic knots” quando você sentava; agora eles lhe dão pão

simples e você tem que pagar pelas coisas boas. No geral, é um vencedor.

`</p>`

[Acompanhe comigo! Antes: [review.html](#), depois: [review-plus-microdata.html](#)]

As propriedades `location` e `geo` não tem nada de novidade. (Se você está mal sintonizando, confira [marcando o endereço de uma pessoa](#), [marcando o endereço de uma empresa](#), e [marcando informação de geolocalização](#) anteriormente nesse capítulo.)

`<p itemprop="location" itemscope`

`itemtype="http://data-vocabulary.org/Address">`

`<span itemprop="street-address">100 North Salem`

`Street</span><br>`

`<span itemprop="locality">Apex</span>,&code>`

`<span itemprop="region">NC</span>`

`<span itemprop="postal-code">27502</span><br>`

`<span itemprop="country-name">USA</span>`

`</p>`

`<span itemprop="geo" itemscope`

`itemtype="http://data-vocabulary.org/Geo">`

`<meta itemprop="latitude" content="35.730796" />`

`<meta itemprop="longitude" content="-78.851426" />`

`</span>`

[Acompanhe comigo! Antes: [review.html](#), depois: [review-plus-microdata.html](#)]

A linha final representa um problema familiar: ela contém duas partes de informação de um elemento. O nome do avaliador é Mark Pilgrim, e a data de avaliação é 31 de March, 2010. Como marcar essas duas propriedades distintas? [Englobe eles em seus próprios elementos](#) e coloque um atributo `itemprop` em cada elemento. Na verdade, em primeiro lugar a data nesse exemplo deveria ser marcada com um elemento `<time>`, assim isso daria um gancho natural sobre para pendurar o nosso atributo `itemprop`. O nome do avaliador pode ser apenas englobado em um fictício elemento `<span>`.

```

<p>– <span itemprop="reviewer">Mark Pilgrim</span>, última
atualização
  <time itemprop="dtreviewed" datetime="2010-03-31">
    31 de Março, 2010
  </time>
</p>
</article>

```

[Acompanhe comigo! Antes: [review.html](#), depois: [review-plus-microdata.html](#)]

Ok, vamos falar de classificações. A parte mais complicada de marcar uma revisão é a classificação. Por padrão, classificação no vocabulário Review está numa escala de 1–5, 1 sendo “terrível” e 5 sendo “maravilhoso.” Se você quer usar uma escala diferente, você definitivamente pode fazer isso. Mas vamos falar sobre a escala padrão primeiro.

```

<p>★★★★☆ (<span itemprop="rating">4</span> stars out of 5)</p>

```

[Acompanhe comigo! Antes: [review.html](#), depois: [review-plus-microdata.html](#)]

Se você está usando a escala padrão 1–5, a única propriedade que você precisa marcar é a própria classificação (4, nesse caso). Mas e se você quiser usar uma escala diferente? Você pode fazer isso; você precisa declarar os limites da escala que você está usando. Por exemplo, se você gostaria de usar uma escala de pontos de 0–10, você ainda declararia a propriedade `itemprop="rating"`, mas em vez de dar o valor da classificação diretamente, você usaria um vocabulário aninhado de <http://data-vocabulary.org/Rating> para declarar o péssimo e o melhor valor na sua escala customizada e o valor real dentro dessa escala.

```

<p itemprop="rating" itemscope
  itemtype="http://data-vocabulary.org/Rating">
  ★★★★★★★★☆☆
  (<span itemprop="value">9</span> numa escala de
   <span itemprop="worst">0</span> até
   <span itemprop="best">10</span>)
</p>

```

Traduzindo, isso quer dizer “o produto que estou avaliando tem um valor de classificação de 9 em uma escala de 0–10.”

Eu mencionei que a microdata review pode afetar a listagem no resultado de busca? Verdade, ela pode. Aqui estão os “dados puros” que a [ferramenta do Google Rich Snippets](#) extraiu da [minha avaliação melhorada com microdata](#):

Item

```
Type: http://data-vocabulary.org/Review
itemreviewed = Anna's Pizzeria
rating = 4
summary = Nova pizza no estilo Nova York logo ali no centro
histórico Apex
description = A comida é de primeira linha. A atmosfera está ali
...
address = Item(__1)
geo = Item(__2)
reviewer = Mark Pilgrim
dtreviewed = 2010-03-31
```

Item

```
Id: __1
Type: http://data-vocabulary.org/Organization
street-address = 100 North Salem Street
locality = Apex
region = NC
postal-code = 27502
country-name = USA
```

Item

```
Id: __2
Type: http://data-vocabulary.org/Geo
latitude = 35.730796
longitude = -78.851426
```

E aqui (módulo dos caprichos do Google, a fase da lua, e assim por diante e assim por diante) está como minha avaliação pode aparecer em uma listagem de resultado de busca:

Anna's Pizzeria: avaliação

★★★★☆ Avaliado por Mark Pilgrim - Mar 31, 2010

Trecho da página aparecerá aqui.

Trecho da página aparecerá aqui..

diveintohtml5.com.br/examples/review-plus-microdata.html - [Cached](#) - [Páginas similares](#)

Colchetes não me impressionam muito, mas eu tenho que admitir, esse é muito legal.



LEITURA COMPLEMENTAR

Microdata resources:

- [Live microdata playground](#)
- [HTML5 microdata specification](#)

Google Rich Snippets resources:

- [About rich snippets and structured data](#)
- [Marking up contact and social networking information](#)
- [Businesses & organizations](#)
- [Events](#)
- [Reviews](#)
- [Review ratings](#)
- [Google Rich Snippets Testing Tool](#)
- [Google Rich Snippets Tips and Tricks](#)

Nº11

MANIPULANDO HISTÓRICO PARA DIVERSÃO & LUCRO

MERGULHANDO



barra de endereços do navegador é talvez a peça mais antiga da interface de usuário no mundo. Há URLs em outdoors, nas laterais de trens, e até mesmo em grafites de rua. Combinado com o botão de voltar — provavelmente o botão mais importante do navegador — você tem uma maneira poderosa para avançar e voltar no vasto conjunto de recursos interligados chamado de Web.

A API de histórico da HTML5 é uma maneira padronizada para manipular o histórico do navegador via script. Parte desta API — navegando pelo histórico — está disponível em versões anteriores do HTML. As novas funcionalidades em HTML5 incluem uma maneira de adicionar entradas ao histórico do navegador, para visivelmente alterar a URL na barra de endereços do navegador (sem precisar atualizar a página), e um evento é acionado quando estas entradas são removidas da pilha do navegador quando o usuário pressiona o botão voltar. Isto quer dizer que a URL na barra de endereços do navegador pode continuar seu trabalho de ser um identificador único para o recurso atual, mesmo em aplicações com scripts pesados que nem sempre necessitam de uma atualização na página toda.



O PORQUÊ



Por que você iria manipular manualmente a barra de endereços do navegador? Afinal, um simples link pode navegar até uma nova URL; essa é a forma que a web tem funcionado nos últimos 20 anos. E isso vai continuar funcionando desta forma. Esta API não tenta revolucionar a web. Muito pelo contrário. Nos últimos anos, os desenvolvedores web têm encontrado novas e excitantes formas de revolucionar a web sem a ajuda de padrões emergentes. A API de histórico da HTML5 foi na verdade criada para garantir que as URLs continuem sendo úteis em aplicações web com scripts pesados.

Voltando aos princípios, o que uma URL faz? Ela identifica um recurso único. Você pode fazer um link direto para ela; você pode marcá-la; motores de busca podem indexá-la; você pode copiar, colar e enviá-la por e-mail para outra pessoa, esta pessoa pode clicar e acabar vendo o mesmo recurso que você viu originalmente. Estas são todas qualidades excelentes. URLs são importantes.

Então nos queremos que recursos únicos tenham URLs únicas. Mas ao mesmo tempo, navegadores sempre tiveram uma limitação fundamental: Se você mudar a URL, mesmo através de script, ele dispara uma requisição ao servidor web remoto e recarrega toda a página. Isso consome tempo e recursos, e parece um desperdício quando você está navegando para uma página que é substancialmente semelhante à sua página atual. Tudo que possui na nova página é baixado, até mesmo as partes que são exatamente as mesmas da página atual. Não tem como alterar a URL em um navegador e este fazer download de apenas metade da página.

A API de histórico do HTML5 permite que você faça isso. Ao invés de desencadear uma atualização na página inteira, você pode utilizar o script para, em essência, baixar metade de uma página. Esta ilusão é um truque difícil, e requer algum trabalho da sua parte. Você está prestando atenção?



Digamos que você possui duas páginas, página A e página B. As duas páginas são 90% idênticas; somente 10% do conteúdo destas páginas é diferente. O usuário navega para a página A, então tenta navegar para a página B. Mas ao invés de desencadear uma atualização na página toda, você interrompe o navegador e segue estes passos manualmente:

1. *Carrega os 10% da página* através da página B que é diferente da página A (provavelmente utilizando XMLHttpRequest). Isso vai exigir algumas modificações no lado servidor de sua aplicação web. Você vai precisar escrever um código que retorne apenas os 10% da página B que é diferente da página A. Isso pode ser uma URL oculta ou um parâmetro de consulta que o usuário final normalmente não vê.
2. *Troque o conteúdo alterado* (utilizando innerHTML ou outro método DOM). Talvez você precise redefinir os manipuladores de eventos que foram alterados junto com o conteúdo.
3. *Atualize a barra de endereços do navegador* com a URL da página B, utilizando um método particular da API de histórico do HTML5 que eu já vou mostrar para você.

Ao final desta ilusão (se você a executar corretamente), o navegador acaba com um DOM que é idêntico a página B, como se você tivesse navegado diretamente para a página B. A barra de endereços do navegador acaba ficando com uma URL que é idêntica a da página B, como se você tivesse navegado diretamente para a página B. Mas na verdade você nunca navegou até a página B, e você nunca precisou atualizar toda a página. Esta é a ilusão. Mas porque a página "compilada" se parece exatamente como a página B e possui a mesma URL, o usuário provavelmente nunca notará a diferença (nem te agradecerá por todo trabalho pesado micro gerenciando suas experiências).



O COMO

A API de histórico do HTML5 é apenas vários métodos no objeto `window.history`, mais um evento no objeto `window`. Você pode utilizar isto para detectar o suporte da API de histórico. O suporte atualmente é limitado para as últimas versões de alguns navegadores, colocando essa técnica diretamente no campo "progressive enhancement".

SUPORTE PARA HISTORY.PUSHSTATE						
IE	FIREFOX	SAFARI	CHROME	OPERA	IPHONE	ANDROID
.	4.0+	5.0+	8.0+	11.50+	4.2.1+	.

dive into dogs é um exemplo simples, mas não trivial de usar a API de histórico do HTML5. Ele demonstra um padrão comum: um longo artigo com uma galeria de fotos. Em um navegador compatível, navegando pelos links "Próximo" e "Anterior" na galeria de fotos irá atualizar apenas foto no lugar *e atualizará o URL na barra de endereços do navegador*, sem desencadear uma atualização na página inteira. Em navegadores sem suporte - ou, de fato, navegadores compatíveis onde o usuário tenha desabilitado scripts - os links simplesmente funcionam como links normais, levando você a uma nova página com uma atualização na página toda.

Isso nos leva a um ponto importante:

Se sua aplicação web falhar em navegadores com script desabilitado, o cachorro de Jakob Nielsen vai até a sua casa e defecará no seu carpete.

Vamos dar uma olhada no exemplo [dive into dogs](#) e ver como ele funciona. Esta é a marcação relevante para uma única foto:

↪ *O penhor*

```
<aside id="gallery">
  <p class="photonav">
    <a id="photonext" href="casey.html">Next &gt;</a>
    <a id="photoprev" href="adagio.html">&lt; Previous</a>
  </p>
  <figure id="photo">
    
    <figcaption>Fer, 1972</figcaption>
  </figure>
</aside>
```

Nada diferente aqui. A foto em si é uma `<img>` dentro de uma `<figure>`, o link é um elemento `<a>` normal, e a coisa toda está colocada dentro de um `<aside>`. É importante que estes links regulares realmente funcionem. Todo o código se passa atrás de um [script de detecção](#). Se um usuário estiver utilizando um navegador sem suporte, nada do nosso código chique da API de histórico será executado. E, claro, sempre há alguns usuários com o script desativado por completo.

A função principal pega cada um destes links e repassa para uma função, `addClicker()`, que faz todo o trabalho de criação e customização do click.

```
function setupHistoryClicks() {
  addClicker(document.getElementById("photonext"));
  addClicker(document.getElementById("photoprev"));
}
```

Esta é a função `addClicker()`. Ela pega um elemento `<a>` e adiciona um click manipulado. E é neste click manipulado onde as coisas ficam interessantes.

```
function addClicker(link) {
```

```

link.addEventListener("click", function(e) {
    swapPhoto(link.href);
    history.pushState(null, null, link.href);
    e.preventDefault();
}, false);
}

```

~ Interessante

A função `swapPhoto()` realiza as duas primeiras etapas de nossa ilusão de três etapas. A primeira metade da função `swapPhoto()` faz parte da URL do link de navegação em si — `casey.html`, `adagio.html`, &c. — e constrói uma URL para uma página oculta que contém nada mais que a marcação exigida pela próxima foto.

```

function swapPhoto(href) {
    var req = new XMLHttpRequest();
    req.open("GET",
        "https://diveintohtml5.com.br/examples/history/gallery/"
+
        href.split("/").pop(),
        false);
    req.send(null);
}

```

Aqui temos um exemplo de marcação retornado por <https://diveintohtml5.com.br/examples/history/gallery/casey.html>. (Você pode verificar isso no seu navegador visitando a URL diretamente.)

```

<p class="photonav">
  <a id="photonext" href="brandy.html">Next &gt;</a>
  <a id="photoprev" href="fer.html">&lt; Previous</a>
</p>
<figure id="photo">
  
  <figcaption>Casey, 1984</figcaption>
</figure>

```

Isso lhe parece familiar? Deveria. Este é a mesma marcação básica utilizada na página original para mostrar a primeira foto.

A segunda metade da função `swapPhoto()` realiza a segunda etapa de nossa ilusão de três etapas: inserindo a nova marcação baixada dentro da página atual. Lembre-se que tínhamos um `<aside>` envolvendo todo a figura, foto e legenda. Então inserindo a nova marcação da foto é uma linha, atribuindo a propriedade `innerHTML` do `<aside>` para a propriedade `responseText` retornada do `XMLHttpRequest`.

```
if (req.status == 200) {
    document.getElementById("gallery").innerHTML = req.responseText;
    setupHistoryClicks();
    return true;
}
return false;
}
```

(Observe também a chamada para o `setupHistoryClicks()`. Isto é necessário para reiniciar os eventos do `click` manipulados nos novos links de navegação.

Atribuindo `innerHTML` remove qualquer traço dos links antigos e seus eventos.)

Agora, vamos voltar para a função `addClicker()`. Depois de alterar a foto com sucesso, temos mais uma etapa para a nossa ilusão de três etapas: atribuir a URL na barra de navegação do navegador sem atualizar a página.

↪ *A troca*

```
history.pushState(null, null, link.href);
```

A função `history.pushState()` recebe três parâmetros:

1. `state` pode ser qualquer dado com estrutura JSON. Isso é passado de volta para o manipulador do evento `popstate`, o qual você irá aprender em algum momento. Nós não precisamos acompanhar nenhum estado nesta demonstração, isto será mantido como `null`.
2. `title` pode ser qualquer string. Este parâmetro não é utilizado atualmente pelos principais navegadores. Se você quiser alterar o título da página, você deve armazenar isto em um argumento `state` e atribuir isto manualmente no callback do seu `popstate`.
3. `url` pode ser, bem, qualquer URL. Isto é a URL que você gostaria que aparecesse na barra de endereços do navegador.

Ao chamar `history.pushState` mudará imediatamente a URL na barra de endereços do navegador. Então, este é o final da ilusão? Bom, não exatamente. Nós ainda precisamos falar sobre o que acontece quando o usuário pressiona o importantíssimo botão de voltar.

Geralmente quando um usuário navega a uma nova página (com uma atualização de página inteira), o navegador joga a nova URL para a lista de histórico, faz os downloads e desenha a nova página. Quando o usuário pressiona o botão voltar, o navegador joga uma página para fora da pilha de histórico e redesenha a página anterior. Mas o que acontece agora que você

gerou um curto-circuito de navegação para evitar a atualização da página toda? Bom, você fingiu o "avançar" para uma nova URL, então agora você pode também fingir um "voltar" para a URL anterior. E a chave para forjar este "retorno" é o evento `popstate`.

↪ *O prestígio*

```
window.addEventListener("popstate", function(e) {  
    swapPhoto(location.pathname);  
})
```

Após você ter utilizado a função `history.pushState()` para jogar a URL para o histórico do navegador, quando o usuário pressionar o botão voltar, o navegador irá disparar um evento `popstate` no objeto `window`. Esta é a sua chance de completar a ilusão de uma vez por todas. Porque fazer alguma coisa desaparecer não é o suficiente; você tem que trazer ela de volta.

Nesta demonstração, “trazer de volta” é tão simples quanto trocando a foto original, que nós fazemos ao chamar o `swapPhoto()` com a localização atual. Quando o retorno do `popstate` é chamado, a URL visível na barra de endereço do navegador foi trocada pela URL antiga. E também, a propriedade `location` global já foi atualizada com a antiga URL.

Para ajudar a visualizar isso, vamos percorrer a ilusão inteira desde do começo ao fim:

- Usuário carrega `https://diveintohtml5.com.br/examples/history/fer.html`, vê a história e a foto de Fer.
- Usuário clica no link “Próximo,” um elemento `<a>` que tem uma propriedade `href` para `https://diveintohtml5.com.br/examples/history/casey.html`.
- Ao invés de navegar para `https://diveintohtml5.com.br/examples/history/casey.html` com uma atualização completa na página, o evento `click` manipulado dentro do elemento `<a>` executa seu próprio código.
- Nosso evento `click` manipulado chama a função `swapPhoto()`, que cria um objeto `XMLHttpRequest` para baixar o fragmento de código HTML localizado em `https://diveintohtml5.com.br/examples/history/gallery/casey.html`.
- A função `swapPhoto()` atribui a propriedade `innerHTML` para o que envolve a galeria de foto (um elemento `<aside>`), substituindo assim a foto e a legenda de Fer pela foto e legenda de Casey.
- Finalmente, nosso `click` manipulado chama a função `history.pushState()` para manualmente alterar a URL na barra de endereços do navegador para `https://diveintohtml5.com.br/examples/history/casey.html`.
- Usuário clica no botão voltar do navegador.
- O navegador percebe que uma URL foi manualmente adicionada na pilha de histórico (pela função `history.pushState()`). Ao invés de navegar para a URL anterior e redesenhar a página toda, o navegador simplesmente atualiza a barra de endereço para

a URL antiga (<https://diveintohtml5.com.br/examples/history/fer.html>) e executa um evento `popstate`.

- Nosso `popstate` customizado manipula chamadas para a função `swapPhoto()` novamente, desta vez com a URL antiga que agora já está visível na barra de endereços do navegador.
- Novamente utilizando `XMLHttpRequest`, a função `swapPhoto()` baixa o fragmento de HTML localizado em <https://diveintohtml5.com.br/examples/history/gallery/fer.html> e atribui a propriedade `innerHTML` do elemento `<aside>`, substituindo assim a foto e a legenda de Casey com uma foto e legenda de Fer.

A ilusão está completa. Todas as evidências visíveis (o conteúdo da página e a URL na barra de endereços) sugerem ao usuário que ele navegou uma página a frente e uma para trás. Mas nenhuma página foi completamente atualizada - isto foi tudo uma ilusão meticulosamente executada.



LEITURA COMPLEMENTAR

- Session history and navigation nas normas de projeto da HTML5
- Manipulating the browser history no centro de desenvolvimento Mozilla
- Simple history API demo
- 20 Things I Learned About Browsers and the Web, uma demonstração avançada que usa a API de histórico e outras técnicas HTML5
- Using HTML5 today descreve a utilização da API de histórico no Facebook
- The Tree Slider descreve a utilização da API de histórico no Github
- History.js, uma meta-API para manipulação de histórico em ambos os navegadores mais novos e mais velhos

APENDICE A:

O ÚNICO

QUASE-ALFABÉTICO

SEM-BESTEIRAS GUIA PARA

DETECTAR TUDO

(Confuso? Leia Detectando Funcionalidades da HTML5: É Elementar, Meu Caro Watson para uma introdução conceitual. Quer uma biblioteca completa? Experimente Modernizr.)

```
<audio> # #  
    return !!document.createElement('audio').canPlayType;
```

```
<audio> no formato MP3 # #  
    var a = document.createElement('audio');  
    return !(a.canPlayType &&  
    a.canPlayType('audio/mpeg;').replace(/no/, ''));
```

```
<audio> no formato Vorbis # #  
    var a = document.createElement('audio');  
    return !(a.canPlayType && a.canPlayType('audio/ogg;  
    codecs="vorbis"').replace(/no/, ''));
```

```
<audio> no formato WAV # #  
    var a = document.createElement('audio');  
    return !(a.canPlayType && a.canPlayType('audio/wav;  
    codecs="1"').replace(/no/, ''));
```

```
<audio> no formato AAC # #  
    var a = document.createElement('audio');  
    return !(a.canPlayType && a.canPlayType('audio/mp4;  
    codecs="mp4a.40.2"').replace(/no/, ''));
```

```
<canvas> # #  
    return !!document.createElement('canvas').getContext;
```

```
<canvas> text API # #  
    var c = document.createElement('canvas');  
    return c.getContext && typeof c.getContext('2d').fillText ==  
    'function';
```



```

<command> # #
    return 'type' in document.createElement('command');
<datalist> # #
    return 'options' in document.createElement('datalist');
<details> # #
    return 'open' in document.createElement('details');
<device> # #
    return 'type' in document.createElement('device');
<form> constraint validation # #
    return 'noValidate' in document.createElement('form');
<iframe sandbox> # #
    return 'sandbox' in document.createElement('iframe');
<iframe srcdoc> # #
    return 'srcdoc' in document.createElement('iframe');
<input autofocus> # #
    return 'autofocus' in document.createElement('input');
<input placeholder> # #
    return 'placeholder' in document.createElement('input');
<textarea placeholder> # #
    return 'placeholder' in document.createElement('textarea');
<input type="color"> # #
    var i = document.createElement('input');
    i.setAttribute('type', 'color');
    return i.type !== 'text';
<input type="email"> # #
    var i = document.createElement('input');
    i.setAttribute('type', 'email');
    return i.type !== 'text';
<input type="number"> # #
    var i = document.createElement('input');
    i.setAttribute('type', 'number');
    return i.type !== 'text';
<input type="range"> # #
    var i = document.createElement('input');
    i.setAttribute('type', 'range');
    return i.type !== 'text';
<input type="search"> # #
    var i = document.createElement('input');
    i.setAttribute('type', 'search');

```

```

    return i.type !== 'text';
<input type="tel"> # #
    var i = document.createElement('input');
    i.setAttribute('type', 'tel');
    return i.type !== 'text';
<input type="url"> # #
    var i = document.createElement('input');
    i.setAttribute('type', 'url');
    return i.type !== 'text';
<input type="date"> # #
    var i = document.createElement('input');
    i.setAttribute('type', 'date');
    return i.type !== 'text';
<input type="time"> # #
    var i = document.createElement('input');
    i.setAttribute('type', 'time');
    return i.type !== 'text';
<input type="datetime"> # #
    var i = document.createElement('input');
    i.setAttribute('type', 'datetime');
    return i.type !== 'text';
<input type="datetime-local"> # #
    var i = document.createElement('input');
    i.setAttribute('type', 'datetime-local');
    return i.type !== 'text';
<input type="month"> # #
    var i = document.createElement('input');
    i.setAttribute('type', 'month');
    return i.type !== 'text';
<input type="week"> # #
    var i = document.createElement('input');
    i.setAttribute('type', 'week');
    return i.type !== 'text';
<meter> # #
    return 'value' in document.createElement('meter');
<output> # #
    return 'value' in document.createElement('output');
<progress> # #
    return 'value' in document.createElement('progress');
<time> # #

```

```

    return 'valueAsDate' in document.createElement('time');
<video> ##
    return !!document.createElement('video').canPlayType;
<video> captions ##
    return 'src' in document.createElement('track');
<video poster> ##
    return 'poster' in document.createElement('video');
<video> no formato WebM ##
    var v = document.createElement('video');
    return !(v.canPlayType && v.canPlayType('video/webm;
    codecs="vp8, vorbis").replace(/no/, ''));
<video> no formato H.264 ##
    var v = document.createElement('video');
    return !(v.canPlayType && v.canPlayType('video/mp4;
    codecs="avc1.42E01E, mp4a.40.2").replace(/no/, ''));
<video> no formato Theora ##
    var v = document.createElement('video');
    return !(v.canPlayType && v.canPlayType('video/ogg;
    codecs="theora").replace(/no/, ''));
contentEditable ##
    return 'isContentEditable' in document.createElement('span');
mensagem Cross-document ##
    return !!window.postMessage;
Drag-and-drop ##
    return 'draggable' in document.createElement('span');
File API ##
    return typeof FileReader != 'undefined';
Geolocalização ##
    return !!navigator.geolocation;
History ##
    return !(window.history && window.history.pushState);
Local storage ##
    try {
        return 'localStorage' in window && window['localStorage']
        != null;
    } catch(e) {
        return false;
    }
Microdata ##
    return !!document.getItems;
Aplicações Web Offline ##

```

```
return !!window.applicationCache;
```

Eventos enviados pelo servidor #

```
return typeof EventSource !== 'undefined';
```

Session storage #

```
try {  
    return 'sessionStorage' in window &&  
    window['sessionStorage'] !== null;  
} catch(e) {  
    return false;  
}
```

SVG #

```
return !(document.createElementNS &&  
document.createElementNS('http://www.w3.org/2000/svg',  
'svg').createSVGRect);
```

SVG in text/html #

```
var e = document.createElement('div');  
e.innerHTML = '<svg></svg>';  
return !(window.SVGSVGElement && e.firstChild instanceof  
window.SVGSVGElement);
```

Undo #

```
return typeof UndoManager !== 'undefined';
```

IndexedDB #

```
return !!window.indexedDB;
```

Web Sockets #

```
return !!window.WebSocket;
```

Web SQL Database #

```
return !!window.openDatabase;
```

Web Workers #

```
return !!window.Worker;
```

Widgets: estou em um? #

```
return typeof widget !== 'undefined';
```

XMLHttpRequest: requisições cross-domain #

```
return "withCredentials" in new XMLHttpRequest;
```

XMLHttpRequest: envio como formData #

```
return !!window.FormData;
```

XMLHttpRequest: eventos de progresso de upload #

```
return "upload" in new XMLHttpRequest;
```



LEITURA COMPLEMENTAR

Especificações e normas:

- [HTML5](#)
- [Geolocalização](#)
- [Server-Sent Events](#)
- [WebSimpleDB](#)
- [Web Sockets](#)
- [Web SQL Database](#)
- [Web Storage](#)
- [Web Workers](#)
- [Widgets](#)
- [XMLHttpRequest Level 2](#)

Bibliotecas JavaScript:

- [Modernizr](#), uma biblioteca HTML5 de detecção